PROJECT RECORD · COMPLETE HISTORY

JAYKUMAR SHEVATE PORTFOLIO

From the first brief to the current release: every request, design, decision, defect, test, measurement, commit and push, with the complete source code of both versions.

Product	Personal portfolio website of Jaykumar Chandrakant Shevate
Repository	github.com/Jayshevate/Portfoilo (public), branch main
Period	3 to 9 November 2026
Versions	v1 "Life of a Request" (3D) and v2 streaming catalogue (current)
Commits and pushes	13 commits in 7 pushes before this document; 0 public deployments
Quality at issue	25 unit tests, 18 browser scenarios, 7 load runs: all passing, 0 failed requests
Lighthouse at issue	Phone 98 / 100 / 100 / 100; desktop 100 / 100 / 100 / 100
Open items	50 facts awaiting the owner; site not yet hosted
Document	Version 1.0, issued 9 November 2026
Prepared by	Claude (AI coding assistant), for the project owner

CONTENTS

Section	Page
1. Document control	3
2. Executive summary	4
3. Key facts at a glance	5
4. The original brief	5
5. Chronology of the engagement	8
6. Phase 1: the plan and the three art directions	10
7. Repository and GitHub setup	11
8. Version 1: "Life of a Request"	11
9. Version 1 visual iterations	13
10. Version 1 defects and fixes	14
11. Version 1 completion and performance round	15
12. The rebuild decision	15
13. Version 2 architecture	16
14. Version 2 component reference	17
15. The energetic update	19
16. The 60-second recruiter brief	19
17. Tailored links, the Making-of and the Pune clock	20
18. Testing	20
19. Performance history	24
20. Hosting, deployment and pushes	25
21. Security and privacy review	26
22. Accessibility statement	27
23. Data integrity and missing content	28
24. Commit log	29
25. Known limitations and items not verified	29
26. Recommended next steps	30
27. Glossary	30
Appendix A. Verbatim test output	31
Appendix B. Complete source code of the current version	33
Appendix C. Complete source code of version 1	62
Appendix D. Screenshot gallery	97
Appendix E. How this document was verified	100

Reading guide

Pages 3 to 30 are the narrative record: what was asked, what was built, what went wrong, how it was fixed, and how it was tested. Pages 31 onwards are evidence: the verbatim output of the final test run (Appendix A), the complete

source code of the current version (Appendix B) and of the first version (Appendix C), screenshots (Appendix D) and the verification of this document itself (Appendix E). Section and page references in the text are exact. Dates and times are in UTC unless stated. Defects are numbered D1 to D32 and test corrections T1 to T4; the same numbers are used wherever they are mentioned.

1. DOCUMENT CONTROL

Item	Detail
Document	Project Record: Jaykumar Shevate Portfolio, from first brief to current release
Version	1.0 (first complete edition)
Date of issue	9 November 2026
Prepared for	Jaykumar Chandrakant Shevate (project owner)
Prepared by	Claude, the AI coding assistant that carried out the work in a Claude Code cloud session
Repository	`https://github.com/Jayshevate/portfoilo` (public, branch `main`, head commit `f63b54a`)
Period covered	3 November 2026 (first brief) to 9 November 2026 (this document)
Status of the product	Code complete and fully tested; content incomplete (50 facts still marked as missing); never deployed to a public host
Classification	Personal project record. Contains no passwords, keys or private data; the owner's email address appears only as the git author identity

1.1 Purpose of this document

This document is the complete written record of the portfolio project: what was asked for, in the words it was asked; what was planned; every design direction that was proposed, chosen, built and replaced; every defect that was found and how it was fixed; every test, audit and measurement that was run, with its real result; every commit and every push to GitHub; and the exact state of the product and its content on the day of issue. It is written so that the owner can explain any part of the project in an interview, so that a future developer (human or AI) can continue the work without guessing, and so that nobody reading it is misled about what was and was not done.

1.2 Sources of evidence

Every number in this document comes from one of five sources, and the source is named wherever it matters. The first source is the git history of the repository, which records 13 commits with their timestamps, authors and file statistics. The second is the saved output of the automated test suites, including a full run of `npm run check` performed on 9 November 2026 specifically for this document; that output is reproduced verbatim in Appendix A. The third is the set of twelve Lighthouse reports saved as JSON files during the project; their scores, timings and timestamps are tabulated in Section 19. The fourth is the build output of the current production bundle, measured file by file for this document. The fifth is the working session itself: the owner's messages, the questions asked and answered, and the actions taken in response. Where a number could not be measured or was not recorded, this document says so instead of estimating it.

1.3 How to read this document

Sections 2 and 3 summarise the whole project in a few pages. Section 4 reproduces the original brief. Section 5 is the chronology: one row per request, in order, with what was done about it. Sections 6 to 11 cover the first version of the site, "Life of a Request", a 3D scroll experience, through its four visual iterations. Sections 12 to 17 cover the second version, the streaming-catalogue site that is live in the repository today, and every feature added to it. Sections 18 to 23 are the quality record: testing, performance, hosting and deployment, security, accessibility and data integrity. Sections 24 to 27 hold the commit log, the known limitations, the recommended next steps and a glossary. The appendices contain the verbatim test logs, the complete source code of the current version, the complete source code of the first version as it stood in its final commit, and a gallery of screenshots.

1.4 A note on honesty

The owner asked for a document in which nothing is left out, and asked, among other things, how many times the site was hosted. The truthful answer to that question is zero: the site has never been deployed to any public host, because the original brief said "Do not deploy anything anywhere; local dev only", and no later instruction changed that. What did happen many times is that the site was served from a temporary local web server inside the cloud session, for testing; those local servers are described accurately in Section 20 and are not presented as hosting. The same principle applies throughout: where something was not tested on real hardware, the document says so; where a screenshot was not preserved, the document says so; where a count was not tracked exactly, the document gives the best verifiable figure and labels it.

2. EXECUTIVE SUMMARY

Jaykumar Shevate, a final-year B.Tech student in Computer Science and Engineering with a Cyber Security specialisation at D. Y. Patil International University in Pune, and a Teaching Associate at the same university since August 2025, asked for a personal portfolio website that would help him win cloud, backend, junior DevOps and security roles and freelance work. The original brief, given on 3 November 2026, asked for an award-level, scroll-driven 3D experience called "Life of a Request" in which the visitor plays an HTTP request travelling through a cloud system, with strict rules that no fact about him may be invented.

The first version was built to that brief with React, react-three-fiber, GSAP ScrollTrigger and Lenis. It contained a particle system that drew his name and collapsed into a "packet", a security gate that burned open for each certification, five project scenes, a terminal overlay, generated ambient sound, a custom cursor, magnetic links, a simple view for recruiters and a README. Its visual design went through three palettes at the owner's request (a blue Cyanotype blueprint, a light Paper and Ink look, and a hand-made Field Notebook look with "boiling" lines). It was pushed to GitHub on 4 November and optimised on 5 November. Lighthouse on a simulated phone then scored 60 for performance and 100 for accessibility, best practices and SEO, measured on a server that had to run WebGL on its processor because it has no graphics card.

On 8 November the owner asked for the site to be reconsidered from a recruiter's point of view and rebuilt as something like a streaming platform, unique, thoroughly tested (including many simultaneous visitors), audited, and written to the "Karpathy guidelines" and "the ponytail". Two questions were asked before starting: "ponytail" was confirmed to mean Tailwind CSS, and the streaming-platform concept was confirmed. The site was rewritten from scratch as an original streaming-style catalogue with Tailwind: a "Who's here?" viewer picker, a billboard, rows of titles, title pages with "episodes", a trailer built only from real facts, and an end-credits contact section. It is pre-rendered to static HTML at build time and hydrated by React. Every 3D, animation and state library was removed. The coding rules were written into a `CLAUDE.md` file in the repository.

Four further rounds followed: a more energetic presentation (kinetic name, highlights ticker, living posters, numbered projects row); a "60-second brief" for recruiters with per-role evidence, logistics and a one-click summary for applicant-tracking systems; tailored links that greet a named company and lead with the role it is hiring for; a "Making of this site" case study; and a live Pune clock.

The current version passes 25 unit tests, 18 browser scenarios (including an automated WCAG 2.1 AA accessibility scan, JavaScript disabled, blocked storage, a code-injection attempt and 25 simultaneous visitors) and a load test of 200 parallel connections with zero errors. Its latest Lighthouse scores are 98 (phone) and 100 (desktop) for performance and 100 for accessibility, best practices and SEO. Its total JavaScript is about 61 KB compressed.

Two things stand between this product and its purpose. First, 50 facts that only the owner can supply are still marked as missing, including his email address, résumé PDF and the technology stack of four of the five projects; until they are filled in, a recruiter cannot contact him from the site and the project pages are mostly empty. Second, the site has not been deployed, so it is not yet reachable by recruiters at all. Both are owner decisions, and Section 26 sets out how to complete them.

3. KEY FACTS AT A GLANCE

Measure	Value	Source
Duration of the engagement	3 to 9 November 2026 (7 calendar days)	Session record, git timestamps
Messages from the owner	20, of which 7 were sent while work was already running; plus 2 sets of answers to multiple-choice questions	Session record (Section 5)
Clarifying questions asked	3 multiple-choice questions in 2 prompts (palette for the first redesign; meaning of "ponytail"; concept for the rebuild), plus the opening request for an art-direction pick and reference sites, which was not answered	Session record
Major versions	2: "Life of a Request" (3D) and the streaming catalogue (current)	Git history
Visual iterations of version 1	3 palettes built (Cyanotype, Paper and Ink, Field Notebook); 3 more proposed but not built	Section 9
Commits	13 before this document, all on `main`; this document and a one-line test rename are committed as the 14th	`git log`
Pushes to GitHub	7 successful pushes	Section 20
Public deployments (hosting)	0 (never deployed, as the brief required)	Section 20
Local test servers	Production preview server on port 4173 (started repeatedly), development server on port 5173 (once)	Section 20
Unit tests (current)	25 tests in 6 files, all passing	Appendix A
Browser end-to-end scenarios (current)	18, all passing	Appendix A
Load test (current)	2 targets x 200 connections x 10 s, 0 errors, 0 timeouts, 0 non-200 responses	Appendix A
Lighthouse reports saved	12	Section 19
Latest Lighthouse, phone	Performance 98, Accessibility 100, Best practices 100, SEO 100	`lh6.json`
Latest Lighthouse, desktop	Performance 100, Accessibility 100, Best practices 100, SEO 100	`lh5-desktop.json`
Production JavaScript	191,758 bytes raw, 61,157 bytes gzip-compressed (one file)	Build output, 9 Nov
Production CSS	30,410 bytes raw, 6,574 bytes compressed	Build output, 9 Nov
Pre-rendered HTML	164,398 bytes raw, 15,251 bytes compressed	Build output, 9 Nov
Tracked source files (excluding media)	45 files, 7,410 lines including the lockfile	`git ls-files`
Facts still missing	50 (49 in `profile.js`, 1 in `brief.js`)	`npm run todo`
Defects found and fixed	32 in the product (17 in version 1, 15 in version 2), plus 4 corrections to the tests themselves	Sections 10, 18.7, 18.8
Load test runs recorded	7 runs, 14 measurements, 0 failed requests	Section 18.4

4. THE ORIGINAL BRIEF

The project began with a single, detailed message on 3 November 2026. It is reproduced here exactly as it was given, including its placeholders, because it remained the governing document for the facts about the owner throughout the

project. The placeholders it contains ([FILL], [PASTE], [URL] and similar) were never filled in during the project, which is why the site still shows "missing" markers in the same places.

```
Act as a senior creative technologist and award-level WebGL developer. Build my
portfolio as a single cinematic, scroll-driven 3D experience called "Life of a
Request". Do not use any template look. Make something that feels art-directed
by a person with a point of view.

CORE IDEA
The visitor is a request travelling through a cloud system. Scroll moves a
camera along a spline through a 3D world:
  Edge/Hero -> Security Gate (certifications) -> Load Balancer (About) ->
  Service districts (one per project) -> Data layer (skills/stack) ->
  Response (Contact, hire-me CTA, resume download).

ABOUT ME (source of truth: the resume below; never invent metrics, employers,
users, repos or outcomes; write [VERIFY] where unknown)
[PASTE FULL RESUME TEXT]
GitHub: [URL] | LinkedIn: [URL] | Email: [EMAIL] | Resume PDF: [PATH]
Target roles: cloud, backend, junior DevOps, security. Also open to freelance work.

PROJECT SCENES (each gets a unique visual world + a case-study panel with
problem, architecture, stack, my role, links; details I provide: [PASTE])
1. CarePilot: healthcare network as a glowing node graph; records "hop" between hospitals
2. RentFlow: a small building whose units light up as rent/status changes
3. Sensorium Nexus: live-looking IoT sensor streams, a real-time dashboard in 3D
4. BazaarLens: products racing across store lanes, the cheapest price wins
5. DYPIU PhD ERP: a doctoral pipeline moving a student through approvals

SIGNATURE MOMENTS (make these exceptional, keep the rest restrained)
- Hero: particles assemble into my name, then dissolve into the packet that starts the journey.
- Security Gate: the packet is "scanned"; each certification (AWS CCP, AI-900) unlocks a gate.
- A terminal overlay (press ~) with commands: whoami, ls projects, open <project>, contact.
- Cursor and scroll feel: custom easing, inertia, magnetic hover on links.
- A sound toggle (off by default) with subtle ambient audio.

TECH
Vite + React + react-three-fiber + drei + GSAP ScrollTrigger + Lenis. Custom GLSL
shaders for at least two effects. No paid services or backends; the contact form
uses a free form service. Do not deploy anything anywhere; local dev only.

ART DIRECTION
Pick ONE distinct palette and ONE font pairing (not Inter, no purple-blue
gradients, no frosted-glass cards). Ask me for 2-3 reference sites first.
Write all copy in a direct first-person voice with specific details, no clichés
like "passionate developer".

PERFORMANCE & QUALITY
- 60fps on mid laptops; lazy-load assets; DPR cap; compressed textures; total
  first load under ~3s on 4G.
- Mobile: simplified scenes, no heavy post-processing. Respect prefers-reduced-motion.
- Real HTML text for SEO/accessibility (not text only in the canvas), keyboard
  navigation, a "skip animation / simple view" button for recruiters in a hurry.
- Clean, commented-by-hand-style code structure and a README explaining
  how every major part works, so I can explain it in interviews.

PROCESS
1. Show me the plan, file structure, and 3 art direction options. Wait for my pick.
2. Build Hero + the first transition only. Wait for feedback.
3. Then one scene at a time, testing performance after each.

=== PORTFOLIO SOURCE OF TRUTH ===
Rules: use ONLY what is written here. Never invent metrics, users, employers,
repos, URLs or outcomes. Where a field says [FILL] or [VERIFY], leave a clearly
marked placeholder instead of guessing.

--- IDENTITY ---
Name: Jaykumar Chandrakant Shevate
Location: Pune, Maharashtra, India
Current role: Teaching Associate, D. Y. Patil International University (since Aug 2025)
Targeting: cloud, backend, junior DevOps, security roles; also open to freelance work
```

```
Languages: English, Hindi, Marathi (learning German, Japanese, Russian)
Tagline / one-line bio: [FILL, in my own voice]
Photo: [FILL path]

--- LINKS ---
GitHub profile: [FILL https://github.com/<username>]
LinkedIn: [FILL https://www.linkedin.com/in/<handle>]
Email: [FILL]
Resume PDF: [FILL path]
Credly / certificate verification links: [FILL, optional]

--- EDUCATION ---
B.Tech, Computer Science & Engineering (Cyber Security),
D. Y. Patil International University, graduating 2026
GATE 2026 (Data Science & AI): qualified

--- CERTIFICATIONS ---
AWS Certified Cloud Practitioner: score 1000/1000, Mar 2026, valid through Mar 2029
  Verification link: [FILL]
Microsoft Azure AI Fundamentals (AI-900): score 929/1000, Mar 2026
  Verification link: [FILL]
AWS re/Start Graduate Program: completed Sep 2025

--- EXPERIENCE ---
1. Teaching Associate, D. Y. Patil International University, Aug 2025 to present
   What I do / achieved: [FILL]
2. Software Developer Intern, Technocia Infotech, May to Aug 2025
   What I built / tech used / outcome: [FILL, add [VERIFY] for any metric]
3. Cybersecurity Research Intern, FirstVidya Pvt. Ltd., Feb to Apr 2025 (remote)
   What I researched / tools / outcome: [FILL, add [VERIFY] for any metric]

--- PROJECTS ---
(For each: one-paragraph description, problem solved, tech stack, my role,
 architecture notes, 2-3 screenshots or a recording, repo link, live/demo link.)

1. DYPIU PhD ERP
   Doctoral programme management system for D Y Patil International University.
   Role: sole developer. Status: UAT / pilot stage.
   Stack: [FILL] | Repo: [FILL, or "private"] | Demo: [FILL] | Screenshots: [FILL]

2. CarePilot
   Healthcare SaaS / national health network.
   Stack: [FILL] | Repo: [FILL] | Demo: [FILL] | Screenshots: [FILL]

3. RentFlow
   Rental property management platform.
   Stack: [FILL] | Repo: [FILL] | Demo: [FILL] | Screenshots: [FILL]

4. Sensorium Nexus
   IoT monitoring platform.
   Stack: [FILL] | Repo: [FILL] | Demo: [FILL] | Screenshots: [FILL]

5. BazaarLens
   AI shopping price comparison for India.
   Stack: [FILL] | Repo: [FILL] | Demo: [FILL] | Screenshots: [FILL]

Other GitHub repos to feature: [FILL, list name + URL + one line each]

--- SKILLS ---
Cloud: AWS, Azure AI [FILL exact services I've used]
Backend / languages / frameworks: [FILL]
Security: [FILL]
Databases / tools / DevOps: [FILL]
Currently learning: Python, German, Japanese, Russian

--- CONTACT SECTION ---
Call to action text: [FILL, e.g. "Hiring for cloud/backend? Let's talk."]
```

4.1 What the brief fixed for the whole project

Several requirements in the brief outlived the 3D concept and still govern the current version. The facts rule ("use ONLY what is written here; never invent metrics, users, employers, repos, URLs or outcomes; leave a clearly marked placeholder") is enforced today by the data file design, by a visible "missing" marker, and by unit tests that pin every score and date. The "local dev only, do not deploy" instruction was followed throughout. The requirements for real HTML text, keyboard navigation, reduced-motion support, a fast route for recruiters in a hurry, a free form service with no backend, and a README that explains every part for interviews all carried over into version 2. The art-direction constraints (one palette, one font pairing, not Inter, no purple-blue gradients, no frosted glass) were respected in every design; a frosted-glass backdrop blur that crept into an early draft of the version 2 title dialog was removed before release for this reason.

4.2 Where the project departed from the brief, and why

- **Process steps 2 and 3** ("build Hero and the first transition only, wait for feedback; then one scene at a time"): the owner instructed on 3 and 4 November to finish everything and to "work non-stop", so the remaining scenes were built in one pass rather than one at a time with a pause after each.
- **Reference sites**: the brief asked for 2 or 3 reference sites before design began. They were requested in the first reply but never supplied, so design decisions were made from the brief and from the owner's later choices.
- **The 3D concept itself** was replaced on 8 November, when the owner asked for the site to be redesigned from a recruiter's perspective, "like a Netflix platform or something like that". The terminal, sound toggle, particle hero and security-gate scene therefore exist only in version 1, which is preserved in git history and reproduced in Appendix C.
- **Tech stack**: the brief named Vite, React, react-three-fiber, drei, GSAP ScrollTrigger and Lenis. Version 2 keeps Vite and React, adds Tailwind CSS at the owner's request, and drops the rest because the new design does not need them (the "simplicity first" rule).

5. CHRONOLOGY OF THE ENGAGEMENT

The table below lists every message the owner sent, in order, with the date (UTC, from the session and from git timestamps), the request in the owner's own words, and what was done in response. Messages marked "while running" arrived while work on the previous request was still in progress and were handled inside that same round of work. Commit hashes refer to Section 24.

#	Date	Request	Response
1	3 Nov	The full brief (Section 4)	Replied with the plan, the file structure and three art directions (Cyanotype, Amber Phosphor, Sodium Night); asked for a pick and for 2 or 3 reference sites. No code was written yet, as the brief's process required.
2	3 Nov	"after finishing push it on my github as repo name as portfilo"	Checked GitHub access from the session: the session could only reach repositories already connected to it and could not create new ones. Asked the owner to create the empty repository. Because no art direction had been picked and the owner wanted the work finished, proceeded with the recommended direction, Cyanotype, and stated that it could be changed later. Scaffolded the project and installed dependencies.
3	4 Nov	"i should create an repo or an project"	Explained that a GitHub repository is needed, not a GitHub Project, with numbered steps: create it empty (no README, .gitignore or licence), then grant the Claude GitHub App access.
4	4 Nov	"done, created repo named portfolio"	Tried to attach `Jayshevate/portfolio`; GitHub reported it as not found or not accessible. Told the owner and continued building.
5	4 Nov, while running	"Portfoilo is the name of the repo check again"	Attached `Jayshevate/Portfoilo` successfully. Cloned it (empty) to `/home/user/portfoilo` and moved the project into it.

#	Date	Request	Response
6	4 Nov, while running	"now create the work given to you an all okay and then give it to me"	Continued building all of version 1: 3D scenes, shaders, HTML sections, terminal, styles.
7	4 Nov, while running	"different colors and design plz"	Asked which direction to switch to, offering four options; the owner chose "Paper and Ink (light)". Rebuilt the palette and fonts.
8	4 Nov, while running	"now worknon-stop and then give me the output and only stop after completing of the entire application or software"	Continued without pausing: tested in a browser, fixed defects, prepared README.
9	4 Nov, while running	"again change the design and colors see choose by yourself but the like should look like its been made by an human ..."	Chose and built the Field Notebook direction (manila paper, graphite, ballpoint blue, red pen, highlighter; Young Serif and Courier Prime; hand-drawn wobble and line boil). Fixed a crash in the terminal. Wrote the README. Five commits pushed (c398f70 to 5d926c9).
10	4 Nov	"100 percent done ???"	Answered honestly: code about 95 percent, content about 40 percent; listed what was missing and what had not been verified (real-GPU frame rate, real 4G timing, real phones, screen readers).
11	4 Nov	"what all things are missing and all in it"	Listed every missing fact as a fill-in form, grouped by section, with the order in which to fill them.
12	4 Nov	"preview"	Recorded a 60-second scroll-through video of the built site and sent it, noting that it was recorded without a GPU and is choppier than real.
-	4 Nov	(Automatic end-of-turn check: "There are untracked files in the repository")	Committed the preview video with a link from the README and pushed (2056503).
13	5 Nov	"complete everything and then wait for the next command"	Added optional photo, screenshot and extra-repository slots and `npm run todo`; ran Lighthouse; fixed contrast, robots.txt, layout shift and first-paint speed; pushed (fd93879). Reported the preview port to the app.
14	8 Nov	Rebuild request: recruiter perspective, "like a Netflix platform or something like that", unique, not AI-looking, test that many recruiters at once are not misled, audit, follow "the Karpathy guidelines and also the ponytail", rewrite everything	Researched the Karpathy guidelines. Asked what "ponytail" meant (answer: Tailwind CSS) and which concept to build (answer: streaming platform). Rewrote the site from scratch; wrote unit, browser, accessibility and load tests; fixed every defect they found; pushed three commits (fe76e38, 1f30dd0, 85bf6ac).
15	8 Nov, while running	"Try again"	Sent after the session's worker process restarted mid-task. Work resumed from where it stopped.
16	8 Nov, while running	"dont stop"	Work continued to completion in the same round.
17	8 Nov	"more engenerejetic" (read as "more energetic")	Added kinetic name, highlights ticker, living posters, numbered projects row, glowing hovers, faster trailer, animated picker; added a reduced-motion test; pushed (6f05b7e).
18	8 Nov	"something like if any recruiter in the world sees it he will definetilitey give me an job for fix"	Explained that no website can guarantee an offer, then built the 60-second recruiter brief, per-role evidence, logistics and an ATS summary; fixed two phone defects; pushed (1818868).
19	9 Nov	"make it even better and more unique"	Built tailored links per application, the "Making of this site" extra and a live Pune clock, with an injection test; pushed (f63b54a).
20	9 Nov	Request for this 100-page document	Re-ran every test suite, gathered every saved report and wrote this record.

5.1 Answers the owner gave to questions

When	Question	Owner's answer
3 Nov	Pick one of three art directions and send 2 or 3 reference sites	Not answered directly; the owner asked for the work to be finished, so the recommended direction was used
4 Nov	Which direction should the whole site switch to? (Amber Phosphor, Sodium Night, Paper and Ink, monochrome with lime)	Paper and Ink (light)
8 Nov	What did you mean by "ponytail"? (Tailwind CSS, skip it, send a link)	Tailwind CSS
8 Nov	Which concept should the whole site be rebuilt as? (Streaming platform, streaming plus the 3D film, something else)	Streaming platform (the recommended option)

6. PHASE 1: THE PLAN AND THE THREE ART DIRECTIONS

Following step 1 of the brief's process, the first reply contained no code. It set out how the experience would work, the file structure, and three art directions, and asked the owner to choose.

6.1 The plan as proposed

One scroll value would drive everything. Lenis would smooth native scrolling, GSAP ScrollTrigger would turn it into a single progress value per section stored outside React, and the camera would read that value every frame to travel along a Catmull-Rom spline, looking slightly ahead so that turns felt like banking. Content would be real HTML sections laid over a fixed canvas, so search engines, screen readers and keyboard users would get the full text even if WebGL failed; drei's ScrollControls was rejected because it moves text into its own scroll container. The route was planned as Edge (hero), Security Gate (certifications), Load Balancer (about), five project districts, Data layer (skills) and Response (contact). The signature moments were planned in detail: a particle name morphing into the packet in a single GPU draw call, a scan band and noise-dissolve gates, a keyboard terminal with an ARIA live region, magnetic links capped at about 8 pixels, and ambient sound generated with the Web Audio API so that no audio file would need downloading. A recruiter mode ("simple view") would hide the canvas and show the same sections as a one-page CV. A performance budget was set: DPR capped at 1.5 on desktop and 1 on mobile, procedural geometry instead of model files, lazy-loaded districts, and a JavaScript target of about 350 KB compressed.

6.2 The three art directions offered

Direction	Palette	Fonts	Reasoning given
A. Cyanotype (recommended)	Prussian paper #0F2D4A, cream ink #EDE6D6, vermilion #E8452C, faded line #7FA3BF	Fraunces with IBM Plex Mono	Says "I think in architecture"; line drawings are cheap to render, safest for 60 fps and mobile
B. Amber Phosphor	Near-black #0C0B09, amber #FFB347, deny-red #FF3B2F, bone #D9D2C3	Bricolage Grotesque with JetBrains Mono	Night-shift operations centre; risk of the "hacker" cliché, so amber not green and little glitch
C. Sodium Night	Concrete #1A1918, sodium orange #F28C28, wet slate #5E6B66, paper #F4EFE6	Clash Display with Geist Mono	Pune after midnight in the monsoon; most personal and most cinematic, but heaviest to render

The reply also listed what was needed before building (a pick, reference sites, the exact name to draw in particles) and what would be needed later (links, résumé, project details, role details, specific services and tools, tagline, choice of form service).

7. REPOSITORY AND GITHUB SETUP

The owner asked for the finished work to be pushed to GitHub under the name "portfilo". The cloud session's GitHub access works per repository: it can only read and write repositories that the owner has connected to the session, and it cannot create new ones. A check confirmed this: the GitHub CLI could identify the account (`Jayshevate`) but could not list or create repositories, and requests for `Jayshevate/portfilo` and `Jayshevate/portfolio` were refused because neither was connected. The owner was therefore asked to create the repository, and to create it empty so that the first push would not conflict with a generated README or licence.

Step	What happened
Requested name	"portfilo" (3 Nov)
Owner asked	Whether to create a "repo" or a "project"; the answer was a repository, with step-by-step instructions (4 Nov)
First report	"created repo named portfolio"; attaching `Jayshevate/portfolio` failed: not found or not accessible to the session
Correction	"Portfoilo is the name of the repo"; attaching `Jayshevate/Portfoilo` succeeded
Clone	The repository was empty (no commits); it was cloned to `/home/user/portfoilo` and the project files, built so far in `/home/user/portfilo`, were moved into it
Visibility	Public (confirmed through the GitHub API on 9 Nov); canonical address `https://github.com/Jayshevate/Portfoilo`; the lower-case form `portfoilo` resolves to the same repository
Branch	All work was committed directly to `main`, the only branch, because the repository was new and empty and the owner asked for the work to be pushed there
Author identity	Commits are authored as Jaykumar Shevate <jaykumarshevate.w@gmail.com>, with a co-author line crediting the AI assistant and a link to the session

The repository name is spelled "Portfoilo". It is not a typing error in this document. If the owner prefers "portfolio", GitHub can rename the repository and will redirect the old address, but the clone command in the README and the repository link inside the "Making of" title would then need to be updated.

8. VERSION 1: "LIFE OF A REQUEST"

Version 1 implemented the brief as written: a single scroll-driven 3D experience in which the visitor is an HTTP request. It was built between 3 and 5 November and lives in commits c398f70 to fd93879. Its full source code, as it stood in its final commit, is reproduced in Appendix C. This section explains how it worked.

8.1 Technology

Package	Version	Role
react, react-dom	18.3.1	User interface
three	0.169.0	WebGL rendering
@react-three/fiber	8.17.10	React renderer for three.js
@react-three/drei	9.114.0	Helpers (PerformanceMonitor)
gsap (with ScrollTrigger)	3.12.5	Scroll progress per section
lenis	1.1.14	Smooth scrolling with inertia
zustand	4.5.5	Small store for rarely changing state
vite	5.4.10	Development server and bundler
vite-plugin-glsl	1.3.0	Importing GLSL shader files with #include support

8.2 The route

The journey was described as data in `src/core/route.js`. Each stop had a camera position, a look-at point (where that stop's scene was placed), a framing mode, a movement style and a section height in viewport heights. Two Catmull-Rom splines ran through the camera and look-at points; stop *k* sat exactly at spline parameter k/(N-1).

Stop	Content	Section height	Camera movement
Edge	Hero: the name in particles, then the packet	230 vh	Holds while the name dissolves, then leaves
Security gate	Certifications; two gates burn open	320 vh	Linear fly-through
Load balancer	About and experience	210 vh	Dwell, then travel
CarePilot	Project district	260 vh	Dwell, then travel
RentFlow	Project district	260 vh	Dwell, then travel
Sensorium Nexus	Project district	260 vh	Dwell, then travel
BazaarLens	Project district	260 vh	Dwell, then travel
DYPIU PhD ERP	Project district	260 vh	Dwell, then travel
Data layer	Skills; the camera dives below the floor	230 vh	Dwell, then travel
Response	Contact form, links, résumé, "200 OK"	Natural height	Holds

8.3 Scroll and camera

Lenis replaced wheel scrolling with an eased version while keeping native scrolling, so the scrollbar, keyboard, find-in-page and screen readers kept working. GSAP's ticker drove Lenis so the page had a single animation clock. One ScrollTrigger per section reported progress from 0 to 1 into a plain JavaScript object; this was deliberately not React state, because it changed on every scroll event and would otherwise have re-rendered the application sixty times a second. The camera rig damped toward the target position with a frame-rate-independent formula and added a few centimetres of mouse parallax. To keep the 3D scene out from under the text panels, the rig shifted the projection with `camera.setViewOffset` rather than moving the camera: right of centre on desktop, upwards on phones.

8.4 Custom shaders

Shader	File(s)	What it did
Particle text	particles.vert, particles.frag	About 11,000 particles carried three positions (scattered, in the name, on the packet sphere). The vertex shader blended them on the GPU in one draw call, staggered each particle by a random value, and pushed particles through a noise field mid-transition so they swirled. The name was sampled from the real web font drawn on a hidden 2D canvas. The same component played backwards to spell "200 OK" at the end.
Packet scan	packet.vert, packet.frag	A Fresnel-lit packet with a bright scanning band and readout stripes that swept twice, once per certification gate, and flashed when accepted.
Gate dissolve	dissolve.vert, dissolve.frag	A hatched membrane in each gate; simplex noise decided which pixels burned away first, with a glowing edge.
Grid floor	ground.vert, ground.frag	A drafting-table grid antialiased with fwidth() so lines stayed one pixel wide at any distance, faded around the camera.
Line boil	Injected into three.js line materials (kit.js)	Added in the Field Notebook iteration: every line vertex jittered by a hashed offset that changed eight times a second, imitating hand-drawn animation; vertices shared by two segments moved together.

8.5 The five project districts

Project	Scene	Technique
CarePilot	Fourteen hospital nodes linked to their two nearest neighbours; records hop along the edges and the node they land on pulses	k-nearest-neighbour graph, instanced meshes on arcs

Project	Scene	Technique
RentFlow	A five-storey building, three units per floor; units change colour by rent state, re-rolling as you scroll	One instanced plane per unit with per-instance colour
Sensorium Nexus	Devices on the floor send readings up wires into six panels whose line charts redraw continuously	Vertex buffers shifted one sample at a time
BazaarLens	Four store lanes; boxes race, then the cheapest lane lights up, not the fastest	Random durations and prices per race
DYPIU PhD ERP	A student token moves through six stations, stamping each one	Token driven by scroll along a curve

All values in these scenes were random or simulated. They illustrated each idea and were never presented as data from the real systems; the README said so explicitly.

8.6 Interface features

- **Terminal**: opened with the tilde key or a button; commands help, whoami, ls projects, ls repos, open (with Tab completion), certs, contact, resume, simple, sound, clear and exit; Up and Down for history; an ARIA live log; focus returned on close.
- **Sound**: off by default; two detuned oscillators, filtered noise and a slow LFO through a low-pass filter, opening slightly at each stop.
- **Cursor**: a dot plus a lagging ring that grew over clickable items; only on mouse or trackpad.
- **Magnetic links**: elements leaned toward the cursor, capped at 8 pixels; first with gsap.quickTo, later with a small spring so that GSAP was not needed on first paint.
- **Simple view**: the first keyboard stop; removed the canvas, smooth scrolling and cursor and showed the same content as a readable page; automatic for `?simple`, missing WebGL or reduced motion.
- **Trace HUD**: a list of hop buttons on the right edge doubling as keyboard navigation.
- **Contact form**: posted to Web3Forms with a key from an environment file; without a key it said so instead of pretending to send.

9. VERSION 1 VISUAL ITERATIONS

The look of version 1 changed three times in two days. Screenshots of the first two looks were taken and reviewed during the work but were overwritten by later captures and were never committed, so they cannot be reproduced in this document; the descriptions below are taken from the code and the session. Screenshots of the final look, the Field Notebook, survive and appear in Appendix D.

Iteration	Trigger	Palette	Fonts	Signature details
1. Cyanotype	Default after no pick was made (3 Nov)	Prussian blue #0F2D4A, cream #EDE6D6, vermilion #E8452C, faded #7FA3BF	Fraunces with IBM Plex Mono	Blueprint grid, engineering-drawing title blocks on every panel (drawing number, title, revision), registration marks
2. Paper and Ink	Owner: "different colors and design plz", then chose this option (4 Nov)	Warm paper #F1ECE2, black ink #141414, red-orange #FF4A1C, pencil grey #8A847A	Instrument Serif with Space Mono	Black ink lines on paper, hard offset shadows, inverted black terminal
3. Field Notebook	Owner: "change the design ... look like its been made by an human ... choose by yourself" (4 Nov)	Manila #ECE3D0, graphite #2B2A28, ballpoint blue #2A44A8, red pen #C8372D, highlighter #F3DF57	Young Serif with Courier Prime	Line boil in 3D, SVG turbulence wobble on outlines, taped pages at deterministic angles, ruled lines, red margin rule, rubber "verified" stamps, crooked highlighter, hand-drawn arrow, circled red "missing" notes

The Field Notebook direction was chosen to answer the request that the site should not look machine-made. Each human-looking detail was produced deterministically, from page numbers or seeded values rather than random numbers, so the page looked the same on every visit and never shifted between loads. To keep text crisp, the wobble filter was applied only to separate outline layers, never to the text itself.

10. VERSION 1 DEFECTS AND FIXES

Version 1 was checked by building it, serving the production build locally and driving it with a headless Chromium browser: screenshots of every stop at desktop and phone sizes, scripted keyboard and terminal sequences, and console error capture. The following defects were found and fixed. Each was confirmed fixed by re-running the same check.

#	Defect	How it was found	Fix
D1	The packet shader failed to compile: `active` is a reserved word in GLSL	Console error on first browser run	Variable renamed to `scanning`
D2	Particle noise and point size were multiplied by the text group's scale (about 11 times), which would have made the swirl far too large	Code review before the first run	Added a `uInvScale` uniform so noise is measured in world units
D3	The scanning ring rotated with the packet instead of staying level	Code review	Moved the ring outside the spinning group
D4	The particle name overlapped the hero text	Screenshot review	Name raised and narrowed
D5	The security gates showed through the hero name	Screenshot review	Gates hidden until the name has mostly dissolved
D6	The BazaarLens scene ran off the right edge of the screen	Screenshot review	Lanes shortened, group repositioned and re-angled
D7	The packet was too large on screen	Screenshot review	Radius reduced from 0.17 to 0.10 world units
D8	Every skill showed "[FILL: function]" because arrays have a built-in `.fill` method and were mistaken for placeholders	Screenshot review	Placeholder check now requires a plain object
D9	The page went blank after several terminal commands: newer Chrome returns a Promise from `scrollIntoView`, an effect returned it, and React tried to call it as a cleanup function ("destroy is not a function")	Scripted terminal test; reproduced on the development server for a readable stack trace	Effect body wrapped in braces so it returns nothing; comment added explaining why
D10	The cursor animation loop touched a removed element after the page was unmounted	Same test (secondary error)	Guard added
D11	Placeholder text was hard to read because the wobble filter distorted the letters	Close-up screenshot	Filter moved to a separate outline layer
D12	On phones the fixed top bar overlapped content	Phone screenshot	Solid bar background with a rule on small screens
D13	Text contrast failed: pencil grey 2.81:1 and red pen 4.08:1 on manila	Lighthouse audit	Text colours darkened to 4.88:1 and 4.86:1 (WCAG AA needs 4.5:1)
D14	No robots.txt: the single-page server returned the HTML page instead, producing 39 Lighthouse errors	Lighthouse audit	Added `public/robots.txt`
D15	Layout shift of 0.078 when the HTML name was hidden after the particles loaded	Lighthouse audit	HTML name hidden visually from the start in the 3D view (kept for search engines and screen readers)

#	Defect	How it was found	Fix
D16	Layout shift of 1.0 (the maximum) in the simple view, because the view was chosen after the first paint	Lighthouse audit	A few lines of inline script now choose the view before anything is drawn
D17	First paint of 3.5 s on a simulated slow phone, because nothing appeared until JavaScript arrived	Lighthouse audit	The hero was rendered into the HTML at build time by a small Vite plugin

11. VERSION 1 COMPLETION AND PERFORMANCE ROUND

On 5 November the owner asked for everything to be completed. The round added optional content slots that stay hidden until filled (a photo shown as a taped polaroid, screenshots per project, a list of extra repositories, also available in the terminal as `ls repos`), the `npm run todo` command that lists every missing fact, and a performance and accessibility programme driven by Lighthouse.

Change	Effect
Magnetic links rewritten as a small spring without GSAP	GSAP and Lenis moved out of the first-paint bundle
New `nav.js` loads the scroll module on demand	The simple view never downloads GSAP or Lenis unless a navigation link is used
3D canvas mounted only after the page has loaded and the browser is idle	Text is readable before three.js downloads; the world fades in afterwards
Hero pre-rendered into index.html at build time	First paint without JavaScript
Inline script chooses simple or 3D view before paint	Simple view layout shift fell from 1.0 to 0.014
Colour contrast fixes and robots.txt	Accessibility and SEO reached 100

First-paint JavaScript fell from about 104 KB to about 56 KB compressed, with roughly 49 KB (scrolling) and 230 KB (3D) loading afterwards. The final Lighthouse results for version 1 were: 3D view on a simulated phone, performance 60, accessibility 100, best practices 100, SEO 100, first paint 2.9 s, layout shift 0; simple view, performance 88 with the same 100s. The low 3D performance score was dominated by "total blocking time" caused by WebGL running on the server's processor, which has no graphics card; it says little about a real laptop, and the README said so. The 60 frames per second target was never measured on real hardware.

12. THE REBUILD DECISION

On 8 November the owner asked for the project to be looked at as a recruiter would look at it, for a design that was "like a Netflix platform or something like that", unique and not AI-looking, completed fully, tested repeatedly (including with many recruiters using it at the same time, so that nobody is misled or meets a broken site), audited, and written according to "the Karpathy guidelines and also the ponytail", with every line rewritten.

12.1 Clarifying before building

A web search established what the Karpathy guidelines are: a widely shared, community-written set of rules for AI coding assistants, published in the repository forrestchang/andrej-karpathy-skills and based on Andrej Karpathy's public observations about the mistakes language models make when writing code. The repository states that it is not by Karpathy himself. Its four principles are: think before coding, simplicity first, surgical changes, and goal-driven execution. No coding guideline called "ponytail" could be found, so the owner was asked, and answered that it meant Tailwind CSS. The owner was also asked to confirm the concept and chose the streaming platform.

12.2 The recruiter analysis

The case for the streaming concept was written from the point of view of a recruiter screening many candidates. Recruiters have very little time per portfolio. A scroll-driven 3D film makes them wait for the story to reach the part

they need, while a streaming catalogue is a layout they already know: they can see the whole offer at a glance, jump straight to credentials or projects, and open any item in one click. The concept also gave natural homes to the owner's material: projects become shows, credentials become titles with their result as the poster, jobs become titles, and the contact section becomes the end credits.

12.3 Avoiding imitation

The new site borrows the interaction pattern of streaming services (profiles, billboard, rows, title pages, a trailer), but not any brand. It has its own name ("Shevate on demand"), its own palette (warm black, cream, marigold and a teal reserved for verified facts), its own fonts, and posters drawn from scratch. It uses no Netflix name, logo, red, sounds or artwork, and no fake engagement numbers such as match percentages, ratings or view counts, because those would be invented metrics and would break the brief's first rule.

12.4 Applying the guidelines

Guideline	How it was applied
Think before coding	Two questions were asked before any code was written; every later change was announced with its reason before it was made.
Simplicity first	Removed three.js, react-three-fiber, drei, GSAP, Lenis, zustand and the GLSL plugin. State is five useState hooks; routing is the URL hash; styling is Tailwind classes. Dead code found in review (an unused helper, a redundant form attribute) was deleted.
Surgical changes	After the rewrite, each later request touched only the files it needed, and commits are separated by purpose (site, tests, documentation).
Goal-driven execution	Every change ended with `npm run check` passing: unit tests, production build, browser tests and load test. When a test failed, the product or the test was fixed and the whole suite re-run.
Tailwind CSS	All styling uses utility classes; colours, fonts and keyframes are defined once in `tailwind.config.js`; a small `index.css` holds only base rules and keyframes that Tailwind cannot generate from class names.

These rules were written into `CLAUDE.md` at the root of the repository, together with the project-specific rules (facts only from `profile.js`; the first client render must match the pre-rendered HTML), so that any future AI-assisted change follows them automatically.

13. VERSION 2 ARCHITECTURE

13.1 One data file, one derived catalogue

`src/content/profile.js` holds every fact about the owner as plain data with no browser code, so Node can run it too (the tests and the pre-render step import it). A missing fact is written as `fill('what is missing')`. `src/content/catalog.js` turns the facts into two shapes the whole interface is built from: *titles* (anything that can be opened: projects, credentials, jobs, skill groups, the about page and the making-of) and *rows*. It also defines `VIEWERS`: each viewer type is only an order of rows. A recruiter sees credentials and experience first; an engineer sees projects first; everyone can reach everything.

13.2 Pre-rendered, then hydrated

`npm run build` runs three steps. Vite builds the client bundle; Vite builds `src/entry-server.jsx` for Node; and `scripts/prerender.mjs` renders the whole application to HTML once and writes it into `dist/index.html`, adding preload links for the fonts the first screen needs. The first response therefore already contains every project, credential and job, which helps slow phones, search engines and link previews, and keeps the whole catalogue readable with JavaScript turned off. React then hydrates that HTML. For hydration to succeed, the first client render must be identical to the server HTML, so everything that depends on the browser (stored choices, the URL hash, the query string, the clock) is read in an effect after the first render.

13.3 State without a state library

State	Where it lives	Why
Viewer type	React state, saved to localStorage	Remembered between visits
Picker open or closed	React state	Opened on first visit and when switching viewer
My list	React state, saved to localStorage	A recruiter's own shortlist
Recently opened	React state, saved to localStorage	Drives "Continue watching"
Search query	React state	Short-lived
Open title	URL hash `#title=id`	Shareable links and a working Back button
Tailoring	URL query `?for=&role=`	One link per application

All storage goes through `src/lib/storage.js`, which never throws: private windows, blocked cookies and full storage all make the browser's storage interface throw, and the site must keep working when that happens. Nothing is shared between visitors; there is no server and no database.

13.4 No flash of the picker

A short inline script in `index.html` runs before the page is painted. If the visitor has picked a viewer before, or arrived on a shared title link or a tailored link, it adds the class `skip-gate` to the page, and a CSS rule hides the picker immediately. Once React is running it adds the class `hydrated` and takes over. A `noscript` style hides the picker entirely for visitors without JavaScript, who would otherwise be stuck behind it.

13.5 Posters without image files

`src/components/Art.jsx` draws every poster in SVG from the title's id: stamped approval stations for the PhD ERP, a hospital network with one record's route for CarePilot, a block of flats with lit windows for RentFlow, sensor waves for Sensorium Nexus, racing price lanes for BazaarLens, the result set large for credentials, initials for jobs, and a browser window with a play button for the making-of. A seeded random generator keyed on the id makes each poster identical on every visit and between server and client, and React's useId gives every gradient a unique id so the same poster can appear many times on a page without duplicate ids.

13.6 Accessibility by construction

- Cards are real buttons, rows are lists under headings, the contact form uses labelled fields, and placeholders are announced as "missing".
- Title pages and the trailer are dialogs: focus moves in, Tab stays inside, Escape closes, focus returns to the element that opened them, and the page behind is made inert.
- The picker is the first element in the document, so after it closes the next Tab reaches "Skip to the catalogue".
- All text meets a 4.5:1 contrast ratio; "dust" grey on the background is about 7:1.
- Every animation stops under prefers-reduced-motion, and the trailer then waits for the visitor instead of advancing.

14. VERSION 2 COMPONENT REFERENCE

File	Responsibility
src/App.jsx	Page shell and all visitor state; decides row order, overlays and where the brief goes
src/main.jsx	Imports fonts and CSS; hydrates pre-rendered HTML or renders fresh in development
src/entry-server.jsx	Renders the app to an HTML string at build time
src/content/profile.js	Every fact; fill() placeholders; isFill and real helpers
src/content/catalog.js	Titles, rows, viewers, search, "more like this"

File	Responsibility
src/content/brief.js	Per-role evidence and the plain-text ATS summary
src/content/making-of.js	The "Making of this site" title and the repository address
src/lib/storage.js	localStorage that never throws
src/lib/hash.js	Reads and writes #title=id; Back closes a title
src/lib/tailor.js	Parses and cleans ?for= and ?role=
src/components/Gate.jsx	"Who's here?" picker
src/components/NavBar.jsx	Wordmark, section links, search with the / shortcut, résumé, viewer switch
src/components/Billboard.jsx	Kinetic name, key credential, tagline, buttons, drifting poster wall, highlights ticker, tailored greeting
src/components/Brief.jsx	The 60-second recruiter brief
src/components/Row.jsx	A horizontal row with paging arrows and optional numerals
src/components/TitleCard.jsx	One card, a real button
src/components/Art.jsx	All posters
src/components/TitlePage.jsx	The title dialog: facts, episodes, links, screenshots, more like this, My list, copy link
src/components/Trailer.jsx	Seven-shot trailer with progress bars, pause and step controls
src/components/Credits.jsx	Contact links, résumé, contact form, credits roll
src/components/PuneClock.jsx	Live time in Pune
src/components/Value.jsx	Renders a fact or a visible "missing" tag
src/index.css	Tailwind layers, the pre-paint picker rule, keyframes for posters, name, ticker and trailer, reduced-motion overrides
index.html	Metadata, structured data, the pre-paint script, the noscript rule, the render placeholder
tailwind.config.js	Palette, fonts, keyframes, hover only on devices that can hover
scripts/prerender.mjs	Writes the rendered HTML and font preloads into the build
scripts/todo.mjs	Lists every missing fact
scripts/link.mjs	Prints a tailored link for one application
tests/unit/*.test.js(x)	Six unit test files (Section 18.2)
tests/e2e/run.mjs	Eighteen browser scenarios (Section 18.3)
tests/load/run.mjs	Load test (Section 18.4)
CLAUDE.md	Coding rules for this repository
README.md	How to run, fill in, check and explain the project

14.1 Palette and type

Token	Value	Use
night	#0D0B09	Page background
soot	#17140F	Cards and panels
ash	#262119	Raised surfaces
line	#3A3328	Hairlines
cream	#F3EDE2	Main text
dust	#A89F90	Secondary text (about 7:1 on night)
marigold	#F2A23A	Actions, focus, the brand

Token	Value	Use
verified	#5CC2B3	Checked facts
missing	#FF8A7A	Placeholders

Headlines use Big Shoulders Display (weights 700 to 900), a condensed face with the feel of a cinema marquee. Body text uses IBM Plex Sans and metadata uses IBM Plex Mono. All fonts are self-hosted through Fontsource, so no request goes to a font service.

15. THE ENERGETIC UPDATE

On 8 November the owner asked for the site to be "more energetic". Every addition was kept to CSS animation, so no animation library returned, and every one stops under prefers-reduced-motion.

Addition	Detail
Kinetic name	Each letter of the name drops in on its own beat, then a marigold rule sweeps underneath; the full name stays in the HTML for screen readers and search engines
Highlights ticker	A tilted marigold strip runs every credential with its result, the PhD ERP role and status, the current job, the degree and the freelance line; all built from profile.js; pauses on hover; the duplicate copy that makes the loop seamless is hidden from assistive technology
Living posters	CarePilot's record route flows, RentFlow's windows flicker, Sensorium's waves scroll, BazaarLens's boxes race, the ERP student token hops
Numbered projects row	Large outlined numerals 1 to 5 beside the project cards, labelled "now showing"; a running order, not a ranking
Hover energy	Cards lift, scale and glow marigold; the Play button glows; picker tiles pop in one after another
Faster trailer	Shots shortened from 3.5 s to 2.8 s, with a zoom on each shot
Poster wall	Faster drift (50 s loop) and brighter

A browser test was added to prove the reduced-motion promise: with the preference set, the number of running animations on the page is zero and the trailer does not advance on its own.

16. THE 60-SECOND RECRUITER BRIEF

When the owner asked for a site that any recruiter would hire from, the reply began with the truth: no website can guarantee an offer, and recruiters decide on evidence. The work therefore aimed at making the evidence fast to find, easy to trust and easy to act on. A new panel, "Your 60-second brief", appears directly under the billboard for recruiters and after the rows for other viewers.

Part	Content
Why me for...	One card per target role. Cloud: AWS Cloud Practitioner 1000/1000, AI-900 929/1000, AWS re/Start (Sep 2025). Backend: sole developer of the DYPIU PhD ERP (UAT / pilot stage), Software Developer Intern at Technocia Infotech. Security: the Cyber Security B.Tech, Cybersecurity Research Intern at FirstVidya. Junior DevOps: AWS re/Start, plus a "missing" marker until real DevOps work is added.
Logistics	Location, local time in Pune, start date, work mode, work authorization (the last three are placeholders until the owner fills them in)
Actions	Email me, download résumé (both appear once filled in), and "Copy summary for your ATS", which copies a clean plain-text profile for pasting into an applicant-tracking system and leaves out anything not yet filled in

A unit test checks that every line of evidence is assembled from text that already exists in profile.js, so the brief cannot drift into claims the résumé does not support. Two phone defects were found while testing this feature and fixed (D30

and D31 in Section 18.7).

17. TAILORED LINKS, THE MAKING-OF AND THE PUNE CLOCK

The final round of features, on 9 November, answered "make it even better and more unique".

17.1 Tailored links

The command `npm run link -- "Infosys" cloud` prints a link such as `?for=Infosys&role=cloud`. A visitor who opens it skips the picker, sees "Prepared for Infosys · cloud role" on the billboard, finds the brief headed "For Infosys" with the Cloud card first and highlighted as "the role you're hiring for", and sees the company in the browser tab title. Accepted roles are cloud, backend, security and devops; anything else is ignored. The values are displayed only as text, cleaned of angle brackets, braces and control characters, collapsed and capped at 40 characters; a browser test confirms that an attempted image-tag injection renders as plain text and never runs. Tailored links only become useful once the site is hosted at a public address, which has not happened yet (Section 20).

17.2 Making of this site

A new Extras row holds one title: the portfolio as its own case study, with episodes for the problem, the architecture, the stack and the quality bar, and a link to the public repository. It turns the site into evidence a recruiter or engineer can check, rather than a claim.

17.3 Pune clock

The brief's logistics and the contact section show the owner's current local time (IST, UTC+5:30), updated every thirty seconds. The pre-rendered HTML shows only the time zone, because the build cannot know when the page will be read; the time appears once the page is running, which keeps hydration exact.

18. TESTING

18.1 Strategy

Testing was organised around the risks that matter for this product. The first risk is misinformation: a wrong score, a wrong date or an invented claim in front of a recruiter, which is worse than no portfolio. The second is a broken experience for some visitors: phones, keyboards, screen readers, blocked storage, JavaScript turned off, or many people at once. The third is slowness, especially on mobile networks. The fourth is regression: a later change silently breaking an earlier promise. Each risk has its own suite, and all of them run with one command, `npm run check`, which runs the unit tests, builds the production site, runs the browser tests against that build, and runs the load test. Version 1 had no automated suites; it was checked with scripted browser sessions, screenshots and Lighthouse, as described in Section 10.

18.2 Unit tests (Vitest, 25 tests in 6 files)

File	Test	What it protects
`facts.test.js`	certification results and dates	AWS CCP 1000/1000, Mar 2026, valid through Mar 2029; AI-900 929/1000, Mar 2026; GATE qualified; re/Start completed Sep 2025
`facts.test.js`	experience titles, employers and dates	All three jobs exactly as on the résumé
`facts.test.js`	education and identity	Degree, school, year, full name, location
`facts.test.js`	the only project with known status/role is the PhD ERP	ERP is "UAT / pilot stage" and "Sole developer"; project names and order
`catalog.test.js`	every title id is unique	No two titles share an id

File	Test	What it protects
`catalog.test.js`	every viewer orders the same rows, each row is non-empty	Every viewer type reaches every row
`catalog.test.js`	recruiters see credentials first, engineers see projects first	Row order per viewer
`catalog.test.js`	search matches every word, case-insensitively	Includes: "kubernetes" finds nothing because it is not on the résumé
`catalog.test.js`	placeholders are never searchable text	Missing facts cannot appear as search hits
`catalog.test.js`	"more like this" stays within the same kind	Suggestions are relevant
`catalog.test.js`	fill / real	Placeholder detection, including the array .fill trap (defect D8)
`catalog.test.js`	parses the title hash	Shared-link parsing
`render.test.jsx`	contains every title and the key facts	The pre-rendered page
`render.test.jsx`	has no rendering garbage	No undefined, NaN, [object Object] or null in the page text
`render.test.jsx`	marks missing facts as missing, never as content	Placeholders are labelled
`render.test.jsx`	has exactly one h1 visible to the page outline per screen	Heading structure
`brief.test.js`	covers every target role	Brief matches the four target roles
`brief.test.js`	every proof is built from facts already in profile.js (or marked missing)	The brief cannot add claims
`brief.test.js`	the ATS summary has the key facts and no placeholders or garbage	Copied summary is clean
`tailor.test.js`	reads company and role	Tailored-link parsing
`tailor.test.js`	ignores unknown roles and empty links	Robustness
`tailor.test.js`	strips markup and caps the length	Injection attempt and 200-character input
`storage.test.js`	round-trips JSON	Normal storage
`storage.test.js`	falls back when storage throws (private mode, blocked cookies)	Storage failure
`storage.test.js`	falls back on corrupted values	Bad stored data

18.3 Browser tests (Playwright with axe-core, 18 scenarios)

The browser suite runs real Chromium against the production build. Each scenario starts a fresh browser context, which behaves like a different person with empty storage. Console errors and uncaught page errors are collected and fail the scenario. Durations are from the run of 9 November 2026.

#	Scenario	Duration (ms)
1	new visitor sees "Who's here?", recruiter gets credentials first, choice is remembered	1,506
2	recruiter brief: shown first, role fit + logistics, copies a clean summary	1,316
3	tailored link: greets the company, leads with their role, no picker, markup stays text	387
4	engineer gets projects first; switching viewer works	445
5	every title opens, shows its name, closes with Esc and returns focus	3,206
6	deep link opens a title; Back closes it	744
7	search finds by word, says when nothing matches, "/" focuses it	415

#	Scenario	Duration (ms)
8	My list: add, see the row, survives reload, remove	2,754
9	trailer plays, advances, pauses, closes	8,181
10	contact form: required fields enforced; without a key it says so, never fakes success	693
11	keyboard: first Tab is the skip link; dialogs trap focus	361
12	accessibility (axe, WCAG 2.1 AA): gate, home, title page, trailer	5,479
13	phone: no sideways scrolling, everything reachable	1,419
14	reduced motion: nothing animates, trailer waits for you	3,841
15	still works when the browser blocks storage	2,238
16	without JavaScript the whole catalogue is still readable	155
17	nothing on the page claims something missing: placeholders are labelled	1,202
18	25 visitors at the same time, each with their own state	6,783

The accessibility scenario runs axe-core with the WCAG 2.0 A, 2.0 AA, 2.1 A and 2.1 AA rule sets against the picker, the home page, a title page and the trailer, and fails on any violation. The concurrency scenario starts 25 visitors in parallel, each choosing a viewer, opening a different title and adding it to My list, then checks that each one saw the right row order and that each list contains only that visitor's own choice.

18.4 Load test (autocannon)

The site is static files with no server code and no database, so the question under load is only whether serving those files fails. The load test sends 200 parallel connections for 10 seconds at the page and then at its main script, and fails on any error, timeout or non-200 response. It runs against `vite preview`, a development-grade server on the same machine as the test, so these figures are a floor for what a real static host or CDN would do.

Run	Date	Build	Target	Requests	Per second	Median	99th pct.	Errors / timeouts / non-200
1	8 Nov	v2 first release candidate	Page	7,459	746	258 ms	301 ms	0 / 0 / 0
1	8 Nov	v2 first release candidate	Main script	40,207	4,021	46 ms	77 ms	0 / 0 / 0
2	8 Nov	v2 after review clean-up	Page	9,816	892	216 ms	274 ms	0 / 0 / 0
2	8 Nov	v2 after review clean-up	Main script	43,102	4,311	44 ms	63 ms	0 / 0 / 0
3	8 Nov	v2 release (full check)	Page	9,029	903	214 ms	288 ms	0 / 0 / 0
3	8 Nov	v2 release (full check)	Main script	42,054	4,206	45 ms	62 ms	0 / 0 / 0
4	8 Nov	Energetic update	Page	8,118	812	242 ms	295 ms	0 / 0 / 0
4	8 Nov	Energetic update	Main script	38,619	3,863	49 ms	69 ms	0 / 0 / 0
5	8 Nov	Recruiter brief	Page	4,803	480	405 ms	483 ms	0 / 0 / 0
5	8 Nov	Recruiter brief	Main script	30,091	3,010	59 ms	129 ms	0 / 0 / 0
6	9 Nov	Tailored links	Page	7,490	749	253 ms	332 ms	0 / 0 / 0
6	9 Nov	Tailored links	Main script	35,999	3,600	52 ms	81 ms	0 / 0 / 0
7	9 Nov	For this document	Page	6,838	684	272 ms	377 ms	0 / 0 / 0
7	9 Nov	For this document	Main script	30,374	3,038	60 ms	107 ms	0 / 0 / 0

Seven load runs were recorded, each with the same two targets, 200 connections and 10 seconds. Throughput varied between runs because the test machine was shared with the browser tests and the build, and because the page grew as features were added (the pre-rendered HTML is now 164 KB raw, 15 KB compressed); run 5 was the slowest because it ran immediately after a full browser suite on the same machine. What did not vary is the result that matters: across all fourteen measurements, not a single request failed, timed out or returned anything but 200.

18.5 Why many recruiters at once is safe

Every visitor downloads the same static files. Viewer choice, My list and Continue watching are stored in that visitor's own browser and never sent anywhere, so one visitor cannot see or change another's state; the 25-visitor test proves this. The only outside call is the contact form, which sends each submission independently to Web3Forms. The only shared limit is therefore the hosting provider's bandwidth and Web3Forms' own rate limits for its free tier, neither of which applies until the site is hosted and the form is connected.

18.6 Run history

When	Suite	Result	What happened next
8 Nov	Unit, first run	18 passed, 1 failed	The render test compared text containing irregular whitespace; the test was corrected to normalise spaces (T1)
8 Nov	Scripted screenshot check before the suite existed	Picker appeared late	Found D18; fixed
8 Nov	Browser, first run	10 passed, 5 failed	Found defects D19, D20 and D21 and two test errors (T2, T3); all addressed
8 Nov	Browser, second run	14 passed, 1 failed	D21 was only partly fixed (focus released, but the next Tab still skipped the skip link); fixed fully by placing the picker first in the document
8 Nov	Full check	19 unit, 15 browser, load 0 errors	Released (commits fe76e38, 1f30dd0, 85bf6ac)
8 Nov	Full check after the energetic update	19 unit, 15 browser, load 0 errors	Reduced-motion scenario added: 16 browser, all passing
8 Nov	Unit, with the brief	21 passed, 1 failed	The role-order test assumed an order; corrected to compare sets (T4)
8 Nov	Browser, with the brief	16 passed, 1 failed (twice)	Found D30 and D31 on phones; fixed
8 Nov	Full check	22 unit, 17 browser, load 0 errors	Released (1818868)
9 Nov	Full check with tailored links	25 unit, 18 browser, load 0 errors	Released (f63b54a)
9 Nov	Full check for this document	25 unit, 18 browser, load 0 errors, exit code 0	Output in Appendix A

18.7 Version 2 defects found and fixed

#	Defect	How it was found	Fix
D18	The picker was hidden in the pre-rendered HTML and only appeared after JavaScript ran, so first-time visitors saw the page and then had it covered	Scripted screenshot check	Picker open in the HTML; hidden before paint by the inline script only for returning visitors
D19	A shared title link opened behind the picker for first-time visitors	Browser test	Shared and tailored links skip the picker
D20	After picking a viewer once, "switch viewer" could not reopen the picker because a CSS rule kept hiding it	Browser test	The hiding rule now applies only before React has started
D21	After picking a viewer, keyboard focus stayed on the vanished picker and the next Tab skipped the skip link	Browser test	Focus released and the picker moved first in the document
D22	Large numbers on credential and job posters overlapped the card titles	Screenshot review	Numbers moved to the top-right of the poster
D23	One row snapped out of line with the page margin	Screenshot review	Scroll padding matches the row padding

#	Defect	How it was found	Fix
D24	On phones the viewer button was cut off	Phone screenshot	Label visually hidden on small screens, still read by screen readers
D25	The trailer's progress bars never animated: Tailwind only generates keyframes for class names it sees, and this one was used from an inline style	Code review	Keyframes defined directly in index.css
D26	"Copy link" failed silently when the browser blocked clipboard access	Code review	The button now says to copy the address from the address bar
D27	Layout shift of 0.159 on phones as headline fonts arrived	Lighthouse	The build adds preload links for the first-screen fonts; shift fell to 0.009
D28	The same poster used several times produced duplicate SVG ids	Code review	Unique ids per instance with useId
D29	A frosted-glass blur behind the title dialog contradicted the brief	Review against the brief	Removed
D30	Long "missing" tags could not wrap, so on phones the page became wider than the screen and the browser zoomed out	Browser test (phone)	Tags wrap; long placeholder texts shortened
D31	Hover lift effects also fired on touch screens, shifting tap targets under the finger	Browser test (phone)	Hover styles only on devices that can hover
D32	Visitors without JavaScript were stuck behind the picker	While designing the no-JavaScript test	A noscript style hides the picker

18.8 Corrections to the tests themselves

#	Test error	Correction
T1	Render test compared raw HTML text that contained irregular whitespace from React's markers	Whitespace normalised before comparing
T2	Expected 16 titles; there were 17 (four credentials, three jobs, five projects, five skill groups); later 18 with the making-of	Expected count corrected each time
T3	The accessibility scan ran while dialogs were still fading in, so semi-transparent text was measured as low contrast	The scan waits 700 ms for entrance animations; the final state has no violations
T4	Brief test assumed the role cards were in the same order as the target list	Compares the sets of roles instead

One further error was in a one-off recording script rather than in the test suites: when recording the preview video it clicked the first element containing "Get in touch", which was a paragraph, not the button. The script was corrected to target the button by its role. No product change was needed.

19. PERFORMANCE HISTORY

Twelve Lighthouse 12 reports were saved during the project. All were run locally against the production build served on port 4173. "Mobile" means Lighthouse's simulated mid-range phone on a slow 4G connection; "desktop" uses its desktop preset. The server has no graphics card, which matters only for version 1, whose 3D scene ran on the processor and inflated total blocking time.

Report	Time (UTC)	Build	Device	Perf.	A11y	Best pr.	SEO	First paint	Largest paint	Blocking	Layout shift
lh	5 Nov 19:21	v1 3D, before fixes	Mobile	51	96	100	91	3.5 s	3.6 s	5,530 ms	0.078
lh-mobile	5 Nov 19:23	v1 3D, contrast and loading fixes	Mobile	53	100	100	100	3.3 s	3.5 s	3,510 ms	0.078

Report	Time (UTC)	Build	Device	Perf.	A11y	Best pr.	SEO	First paint	Largest paint	Blocking	Layout shift
lh-desktop	5 Nov 19:23	v1 3D, same build	Desktop	67	100	100	100	0.7 s	0.7 s	1,390 ms	0.094
lh-m2	5 Nov 19:24	v1 3D, hero name fix	Mobile	57	100	100	100	3.2 s	3.4 s	2,560 ms	0.012
lh-s	5 Nov 19:24	v1 simple view, before pre-render	Mobile	60	100	100	100	3.3 s	3.5 s	0 ms	1.000
(not saved)	5 Nov	v1 3D, hero pre-rendered	Mobile	60	100	100	100	2.9 s	3.3 s	1,730 ms	0
lh-x	5 Nov 19:25	v1 simple view, hero pre-rendered	Mobile	88	100	100	100	3.0 s	3.2 s	0 ms	0.014
lh3-mobile	8 Nov 14:16	v2, first build	Mobile	91	100	100	100	2.0 s	2.0 s	10 ms	0.159
lh3-desktop	8 Nov 14:17	v2, first build	Desktop	100	100	100	100	0.4 s	0.4 s	0 ms	0.016
lh4	8 Nov 14:17	v2, fonts preloaded	Mobile	99	100	100	100	1.6 s	1.9 s	10 ms	0.009
lh5-mobile	8 Nov 14:29	v2, energetic update	Mobile	99	100	100	100	1.6 s	1.9 s	30 ms	0.005
lh5-desktop	8 Nov 14:29	v2, energetic update	Desktop	100	100	100	100	0.4 s	0.4 s	0 ms	0.001
lh6	9 Nov 04:14	v2, tailored links (current)	Mobile	98	100	100	100	1.8 s	2.0 s	10 ms	0.005

The row marked "not saved" was measured in a loop that wrote two reports to the same file; its figures are taken from the session record, and the file holds the simple-view result that followed it. Report times are Lighthouse's own fetch times. They do not line up exactly with commit times, because the two are recorded by different processes and some commits on 8 November were made a few hours after the measurements of the work they contain.

19.1 Bundle sizes

Build	JavaScript before first paint (compressed)	JavaScript loaded later (compressed)	CSS (compressed)
v1, 4 Nov	about 104 KB	about 220 KB (three.js, react-three-fiber, scenes)	about 5 KB
v1, 5 Nov	about 56 KB	about 49 KB (scrolling) and about 230 KB (3D)	about 6 KB
v2, 8 Nov	about 58 KB (all of it)	none	about 6 KB
v2, 9 Nov (current)	61,157 bytes (all of it)	none	6,574 bytes

Version 2 ships no images except the screenshots in the repository's `docs` folder, which are not part of the site; posters are SVG drawn in code. Fonts are the largest assets: each WOFF2 file is 17 to 20 KB, and only the three needed for the first screen are preloaded.

20. HOSTING, DEPLOYMENT AND PUSHES

20.1 Public hosting: none

The site has been deployed to a public host zero times. The brief said "Do not deploy anything anywhere; local dev only", and no later message changed that instruction. No hosting account was created, no domain was configured and

no deployment service was connected. The repository on GitHub is public, so its source code can be read by anyone, but GitHub does not serve it as a website because GitHub Pages has not been enabled.

20.2 Local servers used for testing

Server	Address	When	Purpose
Vite preview (production build)	localhost:4173 inside the cloud session	Started by hand at least four times: 4 Nov, 5 Nov, 8 Nov after the rebuild, and 8 Nov after the worker restart; the browser and load test scripts also start their own copy if none is running	Screenshots, Lighthouse, browser and load tests
Vite development server	localhost:5173 inside the cloud session	Once, on 4 Nov	Reproducing the terminal crash (D9) with a readable stack trace

On 5 and 8 November the port of the preview server was reported to the owner's app so that it could be opened in the app's preview pane while the session was running. These servers existed only inside the temporary cloud container and stopped when it was reclaimed. They were never reachable by recruiters.

20.3 Pushes to GitHub

#	Date (UTC)	Commits pushed	Content
1	4 Nov	c398f70, ad3ce03, 795a77f, 8b25436, 5d926c9	Version 1 complete with README
2	4 Nov 07:03	2056503	Preview video
3	5 Nov 19:28	fd93879	Completion and performance round
4	8 Nov 14:23	fe76e38, 1f30dd0, 85bf6ac	Version 2, tests, documentation
5	8 Nov 19:59	6f05b7e	Energetic update
6	8 Nov 20:12	1818868	Recruiter brief
7	9 Nov 04:14	f63b54a	Tailored links, making-of, clock

The remote head was verified for this document: `refs/heads/main` on GitHub points at `f63b54a`, identical to the local branch. A small follow-up commit made while preparing this document (renaming one test whose title still said "four rows") is described in Section 24.

20.4 How to host it when ready

This section is guidance, not a record of anything done. The production build is a folder of static files (`dist`) produced by `npm run build`, and the site needs no server code, so any static host works: GitHub Pages, Cloudflare Pages, Netlify and Vercel all offer free plans suitable for a portfolio. Before building for hosting, put the résumé PDF in `public`, fill in the missing facts, and set `VITE_WEB3FORMS_KEY` so the contact form works. Because shared links use the URL hash and tailored links use the query string, no server rewrite rules are needed. After hosting, set `SITE_URL` to the public address when running `npm run link`, and run Lighthouse and the browser tests against the public address with `BASE_URL`.

21. SECURITY AND PRIVACY REVIEW

Area	Finding
Server attack surface	None: there is no server code, database or login. The site is static files.
Untrusted input	Three inputs exist: the search box, the URL hash and the tailored-link query. All are rendered as text by React; the tailored values are also stripped of angle brackets, braces and control characters and capped at 40 characters. A browser test attempts an HTML injection through the query and confirms nothing executes.

Area	Finding
Unsafe HTML	A search of the source found no use of dangerouslySetInnerHTML, innerHTML or eval.
Contact form	Posts directly from the visitor's browser to Web3Forms; includes a hidden honeypot field for bots; length limits on every field; never claims success unless the service confirms it.
Secrets	The only secret is the Web3Forms access key, read from `.env.local`, which git ignores (verified). Web3Forms access keys are designed to be public in the browser and only identify where mail is delivered.
External links	Open in a new tab with rel="noreferrer", so the opened page cannot control the portfolio tab.
Privacy	No analytics, trackers, cookies or third-party scripts. Fonts are self-hosted. Viewer choice, My list and recently opened titles stay in the visitor's own browser storage.
Shipped dependencies	`npm audit --omit=dev`: 0 known vulnerabilities in what visitors download (React and React DOM).
Development dependencies	9 known vulnerabilities in the build and test tools (5 moderate, 2 high, 2 critical; detail below). They affect a developer machine running the dev server or test runner, not the published site.

Package	Severity	Affected range	Advisory (abridged)	Fix available
@vitest/mocker	moderate	<=4.1.11	Vitest: Path Traversal / Arbitrary File Read via @vitest/mocker Redirect Mock; Vitest: Code injection via unau	True
autocannon	moderate	2.1.0 - 8.0.0	hyperid	autocannon 9.0.0 (major)
esbuild	moderate	<=0.24.2	esbuild enables any website to send any requests to the development server and read the response	True
hyperid	moderate	<=1.2.0		1.3.1 - 3.3.0
postcss	high	<=8.5.22	PostCSS has XSS via Unescaped </style> in its CSS Stringify Output; PostCSS: Arbitrary file read and informati	True
tinypool	critical	<=2.1.1	Tinypool: Prototype Pollution gadget in worker options leads to Remote Code Execution; Tinypool: Prototype Pol	True
uuid	moderate	<11.1.1	uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided	autocannon 9.0.0 (major)
vite	high	<=6.4.3	Websites were able to send any requests to the development server and read the response in vite; Vite bypasses	True
vitest	critical	<=4.1.10	@vitest/mocker; Vitest allows Remote Code Execution when accessing a malicious website while Vitest API server	True

uuid	autocannon 9.0.0 (major)

Recommendation: upgrade Vite, Vitest and autocannon to their current major versions in a separate change, then re-run `npm run check`. These are major-version upgrades, so they were not applied automatically, in keeping with the "surgical changes" rule. When the site is hosted, also configure security headers at the host (a Content Security Policy allowing only the site itself and api.web3forms.com, plus Referrer-Policy and X-Content-Type-Options).

22. ACCESSIBILITY STATEMENT

The current version was designed and tested against WCAG 2.1 level AA. Automated evidence: Lighthouse accessibility 100 in every version 2 report, and zero axe-core violations across the picker, home page, a title page and the trailer. Behavioural evidence from the browser tests: the first Tab reaches "Skip to the catalogue"; every title can be opened and closed by keyboard and focus returns to its card; dialogs keep focus inside; the slash key focuses search; the whole catalogue is readable with JavaScript off; and nothing animates under reduced motion. What has not been done: testing with real screen readers (NVDA, JAWS, VoiceOver, TalkBack) and with people who rely on assistive technology. Automated tools catch roughly a third to a half of accessibility problems, so a manual screen-reader pass is recommended before the site is promoted widely.

23. DATA INTEGRITY AND MISSING CONTENT

The project's first rule was never to invent a fact. It is enforced in four ways: every fact lives in one file; missing facts are written as `fill()` and appear on the page as a dashed "missing" tag; unit tests pin every score, date, title, employer and project status to the résumé; and derived content (the brief, the ticker, the trailer, the ATS summary) is assembled only from that file, with a test that rejects any line not traceable to it. Visual illustrations, such as the racing boxes on the BazaarLens poster or the windows of the RentFlow building, are random decoration and are never labelled as data.

23.1 The 50 open placeholders

The list below is the verbatim output of `npm run todo` on 9 November 2026: 49 placeholders in `profile.js`. The fiftieth is in `src/content/brief.js`: the evidence for the Junior DevOps role (CI/CD, Docker or Linux work), which only the owner can supply.

```
49 things still missing in src/content/profile.js:
  availability  →  start date
  workMode  →  on-site / hybrid / remote
  workAuthorization  →  work authorization
  links.github  →  GitHub profile URL
  links.linkedin  →  LinkedIn URL
  links.email  →  email address
  links.resume  →  resume PDF path
  credentials[0].verifyUrl  →  Credly link
  credentials[1].verifyUrl  →  verification link
  experience[0].detail  →  what I teach / what I achieved
  experience[1].detail  →  what I built, tech used, outcome
  experience[2].detail  →  what I researched, tools, outcome
  projects[0].problem  →  problem it solves
  projects[0].architecture  →  architecture notes
  projects[0].stack  →  stack
  projects[0].repo  →  repo link, or "private"
  projects[0].demo  →  demo link
  projects[1].status  →  status
  projects[1].role  →  my role
  projects[1].problem  →  problem it solves
  projects[1].architecture  →  architecture notes
  projects[1].stack  →  stack
  projects[1].repo  →  repo link
  projects[1].demo  →  demo link
  projects[2].status  →  status
  projects[2].role  →  my role
  projects[2].problem  →  problem it solves
  projects[2].architecture  →  architecture notes
  projects[2].stack  →  stack
  projects[2].repo  →  repo link
  projects[2].demo  →  demo link
  projects[3].status  →  status
  projects[3].role  →  my role
  projects[3].problem  →  problem it solves
  projects[3].architecture  →  architecture notes
  projects[3].stack  →  stack
  projects[3].repo  →  repo link
  projects[3].demo  →  demo link
  projects[4].status  →  status
  projects[4].role  →  my role
  projects[4].problem  →  problem it solves
  projects[4].architecture  →  architecture notes
  projects[4].stack  →  stack
  projects[4].repo  →  repo link
  projects[4].demo  →  demo link
  skills[0].items[2]  →  exact services used
  skills[1].items[0]  →  languages and frameworks
  skills[2].items[0]  →  tools and areas
  skills[3].items[0]  →  databases, tools, CI/CD
```

23.2 Draft wording to replace

Two pieces of text were drafted from résumé facts and are marked DRAFT in the code: the tagline ("I work on the parts of a system you only notice when they fail: the cloud underneath, the backend in the middle, and the security around both.") and the call to action ("Hiring for cloud, backend, junior DevOps or security? Or need something built freelance? Get in touch."). The brief asked for both to be in the owner's own voice, so both should be rewritten by him.

23.3 Counting note

On 4 November, before the counting script existed, the number of blanks was given as 50 from a manual count of the form; the script later counted 46 in version 1. Version 2 restructured the data and added three logistics fields (start date, work mode, work authorization), giving the current 49 in `profile.js` plus 1 in `brief.js`.

24. COMMIT LOG

Hash	Date (UTC)	Message	Files	Lines added / removed
c398f70	4 Nov 04:46	Set up Vite + React project	6	+2,784
ad3ce03	4 Nov 04:46	Add profile data, scroll pipeline and camera route	7	+517
795a77f	4 Nov 04:46	Add 3D world: shaders, packet, particle name and stop scenes	25	+1,580
8b25436	4 Nov 04:46	Add HTML sections, terminal, simple view and notebook styling	21	+1,377
5d926c9	4 Nov 04:46	Write README explaining how each part works	1	+250
2056503	4 Nov 07:03	Add a scroll-through preview video and link it from the README	2	+3
fd93879	5 Nov 19:28	Pre-render the hero, defer 3D and scroll chunks, fix contrast and layout shift	23	+287 / -57
fe76e38	8 Nov 14:23	Rebuild the portfolio as a streaming catalogue with Tailwind	80	+4,065 / -4,438
1f30dd0	8 Nov 14:23	Add unit, browser, accessibility and load tests	6	+529
85bf6ac	8 Nov 14:23	Document how it works, how it's tested, and the coding rules	8	+183 / -244
6f05b7e	8 Nov 19:59	Make the catalogue more energetic	17	+136 / -29
1818868	8 Nov 20:12	Add a 60-second recruiter brief with role fit and an ATS summary	12	+243 / -6
f63b54a	9 Nov 04:14	Add tailored links, a Making-of extra and a live Pune clock	20	+253 / -23

The first commit's 2,784 added lines are mostly the dependency lockfile. The rebuild commit's 4,438 removed lines are the whole of version 1, which remains available in history at fd93879. Every commit message ends with a co-author line and a link to the working session. The test rename made while preparing this document is committed together with the document itself.

25. KNOWN LIMITATIONS AND ITEMS NOT VERIFIED

- **Not deployed.** Recruiters cannot reach the site yet; tailored links have no public address to point at.
- **Content incomplete.** 50 placeholders, including email, résumé, start date and the stack of four projects. This is the single biggest limitation.
- **Real devices.** Phones were emulated in Chromium (iPhone 13 profile, 390-pixel width). No physical phone, Safari or Firefox was used.

- **Real networks.** "Slow 4G" timings are Lighthouse simulations on a local server, not field measurements.
- **Screen readers.** Covered by automated rules and keyboard tests only (Section 22).
- **Contact form.** The success path has not been exercised, because no Web3Forms key exists yet; only the "not connected" path is tested.
- **Development dependencies.** 9 advisories in build and test tools (Section 21).
- **Version 1 frame rate.** The 60 fps target of the original brief was never measured on real hardware before version 1 was replaced.
- **Lost screenshots.** Images of the Cyanotype and Paper and Ink looks were overwritten during the session and are not in the repository.
- **Browser storage.** My list and Continue watching exist only in the visitor's own browser and do not follow them to another device; this is deliberate, as there is no server.

26. RECOMMENDED NEXT STEPS

Priority	Step	Who	How
1	Add email address and résumé PDF	Owner	Put the PDF in `public`; edit `links` in `profile.js`
2	Add start date, work mode and work authorization	Owner	Edit the three logistics fields in `profile.js`
3	Fill in stack, role, problem, architecture and links for CarePilot, RentFlow, Sensorium Nexus and BazaarLens; problem, architecture and stack for the PhD ERP	Owner	`profile.js`; add screenshots if available
4	Describe the three jobs with tools and outcomes; add DevOps evidence	Owner	`profile.js` and `brief.js`
5	Add GitHub, LinkedIn and certificate verification links	Owner	`profile.js`
6	Rewrite tagline and call to action in own words	Owner	`profile.js`
7	Create a Web3Forms key	Owner	Free sign-up; put it in `.env.local`
8	Run `npm run check`; update `facts.test.js` only if the résumé itself changed	Owner or assistant	One command
9	Decide to host; choose a static host	Owner	Section 20.4
10	After hosting: Lighthouse and browser tests against the public address; real-phone check; screen-reader pass	Owner or assistant	BASE_URL=... npm run test:e2e
11	Upgrade Vite, Vitest and autocannon majors to clear development advisories	Assistant	Separate change, then full check
12	Generate one tailored link per application	Owner	`SITE_URL=... npm run link -- "Company" role`

27. GLOSSARY

Term	Meaning in this document
ATS	Applicant-tracking system: software recruiters use to store candidates; the brief copies a plain-text profile for it
axe-core	An automated accessibility testing engine used inside the browser tests
Billboard	The large first screen of version 2
Catmull-Rom spline	A smooth curve through a list of points; version 1's camera path

Term	Meaning in this document
CLS (layout shift)	How much visible content moves unexpectedly while a page loads; 0 is perfect, above 0.1 is poor
Deployment / hosting	Publishing the built site at a public address; not done in this project
FCP / LCP	First and largest contentful paint: when the first and the largest piece of content appear
fill()	The marker for a fact the owner has not yet supplied
Hydration	React attaching to HTML that was rendered in advance, instead of drawing the page again
Lighthouse	Google's automated audit of performance, accessibility, best practices and SEO
Pre-rendering	Rendering the page to HTML at build time so the first response already contains the content
Reduced motion	An operating-system setting asking websites to minimise animation
Shader	A small program that runs on the graphics processor; version 1 had five custom ones
Tailored link	A link with ?for=Company&role=... that personalises the site for one application
TBT (blocking time)	Total time the page was too busy to respond to input while loading
Title	Anything in version 2 that can be opened: a project, credential, job, skill group, about or making-of
Viewer	The visitor type chosen in "Who's here?": recruiter, engineer or just looking
WCAG 2.1 AA	The Web Content Accessibility Guidelines, level AA, the common legal and industry target

APPENDIX A. VERBATIM TEST OUTPUT

The complete console output of `npm run check` run on 9 November 2026 for this document (unit tests, production build, browser tests and load test, ending with exit code 0), followed by the verbose unit-test report and the dependency audit summary. Colour codes have been removed; nothing else has been changed.

A.1 npm run check

```
> jaykumar-shevate-portfolio@0.1.0 check
> npm run test && npm run build && npm run test:e2e && npm run test:load


> jaykumar-shevate-portfolio@0.1.0 test
> vitest run


 RUN  v2.1.4 /home/user/portfoilo

 ✓ tests/unit/facts.test.js  (4 tests) 3ms
 ✓ tests/unit/catalog.test.js  (8 tests) 20ms
 ✓ tests/unit/render.test.jsx  (4 tests) 4ms
 ✓ tests/unit/brief.test.js  (3 tests) 3ms
 ✓ tests/unit/tailor.test.js  (3 tests) 4ms
 ✓ tests/unit/storage.test.js  (3 tests) 3ms

 Test Files  6 passed (6)
      Tests  25 passed (25)
   Start at  04:47:17
   Duration  1.76s (transform 383ms, setup 0ms, collect 616ms, tests 38ms, environment 2.25s, prepare 343ms)


> jaykumar-shevate-portfolio@0.1.0 build
> vite build && vite build --ssr src/entry-server.jsx --outDir dist-ssr && node scripts/prerender.mjs

vite v5.4.10 building for production...
transforming...
✓ 57 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                                                  2.40 kB │ gzip:  1.23 kB
dist/assets/ibm-plex-mono-latin-400-normal-O6-GRVqx.woff           13.20 kB
dist/assets/big-shoulders-display-latin-900-normal-CW8trzgu.woff2  14.02 kB
dist/assets/big-shoulders-display-latin-700-normal-KM2fueoL.woff2  14.42 kB
dist/assets/big-shoulders-display-latin-800-normal-DDUD9Xuh.woff2  14.56 kB
```

```
dist/assets/ibm-plex-mono-latin-400-normal-Dm_PoFIZ.woff2           14.81 kB
dist/assets/big-shoulders-display-latin-900-normal-DMT-1gsg.woff    17.00 kB
dist/assets/big-shoulders-display-latin-800-normal-CI0lhnTM.woff    17.65 kB
dist/assets/big-shoulders-display-latin-700-normal-CMi6UoP4.woff    17.70 kB
dist/assets/ibm-plex-sans-latin-400-normal-BU83cd9M.woff            17.86 kB
dist/assets/ibm-plex-sans-latin-600-normal-ZykYOGKL.woff            19.13 kB
dist/assets/ibm-plex-sans-latin-400-normal-CdZtFfYS.woff2           19.16 kB
dist/assets/ibm-plex-sans-latin-600-normal-BGOKnPO6.woff2           20.36 kB
dist/assets/index-C_T7AL2J.css                                      30.41 kB │ gzip:  6.56 kB
dist/assets/index-B6yi59Fx.js                                      191.66 kB │ gzip: 61.26 kB
✓ built in 1.06s
vite v5.4.10 building SSR bundle for production...
transforming...
✓ 21 modules transformed.
rendering chunks...
dist-ssr/entry-server.js  70.38 kB
✓ built in 355ms
prerendered dist/index.html

> jaykumar-shevate-portfolio@0.1.0 test:e2e
> node tests/e2e/run.mjs

  ✓ new visitor sees "Who's here?", recruiter gets credentials first, choice is remembered (1506 ms)
  ✓ recruiter brief: shown first, role fit + logistics, copies a clean summary (1316 ms)
  ✓ tailored link: greets the company, leads with their role, no picker, markup stays text (387 ms)
  ✓ engineer gets projects first; switching viewer works (445 ms)
  ✓ every title opens, shows its name, closes with Esc and returns focus (3206 ms)
  ✓ deep link opens a title; Back closes it (744 ms)
  ✓ search finds by word, says when nothing matches, "/" focuses it (415 ms)
  ✓ My list: add, see the row, survives reload, remove (2754 ms)
  ✓ trailer plays, advances, pauses, closes (8181 ms)
  ✓ contact form: required fields enforced; without a key it says so, never fakes success (693 ms)
  ✓ keyboard: first Tab is the skip link; dialogs trap focus (361 ms)
  ✓ accessibility (axe, WCAG 2.1 AA): gate, home, title page, trailer (5479 ms)
  ✓ phone: no sideways scrolling, everything reachable (1419 ms)
  ✓ reduced motion: nothing animates, trailer waits for you (3841 ms)
  ✓ still works when the browser blocks storage (2238 ms)
  ✓ without JavaScript the whole catalogue is still readable (155 ms)
  ✓ nothing on the page claims something missing: placeholders are labelled (1202 ms)
  ✓ 25 visitors at the same time, each with their own state (6783 ms)

18 passed, 0 failed

> jaykumar-shevate-portfolio@0.1.0 test:load
> node tests/load/run.mjs

✓ page: 6838 requests, 684/s, latency p50 272 ms · p99 377 ms, errors 0, timeouts 0, non-2xx 0
✓ main script: 30374 requests, 3038/s, latency p50 60 ms · p99 107 ms, errors 0, timeouts 0, non-2xx 0
exit 0
```

A.2 Unit tests, verbose report

```
 RUN  v2.1.4 /home/user/portfoilo

 ✓ tests/unit/facts.test.js > facts from the resume > certification results and dates
 ✓ tests/unit/facts.test.js > facts from the resume > experience titles, employers and dates
 ✓ tests/unit/facts.test.js > facts from the resume > education and identity
 ✓ tests/unit/facts.test.js > facts from the resume > the only project with known status/role is the PhD ERP
 ✓ tests/unit/catalog.test.js > catalog > every title id is unique
 ✓ tests/unit/catalog.test.js > catalog > every viewer orders the same four rows, each row is non-empty
 ✓ tests/unit/catalog.test.js > catalog > recruiters see credentials first, engineers see projects first
 ✓ tests/unit/catalog.test.js > catalog > search matches every word, case-insensitively
 ✓ tests/unit/catalog.test.js > catalog > placeholders are never searchable text
 ✓ tests/unit/catalog.test.js > catalog > "more like this" stays within the same kind
 ✓ tests/unit/catalog.test.js > helpers > fill / real
 ✓ tests/unit/catalog.test.js > helpers > parses the title hash
 ✓ tests/unit/render.test.jsx > pre-rendered page > contains every title and the key facts
 ✓ tests/unit/render.test.jsx > pre-rendered page > has no rendering garbage
 ✓ tests/unit/render.test.jsx > pre-rendered page > marks missing facts as missing, never as content
 ✓ tests/unit/render.test.jsx > pre-rendered page > has exactly one h1 visible to the page outline per screen
 ✓ tests/unit/brief.test.js > recruiter brief > covers every target role
 ✓ tests/unit/brief.test.js > recruiter brief > every proof is built from facts already in profile.js (or marked missing)
 ✓ tests/unit/brief.test.js > recruiter brief > the ATS summary has the key facts and no placeholders or garbage
 ✓ tests/unit/tailor.test.js > tailored links > reads company and role
 ✓ tests/unit/tailor.test.js > tailored links > ignores unknown roles and empty links
 ✓ tests/unit/tailor.test.js > tailored links > strips markup and caps the length
 ✓ tests/unit/storage.test.js > storage > round-trips JSON
 ✓ tests/unit/storage.test.js > storage > falls back when storage throws (private mode, blocked cookies)
 ✓ tests/unit/storage.test.js > storage > falls back on corrupted values

 Test Files  6 passed (6)
      Tests  25 passed (25)
   Start at  04:48:24
```

```
    Duration  1.33s (transform 258ms, setup 0ms, collect 487ms, tests 26ms, environment 1.76s, prepare 460ms)
```

A.3 npm audit (all dependencies), summarised

```
@vitest/mocker | moderate | <=4.1.11 | Vitest: Path Traversal / Arbitrary File Read via @vitest/mocker Redirect Mock; Vitest:
Code injection via unau | True
autocannon | moderate | 2.1.0 - 8.0.0 | hyperid | autocannon 9.0.0 (major)
esbuild | moderate | <=0.24.2 | esbuild enables any website to send any requests to the development server and read the
response | True
hyperid | moderate | <=1.2.0 || 1.3.1 - 3.3.0 | uuid | autocannon 9.0.0 (major)
postcss | high | <=8.5.22 | PostCSS has XSS via Unescaped </style> in its CSS Stringify Output; PostCSS: Arbitrary file read
and informati | True
tinypool | critical | <=2.1.1 | Tinypool: Prototype Pollution gadget in worker options leads to Remote Code Execution;
Tinypool: Prototype Pol | True
uuid | moderate | <11.1.1 | uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided | autocannon 9.0.0 (major)
vite | high | <=6.4.3 | Websites were able to send any requests to the development server and read the response in vite; Vite
bypasses | True
vitest | critical | <=4.1.10 | @vitest/mocker; Vitest allows Remote Code Execution when accessing a malicious website while
Vitest API server | True
```

APPENDIX B. COMPLETE SOURCE CODE OF THE CURRENT VERSION

Every text file tracked in the repository at the time of writing, in full, except `package-lock.json` (a machine-generated list of 300-plus exact dependency versions that would add dozens of pages without information a reader can use; it is in the repository) and the binary media in `docs` (screenshots and video, shown in Appendix D). Files are ordered configuration first, then source, scripts and tests.

package.json 36 lines

```
{
  "name": "jaykumar-shevate-portfolio",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build && vite build --ssr
src/entry-server.jsx --outDir dist-ssr && node
scripts/prerender.mjs",
    "preview": "vite preview --port 4173",
    "test": "vitest run",
    "test:e2e": "node tests/e2e/run.mjs",
    "test:load": "node tests/load/run.mjs",
    "check": "npm run test && npm run build && npm
run test:e2e && npm run test:load",
    "todo": "node scripts/todo.mjs",
    "link": "node scripts/link.mjs"
  },
  "dependencies": {
    "@fontsource/big-shoulders-display": "^5.1.0",
    "@fontsource/ibm-plex-mono": "^5.1.0",
    "@fontsource/ibm-plex-sans": "^5.1.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@axe-core/playwright": "^4.10.0",
    "@vitejs/plugin-react": "^4.3.3",
    "autocannon": "^7.15.0",
    "autoprefixer": "^10.4.20",
    "jsdom": "^25.0.1",
    "playwright": "^1.63.0",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.14",
    "vite": "^5.4.10",
    "vitest": "^2.1.4"
  }
}
```

index.html 38 lines

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-
width, initial-scale=1.0" />
    <title>Jaykumar Shevate · cloud, backend &
security</title>
    <meta name="description" content="Jaykumar
Shevate: cloud, backend and security. AWS Certified
Cloud Practitioner (1000/1000), Microsoft AI-900
(929/1000), GATE 2026 DS&AI qualified, B.Tech CSE
(Cyber Security) 2026, Pune." />
    <meta name="theme-color" content="#0d0b09" />
    <meta property="og:title" content="Jaykumar
Shevate · cloud, backend & security" />
    <meta property="og:description"
content="Projects, credentials and experience, laid
out like a streaming catalogue." />
    <meta property="og:type" content="profile" />
    <link rel="icon" href="data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32
32'%3E%3Crect width='32' height='32' rx='4'
fill='%230d0b09'/%3E%3Ctext x='16' y='24' text-
anchor='middle' font-family='Impact,sans-serif'
font-size='22'
fill='%23f2a23a'%3ES%3C/text%3E%3C/svg%3E" />
    <script>
      // Before paint: no "Who's here?" screen for
returning visitors or shared links.
      try {
        if (/[#&]title=/.test(location.hash) || /[?
&]for=/.test(location.search) ||
localStorage.getItem('jcs:viewer'))
document.documentElement.classList.add('skip-gate')
      } catch (e) {}
    </script>
    <!-- Without JavaScript the picker can't work,
so skip it and show the catalogue. -->
    <noscript><style>[data-gate] { display: none
!important; }</style></noscript>
    <script type="application/ld+json">
      {
        "@context": "https://schema.org",
        "@type": "Person",
        "name": "Jaykumar Chandrakant Shevate",
        "jobTitle": "Teaching Associate",
        "worksFor": { "@type":
```

```
"CollegeOrUniversity", "name": "D. Y. Patil
International University" },
        "alumniOf": { "@type":
"CollegeOrUniversity", "name": "D. Y. Patil
International University" },
        "address": { "@type": "PostalAddress",
"addressLocality": "Pune", "addressRegion":
"Maharashtra", "addressCountry": "IN" },
        "knowsLanguage": ["English", "Hindi",
"Marathi"]
      }
    </script>
  </head>
  <body>
    <div id="root"><!--app--></div>
    <script type="module" src="/src/main.jsx">
</script>
  </body>
</html>
```

vite.config.js — 10 lines

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    include: ['tests/unit/**/*.test.{js,jsx}'],
    environment: 'jsdom',
  },
})
```

tailwind.config.js — 42 lines

```
/*
 * The whole palette and type scale. A late-night
cinema in Pune:
 * warm black, cream text, marigold for anything you
can act on,
 * and a teal reserved for "verified" facts.
 */
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  // hover: styles only on devices that can hover.
On phones a tap would
  // otherwise leave cards lifted and shift tap
targets under the finger.
  future: { hoverOnlyWhenSupported: true },
  theme: {
    extend: {
      colors: {
        night: '#0d0b09', // page
        soot: '#17140f', // cards
        ash: '#262119', // raised / hover
        line: '#3a3328', // hairlines
        cream: '#f3ede2', // text
        dust: '#a89f90', // secondary text (7:1 on
night)
        marigold: '#f2a23a', // actions, focus, the
brand
        verified: '#5cc2b3', // checked facts
        missing: '#ff8a7a', // [missing: ...]
placeholders
      },
      fontFamily: {
        display: ['"Big Shoulders Display"',
'Impact', 'sans-serif'],
        sans: ['"IBM Plex Sans"', 'system-ui',
'sans-serif'],
        mono: ['"IBM Plex Mono"', 'ui-monospace',
'monospace'],
      },
      keyframes: {
        drift: { from: { transform:
'translate3d(0,0,0)' }, to: { transform:
'translate3d(-50%,0,0)' } },
        rise: { from: { opacity: '0', transform:
'translateY(12px)' }, to: { opacity: '1', transform:
'none' } },
        zoom: { from: { transform: 'scale(1.05)' },
to: { transform: 'scale(1.25)' } },
      },
      animation: {
        drift: 'drift 90s linear infinite',
        rise: 'rise .5s cubic-bezier(.16,1,.3,1)
both',
      },
    },
  },
  plugins: [],
}
```

postcss.config.js — 1 lines

```
export default { plugins: { tailwindcss: {},
autoprefixer: {} } }
```

.gitignore — 7 lines

```
node_modules
dist
dist-ssr
.env
.env.local
.DS_Store
test-results
```

.env.example — 3 lines

```
# Free key from https://web3forms.com (enter your
email, they mail you the key).
# Copy this file to .env.local and paste the key.
Never commit .env.local.
VITE_WEB3FORMS_KEY=
```

CLAUDE.md — 25 lines

```
# Working on this repo

These are the rules this codebase was written to
(adapted from the
Karpathy-inspired coding guidelines). Follow them
for every change.

1. **Think before coding.** Say what you're about to
change and why. If a
   request is ambiguous, ask instead of guessing.
2. **Simplicity first.** The minimum code that
solves the problem. No new
   dependency, abstraction or config option unless
something real needs it.
   Plain React state + Tailwind classes; no state
library, no router.
3. **Surgical changes.** Touch only what the task
needs. Don't reformat or
   "tidy" unrelated code in the same change.
4. **Goal-driven.** Every change ends with the
checks passing:
   `npm run check` (unit tests, build, browser
tests, load test).

Project rules:

- **Facts live only in `src/content/profile.js`.**
Never invent a metric,
  employer, user count, link or outcome. Unknown →
`fill('what is missing')`.
  `tests/unit/facts.test.js` pins the resume facts;
update it only when the
  resume changes.
- Styling is Tailwind utility classes; colours and
fonts are defined once in
  `tailwind.config.js`.
- The first client render must match the pre-
rendered HTML: read
  `window`/`localStorage`/the URL only inside
effects.
```

README.md 243 lines

````
# Jaykumar Shevate · on demand

My portfolio, built as a streaming catalogue. You pick who's watching
(recruiter, engineer, just looking), get a billboard with the essentials,
then rows of "titles": projects, credentials, experience and the stack.
Every title opens into its own page with "episodes" (problem → architecture
→ stack → my part). The trailer is a 20-second cut of the facts. The end
credits are the contact page.

It's meant to feel alive. The name slams in letter by letter, a marigold
ticker runs the real highlights, and every poster moves: CarePilot's record
route flows, RentFlow's windows flicker, Sensorium's waves scroll, BazaarLens's
boxes race, and the PhD ERP's student token hops. Projects get big "now
showing" numerals. All of it switches off under `prefers-reduced-motion`.

The idea: recruiters already know how to use a streaming app. They find what
they need in seconds, and the site is still memorable.

**Tailored links: one per application.** Send a company its own link:

```bash
SITE_URL=https://your-site.example npm run link -- "Infosys" cloud
# → https://your-site.example/?for=Infosys&role=cloud
```

That visitor skips the picker and sees "Prepared for Infosys · cloud role" on
the billboard. The brief opens with Cloud, highlighted as "the role you're
hiring for", and the browser tab title includes the company name. Roles:
`cloud`, `backend`, `security`, `devops`. Values are shown as plain text,
stripped of markup and capped at 40 characters (there's a test that tries to
inject HTML).

**Extras → Making of this site.** The portfolio as its own case study (the
problem, the architecture, the stack, the quality bar) with a link to this
public repo. It's proof of engineering a recruiter can check.

**Local time in Pune.** The brief and the credits show my current time
(IST, UTC+5:30) for recruiters in other time zones.

**For recruiters: the 60-second brief.** Right under the billboard (for
recruiters; after the rows for everyone else):
- **Why me for…** each target role, with only the facts that back it. A role
  with no evidence yet says "missing" instead of being padded.
- **Logistics:** location, start date, work mode, work authorization.
- **Act:** email me, download the résumé, or **copy a clean plain-text summary**
  to paste into an applicant-tracking system. The copied summary leaves out
  anything not filled in yet.

```bash
npm install
npm run dev        # http://localhost:5173
```

Nothing is deployed and there's no backend. The build is plain static files
that any static host can serve.

---

## Before sending the link to anyone

All facts live in **`src/content/profile.js`**. Anything I haven't written yet
is `fill('...')` and shows on the site as a red dashed **missing: …** tag, so
nothing gets made up.

```bash
npm run todo       # lists every missing fact
```

- **Resume PDF:** put it in `public/`, then set `links.resume` to e.g.
  `'/Jaykumar-Shevate-Resume.pdf'`. The Résumé button then appears in the
  top bar and in the credits.
- **Contact form:** get a free key at https://web3forms.com, copy
  `.env.example` to `.env.local` and set `VITE_WEB3FORMS_KEY`. Until then the
  form says it isn't connected; it never pretends to send.
- **Screenshots:** add `screenshots: [{ src: '/shots/x.webp', alt: '…' }]`
  to a project and they appear on its page.
- The tagline and the call-to-action (`cta`) are drafts. Rewrite them in your
  own words.

---

## Checking it

```bash
npm run check      # everything below, in order
npm test           # unit tests (Vitest)
npm run build      # production build + pre-render
npm run test:e2e   # browser tests (Playwright + axe), against the build
npm run test:load  # 200 parallel connections against the build
```

If Playwright can't find a browser on your machine: `npx playwright install chromium`.

**Unit tests** (`tests/unit/`, 25 tests)

- `facts.test.js`: every score, date, title and employer must match the
  resume exactly. If someone edits a number by mistake, this fails.
- `catalog.test.js`: unique ids, every viewer's rows exist, search only finds
  real words (e.g. "kubernetes" finds nothing, because it isn't on the resume).
- `render.test.jsx`: renders the whole page the way the build does and fails
  on `undefined`, `NaN`, `[object Object]` or `null` anywhere in the text.
- `storage.test.js`: the site still works when the browser blocks storage.
- `tailor.test.js`: tailored-link parsing, unknown roles ignored, markup
  stripped, length capped.
````

```
- `brief.test.js`: every "why me" proof is built from text already in
  `profile.js`, and the ATS summary has no placeholders or garbage.

**Browser tests** (`tests/e2e/run.mjs`, 18 scenarios, real Chromium)

- the recruiter brief comes first, and "Copy summary" puts a clean profile on
  the clipboard
- the viewer picker sets the order of the rows and is remembered
- a tailored link greets the company, leads with its role and skips the
  picker, and an HTML/script injection attempt in `?for=` renders as text
- all 18 titles open, show no rendering garbage, close with Esc, and return
  keyboard focus to the card
- shared links (`#title=carepilot`) open straight to the title (no picker),
  Back closes it, and unknown ids are ignored
- search, My list, Continue watching, the trailer (play, pause, step, close)
- the contact form enforces its fields and never fakes success
- keyboard: first Tab is "Skip to the catalogue"; dialogs keep focus inside
- **axe-core WCAG 2.1 AA scan** of the picker, home, a title page and the
  trailer: zero violations
- phone viewport: no sideways scrolling (this caught long "missing" tags
  widening the page)
- reduced motion: zero running animations, and the trailer doesn't auto-advance
- blocked storage, and **JavaScript turned off** (all content still readable)
- **25 visitors at once**, in parallel browser contexts: each gets the right
  rows and their own My list, with no errors

**Load test** (`tests/load/run.mjs`): 200 parallel connections for 10 s on
the page and on the main script. Last run: about 9,800 page requests and
43,000 script requests, **0 errors, 0 timeouts, 0 non-200**, p99 latency
274 ms and 63 ms. That's on `vite preview`, a dev-grade server; a real static
host or CDN does better.

Why many recruiters at once is safe: there is no server code and no shared
state. Every visitor downloads the same static files; "My list", "Continue
watching" and the viewer choice live in that visitor's own browser. The only
outside call is the contact form, and each submission is independent.

**Lighthouse 12** (measured locally):

| | Performance | Accessibility | Best practices | SEO | First paint | Layout shift |
|---|---|---|---|---|---|---|
| Phone (slow 4G) | 98 | 100 | 100 | 100 | 1.8 s | 0.005 |
| Desktop | 100 | 100 | 100 | 100 | 0.4 s | 0.001 |

---

## How it works (the interview version)

### One data file, one derived catalogue
- `src/content/profile.js` holds the facts. It's plain data, so Node can run it
  too (the tests and the pre-render import it).
- `src/content/catalog.js` turns those facts into **titles** (anything you can
  open) and **rows**. It also holds `VIEWERS`: each viewer is just a row order.
  A recruiter sees credentials first, an engineer sees projects first.
  Everyone can reach everything.

### Pre-rendered, then hydrated
`npm run build` builds the app, then builds `src/entry-server.jsx` for Node,
and `scripts/prerender.mjs` renders `<App/>` to HTML into `dist/index.html`. So:
- the first response already contains every project, credential and job
  (good for slow phones, search engines and link previews)
- with JavaScript off, the whole catalogue is still readable
- React then *hydrates* that HTML (`hydrateRoot` in `src/main.jsx`)

For hydration to work, the first client render must match the server HTML
exactly. That's why anything from the browser (storage, the URL hash) is read
in a `useEffect` *after* the first render (see `App.jsx`).

### No flash of the picker
A 3-line inline script in `index.html` runs before paint: if this visitor
already picked a viewer, or arrived on a shared `#title=` link, it adds
`html.skip-gate`, and CSS hides the picker immediately. Once React is running
it adds `.hydrated` and takes over.

### State without a state library
Five `useState`s in `App.jsx`: viewer, picker open, My list, recently opened,
search query. The open title lives in the **URL hash** (`#title=id`,
`src/lib/hash.js`), which gives shareable links and a working Back button for
free. Storage goes through `src/lib/storage.js`, which never throws (private
windows and blocked cookies make `localStorage` throw).

### Posters without images
`src/components/Art.jsx` draws every poster in SVG from the title's id: a
pipeline of stamped approvals for the PhD ERP, a hospital network with one
record's route for CarePilot, a lit-up building for RentFlow, sensor waves for
Sensorium Nexus, racing price lanes for BazaarLens (the cheapest wins, not the
fastest), and big result numbers for credentials. A seeded random generator
keeps each poster identical on every visit and between server and client.
No image downloads at all.

### Accessible by construction
- Cards are real `<button>`s and rows are `<ul>`s under headings.
- Title pages and the trailer are dialogs: focus moves in, Tab stays inside,
  Esc closes, focus returns to where you were. The page behind is `inert`.
- Placeholders are visibly labelled "missing", never styled as content.
- Colours are checked for 4.5:1 contrast; animations respect
  `prefers-reduced-motion` (the trailer stops auto-
```

```
advancing).

### Performance
- About 58 KB of gzipped JS in total. No 3D, no
animation library.
- The fonts the first screen needs are preloaded
(the pre-render step adds
  `<link rel="preload">`), so headlines don't reflow
when fonts arrive.
- Tailwind only ships the classes actually used
(about 6 KB of gzipped CSS).

---

## Files

```
src/
  content/profile.js     every fact (the only file
to edit for content)
  content/catalog.js     titles, rows, viewers,
search
  content/brief.js       role fit + ATS summary
(facts mapped, never added)
  content/making-of.js   the "Making of this site"
extra
  lib/tailor.js          ?for=Company&role=…
tailored links
  lib/storage.js         localStorage that never
throws
  lib/hash.js            #title=id ⇄ open title
  components/
    Gate.jsx             "Who's here?"
    NavBar.jsx           wordmark, sections, search
("/"), résumé, viewer
    Billboard.jsx        hero + drifting poster wall
+ highlights ticker
    Brief.jsx            the recruiter's 60-second
brief
    PuneClock.jsx        my local time (IST)
    Row.jsx · TitleCard.jsx · Art.jsx
    TitlePage.jsx        the title dialog (facts,
episodes, more like this)
    Trailer.jsx          the 7-shot trailer (2.8 s a
shot)
    Credits.jsx          contact + form + credits
roll
    Value.jsx            renders a fact or a
"missing" tag
  App.jsx · main.jsx · entry-server.jsx · index.css
scripts/prerender.mjs    HTML pre-render + font
preloads
scripts/todo.mjs         lists missing facts
scripts/link.mjs         prints a tailored link for
one application
tests/unit · tests/e2e · tests/load
CLAUDE.md                the coding rules this repo
follows
```

Stack: React 18, Vite 5, Tailwind CSS 3, Vitest,
Playwright, axe-core,
autocannon. Fonts: Big Shoulders Display and IBM
Plex Sans/Mono, self-hosted.
```

public/robots.txt 2 lines

```
User-agent: *
Allow: /
```

scripts/link.mjs 14 lines

```
/*
 * npm run link -- "Company"
[cloud|backend|security|devops]
 * Prints a tailored link for one application. Set
SITE_URL to the address
 * the site is hosted at (defaults to the local dev
server).
 */
const [company, role] = process.argv.slice(2)
if (!company) {
  console.log('usage: npm run link -- "Company name"
[cloud|backend|security|devops]')
  process.exit(1)
}
const url = new URL(process.env.SITE_URL ||
'http://localhost:5173/')
url.searchParams.set('for', company)
if (role) url.searchParams.set('role', role)
console.log(url.toString())
```

scripts/prerender.mjs 19 lines

```
/*
 * After `vite build`, render the app to HTML once
and put it in
 * dist/index.html. Visitors (and search engines)
get the full content in
 * the first response; React then hydrates it.
 */
import { readFile, writeFile, rm, readdir } from
'node:fs/promises'
import { render } from '../dist-ssr/entry-server.js'

const file = new URL('../dist/index.html',
import.meta.url)
const html = await readFile(file, 'utf8')
if (!html.includes('<!--app-->')) throw new
Error('dist/index.html has no <!--app-->
placeholder')
// Preload the fonts the first screen uses, so text
doesn't reflow when they arrive.
const fonts = (await readdir(new
URL('../dist/assets', import.meta.url))).filter((f)
=>
  /^(big-shoulders-display-latin-(800|900)|ibm-plex-
sans-latin-400)-normal-.*\.woff2$/.test(f),
)
const preload = fonts.map((f) => `<link
rel="preload" href="/assets/${f}" as="font"
type="font/woff2" crossorigin>`).join('\n    ')
await writeFile(file, html.replace('<!--app-->',
render()).replace('</title>', `</title>\n
${preload}`))
await rm(new URL('../dist-ssr', import.meta.url), {
recursive: true, force: true })
console.log('prerendered dist/index.html')
```

scripts/todo.mjs 15 lines

```
// npm run todo — every fact still missing from
src/content/profile.js.
import { profile, isFill } from
'../src/content/profile.js'

const missing = []
const walk = (v, path) => {
  if (isFill(v)) return missing.push(`${path}  →
${v.fill}`)
  if (Array.isArray(v)) return v.forEach((x, i) =>
walk(x, `${path}[${i}]`))
  if (v && typeof v === 'object') for (const [k, x]
of Object.entries(v)) walk(x, path ? `${path}.${k}`
: k)
}
walk(profile, '')

console.log(`\n${missing.length} things still
missing in src/content/profile.js:\n`)
for (const m of missing) console.log('  ' + m)
console.log('\nOptional: screenshots per project,
otherRepos, VITE_WEB3FORMS_KEY in .env.local.')
console.log('Also: rewrite the DRAFT tagline and cta
in your own words.\n')
```

src/App.jsx 155 lines

```
/*
 * App — the whole site is one page: billboard,
rows, credits. Overlays on
 * top of it: the "Who's here?" gate, a title page,
the trailer.
 *
 * State, all local to this visitor:
 *   viewer   who's watching (orders the rows)
→ localStorage
 *   saved    My List
→ localStorage
 *   recent   titles opened, newest first
→ localStorage
 *   query    search box
 *   open     the title in the URL hash (#title=id),
so links are shareable
 *
 * The first render must match the pre-rendered HTML
exactly, so anything
 * read from the browser (storage, hash) is loaded
in an effect afterwards.
 */
import { useCallback, useEffect, useState } from
'react'
import { ROWS, VIEWERS, DEFAULT_VIEWER, titleById,
search } from './content/catalog'
import { read, write } from './lib/storage'
import { parseHash, setTitleHash, useTitleHash }
from './lib/hash'
import { useTailor } from './lib/tailor'
import NavBar from './components/NavBar'
import Billboard from './components/Billboard'
import Row from './components/Row'
import Gate from './components/Gate'
import TitlePage from './components/TitlePage'
import Trailer from './components/Trailer'
import Credits from './components/Credits'
import Brief from './components/Brief'
import TitleCard from './components/TitleCard'

const MAX_RECENT = 8

export default function App() {
  const [ready, setReady] = useState(false) // true
once browser state is loaded
  const [viewer, setViewer] = useState(null)
  // Open in the pre-rendered HTML; CSS hides it
before paint for returning visitors.
  const [gate, setGate] = useState(true)
  const [saved, setSaved] = useState([])
  const [recent, setRecent] = useState([])
  const [query, setQuery] = useState('')
  const [trailer, setTrailer] = useState(false)
  const tailor = useTailor() // ?
for=Company&role=cloud
  const openId = useTitleHash()
  const open = openId ? titleById[openId] : null

  useEffect(() => {
    const v = read('viewer', null)
    setViewer(VIEWERS[v] ? v : null)
    // Someone who followed a shared link
(#title=...) goes straight to it.
    // Shared links (#title=...) and tailored links
(?for=...) skip the picker.
    const direct = parseHash(window.location.hash)
|| new
URLSearchParams(window.location.search).has('for')
    setGate(!VIEWERS[v] && !direct)

document.documentElement.classList.add('hydrated')
    setSaved(read('saved', []).filter((id) =>
titleById[id]))
    setRecent(read('recent', []).filter((id) =>
titleById[id]))
    setReady(true)
  }, [])

  // A tailored link says who it's for in the tab
title too.
  useEffect(() => {
    if (tailor?.company) document.title = `Jaykumar
Shevate for ${tailor.company}`
  }, [tailor])

  // Remember what this visitor opened (for
"Continue watching").
  useEffect(() => {
    if (!open) return
    setRecent((r) => {
      const next = [open.id, ...r.filter((x) => x
!== open.id)].slice(0, MAX_RECENT)
      write('recent', next)
      return next
    })
  }, [open])

  const pickViewer = (v) => {
    setViewer(v)
    write('viewer', v)
    setGate(false)
    window.scrollTo(0, 0)
    // The button just clicked disappears with the
gate; drop focus so the
    // next Tab starts from there (the gate is first
in the DOM).
    document.activeElement?.blur()
  }

  const toggleSave = (id) =>
    setSaved((s) => {
      const next = s.includes(id) ? s.filter((x) =>
x !== id) : [id, ...s]
      write('saved', next)
      return next
    })

  const openTitle = useCallback((id) =>
setTitleHash(id), [])
  const closeTitle = useCallback(() =>
setTitleHash(null), [])
  const closeTrailer = useCallback(() =>
setTrailer(false), [])
  const toContact = useCallback(() => {
    setTrailer(false)

document.getElementById('credits')?.scrollIntoView()
  }, [])

  const rows = VIEWERS[viewer ??
DEFAULT_VIEWER].rows
  const results = query.trim() ? search(query) :
null
  const overlay = (ready && gate) || open || trailer
  const list = (ids) => ids.map((id) =>
titleById[id]).filter(Boolean)

  return (
    <>
      {/* Gate first in the DOM: after it closes,
the next Tab lands on the skip link. */}
      <Gate open={gate} onPick={pickViewer} />

      <a href="#rows" className="fixed left-3 top-3
z-[60] -translate-y-24 rounded-sm bg-marigold px-3
py-2 font-semibold text-night focus:translate-y-0">
        Skip to the catalogue
      </a>

      <div inert={overlay ? '' : undefined} aria-
hidden={overlay ? true : undefined}>
        <NavBar query={query} onQuery={setQuery}
viewer={viewer} onSwitchViewer={() => setGate(true)}
/>
```

```
        <main>
          {!results && <Billboard tailor={tailor}
onPlay={() => setTrailer(true)} onInfo={() =>
openTitle('about')} />}

          <div id="rows" className={`relative z-10
space-y-8 sm:space-y-12 ${results ? 'pt-24' : 'pt-
10'}`}>
            {results ? (
              <section aria-labelledby="results"
className="px-4 sm:px-10">
                <h2 id="results" className="font-
display text-3xl font-bold uppercase tracking-wide
text-cream">
                  {results.length ? `Results for
“${query.trim()}”` : `Nothing matches
“${query.trim()}”`}
                </h2>
                {!results.length && <p
className="mt-2 text-dust">Try “aws”, “iot”,
“healthcare” or “intern”.</p>}
                <ul className="mt-5 grid grid-cols-1
gap-3 sm:grid-cols-2 lg:grid-cols-4">
                  {results.map((t) => (
                    <li key={t.id}>
                      <TitleCard title={t} onOpen=
{openTitle} saved={saved.includes(t.id)} />
                    </li>
                  ))}
                </ul>
              </section>
            ) : (
              <>
                {ready && <Row id="mylist" name="My
list" note="saved in your browser only" titles=
{list(saved)} onOpen={openTitle} savedIds={saved}
/>}
                {ready && <Row id="recent"
name="Continue watching" titles={list(recent)}
onOpen={openTitle} savedIds={saved} />}
                {/* Recruiters get the brief first;
everyone else after the rows they came for. */}
                {viewer !== 'engineer' && viewer !==
'curious' && <Brief tailor={tailor} />}
                {rows.map((key) => (
                  <Row key={key} id={key} name=
{ROWS[key].name} numbered={key === 'projects'} note=
{key === 'projects' ? 'now showing' : undefined}
titles={list(ROWS[key].ids)} onOpen={openTitle}
savedIds={saved} />
                ))}
                {(viewer === 'engineer' || viewer
=== 'curious') && <Brief tailor={tailor} />}
              </>
            )}
          </div>
        </main>

        <Credits />
      </div>

      {open && <TitlePage key={open.id} title={open}
onClose={closeTitle} onOpen={openTitle} saved=
{saved.includes(open.id)} onToggleSave={toggleSave}
/>}
      {trailer && <Trailer onClose={closeTrailer}
onContact={toContact} />}
    </>
  )
}
```

src/components/Art.jsx — 215 lines

```
/*
 * Art — every title's "poster", drawn in SVG from
its id. No image files:
 * each project gets a small illustration of what it
does, credentials get
 * their result set big like a film title, jobs and
skill groups get type.
 *
 * Random-looking details come from a seeded
generator, so a poster looks
 * the same on every visit and on the server and the
client (no hydration
 * mismatch).
 */
import { useId } from 'react'

const W = 320
const H = 180
const C = { cream: '#f3ede2', dust: '#a89f90', line:
'#3a3328', gold: '#f2a23a', teal: '#5cc2b3', ash:
'#262119' }

function rng(seed) {
  let s = 0
  for (const ch of seed) s = (s * 31 +
ch.charCodeAt(0)) >>> 0
  return () => ((s = (s * 1664525 + 1013904223) >>>
0) / 4294967296)
}

/* --- one motif per project --- */

function Pipeline() {
  // Doctoral pipeline: stations get stamped as the
student token moves along.
  const xs = [40, 88, 136, 184, 232, 280]
  return (
    <g>
      <path d={`M${xs[0]} 72 H${xs.at(-1)}`} stroke=
{C.line} strokeWidth="2" />
      <path d={`M${xs[0]} 72 H${xs[3]}`} stroke=
{C.gold} strokeWidth="2" />
      {xs.map((x, i) => (
        <g key={x}>
          <circle cx={x} cy="72" r="11" fill={C.ash}
stroke={i <= 3 ? C.gold : C.line} strokeWidth="2" />
          {i < 3 && <path d={`M${x - 4} 72 l3 3 l6
-7`} stroke={C.gold} strokeWidth="2" fill="none" />}
        </g>
      ))}
      <rect x={xs[3] - 6} y="40" width="12"
height="12" fill={C.gold} className="art-hop" />
    </g>
  )
}

function Graph({ r }) {
  // Hospitals as a network; one record's route is
highlighted.
  const nodes = Array.from({ length: 11 }, () => [30
+ r() * 260, 30 + r() * 120])
  const edges = nodes.map((n, i) => {
    let best = 0
    let d = Infinity
    nodes.forEach((m, j) => {
      const dd = (n[0] - m[0]) ** 2 + (n[1] - m[1])
** 2
      if (j !== i && dd < d) (d = dd), (best = j)
    })
    return [i, best]
  })
  const route = [0, 3, 6, 9]
  return (
    <g>
      {edges.map(([a, b], k) => (
        <line key={k} x1={nodes[a][0]} y1={nodes[a]
[1]} x2={nodes[b][0]} y2={nodes[b][1]} stroke=
{C.line} strokeWidth="1.5" />
      ))}
      <polyline points={route.map((i) =>
nodes[i].join(',')).join(' ')} fill="none" stroke=
{C.gold} strokeWidth="2" strokeDasharray="5 4"
className="art-flow" />
      {nodes.map(([x, y], i) => (
```

```
        <g key={i}>
          <circle cx={x} cy={y} r={route.includes(i)
? 6 : 4} fill={route.includes(i) ? C.gold : C.ash}
stroke={C.dust} strokeWidth="1.2" />
        </g>
      ))}
    </g>
  )
}

function Building({ r }) {
  // A block of flats; each window is a unit, lit by
its rent status.
  const cells = []
  for (let f = 0; f < 5; f++)
    for (let c = 0; c < 4; c++) {
      const v = r()
      const lit = v > 0.82
      cells.push(
        <rect
          key={`${f}-${c}`}
          x={108 + c * 27}
          y={34 + f * 25}
          width="19"
          height="16"
          fill={lit ? C.gold : v > 0.55 ? C.dust :
C.ash}
          opacity={v > 0.55 ? 0.9 : 1}
          className={lit || v < 0.12 ? 'art-flicker'
: undefined}
          style={{ animationDelay: `${(v *
4).toFixed(2)}s` }}
        />,
      )
    }
  return (
    <g>
      <rect x="98" y="24" width="124" height="134"
fill="none" stroke={C.dust} strokeWidth="1.5" />
      <path d="M90 24 L160 4 L230 24" fill="none"
stroke={C.dust} strokeWidth="1.5" />
      {cells}
      <line x1="20" y1="158" x2="300" y2="158"
stroke={C.line} strokeWidth="2" />
    </g>
  )
}

function Waves({ r }) {
  // Sensor streams, one line per device.
  const lines = [0, 1, 2, 3].map((k) => {
    const f = 0.03 + r() * 0.05
    const ph = r() * 6
    const y0 = 42 + k * 34
    const pts = Array.from({ length: 65 }, (_, i) =>
{
      const x = 16 + i * 4.5
      return `${x.toFixed(1)},${(y0 + Math.sin(x * f
+ ph) * 9 + Math.sin(x * f * 3.1) * 3).toFixed(1)}`
    })
    return (
      <polyline key={k} points={pts.join(' ')}
fill="none" stroke={k === 1 ? C.gold : C.dust}
strokeWidth={k === 1 ? 2 : 1.4} className="art-wave"
style={{ animationDuration: `${3 + k}s` }} />
    )
  })
  return <g>{lines}</g>
}

function Lanes({ r }) {
  // Stores racing in lanes; the cheapest one is
marked, not the fastest.
  const runs = [0, 1, 2, 3].map(() => ({ reach: 0.55
+ r() * 0.45, price: 20 + r() * 50 }))
  const cheapest = runs.reduce((b, x, i) => (x.price
< runs[b].price ? i : b), 0)
  return (
    <g>
      {runs.map((run, i) => {
        const y = 38 + i * 34
        const x = 24 + run.reach * 250
        const win = i === cheapest
        return (
          <g key={i}>
            <line x1="24" y1={y} x2="296" y2={y}
stroke={C.line} strokeWidth="1.5" />
            <rect x={x - 9} y={y - 9} width="18"
height="18" fill={win ? C.gold : C.ash} stroke={win
? C.gold : C.dust} strokeWidth="1.4" className="art-
race" style={{ animationDuration: `${(1.6 + (1 -
run.reach) * 2.4).toFixed(2)}s` }} />
            <rect x={x + 14} y={y - 4} width=
{run.price} height="8" fill={win ? C.gold : C.line}
/>
          </g>
        )
      })}
      <line x1="296" y1="22" x2="296" y2="150"
stroke={C.dust} strokeDasharray="3 3" />
    </g>
  )
}

function Screen() {
  // This site: a browser window with rows of cards
and a play button.
  return (
    <g>
      <rect x="70" y="18" width="180" height="112"
rx="6" fill={C.ash} stroke={C.dust}
strokeWidth="1.5" />
      <line x1="70" y1="34" x2="250" y2="34" stroke=
{C.line} strokeWidth="1.5" />
      {[0, 1, 2].map((i) => (
        <circle key={i} cx={82 + i * 9} cy="26"
r="2.5" fill={i === 0 ? C.gold : C.line} />
      ))}
      {[0, 1].map((r) =>
        [0, 1, 2, 3].map((c) => <rect key=
{`${r}${c}`} x={82 + c * 41} y={78 + r * 22}
width="35" height="16" rx="2" fill={c === 1 && r ===
0 ? C.gold : C.line} />),
      )}
      <circle cx="160" cy="56" r="14" fill={C.gold}
className="art-hop" />
      <path d="M155 49 L167 56 L155 63 Z"
fill="#0d0b09" className="art-hop" />
    </g>
  )
}

const MOTIF = { 'phd-erp': Pipeline, carepilot:
Graph, rentflow: Building, sensorium: Waves,
bazaarlens: Lanes, 'this-site': Screen }

/* --- the poster --- */

export default function Art({ title, className = ''
}) {
  const r = rng(title.id)
  // Unique per instance: the same poster can appear
many times on a page.
  const fadeId = `fade${useId().replace(/[^a-zA-Z0-
9]/g, '')}`
  const Motif = MOTIF[title.id]
  const tilt = (r() - 0.5) * 30

  return (
    <svg viewBox={`0 0 ${W} ${H}`} className=
{className} role="img" aria-label={`${title.name}
poster`} preserveAspectRatio="xMidYMid slice">
      <rect width={W} height={H} fill="#17140f" />
      {/* a faint grid, rotated a little differently
on every poster */}
      <g opacity="0.35" transform={`rotate(${tilt}
```

```
${W / 2} ${H / 2})`}>
        {Array.from({ length: 14 }, (_, i) => (
          <line key={i} x1={-80 + i * 36} y1="-80"
x2={-80 + i * 36} y2="260" stroke={C.line} />
        ))}
      </g>

      {Motif && <Motif r={r} />}

      {title.kind === 'credential' && (
        <text x="300" y="66" textAnchor="end" fill=
{C.teal} fontFamily="Big Shoulders Display"
fontWeight="900" fontSize={title.result.length > 6 ?
40 : 52}>
          {title.result.toUpperCase()}
        </text>
      )}

      {title.kind === 'experience' && (
        <g>
          <text x="300" y="64" textAnchor="end"
fill={C.gold} fontFamily="Big Shoulders Display"
fontWeight="900" fontSize="50" opacity="0.9">
            {title.name.split(' ').map((w) =>
w[0]).join('')}
          </text>
          <line x1="20" y1="82" x2="300" y2="82"
stroke={C.line} strokeWidth="2" />
        </g>
      )}

      {title.kind === 'skill' && (
        <g>
          {title.items.slice(0, 4).map((it, i) => (
            <rect key={i} x={230 + i * 18} y={30 +
(i % 2) * 8} width="14" height="14" fill={i === 0 ?
C.gold : C.ash} stroke={C.dust} />
          ))}
        </g>
      )}

      {/* bottom fade so the title on the card stays
readable */}
      <defs>
        <linearGradient id={fadeId} x1="0" y1="0"
x2="0" y2="1">
          <stop offset="0.45" stopColor="#0d0b09"
stopOpacity="0" />
          <stop offset="1" stopColor="#0d0b09"
stopOpacity="0.92" />
        </linearGradient>
      </defs>
      <rect width={W} height={H} fill=
{`url(#${fadeId})`} />
    </svg>
  )
}
```

src/components/Billboard.jsx 109 lines

```
/*
 * The billboard: who I am in one screen, with the
two things a recruiter
 * checks first (strongest credential, what I'm
looking for) and two buttons.
 * Behind it, the five project posters drift past on
a slow loop.
 */
import { profile } from '../content/profile'
import { titles } from '../content/catalog'
import Art from './Art'

const projects = titles.filter((t) => t.kind ===
'project')
const ccp = profile.credentials[0]
const erp = profile.projects.find((p) => p.id ===
'phd-erp')

// Everything in the ticker comes straight from
profile.js.
const highlights = [
  ...profile.credentials.map((c) => `${c.name} ·
${c.result}`),
  `${erp.role} · ${erp.name} · ${erp.status}`,
  `${profile.experience[0].title} ·
${profile.experience[0].org}`,
  `${profile.education.degree} ·
${profile.education.year}`,
  profile.freelance ? 'Open to freelance work' :
null,
].filter(Boolean)

function Ticker() {
  const items = (hidden) =>
    highlights.map((h) => (
      <li key={h + hidden} aria-hidden={hidden ||
undefined} className="flex shrink-0 items-center
gap-6 pr-6">
        <span>{h}</span>
        <span aria-hidden="true" className="text-
night/50">✦</span>
      </li>
    ))
  return (
    <div className="absolute inset-x-0 bottom-0 -
rotate-1 overflow-hidden bg-marigold py-2 shadow-
[0_10px_30px_-10px_rgba(242,162,58,.6)]">
      <ul aria-label="Highlights" className="ticker
flex w-max font-mono text-xs font-semibold uppercase
tracking-[0.18em] text-night sm:text-sm">
        {items(false)}
        {items(true)}
      </ul>
    </div>
  )
}

export default function Billboard({ tailor, onPlay,
onInfo }) {
  return (
    <section id="top" aria-labelledby="bill-title"
className="relative isolate flex min-h-[86svh]
items-end overflow-hidden pb-24 pt-28 sm:min-h-
[92svh] sm:pb-32">
      {/* Poster wall: two copies side by side so
the loop has no seam. */}
      <div aria-hidden="true" className="absolute
inset-0 -z-10 opacity-75 [perspective:900px]">
        <div className="absolute -inset-x-20 top-6
flex w-max motion-safe:animate-drift
[transform:rotateX(14deg)_rotateZ(-4deg)]" style={{
animationDuration: '50s' }}>
          {[0, 1].map((copy) => (
            <div key={copy} className="grid grid-
flow-col grid-rows-2 gap-3 pr-3">
              {[...projects, ...projects].map((t, i)
=> (
                <Art key={`${copy}-${i}`} title={t}
className="h-[30vh] w-auto rounded-md" />
              ))}
            </div>
          ))}
        </div>
        <div className="absolute inset-0 bg-
gradient-to-r from-night via-night/80 to-night/10"
/>
        <div className="absolute inset-0 bg-
gradient-to-t from-night via-transparent to-
night/40" />
      </div>

      <div className="max-w-3xl px-4 sm:px-10">
        {tailor?.company && (
          <p data-tailored className="mb-4 inline-
flex items-center gap-2 rounded-sm bg-marigold px-3
py-1.5 font-mono text-xs font-semibold uppercase
tracking-[0.18em] text-night motion-safe:animate-
```

```
rise">
            <span aria-hidden="true">●</span>
Prepared for {tailor.company}
            {tailor.role && <span className="text-
night/70">· {tailor.role === 'devops' ? 'junior
DevOps' : tailor.role} role</span>}
          </p>
        )}
        <p className="motion-safe:animate-rise font-
mono text-xs uppercase tracking-[0.25em] text-
marigold">
          A one-person catalogue · {profile.city}
        </p>
        <h1 id="bill-title" className="mt-3 font-
display text-6xl font-black uppercase leading-[0.85]
tracking-tight text-cream sm:text-8xl lg:text-9xl">
          <span className="sr-only">
{profile.displayName}</span>
          {/* Each letter is its own span so it can
slam in on its own beat. */}
          {profile.displayName.split(' ').map((word,
w) => (
            <span key={word} aria-hidden="true"
className="block">
              {[...word].map((ch, i) => (
                <span key={i} className="slam"
style={{ animationDelay: `${(w * 8 + i) * 45}ms` }}>
                  {ch}
                </span>
              ))}
            </span>
          ))}
          <span aria-hidden="true" className="sweep
mt-3 block h-1.5 w-40 bg-marigold sm:w-64" />
        </h1>

        <p className="mt-4 flex flex-wrap items-
center gap-x-3 gap-y-1 font-mono text-xs text-dust
sm:text-sm">
          <span className="rounded-sm border border-
verified/60 px-1.5 py-0.5 text-verified">
{ccp.name.replace('AWS Certified ', 'AWS ')} ·
{ccp.result}</span>
          <span>
{profile.education.degree.replace('Computer Science
& Engineering', 'CSE')}</span>
          <span>· {profile.education.year}</span>
        </p>

        <p className="mt-5 max-w-xl text-lg leading-
relaxed text-cream/90 sm:text-xl">{profile.tagline}
</p>

        <p className="mt-3 text-sm text-dust">
          Looking for {profile.targeting.slice(0,
-1).join(', ')} or {profile.targeting.at(-1)} roles
          {profile.freelance ? ', and open to
freelance work.' : '.'}
        </p>

        <div className="mt-7 flex flex-wrap gap-3">
          <button type="button" onClick={onPlay}
className="flex items-center gap-2 rounded-sm bg-
cream px-6 py-2.5 font-semibold text-night
transition hover:bg-marigold hover:shadow-
[0_0_30px_rgba(242,162,58,.55)] motion-
safe:hover:scale-105">
            <span aria-hidden="true">▶</span> Play
trailer
          </button>
          <button type="button" onClick={onInfo}
className="flex items-center gap-2 rounded-sm bg-
ash/90 px-6 py-2.5 font-semibold text-cream ring-1
ring-line hover:bg-ash hover:ring-marigold">
            <span aria-hidden="true" className="grid
h-5 w-5 place-items-center rounded-full border
border-current text-xs">i</span> More info
          </button>
        </div>
      </div>
      <Ticker />
    </section>
  )
}
```

src/components/Brief.jsx — 107 lines

```
/*
 * "Your 60-second brief" — everything a recruiter
screens for, on one
 * screen: the fit for each role (with the proof),
the logistics, and the
 * three ways to act (email, résumé, copy a summary
into their ATS).
 */
import { useState } from 'react'
import { profile, real } from '../content/profile'
import { fit, summary } from '../content/brief'
import Value from './Value'
import PuneClock from './PuneClock'

export default function Brief({ tailor }) {
  const [copy, setCopy] = useState('Copy summary for
your ATS')
  const email = real(profile.links.email)
  const resume = real(profile.links.resume)

  const copySummary = async () => {
    try {
      await navigator.clipboard.writeText(summary())
      setCopy('Copied: paste it anywhere')
    } catch {
      setCopy('Copy blocked by the browser: select
the text in “More info”')
    }
    setTimeout(() => setCopy('Copy summary for your
ATS'), 3000)
  }

  // On a tailored link, the role they're hiring for
goes first.
  const wanted = tailor?.role
  const roles = wanted ? [...fit].sort((a, b) =>
(b.role.toLowerCase().includes(wanted) ? 1 : 0) -
(a.role.toLowerCase().includes(wanted) ? 1 : 0)) :
fit

  const logistics = [
    ['Location', profile.location],
    ['Local time', <PuneClock key="clock" />],
    ['Available', profile.availability],
    ['Work mode', profile.workMode],
    ['Work authorization',
profile.workAuthorization],
  ]

  return (
    <section id="brief" aria-labelledby="brief-
title" className="px-4 sm:px-10">
      <div className="rounded-md bg-soot p-5 ring-1
ring-line sm:p-8">
        <div className="flex flex-wrap items-end
justify-between gap-4">
          <div>
            <p className="font-mono text-xs
uppercase tracking-[0.25em] text-marigold">
{tailor?.company ? `For ${tailor.company}` : 'For
recruiters'}</p>
            <h2 id="brief-title" className="mt-1
font-display text-4xl font-black uppercase leading-
none text-cream sm:text-5xl">
              Your 60-second brief
            </h2>
          </div>
          <div className="flex flex-wrap gap-2">
            {email ? (
              <a href={`mailto:${email}`}
```

```
className="rounded-sm bg-marigold px-5 py-2.5 font-
semibold text-night hover:bg-cream">Email me</a>
            ) : (
              <span className="rounded-sm border
border-dashed border-missing/70 px-3 py-2.5 font-
mono text-xs text-missing">missing: email</span>
            )}
            {resume && (
              <a href={resume} download
className="rounded-sm bg-cream px-5 py-2.5 font-
semibold text-night hover:bg-marigold">Résumé (PDF)
</a>
            )}
            <button type="button" onClick=
{copySummary} className="rounded-sm bg-ash px-5 py-
2.5 text-cream ring-1 ring-line hover:ring-
marigold">
              {copy}
            </button>
          </div>
        </div>

        <div className="mt-8 grid gap-8 lg:grid-
cols-[1.7fr_1fr]">
          <div>
            <h3 className="font-mono text-[11px]
uppercase tracking-[0.2em] text-dust">Why me for…
</h3>
            <ul className="mt-3 grid gap-3 sm:grid-
cols-2">
              {roles.map((f) => {
                const match = wanted &&
f.role.toLowerCase().includes(wanted)
                return (
                <li key={f.role} data-role={f.role}
className={`rounded-sm bg-night p-4 ring-1 ${match ?
'ring-2 ring-marigold shadow-
[0_0_30px_-8px_rgba(242,162,58,.5)] sm:col-span-2' :
'ring-line'}`}>
                  {match && <p className="mb-1 font-
mono text-[11px] uppercase tracking-[0.2em] text-
marigold">The role you're hiring for</p>}
                  <p className="font-display text-
2xl font-bold uppercase tracking-wide text-cream">
{f.role}</p>
                  <ul className="mt-2 space-y-1.5
text-sm text-dust">
                    {f.proof.map((p, i) => (
                      <li key={i} className="flex
gap-2">
                        <span aria-hidden="true"
className="text-verified">✓</span>
                        <span>
                          <Value v={p} />
                        </span>
                      </li>
                    ))}
                  </ul>
                </li>
                )
              })}
            </ul>
          </div>

          <div>
            <h3 className="font-mono text-[11px]
uppercase tracking-[0.2em] text-dust">Logistics</h3>
            <dl className="mt-3 divide-y divide-
line">
              {logistics.map(([k, v]) => (
                <div key={k} className="grid grid-
cols-[9rem_1fr] gap-3 py-2.5 text-sm">
                  <dt className="text-dust">{k}</dt>
                  <dd className="text-cream">
                    <Value v={v} />
                  </dd>
                </div>
              ))}
            </dl>
          </div>
        </div>
      </div>
    </section>
  )
}
```

src/components/Credits.jsx — 135 lines

```
/*
 * End credits = the contact section. Links, résumé,
a form, then the
 * credits roll, which is just the facts about me
set like film credits.
 *
 * The form posts to Web3Forms (free, no server of
mine). The key comes from
 * .env.local as VITE_WEB3FORMS_KEY. Without a key
the form says so instead
 * of pretending to send. Each visitor's submission
is independent: there
 * is no shared state, so many people can use it at
the same time.
 */
import { useState } from 'react'
import { profile, real } from '../content/profile'
import Value from './Value'
import PuneClock from './PuneClock'

const KEY = import.meta.env.VITE_WEB3FORMS_KEY

function Contact({ label, value, href }) {
  const url = real(value)
  return (
    <li className="flex items-baseline gap-4 border-
b border-line py-3">
      <span className="w-20 shrink-0 font-mono text-
[11px] uppercase tracking-[0.18em] text-dust">
{label}</span>
      {url ? (
        <a href={href ? href(url) : url} target=
{href ? undefined : '_blank'} rel="noreferrer"
className="break-all text-cream underline
decoration-marigold/60 underline-offset-4
hover:text-marigold">
          {url.replace(/^(https?:\/\/|mailto:)/,
'')}
        </a>
      ) : (
        <Value v={value} />
      )}
    </li>
  )
}

export function ContactForm() {
  const [status, setStatus] = useState('idle')

  async function submit(e) {
    e.preventDefault()
    if (!KEY) return setStatus('nokey')
    const form = e.currentTarget
    setStatus('sending')
    try {
      const res = await
fetch('https://api.web3forms.com/submit', {
        method: 'POST',
        headers: { 'Content-Type':
'application/json', Accept: 'application/json' },
        body: JSON.stringify({
...Object.fromEntries(new FormData(form)),
access_key: KEY, subject: 'Portfolio: new message'
}),
      })
      const json = await res.json().catch(() =>
({}))
      if (res.ok && json.success) {
        setStatus('sent')
```

```
        form.reset()
      } else setStatus('error')
    } catch {
      setStatus('error')
    }
  }

  const field = 'mt-1 w-full rounded-sm border
border-line bg-night px-3 py-2 text-cream
placeholder:text-dust focus:border-marigold
focus:outline-none'
  return (
    <form onSubmit={submit} className="grid gap-4">
      <label className="text-sm text-dust">
        Name
        <input name="name" required
autoComplete="name" maxLength={120} className=
{field} />
      </label>
      <label className="text-sm text-dust">
        Email
        <input name="email" type="email" required
autoComplete="email" maxLength={200} className=
{field} />
      </label>
      <label className="text-sm text-dust">
        Message
        <textarea name="message" required rows={4}
maxLength={4000} className={field} />
      </label>
      {/* Honeypot: bots fill it, people never see
it. */}
      <input type="checkbox" name="botcheck"
className="hidden" tabIndex={-1} aria-hidden="true"
/>
      <div className="flex items-center gap-4">
        <button type="submit" disabled={status ===
'sending'} className="rounded-sm bg-marigold px-6
py-2.5 font-semibold text-night hover:bg-cream
disabled:opacity-60">
          {status === 'sending' ? 'Sending…' :
'Send'}
        </button>
        <p role="status" className="text-sm">
          {status === 'sent' && <span
className="text-verified">Sent. I'll reply to the
email you gave.</span>}
          {status === 'error' && <span
className="text-missing">That didn't go through. Try
again, or email me directly.</span>}
          {status === 'nokey' && <span
className="text-missing">The form isn't connected
yet (missing VITE_WEB3FORMS_KEY).</span>}
        </p>
      </div>
    </form>
  )
}

export default function Credits() {
  const resume = real(profile.links.resume)
  const credits = [
    ['Written, directed and engineered by',
profile.name],
    ['Currently', profile.role],
    ['Education', `${profile.education.degree},
${profile.education.school},
${profile.education.year}`],
    ['Filmed on location in', profile.location],
    ['Languages', profile.spoken.join(', ')],
    ['In production', `Learning
${profile.learning.join(', ')}`],
  ]

  return (
    <footer id="credits" className="mt-16 border-t
border-line px-4 pb-16 pt-14 sm:px-10">
      <div className="grid gap-12 lg:grid-cols-2">
        <div>
          <h2 className="font-display text-5xl font-
black uppercase leading-none text-cream sm:text-
7xl">Roll credits</h2>
          <p className="mt-4 max-w-lg text-lg text-
cream/90">{profile.cta}</p>
          <p className="mt-2 text-sm text-dust">It's
<PuneClock /> in {profile.city} right now.</p>
          <ul className="mt-6 max-w-lg">
            <Contact label="Email" value=
{profile.links.email} href={(u) => `mailto:${u}`} />
            <Contact label="GitHub" value=
{profile.links.github} />
            <Contact label="LinkedIn" value=
{profile.links.linkedin} />
          </ul>
          <div className="mt-6">
            {resume ? (
              <a href={resume} download
className="inline-block rounded-sm bg-cream px-6 py-
2.5 font-semibold text-night hover:bg-
marigold">Download résumé (PDF)</a>
            ) : (
              <p className="text-sm text-
dust">Résumé: <Value v={profile.links.resume} /></p>
            )}
          </div>
        </div>
        <ContactForm />
      </div>

      <dl className="mx-auto mt-20 max-w-xl space-y-
5 text-center">
        {credits.map(([role, who]) => (
          <div key={role}>
            <dt className="font-mono text-[11px]
uppercase tracking-[0.25em] text-dust">{role}</dt>
            <dd className="mt-1 font-display text-
2xl font-bold uppercase tracking-wide text-cream">
{who}</dd>
          </div>
        ))}
      </dl>
      <p className="mt-16 text-center font-mono
text-xs text-dust">
        Built with React, Tailwind and Vite. No
trackers, no cookies. Your list and history stay in
your own browser.
      </p>
    </footer>
  )
}
```

src/components/Gate.jsx — 61 lines

```
/*
 * "Who's here?" — the first screen. Picking a
viewer only changes the order
 * of the rows (see VIEWERS in catalog.js). "Skip"
goes straight in.
 * Returning visitors never see it: an inline script
in index.html hides it
 * before paint if a choice is stored or the URL is
a shared link (html.skip-gate).
 */
import { VIEWERS, DEFAULT_VIEWER } from
'../content/catalog'

const FACE = {
  // Each viewer gets a simple drawn mark instead of
a stock avatar.
  recruiter: <path d="M14 34h36M14 26h36M14 18h22"
strokeWidth="4" />,
  engineer: <path d="M22 18l-10 14 10 14M42 18l10
14-10 14M36 14l-8 36" strokeWidth="4" />,
  curious: <circle cx="32" cy="32" r="13"
strokeWidth="4" />,
}
```

```
export default function Gate({ open, onPick }) {
  return (
    <div
      data-gate
      role="dialog"
      aria-modal="true"
      aria-labelledby="gate-title"
      className={`fixed inset-0 z-50 flex flex-col
items-center justify-center bg-night px-6 ${open ?
'' : 'hidden'}`}
    >
      <h1 id="gate-title" className="font-display
text-5xl font-extrabold uppercase tracking-wide
text-cream sm:text-7xl">
        Who's here?
      </h1>
      <p className="mt-3 max-w-md text-center text-
dust">
        Everything is open to everyone. This only
decides what you see first.
      </p>

      <ul className="mt-10 grid grid-cols-3 gap-4
sm:gap-8">
        {Object.entries(VIEWERS).map(([key, v], i)
=> (
          <li key={key} className="motion-
safe:animate-rise" style={{ animationDelay: `${120 +
i * 110}ms` }}>
            <button
              type="button"
              onClick={() => onPick(key)}
              className="group flex w-24 flex-col
items-center text-center outline-none sm:w-36"
            >
              <span className="grid aspect-square w-
full place-items-center rounded-md bg-soot ring-1
ring-line transition group-hover:ring-2 group-
hover:ring-marigold group-hover:shadow-
[0_0_40px_-6px_rgba(242,162,58,.6)] group-focus-
visible:ring-2 group-focus-visible:ring-marigold
motion-safe:group-hover:-translate-y-1.5 motion-
safe:group-hover:scale-105">
                <svg viewBox="0 0 64 64"
className="w-1/2 fill-none stroke-marigold" aria-
hidden="true">
                  {FACE[key]}
                </svg>
              </span>
              <span className="mt-3 font-display
text-xl font-bold uppercase tracking-wide text-cream
sm:text-2xl">{v.label}</span>
              <span className="mt-1 hidden text-xs
text-dust sm:block">{v.hint}</span>
            </button>
          </li>
        ))}
      </ul>

      <button
        type="button"
        onClick={() => onPick(DEFAULT_VIEWER)}
        className="mt-12 font-mono text-xs uppercase
tracking-[0.2em] text-dust underline-offset-4
hover:text-cream hover:underline"
      >
        Skip — show me everything
      </button>
    </div>
  )
}
```

src/components/NavBar.jsx 95 lines

```
/*
 * Top bar: wordmark, section links, search, résumé,
and who's watching.
 * "/" focuses search from anywhere (unless you're
already typing).
 */
import { useEffect, useRef, useState } from 'react'
import { profile, real } from '../content/profile'
import { VIEWERS } from '../content/catalog'

const LINKS = [
  ['#brief', 'Brief'],
  ['#row-projects', 'Projects'],
  ['#row-credentials', 'Credentials'],
  ['#row-experience', 'Experience'],
  ['#row-stack', 'Stack'],
  ['#credits', 'Contact'],
]

export default function NavBar({ query, onQuery,
viewer, onSwitchViewer }) {
  const input = useRef(null)
  const [scrolled, setScrolled] = useState(false)
  const resume = real(profile.links.resume)

  useEffect(() => {
    const onScroll = () =>
setScrolled(window.scrollY > 40)
    const onKey = (e) => {
      const typing =
/INPUT|TEXTAREA/.test(document.activeElement?.tagNam
e)
      if (e.key === '/' && !typing) {
        e.preventDefault()
        input.current?.focus()
      }
    }
    onScroll()
    window.addEventListener('scroll', onScroll, {
passive: true })
    window.addEventListener('keydown', onKey)
    return () => {
      window.removeEventListener('scroll', onScroll)
      window.removeEventListener('keydown', onKey)
    }
  }, [])

  return (
    <header className={`fixed inset-x-0 top-0 z-30
transition-colors duration-300 ${scrolled ? 'bg-
night/95 shadow-[0_1px_0_#3a3328]' : 'bg-gradient-
to-b from-night/90 to-transparent'}`}>
      <div className="flex items-center gap-4 px-4
py-3 sm:gap-8 sm:px-10">
        <a href="#top" className="shrink-0 font-
display text-2xl font-black uppercase tracking-
[0.12em] text-marigold sm:text-3xl">
          Shevate
          <span className="ml-2 align-middle font-
mono text-[10px] font-normal tracking-[0.2em] text-
dust">on demand</span>
        </a>

        <nav aria-label="Sections" className="hidden
gap-5 text-sm text-dust lg:flex">
          {LINKS.map(([href, label]) => (
            <a key={href} href={href}
className="hover:text-cream">
              {label}
            </a>
          ))}
        </nav>

        <div className="ml-auto flex items-center
gap-2 sm:gap-3">
          <label className="relative">
            <span className="sr-only">Search
titles</span>
            <input
              ref={input}
              type="search"
              value={query}
              onChange={(e) =>
```

```
onQuery(e.target.value)}
              onKeyDown={(e) => e.key === 'Escape'
&& onQuery('')}
              placeholder="Search  /"
              className="w-24 rounded-sm border
border-line bg-soot px-3 py-1.5 text-sm text-cream
placeholder:text-dust focus:w-44 focus:border-
marigold focus:outline-none sm:w-44 sm:focus:w-60
motion-safe:transition-all"
            />
          </label>

          {resume ? (
            <a href={resume} download
className="hidden rounded-sm bg-marigold px-3 py-1.5
text-sm font-semibold text-night hover:bg-cream
sm:block">
              Résumé
            </a>
          ) : (
            <span className="hidden rounded-sm
border border-dashed border-missing/70 px-2 py-1.5
font-mono text-[11px] text-missing sm:block"
title="Add the PDF path in profile.js">
              missing: résumé
            </span>
          )}

          <button
            type="button"
            onClick={onSwitchViewer}
            className="rounded-sm border border-line
px-2.5 py-1.5 text-sm text-dust hover:border-
marigold hover:text-cream"
          >
            <span className="sr-only">Watching as
</span>
            <span className="hidden sm:inline">
{VIEWERS[viewer]?.label ?? 'Who's here?'}</span>
            <span className="sr-only sm:hidden">
{VIEWERS[viewer]?.label ?? 'Who's here?'}</span>
            <span aria-hidden="true" className="ml-1
text-marigold">⇄</span>
          </button>
        </div>
      </div>
    </header>
  )
}
```

src/components/PuneClock.jsx 24 lines

```
/*
 * My local time, for recruiters in other time
zones. The server render
 * shows only the zone (it can't know when you'll
read it); the time
 * appears once the page is running and ticks every
30 seconds.
 */
import { useEffect, useState } from 'react'

const fmt = () =>
  new Intl.DateTimeFormat('en-IN', { timeZone:
'Asia/Kolkata', hour: '2-digit', minute: '2-digit',
hour12: false }).format(new Date())

export default function PuneClock() {
  const [time, setTime] = useState(null)
  useEffect(() => {
    setTime(fmt())
    const id = setInterval(() => setTime(fmt()),
30000)
    return () => clearInterval(id)
  }, [])
  return (
    <span data-clock>
      {time && <span className="font-mono text-
cream">{time} </span>}
      <span className="text-dust">IST ·
UTC+5:30</span>
    </span>
  )
}
```

src/components/Row.jsx 58 lines

```
/*
 * A horizontal row of titles. Native horizontal
scrolling (touch, trackpad,
 * Shift+wheel all work for free); the arrow buttons
just scroll by one
 * screen. The list is a real <ul> under a heading,
so it reads properly
 * without any of the visuals.
 */
import { useRef } from 'react'
import TitleCard from './TitleCard'

export default function Row({ id, name, titles,
onOpen, savedIds = [], note, numbered = false }) {
  const track = useRef(null)
  const page = (dir) => track.current?.scrollBy({
left: dir * track.current.clientWidth * 0.9,
behavior: 'smooth' })

  if (!titles.length) return null
  return (
    <section aria-labelledby={`row-${id}`}
className="group/row relative">
      <div className="mb-2 flex items-baseline gap-3
px-4 sm:px-10">
        <h2 id={`row-${id}`} className="font-display
text-2xl font-bold uppercase tracking-wide text-
cream sm:text-3xl">
          {name}
        </h2>
        {note && <p className="font-mono text-xs
text-dust">{note}</p>}
      </div>

      <ul
        ref={track}
        className="flex snap-x snap-mandatory
scroll-px-4 gap-3 overflow-x-auto scroll-smooth px-4
sm:scroll-px-10 pb-4 pt-2 [scrollbar-width:none]
sm:px-10 [&::-webkit-scrollbar]:hidden"
      >
        {titles.map((t, i) => (
          <li key={t.id} className={`flex shrink-0
snap-start items-end ${numbered ? 'w-[86vw] sm:w-
[52vw] md:w-[38vw] lg:w-[28vw] xl:w-[24vw]' : 'w-
[72vw] sm:w-[42vw] md:w-[30vw] lg:w-[22vw] xl:w-
[19vw]'}`}>
            {numbered && (
              // Outlined numerals, just a running
order, not a ranking.
              <span aria-hidden="true" className="-
mr-3 select-none font-display text-[7rem] font-black
leading-[0.75] text-transparent [-webkit-text-
stroke:2px_#f2a23a] sm:text-[9rem]">
                {i + 1}
              </span>
            )}
            <div className="min-w-0 flex-1">
              <TitleCard title={t} onOpen={onOpen}
saved={savedIds.includes(t.id)} />
            </div>
          </li>
        ))}
      </ul>

      {/* Paging arrows: desktop only, keyboard
users have Tab. */}
      {['left', 'right'].map((side) => (
        <button
          key={side}
          type="button"
```

```
          tabIndex={-1}
          aria-hidden="true"
          onClick={() => page(side === 'left' ? -1 :
1)}
          className={`absolute top-10 bottom-4
hidden w-10 items-center justify-center bg-night/70
text-3xl text-cream opacity-0 transition group-
hover/row:opacity-100 sm:flex ${side === 'left' ?
'left-0' : 'right-0'}`}
        >
          {side === 'left' ? '‹' : '›'}
        </button>
      ))}
    </section>
  )
}
```

src/components/TitleCard.jsx 30 lines

```
/*
 * One card in a row. It's a real <button>, so it
works with Tab/Enter and
 * screen readers; the hover lift is decoration on
top.
 */
import Art from './Art'

const KIND = { project: 'Project', credential:
'Credential', experience: 'Experience', skill:
'Stack', extra: 'Extra' }

export default function TitleCard({ title, onOpen,
saved }) {
  return (
    <button
      type="button"
      onClick={() => onOpen(title.id)}
      className="group relative block w-full
overflow-hidden rounded-md bg-soot text-left
outline-none ring-marigold transition duration-300
focus-visible:ring-2 hover:shadow-
[0_0_0_2px_#f2a23a,0_18px_40px_-12px_rgba(242,162,58
,.45)] motion-safe:hover:-translate-y-2 motion-
safe:hover:scale-[1.06]"
      data-title={title.id}
    >
      <Art title={title} className="block aspect-
video w-full" />
      <span className="absolute inset-x-0 bottom-0
p-3">
        <span className="block font-mono text-[10px]
uppercase tracking-[0.18em] text-dust">
          {KIND[title.kind]}
          {saved && <span className="ml-2 text-
marigold">· on my list</span>}
        </span>
        <span className="block font-display text-xl
font-extrabold uppercase leading-none tracking-wide
text-cream sm:text-2xl">
          {title.name}
        </span>
      </span>
      <span className="pointer-events-none absolute
inset-0 rounded-md ring-1 ring-inset ring-line
group-hover:ring-marigold/60" />
    </button>
  )
}
```

src/components/TitlePage.jsx 191 lines

```
/*
 * The page a title opens into (a modal over the
catalogue): banner, the
 * facts, "episodes" (problem → architecture → stack
→ my part), links,
 * and "More like this". It's a proper dialog: focus
moves in, Tab stays
 * inside, Esc or Back closes it, focus returns to
the card you came from.
 */
import { useEffect, useRef, useState } from 'react'
import { isFill, real } from '../content/profile'
import { similar } from '../content/catalog'
import Art from './Art'
import Value from './Value'
import TitleCard from './TitleCard'

const LABEL = { project: 'Project', credential:
'Credential', experience: 'Experience', skill:
'Stack', about: 'About', extra: 'Extra' }

function useFocusTrap(ref, onClose) {
  useEffect(() => {
    const back = document.activeElement
    const el = ref.current
    el?.focus()
    const onKey = (e) => {
      if (e.key === 'Escape') return onClose()
      if (e.key !== 'Tab' || !el) return
      const f = el.querySelectorAll('a[href],
button:not([disabled]), input, textarea,
[tabindex="0"]')
      if (!f.length) return
      const first = f[0]
      const last = f[f.length - 1]
      if (e.shiftKey && document.activeElement ===
first) (e.preventDefault(), last.focus())
      else if (!e.shiftKey && document.activeElement
=== last) (e.preventDefault(), first.focus())
    }
    document.addEventListener('keydown', onKey)
    return () => {
      document.removeEventListener('keydown', onKey)
      back?.focus?.()
    }
  }, [ref, onClose])
}

function LinkOut({ label, value }) {
  const url = real(value)
  if (url && /^(https?:|\/)/.test(url))
    return (
      <a href={url} target="_blank" rel="noreferrer"
className="rounded-sm bg-cream px-4 py-2 text-sm
font-semibold text-night hover:bg-marigold">
        {label} ↗
      </a>
    )
  return (
    <span className="rounded-sm border border-line
px-4 py-2 text-sm text-dust">
      {label}: <Value v={value} />
    </span>
  )
}

export default function TitlePage({ title, onClose,
onOpen, saved, onToggleSave }) {
  const box = useRef(null)
  const [copy, setCopy] = useState('Copy link') //
button label doubles as feedback
  useFocusTrap(box, onClose)

  // Lock the page behind the dialog.
  useEffect(() => {
    const prev = document.body.style.overflow
    document.body.style.overflow = 'hidden'
    return () => {
      document.body.style.overflow = prev
    }
  }, [])

  const share = async () => {
    try {
      await
navigator.clipboard.writeText(window.location.href)
```

```
      setCopy('Link copied')
    } catch {
      // Clipboard blocked (insecure origin or
permissions): the URL bar has it.
      setCopy('Copy the URL from the address bar')
    }
    setTimeout(() => setCopy('Copy link'), 2500)
  }

  const more = similar(title.id)

  return (
    <div className="fixed inset-0 z-40 overflow-y-
auto bg-night/80 sm:py-10" onMouseDown={(e) =>
e.target === e.currentTarget && onClose()}>
      <article
        ref={box}
        tabIndex={-1}
        role="dialog"
        aria-modal="true"
        aria-labelledby="title-name"
        className="relative mx-auto min-h-full w-
full max-w-4xl bg-soot outline-none ring-1 ring-line
motion-safe:animate-rise sm:min-h-0 sm:rounded-md"
      >
        <button type="button" onClick={onClose}
aria-label="Close" className="absolute right-3 top-3
z-10 grid h-10 w-10 place-items-center rounded-full
bg-night/80 text-xl text-cream ring-1 ring-line
hover:ring-marigold">
          ×
        </button>

        <div className="relative">
          <Art title={title} className="block
aspect-[16/7] w-full sm:rounded-t-md" />
          <div className="absolute bottom-0 left-0
p-5 sm:p-8">
            <p className="font-mono text-xs
uppercase tracking-[0.2em] text-marigold">
{LABEL[title.kind]}</p>
            <h2 id="title-name" className="font-
display text-4xl font-black uppercase leading-none
text-cream sm:text-6xl">
              {title.name}
            </h2>
          </div>
        </div>

        <div className="grid gap-8 p-5 sm:grid-cols-
[1.6fr_1fr] sm:p-8">
          <div>
            {title.tags.length > 0 && (
              <p className="mb-3 flex flex-wrap gap-
2 font-mono text-xs text-dust">
                {title.tags.map((t) => (
                  <span key={t} className="rounded-
sm border border-line px-1.5 py-0.5">{t}</span>
                ))}
              </p>
            )}
            <p className="text-lg leading-relaxed
text-cream">{title.logline}</p>

            <div className="mt-6 flex flex-wrap gap-
2">
              {title.kind !== 'about' && (
                <button type="button" onClick={() =>
onToggleSave(title.id)} aria-pressed={saved}
className="rounded-sm bg-ash px-4 py-2 text-sm text-
cream ring-1 ring-line hover:ring-marigold">
                  {saved ? '✓ On my list' : '+ My
list'}
                </button>
              )}
              <button type="button" onClick={share}
className="rounded-sm bg-ash px-4 py-2 text-sm text-
cream ring-1 ring-line hover:ring-marigold">
                {copy}
              </button>
              {title.links.map((l) => (
                <LinkOut key={l.label} {...l} />
              ))}
            </div>
          </div>

          {title.facts.length > 0 && (
            <dl className="grid content-start gap-3
text-sm">
              {title.facts.map((f) => (
                <div key={f.label}>
                  <dt className="font-mono text-
[11px] uppercase tracking-[0.18em] text-dust">
{f.label}</dt>
                  <dd className={isFill(f.value) ?
'' : 'text-cream'}>
                    <Value v={f.value} />
                  </dd>
                </div>
              ))}
            </dl>
          )}
        </div>

        {title.episodes.length > 0 && (
          <section aria-labelledby="eps"
className="border-t border-line p-5 sm:p-8">
            <h3 id="eps" className="font-display
text-2xl font-bold uppercase tracking-wide text-
cream">Episodes</h3>
            <ol className="mt-4 divide-y divide-
line">
              {title.episodes.map((ep, i) => (
                <li key={ep.name} className="grid
grid-cols-[2.5rem_1fr] gap-3 py-4 sm:grid-cols-
[3.5rem_1fr]">
                  <span className="font-display
text-3xl font-black text-dust sm:text-4xl">{i + 1}
</span>
                  <div>
                    <h4 className="font-semibold
text-cream">{ep.name}</h4>
                    <p className="mt-1 text-dust">
                      <Value v={ep.body} />
                    </p>
                  </div>
                </li>
              ))}
            </ol>
          </section>
        )}

        {title.screenshots.length > 0 && (
          <section aria-label="Screenshots"
className="border-t border-line p-5 sm:p-8">
            <ul className="grid grid-cols-2 gap-3
sm:grid-cols-3">
              {title.screenshots.map((s) => (
                <li key={s.src}>
                  <img src={s.src} alt={s.alt}
loading="lazy" decoding="async" className="aspect-
video w-full rounded-sm object-cover ring-1 ring-
line" />
                </li>
              ))}
            </ul>
          </section>
        )}

        {more.length > 0 && (
          <section aria-labelledby="more"
className="border-t border-line p-5 sm:p-8">
            <h3 id="more" className="font-display
text-2xl font-bold uppercase tracking-wide text-
cream">More like this</h3>
            <ul className="mt-4 grid grid-cols-2
```

```
gap-3 sm:grid-cols-3">
              {more.map((t) => (
                <li key={t.id}>
                  <TitleCard title={t} onOpen=
{onOpen} />
                </li>
              ))}
            </ul>
          </section>
        )}
      </article>
    </div>
  )
}
```

src/components/Trailer.jsx 109 lines

```
/*
 * The trailer: seven title cards, about 2.8 seconds
each, built only from
 * facts in profile.js. Space pauses, ←/→ step, Esc
closes. With reduced
 * motion it doesn't auto-advance; you step through
it yourself.
 */
import { useEffect, useMemo, useRef, useState } from
'react'
import { profile } from '../content/profile'
import { titleById } from '../content/catalog'
import Art from './Art'

const SHOT_MS = 2800

function useShots() {
  return useMemo(() => {
    const [ccp, ai] = profile.credentials
    return [
      { kicker: profile.city, line:
`${profile.education.year}.`, art: 'carepilot' },
      { kicker: 'One engineer', line: 'Cloud.
Backend. Security.', art: 'sensorium' },
      { kicker: ccp.name, line:
`${ccp.result.replace('/', ' out of ')}.`, art:
'aws-ccp' },
      { kicker: ai.name, line:
`${ai.result.split('/')[0]}.`, art: 'ai-900' },
      { kicker: 'DYPIU PhD ERP', line: 'Sole
developer. Now in pilot.', art: 'phd-erp' },
      { kicker: 'Also showing', line: 'CarePilot ·
RentFlow · Sensorium Nexus · BazaarLens', art:
'bazaarlens' },
      { kicker: 'Now casting', line:
`${profile.targeting.join(' · ')}.`, art:
'rentflow', end: true },
    ]
  }, [])
}

export default function Trailer({ onClose, onContact
}) {
  const shots = useShots()
  const [i, setI] = useState(0)
  const reduced = useRef(typeof window !==
'undefined' && window.matchMedia('(prefers-reduced-
motion: reduce)').matches)
  const [paused, setPaused] =
useState(reduced.current)
  const box = useRef(null)

  useEffect(() => {
    if (paused || i === shots.length - 1) return
    const t = setTimeout(() => setI((n) => n + 1),
SHOT_MS)
    return () => clearTimeout(t)
  }, [i, paused, shots.length])

  useEffect(() => {
    const back = document.activeElement
    box.current?.focus()
    const onKey = (e) => {
      if (e.key === 'Escape') onClose()
      else if (e.key === 'ArrowRight') setI((n) =>
Math.min(shots.length - 1, n + 1))
      else if (e.key === 'ArrowLeft') setI((n) =>
Math.max(0, n - 1))
      else if (e.key === ' ') (e.preventDefault(),
setPaused((p) => !p))
    }
    document.addEventListener('keydown', onKey)
    document.body.style.overflow = 'hidden'
    return () => {
      document.removeEventListener('keydown', onKey)
      document.body.style.overflow = ''
      back?.focus?.()
    }
  }, [onClose, shots.length])

  const shot = shots[i]
  return (
    <div ref={box} tabIndex={-1} role="dialog" aria-
modal="true" aria-label="Trailer" className="fixed
inset-0 z-50 overflow-hidden bg-night outline-none">
      <div key={i} aria-hidden="true"
className="absolute inset-0 opacity-50 motion-
safe:animate-[rise_1.2s_ease-out_both]">
        <Art title={titleById[shot.art]}
className="h-full w-full motion-safe:animate-
[zoom_2.8s_linear_both]" />
        <div className="absolute inset-0 bg-
gradient-to-t from-night via-night/60 to-night/30"
/>
      </div>

      {/* Story-style progress bars, one per shot.
*/}
      <ol className="absolute inset-x-4 top-4 flex
gap-1.5 sm:inset-x-10" aria-hidden="true">
        {shots.map((_, n) => (
          <li key={n} className="h-0.5 flex-1
overflow-hidden bg-line">
            <span
              className="block h-full origin-left
bg-marigold"
              style={
                n < i
                  ? { transform: 'scaleX(1)' }
                  : n === i && !paused
                    ? { animation: `shot-progress
${SHOT_MS}ms linear both` }
                    : { transform: n === i && paused
? 'scaleX(0.5)' : 'scaleX(0)' }
              }
            />
          </li>
        ))}
      </ol>

      <div className="relative flex h-full flex-col
items-start justify-center px-6 sm:px-16" aria-
live="polite">
        <p key={`k${i}`} className="font-mono text-
xs uppercase tracking-[0.3em] text-marigold motion-
safe:animate-rise sm:text-sm">{shot.kicker}</p>
        <p key={`l${i}`} className="mt-3 max-w-5xl
font-display text-5xl font-black uppercase leading-
[0.9] text-cream motion-safe:animate-rise sm:text-
8xl">
          {shot.line}
        </p>
        {shot.end && (
          <div className="mt-8 flex flex-wrap gap-3
motion-safe:animate-rise">
            <button type="button" onClick=
{onContact} className="rounded-sm bg-marigold px-6
py-2.5 font-semibold text-night hover:bg-cream">Get
in touch</button>
            <button type="button" onClick={onClose}
```

```
className="rounded-sm bg-ash px-6 py-2.5 text-cream ring-1 ring-line hover:ring-marigold">Browse the catalogue</button>
          </div>
        )}
      </div>

      <div className="absolute inset-x-4 bottom-5 flex items-center justify-between font-mono text-xs text-dust sm:inset-x-10">
        <div className="flex gap-2">
          <button type="button" onClick={() => setI((n) => Math.max(0, n - 1))} className="rounded-sm px-2 py-1 ring-1 ring-line hover:text-cream" aria-label="Previous">←</button>
          <button type="button" onClick={() => setPaused((p) => !p)} className="rounded-sm px-3 py-1 ring-1 ring-line hover:text-cream">{paused ? 'Play' : 'Pause'}</button>
          <button type="button" onClick={() => setI((n) => Math.min(shots.length - 1, n + 1))} className="rounded-sm px-2 py-1 ring-1 ring-line hover:text-cream" aria-label="Next">→</button>
        </div>
        <span>{i + 1} / {shots.length}</span>
        <button type="button" onClick={onClose} className="rounded-sm px-3 py-1 ring-1 ring-line hover:text-cream">Skip trailer ×</button>
      </div>
    </div>
  )
}
```

src/components/Value.jsx — 22 lines

```
/*
 * Renders anything from profile.js. A placeholder renders as a visible,
 * dashed "[missing: ...]" tag so nobody can mistake it for a real claim.
 */
import { isFill } from '../content/profile'

export default function Value({ v }) {
  if (isFill(v))
    return (
      <span className="rounded-sm border border-dashed border-missing/70 px-1.5 font-mono text-[0.8em] text-missing [overflow-wrap:anywhere] [box-decoration-break:clone]">
        missing: {v.fill}
      </span>
    )
  if (Array.isArray(v))
    return v.map((x, i) => (
      <span key={i}>
        {i > 0 && ', '}
        <Value v={x} />
      </span>
    ))
  return v ?? null
}
```

src/content/brief.js — 65 lines

```
/*
 * brief.js — the recruiter's view of profile.js:
 *   fit       one line per target role, with only the facts that back it
 *   summary   plain text for pasting into an applicant-tracking system
 *
 * It maps existing facts onto roles; it never adds new ones. A role with no
 * evidence yet says so instead of being padded.
 */
import { profile, real, fill } from './profile.js'

const cred =
Object.fromEntries(profile.credentials.map((c) => [c.id, c]))
const job =
Object.fromEntries(profile.experience.map((x) => [x.id, x]))
const erp = profile.projects.find((p) => p.id === 'phd-erp')

export const fit = [
  {
    role: 'Cloud',
    proof: [
      `${cred['aws-ccp'].name}: ${cred['aws-ccp'].result}`,
      `${cred['ai-900'].name}: ${cred['ai-900'].result}`,
      `${cred['aws-restart'].name} (${cred['aws-restart'].date})`,
    ],
  },
  {
    role: 'Backend',
    proof: [`${erp.role}, ${erp.name} (${erp.status})`, `${job.technocia.title}, ${job.technocia.org}`],
  },
  {
    role: 'Security',
    proof: [profile.education.degree, `${job.firstvidya.title}, ${job.firstvidya.org}`],
  },
  {
    role: 'Junior DevOps',
    proof: [cred['aws-restart'].name, fill('DevOps work (CI/CD, Docker, Linux)')],
  },
]

/** Plain-text profile. Missing fields are skipped, never printed as placeholders. */
export function summary() {
  const line = (label, v) => (real(v) ? `${label}: ${v}` : null)
  return [
    `${profile.name}`,
    `${profile.role} · ${profile.location}`,
    `Looking for: ${profile.targeting.join(', ')} roles${profile.freelance ? '; open to freelance' : ''}`,
    line('Available', profile.availability),
    line('Work mode', profile.workMode),
    line('Work authorization', profile.workAuthorization),
    '',
    `Education: ${profile.education.degree}, ${profile.education.school}, ${profile.education.year}`,
    'Credentials:',
    ...profile.credentials.map((c) => `- ${c.name}: ${c.result} (${c.date})`),
    'Experience:',
    ...profile.experience.map((x) => `- ${x.title}, ${x.org}, ${x.when}`),
    'Projects:',
    ...profile.projects.map((p) => `- ${p.name}: ${p.logline}${real(p.role) ? ` Role: ${p.role}.` : ''}${real(p.status) ? ` Status: ${p.status}.` : ''}`),
    `Languages: ${profile.spoken.join(', ')}`,
    '',
    line('Email', profile.links.email),
    line('GitHub', profile.links.github),
    line('LinkedIn', profile.links.linkedin),
  ]
    .filter((l) => l !== null)
    .join('\n')
    .trim()
}
```

src/content/catalog.js 150 lines

```
/*
 * catalog.js — turns profile.js into "titles" and "rows", the two shapes the
 * whole UI is built from. A title is anything you can open: a project, a
 * credential, a job, a skill group. Nothing new is invented here; every
 * string comes from profile.js.
 */
import { profile, real } from './profile.js'
import { makingOf } from './making-of.js'

const projectTitle = (p) => ({
  id: p.id,
  kind: 'project',
  name: p.name,
  tags: p.genre,
  logline: p.logline,
  facts: [
    { label: 'Status', value: p.status },
    { label: 'Role', value: p.role },
  ],
  episodes: [
    { name: 'The problem', body: p.problem },
    { name: 'The architecture', body: p.architecture },
    { name: 'The stack', body: p.stack },
    { name: 'My part in it', body: p.role },
  ],
  links: [
    { label: 'Repository', value: p.repo },
    { label: 'Live demo', value: p.demo },
  ],
  screenshots: p.screenshots,
  searchText: [p.name, p.logline, ...p.genre, real(p.stack), real(p.role)],
})

const credentialTitle = (c) => ({
  id: c.id,
  kind: 'credential',
  name: c.name,
  tags: [c.issuer, c.date],
  logline: [c.result, c.note].filter(Boolean).join(' · '),
  result: c.result,
  facts: [
    { label: 'Result', value: c.result },
    { label: 'Issued', value: c.date },
    ...(c.note ? [{ label: 'Validity', value: c.note }] : []),
  ],
  episodes: [],
  links: c.verifyUrl !== undefined ? [{ label: 'Verify', value: c.verifyUrl }] : [],
  screenshots: [],
  searchText: [c.name, c.issuer, c.result],
})

const experienceTitle = (x) => ({
  id: x.id,
  kind: 'experience',
  name: x.title,
  tags: [x.org, x.when],
  logline: `${x.org} · ${x.when}`,
  facts: [
    { label: 'Where', value: x.org },
    { label: 'When', value: x.when },
  ],
  episodes: [{ name: 'What I did', body: x.detail }],
  links: [],
  screenshots: [],
  searchText: [x.title, x.org],
})

const skillTitle = (s) => ({
  id: `skills-${s.group.toLowerCase().replace(/[^a-z]+/g, '-')}`,
  kind: 'skill',
  name: s.group,
  tags: [],
  logline: s.items.filter((i) => typeof i === 'string').join(', '),
  items: s.items,
  facts: [],
  episodes: [{ name: 'In this group', body: s.items }],
  links: [],
  screenshots: [],
  searchText: [s.group, ...s.items.filter((i) => typeof i === 'string')],
})

const about = {
  id: 'about',
  kind: 'about',
  name: profile.displayName,
  tags: [profile.city, profile.education.year],
  logline: profile.tagline,
  facts: [
    { label: 'Now', value: profile.role },
    { label: 'Studying', value: `${profile.education.degree}, ${profile.education.school}, ${profile.education.year}` },
    { label: 'Based in', value: profile.location },
    { label: 'Looking for', value: `${profile.targeting.join(', ')} roles${profile.freelance ? '; open to freelance' : ''}` },
    { label: 'Speaks', value: profile.spoken.join(', ') },
    { label: 'Learning', value: profile.learning.join(', ') },
  ],
  episodes: [],
  links: [
    { label: 'GitHub', value: profile.links.github },
    { label: 'LinkedIn', value: profile.links.linkedin },
  ],
  screenshots: [],
  searchText: [profile.name, profile.city, ...profile.spoken, ...profile.learning],
}

export const titles = [
  ...profile.projects.map(projectTitle),
  ...profile.credentials.map(credentialTitle),
  ...profile.experience.map(experienceTitle),
  ...profile.skills.map(skillTitle),
  makingOf,
]

// `about` can be opened (billboard "More info") and searched, but isn't in a row.
export const titleById = Object.fromEntries([...titles, about].map((t) => [t.id, t]))

export const ROWS = {
  projects: { name: 'Projects', ids: titles.filter((t) => t.kind === 'project').map((t) => t.id) },
  credentials: { name: 'Credentials', ids: titles.filter((t) => t.kind === 'credential').map((t) => t.id) },
  experience: { name: 'Experience', ids: titles.filter((t) => t.kind === 'experience').map((t) => t.id) },
  stack: { name: 'The stack', ids: titles.filter((t) => t.kind === 'skill').map((t) => t.id) },
  extras: { name: 'Extras', ids: [makingOf.id] },
```

```
}

/*
 * Who's watching decides the order of the rows,
nothing else: every viewer
 * can reach everything. A recruiter sees proof
first, an engineer sees the
 * work first.
 */
export const VIEWERS = {
  recruiter: { label: 'Recruiter', hint:
'Credentials and experience first', rows:
['credentials', 'experience', 'projects', 'stack',
'extras'] },
  engineer: { label: 'Engineer', hint: 'The projects
and the stack first', rows: ['projects', 'extras',
'stack', 'credentials', 'experience'] },
  curious: { label: 'Just looking', hint: 'The
highlights', rows: ['projects', 'credentials',
'experience', 'stack', 'extras'] },
}
export const DEFAULT_VIEWER = 'recruiter'

/** Case-insensitive search over the words each
title is made of. */
export function search(query) {
  const words =
query.toLowerCase().split(/\s+/).filter(Boolean)
  if (!words.length) return []
  return [...titles, about].filter((t) => {
    const hay = t.searchText.filter(Boolean).join('
').toLowerCase()
    return words.every((w) => hay.includes(w))
  })
}

/** Titles of the same kind, for "More like this".
*/
export const similar = (id) => {
  const t = titleById[id]
  return t ? titles.filter((x) => x.kind === t.kind
&& x.id !== id) : []
}
```

src/content/making-of.js 39 lines

```
/*
 * The "Making of" extra: this site as its own case
study. Everything here
 * is checkable in the public repository; the
numbers are from the test runs
 * described in the README (re-run them with `npm
run check`).
 */
export const REPO =
'https://github.com/Jayshevate/portfoilo'

export const makingOf = {
  id: 'this-site',
  kind: 'extra',
  name: 'Making of this site',
  tags: ['React', 'Tailwind', 'Testing'],
  logline: 'The portfolio you are looking at, as a
case study: built, tested and documented like
production work.',
  facts: [
    { label: 'Role', value: 'Designed, built and
tested it' },
    { label: 'Backend', value: 'None: static files,
works on any host' },
  ],
  episodes: [
    {
      name: 'The problem',
      body: 'A recruiter gives a portfolio under a
minute. It has to answer "can this person do the
job?" fast, and prove it, without inventing
anything.',
    },
    {
      name: 'The architecture',
      body: 'Every fact lives in one data file. The
app is rendered to static HTML at build time and
hydrated by React, so it loads instantly and still
reads with JavaScript off. Each visitor’s choices
stay in their own browser: no server, no shared
state.',
    },
    {
      name: 'The stack',
      body: 'React 18, Vite 5, Tailwind CSS 3.
Tests: Vitest, Playwright, axe-core, autocannon.',
    },
    {
      name: 'The quality bar',
      body: 'Unit tests pin every score and date to
the resume. Browser tests cover every flow, an
automated WCAG 2.1 AA accessibility scan, 25
visitors at once, and 200 parallel connections with
zero errors. Lighthouse: 99–100 across the board.',
    },
  ],
  links: [{ label: 'Source code', value: REPO }],
  screenshots: [],
  searchText: ['making of', 'this site', 'react',
'tailwind', 'vite', 'testing', 'playwright',
'accessibility'],
}
```

src/content/profile.js 202 lines

```
/*
 * profile.js — every fact on the site lives here,
and nowhere else.
 *
 * Rule: only what is on my resume. If I don't have
something yet, it is
 * `fill('what is missing')`, which the site shows
as a clearly marked
 * placeholder instead of a guess. `npm run todo`
lists all of them.
 *
 * Nothing in this file touches the browser, so it
also runs in Node
 * (the pre-render step and the tests import it).
 */

export const fill = (what) => ({ fill: what })

export const profile = {
  name: 'Jaykumar Chandrakant Shevate',
  displayName: 'Jaykumar Shevate',
  location: 'Pune, Maharashtra, India',
  city: 'Pune',
  role: 'Teaching Associate, D. Y. Patil
International University',
  targeting: ['cloud', 'backend', 'junior DevOps',
'security'],
  freelance: true,

  // What recruiters screen for first. Fill these
in; until then the brief
  // shows them as missing (and the copied summary
leaves them out).
  availability: fill('start date'), // e.g. 'July
2026, after graduation'
  workMode: fill('on-site / hybrid / remote'), //
and whether I'd relocate
  workAuthorization: fill('work authorization'), //
e.g. 'Indian citizen'
  spoken: ['English', 'Hindi', 'Marathi'],
  learning: ['German', 'Japanese', 'Russian'],

  // DRAFT, written only from resume facts. Rewrite
in my own words.
  tagline:
    'I work on the parts of a system you only notice
when they fail: the cloud underneath, the backend in
```

```
the middle, and the security around both.',

  links: {
    github: fill('GitHub profile URL'),
    linkedin: fill('LinkedIn URL'),
    email: fill('email address'),
    // Put the PDF in /public and write e.g.
'/Jaykumar-Shevate-Resume.pdf'
    resume: fill('resume PDF path'),
  },

  education: {
    degree: 'B.Tech, Computer Science & Engineering
(Cyber Security)',
    school: 'D. Y. Patil International University',
    year: '2026',
  },

  credentials: [
    {
      id: 'aws-ccp',
      name: 'AWS Certified Cloud Practitioner',
      issuer: 'Amazon Web Services',
      result: '1000/1000',
      date: 'Mar 2026',
      note: 'Valid through Mar 2029',
      verifyUrl: fill('Credly link'),
    },
    {
      id: 'ai-900',
      name: 'Microsoft Azure AI Fundamentals (AI-
900)',
      issuer: 'Microsoft',
      result: '929/1000',
      date: 'Mar 2026',
      verifyUrl: fill('verification link'),
    },
    {
      id: 'gate-2026',
      name: 'GATE 2026, Data Science & AI',
      issuer: 'GATE',
      result: 'Qualified',
      date: '2026',
    },
    {
      id: 'aws-restart',
      name: 'AWS re/Start Graduate Program',
      issuer: 'Amazon Web Services',
      result: 'Completed',
      date: 'Sep 2025',
    },
  ],

  experience: [
    {
      id: 'dypiu-ta',
      title: 'Teaching Associate',
      org: 'D. Y. Patil International University',
      when: 'Aug 2025 – present',
      detail: fill('what I teach / what I
achieved'),
    },
    {
      id: 'technocia',
      title: 'Software Developer Intern',
      org: 'Technocia Infotech',
      when: 'May – Aug 2025',
      detail: fill('what I built, tech used,
outcome'),
    },
    {
      id: 'firstvidya',
      title: 'Cybersecurity Research Intern',
      org: 'FirstVidya Pvt. Ltd. (remote)',
      when: 'Feb – Apr 2025',
      detail: fill('what I researched, tools,
outcome'),
    },
  ],

  // In the order they appear in the Projects row.
  projects: [
    {
      id: 'phd-erp',
      name: 'DYPIU PhD ERP',
      genre: ['Education', 'ERP'],
      logline: 'Doctoral programme management system
for D Y Patil International University.',
      status: 'UAT / pilot stage',
      role: 'Sole developer',
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link, or "private"'),
      demo: fill('demo link'),
      screenshots: [], // optional: [{ src:
'/shots/x.webp', alt: '...' }]
    },
    {
      id: 'carepilot',
      name: 'CarePilot',
      genre: ['Healthcare', 'SaaS'],
      logline: 'Healthcare SaaS / national health
network.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      screenshots: [],
    },
    {
      id: 'rentflow',
      name: 'RentFlow',
      genre: ['Property', 'Platform'],
      logline: 'Rental property management
platform.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      screenshots: [],
    },
    {
      id: 'sensorium',
      name: 'Sensorium Nexus',
      genre: ['IoT', 'Monitoring'],
      logline: 'IoT monitoring platform.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      screenshots: [],
    },
    {
      id: 'bazaarlens',
      name: 'BazaarLens',
      genre: ['AI', 'Shopping'],
      logline: 'AI shopping price comparison for
India.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      screenshots: [],
```

```
    },
  ],

  skills: [
    { group: 'Cloud', items: ['AWS', 'Azure AI',
fill('exact services used')] },
    { group: 'Backend', items: [fill('languages and
frameworks')] },
    { group: 'Security', items: [fill('tools and
areas')] },
    { group: 'Data & DevOps', items:
[fill('databases, tools, CI/CD')] },
    { group: 'Learning now', items: ['Python',
'German', 'Japanese', 'Russian'] },
  ],

  // Optional: other repos worth a look. [{ name,
url, line }]. Hidden while empty.
  otherRepos: [],

  // DRAFT call to action. Replace with my own line.
  cta: 'Hiring for cloud, backend, junior DevOps or
security? Or need something built freelance? Get in
touch.',
}

/* ---------- helpers ---------- */

// Arrays have a .fill() method, so check for a
plain object first.
export const isFill = (v) => !!v && typeof v ===
'object' && !Array.isArray(v) && 'fill' in v

/** A real string value, or null while it's still a
placeholder. */
export const real = (v) => (typeof v === 'string' &&
v.trim() ? v : null)
```

src/entry-server.jsx 5 lines

```
// Used only at build time by scripts/prerender.mjs
to turn <App/> into HTML.
import { renderToString } from 'react-dom/server'
import App from './App'

export const render = () => renderToString(<App />)
```

src/index.css 47 lines

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html { background: theme('colors.night'); color:
theme('colors.cream'); scroll-behavior: smooth;
scroll-padding-top: 5rem; }
  body { @apply font-sans antialiased; -webkit-tap-
highlight-color: transparent; }
  ::selection { background:
theme('colors.marigold'); color:
theme('colors.night'); }
  :focus-visible { outline: 2px solid
theme('colors.marigold'); outline-offset: 2px; }
}

/* Returning visitors and shared links: an inline
script in index.html adds
   .skip-gate before paint, so the "Who's here?"
screen never flashes up.
   Once React is running (.hydrated) it decides on
its own. */
html.skip-gate:not(.hydrated) [data-gate] { display:
none; }

/* ---------- living posters (all off with reduced
motion, see below) ---------- */
.art-flow { animation: art-flow 1.1s linear
infinite; }
.art-wave { animation: art-wave 4s linear infinite;
}
.art-flicker { animation: art-flicker 4.5s steps(1)
infinite; }
.art-race { animation: art-race 2.2s cubic-
bezier(.2,.8,.2,1) infinite; }
.art-hop { animation: art-hop 1.4s cubic-
bezier(.3,1.6,.5,1) infinite; transform-box: fill-
box; transform-origin: center; }
@keyframes art-flow { to { stroke-dashoffset: -18; }
}
@keyframes art-wave { to { transform:
translateX(-36px); } }
@keyframes art-flicker { 0%, 100% { opacity: 1; }
47% { opacity: .25; } 53% { opacity: 1; } 71% {
opacity: .5; } }
@keyframes art-race { 0% { transform:
translateX(-220px); } 60%, 100% { transform: none; }
}
@keyframes art-hop { 0%, 100% { transform:
translateY(0); } 50% { transform: translateY(-10px);
} }

/* Name slams in letter by letter, then the marigold
rule sweeps under it. */
.slam { display: inline-block; animation: slam .55s
cubic-bezier(.2,1.4,.4,1) both; }
@keyframes slam { from { opacity: 0; transform:
translateY(.45em) scale(1.25) skewX(-8deg); } }
.sweep { transform-origin: left; animation: sweep
.9s cubic-bezier(.16,1,.3,1) .7s both; }
@keyframes sweep { from { transform: scaleX(0); } }

/* Highlights ticker. Two copies of the content, so
the loop has no seam. */
.ticker { animation: ticker 28s linear infinite; }
.ticker:hover { animation-play-state: paused; }
@keyframes ticker { to { transform:
translateX(-50%); } }

/* Trailer progress bar (used from an inline style,
so it lives here rather
   than in tailwind.config.js, which only emits
keyframes for used classes). */
@keyframes shot-progress { from { transform:
scaleX(0); } to { transform: scaleX(1); } }

@media (prefers-reduced-motion: reduce) {
  html { scroll-behavior: auto; }
  .art-flow, .art-wave, .art-flicker, .art-race,
.art-hop, .slam, .sweep, .ticker { animation: none
!important; }
}
```

src/lib/hash.js 30 lines

```
/*
 * The only "routing" the site needs: #title=<id>
opens a title, so a
 * recruiter can paste a link to one project.
Everything else is one page.
 */
import { useEffect, useState } from 'react'

export const parseHash = (hash) => new
URLSearchParams(hash.replace(/^#/, '')).get('title')

export function setTitleHash(id) {
  const url = id ?
`#title=${encodeURIComponent(id)}` :
window.location.pathname + window.location.search
  // replaceState for close, pushState for open, so
Back closes the title.
  if (id) window.history.pushState(null, '', url)
  else window.history.replaceState(null, '', url)
  window.dispatchEvent(new
HashChangeEvent('hashchange'))
}
```

```
export function useTitleHash() {
  const [id, setId] = useState(null) // null on the
server and first client render
  useEffect(() => {
    const sync = () =>
setId(parseHash(window.location.hash))
    sync()
    window.addEventListener('hashchange', sync)
    window.addEventListener('popstate', sync)
    return () => {
      window.removeEventListener('hashchange', sync)
      window.removeEventListener('popstate', sync)
    }
  }, [])
  return id
}
```

src/lib/storage.js 24 lines

```
/*
 * localStorage that never throws. Private windows,
blocked cookies and
 * full storage all make the real API throw; the
site must still work, it
 * just won't remember anything. Only per-visitor
conveniences live here:
 * who's watching, My List, recently opened.
 */
const KEY = 'jcs:'

export function read(name, fallback) {
  try {
    const raw = window.localStorage.getItem(KEY +
name)
    return raw === null ? fallback : JSON.parse(raw)
  } catch {
    return fallback
  }
}

export function write(name, value) {
  try {
    window.localStorage.setItem(KEY + name,
JSON.stringify(value))
  } catch {
    /* storage unavailable: forget silently */
  }
}
```

src/lib/tailor.js 34 lines

```
/*
 * Tailored links: ?for=<company>&role=<role>
 *
 * When I apply somewhere I send a link like
 *   https://…/?for=Infosys&role=cloud
 * and the site greets that company and leads with
that role. The values are
 * shown as plain text (React escapes them),
trimmed, stripped of anything
 * that looks like markup, and capped at 40
characters.
 */
import { useEffect, useState } from 'react'

const ROLES = ['cloud', 'backend', 'security',
'devops']

const clean = (s) =>
  (s || '')
    .replace(/[<>{}\u0000-\u001f]/g, '')
    .replace(/\s+/g, ' ')
    .trim()
    .slice(0, 40)

export function parseTailor(search) {
  const q = new URLSearchParams(search)
  const company = clean(q.get('for')) || null
  const r = clean(q.get('role')).toLowerCase()
  const role = ROLES.find((x) => r && (r === x ||
r.includes(x))) || null
  return company || role ? { company, role } : null
}

/** null on the server and first render (keeps
hydration exact), then the real value. */
export function useTailor() {
  const [t, setT] = useState(null)
  useEffect(() =>
setT(parseTailor(window.location.search)), [])
  return t
}
```

src/main.jsx 15 lines

```
import { createRoot, hydrateRoot } from 'react-
dom/client'
import '@fontsource/big-shoulders-display/latin-
700.css'
import '@fontsource/big-shoulders-display/latin-
800.css'
import '@fontsource/big-shoulders-display/latin-
900.css'
import '@fontsource/ibm-plex-sans/latin-400.css'
import '@fontsource/ibm-plex-sans/latin-600.css'
import '@fontsource/ibm-plex-mono/latin-400.css'
import './index.css'
import App from './App'

// The production build ships pre-rendered HTML
(scripts/prerender.mjs), so
// React attaches to it. In dev the root is empty
and React renders fresh.
const root = document.getElementById('root')
if (root.hasChildNodes()) hydrateRoot(root, <App />)
else createRoot(root).render(<App />)
```

tests/e2e/run.mjs 377 lines

```
/*
 * End-to-end tests in a real browser, against the
production build.
 *
 *   npm run build && npm run test:e2e
 *
 * Starts `vite preview` if nothing is listening on
:4173, runs every
 * scenario below, prints a pass/fail line each,
exits non-zero on failure.
 * Uses Playwright + axe-core (accessibility rules,
WCAG 2.1 AA).
 * If Playwright can't find a browser: npx
playwright install chromium
 */
import { spawn } from 'node:child_process'
import { chromium, devices } from 'playwright'
import AxeBuilder from '@axe-core/playwright'

const BASE = process.env.BASE_URL ||
'http://localhost:4173/'
const results = []
let server

async function up() {
  try {
    return (await fetch(BASE)).ok
  } catch {
    return false
  }
}

async function ensureServer() {
  if (await up()) return
  server = spawn('npx', ['vite', 'preview', '--
port', '4173', '--strictPort'], { stdio: 'ignore' })
  for (let i = 0; i < 50 && !(await up()); i++)
await new Promise((r) => setTimeout(r, 200))
  if (!(await up())) throw new Error('preview server
```

```
did not start (did you run npm run build?)')
}

function assert(cond, msg) {
  if (!cond) throw new Error(msg)
}

/** A fresh visitor: new context = empty storage,
like a different person. */
async function visitor(browser, opts = {}) {
  const ctx = await browser.newContext({ viewport: {
width: 1366, height: 860 }, ...opts })
  const page = await ctx.newPage()
  const errors = []
  page.on('pageerror', (e) =>
errors.push(e.message))
  page.on('console', (m) => m.type() === 'error' &&
errors.push(m.text()))
  page.errors = errors
  return { ctx, page }
}

const rowOrder = (page) =>
page.$$eval('section[aria-labelledby^="row-"] h2',
(hs) => hs.map((h) => h.textContent.trim()))
const visibleText = (page) => page.evaluate(() =>
document.body.innerText)

async function axe(page, label) {
  await page.waitForTimeout(700) // let entrance
animations finish; axe measures what's on screen
  const r = await new AxeBuilder({ page
}).withTags(['wcag2a', 'wcag2aa', 'wcag21a',
'wcag21aa']).analyze()
  assert(!r.violations.length, `${label}:
${r.violations.map((v) => `${v.id}
(${v.nodes.length})`).join(', ')}`)
}

/* ---------------- scenarios ---------------- */

const tests = {
  async 'new visitor sees "Who\'s here?", recruiter
gets credentials first, choice is remembered'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('dialog', { name: "Who's
here?" }).waitFor()
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    assert((await rowOrder(page))[0] ===
'Credentials', `order was ${await rowOrder(page)}`)
    await page.reload()
    assert(!(await page.locator('[data-
gate]').isVisible()), 'gate shown again after
reload')
    assert((await rowOrder(page))[0] ===
'Credentials', 'order not remembered')
    assert(!page.errors.length,
page.errors.join('\n'))
    await ctx.close()
  },

  async 'recruiter brief: shown first, role fit +
logistics, copies a clean summary'(b) {
    const { ctx, page } = await visitor(b)
    await ctx.grantPermissions(['clipboard-read',
'clipboard-write'])
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    const brief = page.locator('#brief')
    await brief.waitFor()
    const briefTop = await brief.evaluate((e) =>
e.getBoundingClientRect().top + scrollY)
    const rowTop = await page.locator('#row-
credentials').evaluate((e) =>
e.getBoundingClientRect().top + scrollY)
    assert(briefTop < rowTop, 'brief should come
before the rows for recruiters')
    const text = await brief.innerText()
    for (const s of ['CLOUD', 'BACKEND', 'SECURITY',
'JUNIOR DEVOPS', '1000/1000',
'Logistics'.toUpperCase()])
      assert(text.toUpperCase().includes(s), `brief
is missing ${s}`)
    await page.getByRole('button', { name: /Copy
summary/ }).click()
    await page.getByRole('button', { name: /Copied/
}).waitFor()
    const clip = await page.evaluate(() =>
navigator.clipboard.readText())
    assert(clip.startsWith('Jaykumar Chandrakant
Shevate') && clip.includes('1000/1000') &&
!clip.includes('missing'), `bad summary:
${clip.slice(0, 120)}`)
    await ctx.close()
  },

  async 'tailored link: greets the company, leads
with their role, no picker, markup stays text'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE + '?
for=Infosys&role=security')
    await page.locator('[data-tailored]', { hasText:
'Prepared for Infosys' }).waitFor()
    assert(!(await page.locator('[data-
gate]').isVisible()), 'picker shown on a tailored
link')
    assert((await page.title()).includes('Infosys'),
'tab title not tailored')
    const first = await page.$eval('#brief [data-
role]', (e) => e.dataset.role)
    assert(first === 'Security', `first role card is
${first}`)
    await page.locator('[data-
clock]').first().getByText(/\d\d:\d\d/).waitFor()

    let fired = false
    page.on('dialog', async (d) => ((fired = true),
d.dismiss()))
    await page.goto(BASE + '?for=' +
encodeURIComponent('<img src=x
onerror=alert(1)>Evil'))
    await page.locator('[data-tailored]').waitFor()
    assert(!(await page.locator('[data-tailored]
img').count()) && !fired, 'markup in ?for= was
rendered')
    assert(!page.errors.length,
page.errors.join('\n'))
    await ctx.close()
  },

  async 'engineer gets projects first; switching
viewer works'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Engineer/ }).click()
    assert((await rowOrder(page))[0] === 'Projects',
`order was ${await rowOrder(page)}`)
    await page.getByRole('button', { name: /Watching
as/ }).click()
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    assert((await rowOrder(page))[0] ===
'Credentials', 'switch did not reorder')
    await ctx.close()
  },

  async 'every title opens, shows its name, closes
with Esc and returns focus'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name: /Skip —
show me everything/ }).click()
```

```
    const ids = await page.$$eval('#rows [data-
title]', (els) => [...new Set(els.map((e) =>
e.dataset.title))])
    assert(ids.length === 18, `expected 18 titles,
found ${ids.length}`)
    for (const id of ids) {
      const card = page.locator(`#rows [data-
title="${id}"]`).first()
      await card.scrollIntoViewIfNeeded()
      await card.click()
      const dlg = page.getByRole('dialog').filter({
has: page.locator('#title-name') })
      await dlg.waitFor()
      assert(page.url().endsWith(`#title=${id}`),
`hash not set for ${id}`)
      const text = await dlg.innerText()
      for (const bad of ['undefined', 'NaN',
'[object Object]']) assert(!text.includes(bad),
`${id} shows "${bad}"`)
      await page.keyboard.press('Escape')
      await dlg.waitFor({ state: 'detached' })
      const focused = await page.evaluate(() =>
document.activeElement?.dataset?.title)
      assert(focused === id, `focus went to
${focused}, not ${id}`)
    }
    assert(!page.errors.length,
page.errors.join('\n'))
    await ctx.close()
  },

  async 'deep link opens a title; Back closes it'(b)
{
    const { ctx, page } = await visitor(b)
    await page.goto(BASE + '#title=phd-erp')
    await page.locator('#title-name', { hasText:
'DYPIU PhD ERP' }).waitFor()
    assert(!(await page.locator('[data-
gate]').isVisible()), 'gate covers a shared link')
    assert((await visibleText(page)).includes('Sole
developer'), 'ERP role missing')
    await page.keyboard.press('Escape')
    await page.locator('#title-name').waitFor({
state: 'detached' })
    // open from a card, then the browser Back
button closes it
    await page.locator('#rows [data-
title="carepilot"]').first().click()
    await page.locator('#title-name', { hasText:
'CarePilot' }).waitFor()
    await page.goBack()
    await page.locator('#title-name').waitFor({
state: 'detached' })
    // a bogus id must not crash or show a broken
dialog
    await page.goto(BASE + '#title=does-not-exist')
    await page.waitForTimeout(300)
    assert(!(await page.locator('#title-
name').count()), 'dialog opened for unknown id')
    assert(!page.errors.length,
page.errors.join('\n'))
    await ctx.close()
  },

  async 'search finds by word, says when nothing
matches, "/" focuses it'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await page.keyboard.press('/')
    assert(await page.evaluate(() =>
document.activeElement?.type === 'search'), '"/" did
not focus search')
    await page.keyboard.type('aws')
    await page.getByRole('heading', { name: /Results
for “aws”/ }).waitFor()
    assert((await page.locator('#rows [data-
title]').count()) === 3, 'expected 3 results for
aws')
    await page.fill('input[type=search]',
'kubernetes')
    await page.getByRole('heading', { name: /Nothing
matches/ }).waitFor()
    await page.keyboard.press('Escape')
    await page.getByRole('heading', { name:
'Credentials' }).waitFor()
    await ctx.close()
  },

  async 'My list: add, see the row, survives reload,
remove'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Engineer/ }).click()
    await page.goto(BASE + '#title=sensorium')
    await page.getByRole('button', { name: '+ My
list' }).click()
    await page.keyboard.press('Escape')
    await page.getByRole('heading', { name: 'My
list' }).waitFor()
    await page.reload()
    await page.getByRole('heading', { name: 'My
list' }).waitFor()
    await page.goto(BASE + '#title=sensorium')
    await page.getByRole('button', { name: '✓ On my
list' }).click()
    await page.keyboard.press('Escape')
    await page.waitForTimeout(200)
    assert(!(await page.getByRole('heading', { name:
'My list' }).count()), 'My list row still there
after removing')
    assert((await page.getByRole('heading', { name:
'Continue watching' }).count()) === 1, 'Continue
watching row missing')
    await ctx.close()
  },

  async 'trailer plays, advances, pauses, closes'(b)
{
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await page.getByRole('button', { name: /Play
trailer/ }).click()
    const dlg = page.getByRole('dialog', { name:
'Trailer' })
    await dlg.waitFor()
    await page.getByText('1 / 7').waitFor()
    await page.getByText('2 / 7').waitFor({ timeout:
6000 })
    await page.keyboard.press(' ')
    await page.waitForTimeout(4000)
    assert(await page.getByText('2 /
7').isVisible(), 'trailer kept going while paused')
    await page.keyboard.press('ArrowRight')
    await page.getByText('3 / 7').waitFor()
    assert((await dlg.innerText()).includes('1000
OUT OF 1000') || (await
dlg.innerText()).toLowerCase().includes('1000 out of
1000'), 'shot 3 should state the AWS score')
    await page.keyboard.press('Escape')
    await dlg.waitFor({ state: 'detached' })
    await ctx.close()
  },

  async 'contact form: required fields enforced;
without a key it says so, never fakes success'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await
page.locator('#credits').scrollIntoViewIfNeeded()
```

```
    await page.getByRole('button', { name: 'Send'
}).click()
    assert(!(await
page.getByText(/Sent\./).count()), 'empty form
claimed success')
    await page.getByLabel('Name').fill('A
Recruiter')
    await
page.getByLabel('Email').fill('recruiter@example.com
')
    await page.getByLabel('Message').fill('Hello')
    await page.getByRole('button', { name: 'Send'
}).click()
    await page.getByText(/isn't connected
yet|Sent\.|didn't go through/).waitFor()
    const status = await
page.getByRole('status').innerText()
    if (!process.env.VITE_WEB3FORMS_KEY)
assert(status.includes("isn't connected yet"),
`unexpected status: ${status}`)
    await ctx.close()
  },

  async 'keyboard: first Tab is the skip link;
dialogs trap focus'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await page.keyboard.press('Tab')
    assert((await page.evaluate(() =>
document.activeElement.textContent)).includes('Skip
to the catalogue'), 'first Tab is not the skip
link')
    await page.goto(BASE + '#title=rentflow')
    await page.locator('#title-name').waitFor()
    for (let i = 0; i < 25; i++) await
page.keyboard.press('Tab')
    assert(await page.evaluate(() =>
!!document.activeElement.closest('[role=dialog]')),
'focus escaped the dialog')
    await ctx.close()
  },

  async 'accessibility (axe, WCAG 2.1 AA): gate,
home, title page, trailer'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await axe(page, 'gate')
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await axe(page, 'home')
    await page.goto(BASE + '#title=carepilot')
    await page.locator('#title-name').waitFor()
    await axe(page, 'title page')
    await page.keyboard.press('Escape')
    await page.getByRole('button', { name: /Play
trailer/ }).click()
    await page.getByRole('dialog', { name: 'Trailer'
}).waitFor()
    await page.keyboard.press(' ')
    await axe(page, 'trailer')
    await ctx.close()
  },

  async 'phone: no sideways scrolling, everything
reachable'(b) {
    const { ctx, page } = await visitor(b, {
...devices['iPhone 13'] })
    await page.goto(BASE)
    await page.getByRole('button', { name: /Just
looking/ }).click()
    const overflow = await page.evaluate(() =>
document.documentElement.scrollWidth -
window.innerWidth)
    assert(overflow <= 0, `page is ${overflow}px
wider than the screen`)
    await page.locator('[data-
title="bazaarlens"]').first().scrollIntoViewIfNeeded
()
    await page.locator('[data-
title="bazaarlens"]').first().click()
    await page.locator('#title-name', { hasText:
'BazaarLens' }).waitFor()
    await page.getByRole('button', { name: 'Close'
}).click()
    await ctx.close()
  },

  async 'reduced motion: nothing animates, trailer
waits for you'(b) {
    const { ctx, page } = await visitor(b, {
reducedMotion: 'reduce' })
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    const running = await page.evaluate(() =>
document.getAnimations().filter((a) => a.playState
=== 'running').length)
    assert(running === 0, `${running} animations
still running`)
    await page.getByRole('button', { name: /Play
trailer/ }).click()
    await page.waitForTimeout(3500)
    assert(await page.getByText('1 /
7').isVisible(), 'trailer auto-advanced under
reduced motion')
    await ctx.close()
  },

  async 'still works when the browser blocks
storage'(b) {
    const { ctx, page } = await visitor(b)
    await ctx.addInitScript(() => {
      Object.defineProperty(window, 'localStorage',
{ get() { throw new Error('blocked') } })
    })
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    await page.goto(BASE + '#title=ai-900')
    await page.locator('#title-name').waitFor()
    await page.getByRole('button', { name: '+ My
list' }).click()
    assert(!page.errors.length,
page.errors.join('\n'))
    await ctx.close()
  },

  async 'without JavaScript the whole catalogue is
still readable'(b) {
    const { ctx, page } = await visitor(b, {
javaScriptEnabled: false })
    await page.goto(BASE)
    assert(!(await page.locator('[data-
gate]').isVisible()), 'gate blocks the page without
JS')
    const text = await visibleText(page)
    for (const s of ['Jaykumar Shevate',
'1000/1000', '929/1000', 'CarePilot', 'Teaching
Associate', 'Roll credits', '60-second brief'])
      assert(text.includes(s) ||
text.toUpperCase().includes(s.toUpperCase()),
`missing "${s}" without JS`)
    await ctx.close()
  },

  async 'nothing on the page claims something
missing: placeholders are labelled'(b) {
    const { ctx, page } = await visitor(b)
    await page.goto(BASE)
    await page.getByRole('button', { name:
/Recruiter/ }).click()
    const text = await visibleText(page)
    for (const bad of ['undefined', 'NaN', '[object
Object]', 'Lorem']) assert(!text.includes(bad),
```

```
`page shows "${bad}"`)
    const hrefs = await page.$$eval('a[href]', (as)
=> as.map((a) => a.getAttribute('href')))
    for (const h of hrefs) assert(h &&
!h.includes('[object') && !h.includes('undefined'),
`broken link: ${h}`)
    await ctx.close()
  },

  async '25 visitors at the same time, each with
their own state'(b) {
    const N = 25
    const viewers = ['Recruiter', 'Engineer', 'Just
looking']
    const first = { Recruiter: 'Credentials',
Engineer: 'Projects', 'Just looking': 'Projects' }
    const ids = ['carepilot', 'rentflow',
'sensorium', 'bazaarlens', 'phd-erp', 'aws-ccp',
'ai-900']
    const runs = Array.from({ length: N }, async (_,
i) => {
      const { ctx, page } = await visitor(b)
      const who = viewers[i % 3]
      const pick = ids[i % ids.length]
      await page.goto(BASE)
      await page.getByRole('button', { name: new
RegExp(who) }).click()
      assert((await rowOrder(page))[0] ===
first[who], `visitor ${i}: wrong first row`)
      await page.goto(BASE + `#title=${pick}`)
      await page.locator('#title-name').waitFor()
      await page.getByRole('button', { name: '+ My
list' }).click()
      await page.keyboard.press('Escape')
      const mine = await page.$$eval('[aria-
labelledby="row-mylist"] [data-title]', (els) =>
els.map((e) => e.dataset.title))
      assert(mine.length === 1 && mine[0] === pick,
`visitor ${i}: My list is ${mine}, expected only
${pick}`)
      assert(!page.errors.length, `visitor ${i}:
${page.errors.join(' | ')}`)
      await ctx.close()
    })
    await Promise.all(runs)
  },
}

/* ---------------- runner ---------------- */

await ensureServer()
const browser = await chromium.launch()
const only = process.argv[2]
for (const [name, fn] of Object.entries(tests)) {
  if (only && !name.includes(only)) continue
  const t0 = Date.now()
  try {
    await fn(browser)
    results.push([true, name, Date.now() - t0])
    console.log(`  ✓ ${name} (${Date.now() - t0}
ms)`)
  } catch (e) {
    results.push([false, name, e.message])
    console.log(`  ✗ ${name}\n
${e.message.split('\n').join('\n      ')}`)
  }
}
await browser.close()
server?.kill()
const failed = results.filter((r) => !r[0]).length
console.log(`\n${results.length - failed} passed,
${failed} failed`)
process.exit(failed ? 1 : 0)
```

tests/load/run.mjs — 41 lines

```
/*
 * Load test: many visitors hitting the built site
at once.
 *
 *   npm run build && npm run test:load
 *
 * The site is static files (no server code of mine,
no database), so the
 * question is only "does serving it break under
concurrent requests?".
 * This hammers the page and its biggest asset with
200 parallel connections
 * for 10 s each and fails on any error or non-200
response.
 * Note: `vite preview` is a dev-grade server. A
real static host / CDN is
 * faster still; this is a floor, not a ceiling.
 */
import { spawn } from 'node:child_process'
import { readFile } from 'node:fs/promises'
import autocannon from 'autocannon'

const BASE = process.env.BASE_URL ||
'http://localhost:4173/'
const up = () => fetch(BASE).then((r) => r.ok, () =>
false)

let server
if (!(await up())) {
  server = spawn('npx', ['vite', 'preview', '--
port', '4173', '--strictPort'], { stdio: 'ignore' })
  for (let i = 0; i < 50 && !(await up()); i++)
await new Promise((r) => setTimeout(r, 200))
}

const html = await readFile(new
URL('../../dist/index.html', import.meta.url),
'utf8')
const js = html.match(/src="
(\/assets\/[^"]+\.js)"/)?.[1]
const targets = [['page', BASE], ['main script', new
URL(js, BASE).href]]

let failed = false
for (const [name, url] of targets) {
  const r = await autocannon({ url, connections:
200, duration: 10 })
  const bad = r.errors + r.timeouts + r.non2xx
  failed ||= bad > 0
  console.log(
    `${bad ? '✗' : '✓'} ${name}: ${r.requests.total}
requests, ${Math.round(r.requests.average)}/s, ` +
      `latency p50 ${r.latency.p50} ms · p99
${r.latency.p99} ms, errors ${r.errors}, timeouts
${r.timeouts}, non-2xx ${r.non2xx}`,
  )
}
server?.kill()
process.exit(failed ? 1 : 0)
```

tests/unit/brief.test.js — 25 lines

```
import { describe, expect, it } from 'vitest'
import { fit, summary } from
'../../src/content/brief.js'
import { profile, isFill } from
'../../src/content/profile.js'

describe('recruiter brief', () => {
  it('covers every target role', () => {
    expect(fit.map((f) =>
f.role.toLowerCase()).sort()).toEqual(profile.target
ing.map((t) => t.toLowerCase()).sort())
  })

  it('every proof is built from facts already in
profile.js (or marked missing)', () => {
    const facts = JSON.stringify(profile)
    for (const f of fit)
      for (const p of f.proof) {
        if (isFill(p)) continue
        // each comma/colon-separated piece must
```

```
appear verbatim in the profile
       for (const piece of p.split(/: |, | \
(|\)/).filter(Boolean)) expect(facts,
`"${piece}"`).toContain(piece)
      }
  })

  it('the ATS summary has the key facts and no
placeholders or garbage', () => {
    const s = summary()
    for (const want of [profile.name, '1000/1000',
'929/1000', 'Qualified', 'DYPIU PhD ERP', 'Sole
developer']) expect(s).toContain(want)
    for (const bad of ['missing', '[object',
'undefined', 'NaN', 'null'])
expect(s).not.toContain(bad)
  })
})
```

tests/unit/catalog.test.js 58 lines

```
import { describe, expect, it } from 'vitest'
import { titles, titleById, ROWS, VIEWERS,
DEFAULT_VIEWER, search, similar } from
'../../src/content/catalog.js'
import { isFill, real, fill } from
'../../src/content/profile.js'
import { parseHash } from '../../src/lib/hash.js'

describe('catalog', () => {
  it('every title id is unique', () => {
    const ids = [...titles.map((t) => t.id),
'about']
    expect(new Set(ids).size).toBe(ids.length)
  })

  it('every viewer orders the same rows, each row is
non-empty', () => {
    for (const v of Object.values(VIEWERS))
expect([...v.rows].sort()).toEqual(Object.keys(ROWS)
.sort())
    for (const r of Object.values(ROWS)) {
      expect(r.ids.length).toBeGreaterThan(0)
      for (const id of r.ids)
expect(titleById[id]).toBeTruthy()
    }
    expect(VIEWERS[DEFAULT_VIEWER]).toBeTruthy()
  })

  it('recruiters see credentials first, engineers
see projects first', () => {

expect(VIEWERS.recruiter.rows[0]).toBe('credentials'
)

expect(VIEWERS.engineer.rows[0]).toBe('projects')
  })

  it('search matches every word, case-
insensitively', () => {
    expect(search('AWS').map((t) =>
t.id).sort()).toEqual(['aws-ccp', 'aws-restart',
'skills-cloud'])
    expect(search('aws practitioner').map((t) =>
t.id)).toEqual(['aws-ccp'])
    expect(search('iot').map((t) =>
t.id)).toEqual(['sensorium'])
    expect(search('   ')).toEqual([])
    expect(search('kubernetes')).toEqual([]) // not
on the resume, must not appear
  })

  it('placeholders are never searchable text', () =>
{
    for (const t of titles) for (const s of
t.searchText) expect(isFill(s)).toBe(false)
  })

  it('"more like this" stays within the same kind',
() => {
    for (const t of titles) for (const s of
similar(t.id)) expect(s.kind).toBe(t.kind)
  })
})

describe('helpers', () => {
  it('fill / real', () => {
    expect(isFill(fill('x'))).toBe(true)
    expect(isFill(['a'])).toBe(false) // arrays have
a .fill method
    expect(real(fill('x'))).toBe(null)
    expect(real('  ')).toBe(null)
    expect(real('https://a.b')).toBe('https://a.b')
  })

  it('parses the title hash', () => {

expect(parseHash('#title=carepilot')).toBe('carepilo
t')
    expect(parseHash('#title=a%20b')).toBe('a b')
    expect(parseHash('')).toBe(null)
    expect(parseHash('#row-projects')).toBe(null)
  })
})
```

tests/unit/facts.test.js 41 lines

```
/*
 * The facts on the site must match the resume
exactly. If someone edits
 * profile.js and a score or date drifts, this fails
before it ships.
 */
import { describe, expect, it } from 'vitest'
import { profile } from
'../../src/content/profile.js'

const byId =
Object.fromEntries(profile.credentials.map((c) =>
[c.id, c]))

describe('facts from the resume', () => {
  it('certification results and dates', () => {
    expect(byId['aws-ccp']).toMatchObject({ result:
'1000/1000', date: 'Mar 2026', note: 'Valid through
Mar 2029' })
    expect(byId['ai-900']).toMatchObject({ result:
'929/1000', date: 'Mar 2026' })
    expect(byId['gate-2026']).toMatchObject({
result: 'Qualified' })
    expect(byId['aws-restart']).toMatchObject({
result: 'Completed', date: 'Sep 2025' })
  })

  it('experience titles, employers and dates', () =>
{
    expect(profile.experience.map((x) => [x.title,
x.org, x.when])).toEqual([
      ['Teaching Associate', 'D. Y. Patil
International University', 'Aug 2025 – present'],
      ['Software Developer Intern', 'Technocia
Infotech', 'May – Aug 2025'],
      ['Cybersecurity Research Intern', 'FirstVidya
Pvt. Ltd. (remote)', 'Feb – Apr 2025'],
    ])
  })

  it('education and identity', () => {
    expect(profile.education).toEqual({
      degree: 'B.Tech, Computer Science &
Engineering (Cyber Security)',
      school: 'D. Y. Patil International
University',
      year: '2026',
    })
    expect(profile.name).toBe('Jaykumar Chandrakant
Shevate')
    expect(profile.location).toBe('Pune,
```

```
Maharashtra, India')
  })

  it('the only project with known status/role is the
PhD ERP', () => {
    const erp = profile.projects.find((p) => p.id
=== 'phd-erp')
    expect(erp).toMatchObject({ status: 'UAT / pilot
stage', role: 'Sole developer' })
    expect(profile.projects.map((p) =>
p.name)).toEqual(['DYPIU PhD ERP', 'CarePilot',
'RentFlow', 'Sensorium Nexus', 'BazaarLens'])
  })
})
```

tests/unit/render.test.jsx 36 lines

```
/*
 * Render the whole app to HTML (exactly what the
pre-render step ships) and
 * check nothing broken or misleading leaks into the
page.
 */
import { describe, expect, it } from 'vitest'
import { renderToString } from 'react-dom/server'
import App from '../../src/App.jsx'
import { titles } from
'../../src/content/catalog.js'
import { profile } from
'../../src/content/profile.js'

const html = renderToString(<App />)
const text = html.replace(/<[^>]+>/g, '
').replace(/&#x27;/g, "'").replace(/&amp;/g,
'&').replace(/\s+/g, ' ')

describe('pre-rendered page', () => {
  it('contains every title and the key facts', () =>
{
    for (const t of titles)
expect(text).toContain(t.name.replace(/&/g, '&'))
    expect(text).toContain(profile.displayName)
    expect(text).toContain('1000/1000')
    expect(text).toContain('929/1000')
  })

  it('has no rendering garbage', () => {
    for (const bad of ['undefined', 'NaN', '[object
Object]', 'null'])
      expect(text, `found "${bad}"`).not.toMatch(new
RegExp(`\\b${bad.replace(/[[\]]/g, '\\$&')}\\b`))
  })

  it('marks missing facts as missing, never as
content', () => {
    expect(text).toContain('missing: GitHub profile
URL')
    expect(html).not.toMatch(/href="\[object/)
  })

  it('has exactly one h1 visible to the page outline
per screen', () => {
    // gate + billboard: two h1s, the gate is a
dialog of its own
    expect((html.match(/<h1/g) ||
[]).length).toBe(2)
  })
})
```

tests/unit/storage.test.js 27 lines

```
import { afterEach, describe, expect, it, vi } from
'vitest'
import { read, write } from
'../../src/lib/storage.js'

describe('storage', () => {
  afterEach(() => vi.restoreAllMocks())

  it('round-trips JSON', () => {
    write('t', { a: [1, 2] })
    expect(read('t', null)).toEqual({ a: [1, 2] })
  })

  it('falls back when storage throws (private mode,
blocked cookies)', () => {
    vi.spyOn(Storage.prototype,
'getItem').mockImplementation(() => {
      throw new Error('denied')
    })
    vi.spyOn(Storage.prototype,
'setItem').mockImplementation(() => {
      throw new Error('denied')
    })
    expect(() => write('x', 1)).not.toThrow()
    expect(read('x', 'fallback')).toBe('fallback')
  })

  it('falls back on corrupted values', () => {
    window.localStorage.setItem('jcs:bad', '{not
json')
    expect(read('bad', 7)).toBe(7)
  })
})
```

tests/unit/tailor.test.js 22 lines

```
import { describe, expect, it } from 'vitest'
import { parseTailor } from
'../../src/lib/tailor.js'

describe('tailored links', () => {
  it('reads company and role', () => {
    expect(parseTailor('?
for=Infosys&role=cloud')).toEqual({ company:
'Infosys', role: 'cloud' })
    expect(parseTailor('?
for=Acme%20Corp&role=Junior%20DevOps')).toEqual({
company: 'Acme Corp', role: 'devops' })
    expect(parseTailor('?
role=Security%20Engineer')).toEqual({ company: null,
role: 'security' })
  })

  it('ignores unknown roles and empty links', () =>
{
    expect(parseTailor('?
for=Acme&role=astronaut')).toEqual({ company:
'Acme', role: null })
    expect(parseTailor('')).toBe(null)
    expect(parseTailor('?for=%20%20')).toBe(null)
  })

  it('strips markup and caps the length', () => {
    const t = parseTailor('?for=' +
encodeURIComponent('<img src=x
onerror=alert(1)>Evil'))
    expect(t.company).not.toMatch(/[<>]/)
    expect(parseTailor('?for=' +
'x'.repeat(200)).company.length).toBe(40)
  })
})
```

APPENDIX C. COMPLETE SOURCE CODE OF VERSION 1

Version 1, "Life of a Request", as it stood in its final commit `fd93879` (5 November 2026) before the rebuild removed it. Reproduced from git history so that the 3D work, including its five custom shaders, remains readable in this record. The lockfile and the preview video are omitted for the reasons given in Appendix B.

package.json 29 lines

```
{
  "name": "life-of-a-request",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "todo": "node scripts/todo.mjs"
  },
  "dependencies": {
    "@fontsource/courier-prime": "^5.1.0",
    "@fontsource/young-serif": "^5.1.0",
    "@react-three/drei": "^9.114.0",
    "@react-three/fiber": "^8.17.10",
    "gsap": "^3.12.5",
    "lenis": "^1.1.14",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "three": "^0.169.0",
    "zustand": "^4.5.5"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.3",
    "vite": "^5.4.10",
    "vite-plugin-glsl": "^1.3.0"
  }
}
```

index.html 49 lines

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-
width, initial-scale=1.0" />
    <title>Jaykumar Shevate · Life of a
Request</title>
    <meta name="description" content="Jaykumar
Shevate: cloud, backend and security. AWS Certified
Cloud Practitioner (1000/1000), Azure AI-900, B.Tech
CSE (Cyber Security) 2026, Pune. Portfolio told as
the journey of a request through a cloud system." />
    <meta name="theme-color" content="#ece3d0" />
    <meta property="og:title" content="Jaykumar
Shevate · Life of a Request" />
    <meta property="og:description" content="Cloud,
backend and security work, told as one request
travelling through a system." />
    <meta property="og:type" content="website" />
    <link rel="icon" href="data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32
32'%3E%3Crect width='32' height='32'
fill='%23ece3d0'/%3E%3Ccircle cx='16' cy='16' r='7'
fill='%23c8372d'/%3E%3C/svg%3E" />
    <!--
      Pick the view before anything paints, so the
page never jumps:
      ?simple, no WebGL, or reduced motion (unless ?
3d) → simple view.
      App reads this class to set its starting mode.
    -->
    <script>
      (function () {
        var q = location.search, gl = false
        try { var c =
document.createElement('canvas'); gl = !!
(c.getContext('webgl2') || c.getContext('webgl')) }
catch (e) {}
        var rm = window.matchMedia('(prefers-
reduced-motion: reduce)').matches
        if (/[?&]simple/.test(q) || !gl || (rm &&
!/[?&]3d/.test(q)))
document.documentElement.classList.add('is-simple')
      })()
    </script>
    <script type="application/ld+json">
      {
        "@context": "https://schema.org",
        "@type": "Person",
        "name": "Jaykumar Chandrakant Shevate",
        "jobTitle": "Teaching Associate",
        "worksFor": { "@type":
"CollegeOrUniversity", "name": "D. Y. Patil
International University" },
        "alumniOf": { "@type":
"CollegeOrUniversity", "name": "D. Y. Patil
International University" },
        "address": { "@type": "PostalAddress",
"addressLocality": "Pune", "addressRegion":
"Maharashtra", "addressCountry": "IN" },
        "knowsLanguage": ["English", "Hindi",
"Marathi"]
      }
    </script>
  </head>
  <body>
    <div id="root"></div>
    <noscript>
      <main style="font-family: monospace; padding:
2rem; color: #2b2a28; background: #ece3d0">
        <h1>Jaykumar Shevate</h1>
        <p>Cloud, backend and security. AWS
Certified Cloud Practitioner (1000/1000), Microsoft
AI-900 (929/1000). B.Tech CSE (Cyber Security), D.
Y. Patil International University, 2026. Pune,
India.</p>
      </main>
    </noscript>
    <script type="module" src="/src/main.jsx">
</script>
  </body>
</html>
```

vite.config.js 47 lines

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import glsl from 'vite-plugin-glsl'
import { hero, ARROW_PATHS } from
'./src/content/hero.js'
import { STOPS } from './src/core/route.js'

const esc = (s) => String(s).replace(/&/g,
'&amp;').replace(/</g, '&lt;').replace(/>/g,
'&gt;').replace(/"/g, '&quot;')

/*
 * Writes the hero into index.html at build/dev
time, so the first thing a
 * visitor sees (on a slow phone, before ~60 KB of
JS arrives) is real text,
 * not a blank page. React replaces it when it
mounts; the markup matches the
```

```
 * Hero component so nothing moves when that
happens.
 */
function heroShell() {
  const vh = STOPS[0].vh
  return `<main id="main" class="shell-main">
<section class="stop hero is-on shell" style="--
shell-h:${vh}vh">
<div class="stop__sticky"><div class="hero__inner">
<p class="kicker">${esc(hero.kicker)}</p>
<h1 class="hero__title">${esc(hero.title)}</h1>
<p class="hero__line">${esc(hero.line)}</p>
<p class="hero__meta">${esc(hero.meta1)}
<br>${esc(hero.meta2)}</p>
<p class="hero__hint" aria-hidden="true"><svg
width="54" height="40" viewBox="0 0 54 40"
class="hero__arrow">${ARROW_PATHS.map((d) => `<path
d="${d}"/>`).join('')}</svg>${esc(hero.hint)}</p>
</div></div></section></main>`
}

const prerenderHero = () => ({
  name: 'prerender-hero',
  transformIndexHtml: (html) => html.replace('<div
id="root"></div>', `<div id="root">${heroShell()}
</div>`),
})

// glsl() lets us `import frag from './x.frag'` as a
string and supports #include.
export default defineConfig({
  plugins: [react(), glsl({ compress: true }),
prerenderHero()],
  build: {
    target: 'es2020',
    // three.js is big; keep it in its own long-
cached chunk.
    rollupOptions: {
      output: {
        manualChunks: {
          three: ['three'],
          r3f: ['@react-three/fiber'],
        },
      },
    },
  },
})
```

README.md 277 lines

```
# Life of a Request

My portfolio, told as one HTTP request travelling
through a cloud system.
You scroll, the request moves: edge → security gate
→ load balancer → five
project "services" → data layer → back to you as a
`200 OK`.

**Preview:** [`preview/life-of-a-request-
preview.mp4`](preview/life-of-a-request-preview.mp4)
(60 s scroll-through, recorded headless without a
GPU, so it's choppier than the real thing).

It's drawn like pages from a field notebook: manila
paper, graphite text,
blue ballpoint for the 3D drawings, red pen for the
request itself, and a
highlighter I only use on facts I can prove (scores,
dates).

```bash
npm install
npm run dev        # http://localhost:5173
npm run build      # production build into dist/
npm run preview    # serve that build locally
```

Useful URLs while developing:

- `/?simple` forces the simple, no-3D view (what
recruiters in a hurry get)
- `/?3d` forces the 3D version even if the OS asks
for reduced motion

Nothing is deployed. There's no backend.

Before this goes in front of anyone: fill in the
blanks

All facts live in **`src/content/profile.js`**.
Anything I haven't written yet
is wrapped in `fill('...')` and shows up on the page
as a red circled
`[FILL: ...]` note, so nothing gets made up. To list
what's left:

```bash
npm run todo
```

Optional extras in the same file stay hidden until
they're filled in:
`photo` (shows as a taped polaroid on the About
page), `screenshots` on each
project (a small strip under the links), and
`otherRepos` (listed under the
skills, and in the terminal as `ls repos`).

What's still missing: GitHub / LinkedIn / email /
resume links, cert
verification links, what I did in each role, and for
each project its status,
role, problem, architecture, stack, and repo/demo
links. The tagline and the
contact call-to-action are drafts written from
resume facts; I should rewrite
them in my own words.

- **Resume PDF:** drop it in `public/` (e.g.
`public/Jaykumar-Shevate-Resume.pdf`)
  and set `links.resume` to `'/Jaykumar-Shevate-
Resume.pdf'`.
- **Contact form:** get a free key at
https://web3forms.com, copy
  `.env.example` to `.env.local` and paste it in as
`VITE_WEB3FORMS_KEY`. Until
  then the form tells visitors it isn't wired up,
rather than pretending to send.

How it works (the version I'd give in an
interview)

1. One number drives everything

`src/core/scroll.js`

The page is a stack of real HTML `<section>`s, one
per stop. Each is tall
(height comes from `vh` in `src/core/route.js`), and
its panel is
`position: sticky`, so the text stays still while
you scroll "through" it.

- **Lenis** smooths wheel scrolling (inertia, with
my own easing curve in
  `src/core/ease.js`) but still uses native scroll,
so keyboard, scrollbar,
  find-in-page and screen readers all keep working.
- **GSAP ScrollTrigger** watches every `[data-stop]`
section and reports its
  progress from 0 to 1. Those numbers go into a
plain object, `scroll.local`.
```

```
- GSAP's ticker drives Lenis, so there's exactly one
animation clock.

`scroll` is deliberately **not** React state
(`src/core/store.js`). It changes
on every scroll event; putting it in state would re-
render the whole app 60
times a second. The 3D code just reads it inside
`useFrame`. Things that change
rarely (which stop is active, terminal open, sound
on) live in a small
**zustand** store.

### 2. The camera rides a spline

`src/core/route.js`, `src/canvas/CameraRig.jsx`

Every stop has a camera position and a look-at
point. Two Catmull-Rom curves
run through them. Stop *k* sits exactly at parameter
`k / (N-1)`, so
"progress inside section k" turns into "how far from
stop k to stop k+1" via
a per-stop easing:

- `hero`: hold still while the name dissolves, then
leave
- `linear`: keep moving (the security gate is a fly-
through)
- `dwell`: drift slightly while you read, then
travel in the last third

The rig doesn't jump to the target. It damps toward
it (`1 - exp(-λ·dt)`, so
it's frame-rate independent), which adds a second
layer of inertia on top of
Lenis. The mouse adds a little parallax.

To keep the 3D scene out from under the text panel,
I don't move the camera.
I shift the **projection** with
`camera.setViewOffset`: on desktop the world
renders right of centre, on phones it renders higher
up.

### 3. The hero: text made of particles (custom GLSL
#1)

`src/canvas/ParticleText.jsx`,
`src/canvas/shaders/particles.vert`

1. Draw my name with the real web font onto a hidden
2D canvas.
2. Read the pixels back and keep the inked ones.
3. Pick N of them at random. Those are the particle
positions.

Each particle carries three positions as attributes:
scattered, in the name,
and on a sphere (the packet). The vertex shader
blends scattered → name over
time when the page loads, then name → packet as you
scroll. Each particle
starts at a slightly different moment (`aRand`), and
halfway through the
collapse they're pushed through a noise field so
they swirl instead of sliding
in a straight line. It's 11k particles in **one draw
call**. The same component
plays backwards at the end to spell `200 OK`.

### 4. The packet and the security scan (custom GLSL
#2)

`src/canvas/Packet.jsx`,
`src/canvas/shaders/packet.frag`

The packet sits at a fixed spot in front of the
camera, but in world space, so
it actually passes through the gates and leaves a
trail (a ring buffer of its
last 48 positions). In the gate section a uniform
`uScan` sweeps a bright band
from top to bottom, with "readout" stripes behind
it, then the packet flashes
when it's accepted.

### 5. Gates that burn open (custom GLSL #3)

`src/canvas/scenes/Gate.jsx`,
`src/canvas/shaders/dissolve.frag`

Each certification has a gate with a hatched
membrane. When its scan finishes,
`uDissolve` goes from 0 to 1. Simplex noise decides
which pixels are discarded
first, and a thin rim glows along the burning edge.
The HTML cert list switches
from "scanning…" to a stamped "verified" at the same
scroll point (it reads
`gateStage` from the store).

### 6. Hand-drawn lines that "boil" (custom GLSL #4)

`src/canvas/kit.js`

In hand-drawn animation every frame is re-traced, so
lines shimmer slightly. I
patch three.js's line material with
`onBeforeCompile` to add a tiny random
offset per vertex that changes **8 times a second**,
not every frame; the
stepping is what makes it read as drawn. The offset
is hashed from the vertex
position, so two segments sharing a corner move
together and never split. It's
turned off for reduced motion.

Plus a blueprint grid floor (`ground.frag`) that
uses `fwidth()` so lines stay
1px wide at any distance.

### 7. The five project scenes

`src/canvas/scenes/*.jsx`. Each is small and self-
contained:

| Project | Metaphor | Technique |
|---|---|---|
| CarePilot | hospitals as a graph, records hop
between them | k-nearest-neighbour graph,
`InstancedMesh` dots on arcs |
| RentFlow | a building whose units change state |
one instanced plane per unit, `setColorAt` |
| Sensorium Nexus | devices streaming readings into
live charts | line buffers shifted one sample at a
time |
| BazaarLens | stores race; the cheapest wins, not
the fastest | per-race random durations and prices |
| DYPIU PhD ERP | a student token stamped at each
station | token moves along a curve driven by scroll
|

All the values in these are random or simulated.
They illustrate the idea;
they aren't data from the real systems.

### 8. Performance

- **Text before JavaScript.** A small Vite plugin in
`vite.config.js` writes
  the hero (built from `profile.js`) straight into
`index.html`, so a slow phone
  shows real text before any JS arrives. React
replaces it with identical
  markup, so nothing moves.
```

```
- **The view is chosen before paint.** A few lines of inline script in
  `index.html` decide simple vs 3D (`?simple`, no WebGL, reduced motion) and set
  a class on `<html>`, so the layout never switches after it has been drawn.
- **The 3D waits its turn.** The canvas is a `React.lazy` chunk that only
  starts downloading after the page has loaded and the browser is idle. GSAP +
  Lenis are a separate chunk too (`core/nav.js` loads them on demand), so the
  simple view never downloads them unless you click a nav link.
- **Each scene is its own chunk** (0.5 to 2 KB each). Only the active stop, one
  behind and two ahead are mounted; the next ones are prefetched.
- **No models, no textures.** Everything is built from code (lines, a few
  instanced meshes), so there's nothing to download beyond JS and a few small
  font files. Paper grain is an SVG filter in CSS.
- **Shared materials.** Every line uses one of three materials, so the GPU
  compiles a handful of shader programs, not hundreds.
- **DPR capped** at 1.5 (1 on low-end), and drei's `PerformanceMonitor` drops
  the resolution if fps falls and raises it again when it recovers.
- **Device tiers** (`src/core/quality.js`): phones get a third of the
  particles, no antialiasing and fewer animated objects. There's no
  post-processing anywhere.

Current production build: about 56 KB gzipped of JS for first paint, then
about 49 KB (scroll) and 230 KB (3D) loaded after it.

Lighthouse 12, measured here on a server **without a GPU** (WebGL runs on the
CPU, which inflates "total blocking time" in the 3D view):

| | Performance | Accessibility | Best practices | SEO | First paint (slow 4G phone) | Layout shift |
|---|---|---|---|---|---|---|
| 3D view | 60 | 100 | 100 | 100 | 2.9 s | 0 |
| Simple view | 88 | 100 | 100 | 100 | 3.0 s | 0.01 |

The 60fps target still needs checking on a real laptop with a graphics card.

### 9. Accessibility and the simple view

- All content is real HTML: headings, lists, a table, a form. The canvas is
  `aria-hidden`. The `<h1>` stays in the DOM after the particles take over
  visually.
- The first Tab stop is **"Skip the animation: simple view"**. Simple view
  removes the canvas, smooth scroll and custom cursor, and shows the same
  content as one readable page.
- `prefers-reduced-motion` starts in the simple view (with a button to play the
  3D anyway), and turns off line boil and animations.
- The trace on the right edge is a list of buttons, so every stop can be
  reached by keyboard. Panels become visible when anything inside them has
  focus.

### 10. The little things

- **Terminal** (`~` or the button): `help`, `whoami`, `ls projects`,
  `open <project>` (Tab completes), `certs`, `contact`, `resume`, `simple`,
  `sound on|off`. Up/Down for history. It's a dialog with focus return and
  `aria-live` output. `src/dom/Terminal.jsx`
- **Magnetic links** (`src/dom/Magnetic.jsx`): `gsap.quickTo` leans the element
  toward the cursor, capped at 8px.
- **Cursor** (`src/dom/Cursor.jsx`): a red pen dot plus a pencil ring that
  lags behind it. Only on mouse/trackpad.
- **Sound** (`src/core/sound.js`): off by default and made with the Web Audio
  API (oscillators, filtered noise, a slow LFO), so there's no audio file. The
  filter opens a little at each stop, so the hum brightens as the request goes
  deeper.
- **Hand-drawn edges** (`src/styles/notebook.css`): page outlines, buttons and
  the "fill" circles go through an SVG displacement filter. It's applied to a
  separate border layer so the text inside stays sharp. Each page is tilted by
  an angle derived from its page number, so it's the same on every visit.

---

## File map

```
src/
  content/profile.js   every fact on the site (edit this one)
  core/
    route.js   the stops, the splines, the pacing
    scroll.js   Lenis + ScrollTrigger → scroll.local
    store.js   zustand store + the non-reactive scroll object
    quality.js   device tier, DPR, reduced motion, WebGL check
    ease.js   the house easing curve
    sound.js   Web Audio ambient
  canvas/
    Experience.jsx   <Canvas>, lazy scenes, perf monitor, boil clock
    CameraRig.jsx   camera on the spline + view offset
    Packet.jsx   the request + its trail
    ParticleText.jsx   text → particles → packet
    Ground.jsx   grid floor
    kit.js   palette, shared line materials, line boil, helpers
    shaders/   GLSL (particles, packet, dissolve, ground, noise)
    scenes/   one file per stop
  dom/
    sections/   the real HTML for each stop
    Stop.jsx   tall sticky section vs simple section
    Sheet.jsx   the taped notebook page
    Terminal.jsx · TopBar.jsx · Trace.jsx · Cursor.jsx · Magnetic.jsx · SoundToggle.jsx · Txt.jsx
  styles/
    tokens.css   colours + fonts (change the look here)
    base.css · sections.css · notebook.css
```
```

```
## Stack

Vite, React 18, three.js, @react-three/fiber,
@react-three/drei, GSAP +
ScrollTrigger, Lenis, zustand, vite-plugin-glsl.
Fonts: Young Serif and
Courier Prime (self-hosted via Fontsource).
```

.env.example 3 lines

```
# Free key from https://web3forms.com (enter your
email, they mail you the key).
# Copy this file to .env.local and paste the key.
Never commit .env.local.
VITE_WEB3FORMS_KEY=
```

.gitignore 5 lines

```
node_modules
dist
.env
.env.local
.DS_Store
```

public/robots.txt 2 lines

```
User-agent: *
Allow: /
```

scripts/todo.mjs 24 lines

```
// npm run todo — lists every fact still missing
from src/content/profile.js,
// so I know exactly what to fill in before sending
the link to anyone.
import { profile, isFill } from
'../src/content/profile.js'

const missing = []
const walk = (v, path) => {
  if (isFill(v)) return missing.push(`${path}  →
${v.fill}`)
  if (Array.isArray(v)) return v.forEach((x, i) =>
walk(x, `${path}[${i}]`))
  if (v && typeof v === 'object') for (const [k, x]
of Object.entries(v)) walk(x, path ? `${path}.${k}`
: k)
}
walk(profile, '')

const extras = []
if (!profile.photo) extras.push('photo (optional)')
if (!profile.otherRepos.length)
extras.push('otherRepos (optional)')
const noShots = profile.projects.filter((p) =>
!p.screenshots.length).map((p) => p.slug)
if (noShots.length) extras.push(`screenshots for:
${noShots.join(', ')} (optional)`)
if (!process.env.VITE_WEB3FORMS_KEY)
extras.push('VITE_WEB3FORMS_KEY in .env.local
(contact form)')

console.log(`\n${missing.length} blanks still in
profile.js:\n`)
missing.forEach((m) => console.log('  ' + m))
console.log(`\nAlso not set yet:\n`)
extras.forEach((m) => console.log('  ' + m))
console.log('\nPlus: rewrite the DRAFT tagline and
contact.cta in your own words.\n')
```

src/App.jsx 132 lines

```
/*
 * App — decides which version of the site you get,
and lays out the page.
 *
 *  - The content (all the <section>s) is the same
in both modes.
 *  - "experience": tall sticky sections + the lazy-
loaded 3D canvas behind.
 *  - "simple": normal-height sections, no canvas,
no smooth scroll. Chosen
 *    automatically for ?simple, no WebGL, or
prefers-reduced-motion (that
 *    choice is made by a tiny inline script in
index.html, before paint).
 */
import { Suspense, lazy, useEffect, useState } from
'react'
import { profile } from './content/profile'
import { useStore } from './core/store'
import { quality } from './core/quality'
import { loadScroll } from './core/nav'
import { setSound } from './core/sound'
import TopBar from './dom/TopBar'
import Trace from './dom/Trace'
import Terminal from './dom/Terminal'
import Cursor from './dom/Cursor'
import Hero from './dom/sections/Hero'
import Certs from './dom/sections/Certs'
import About from './dom/sections/About'
import Project from './dom/sections/Project'
import Skills from './dom/sections/Skills'
import Contact from './dom/sections/Contact'

const Experience = lazy(() =>
import('./canvas/Experience'))

// GSAP + Lenis live in their own chunk
(core/scroll.js); this holds it once loaded.
let scrollMod = null

/*
 * Wait until the page has painted and the browser
is idle before starting the
 * 3D download. The text is readable first; the
world fades in after.
 */
function useAfterFirstPaint(enabled) {
  const [ok, setOk] = useState(false)
  useEffect(() => {
    if (!enabled) return
    const idle = window.requestIdleCallback || ((f)
=> setTimeout(f, 200))
    const start = () => idle(() => setOk(true), {
timeout: 1500 })
    if (document.readyState === 'complete') start()
    else window.addEventListener('load', start, {
once: true })
    return () => window.removeEventListener('load',
start)
  }, [enabled])
  return ok
}


export default function App() {
  const mode = useStore((s) => s.mode)
  const ready = useStore((s) => s.canvasReady)
  const setMode = useStore((s) => s.setMode)
  const simple = mode === 'simple'
  const show3d = useAfterFirstPaint(!simple) &&
!simple

  // (Re)build scroll handling whenever the layout
changes between modes.
  useEffect(() => {
    window.scrollTo(0, 0)
    document.documentElement.classList.toggle('is-
simple', simple)
    if (simple) {
      scrollMod?.destroyScroll()
      useStore.getState().setCanvasReady(false)
      if (useStore.getState().sound) {
        setSound(false)
        useStore.getState().setSound(false)
      }
      return
    }
```

```
    let alive = true
    loadScroll().then((m) => {
      scrollMod = m
      if (alive) m.initScroll({ smooth:
!quality.reducedMotion })
    })
    return () => {
      alive = false
      scrollMod?.destroyScroll()
    }
  }, [simple])

  useEffect(() => {

document.documentElement.classList.toggle('canvas-
ready', ready && !simple)
  }, [ready, simple])

  return (
    <>
      {/* First thing a keyboard user reaches. */}
      <button type="button" className="skip"
onClick={() => setMode(simple ? 'experience' :
'simple')}>
        {simple ? 'Play the 3D version' : 'Skip the
animation: simple view'}
      </button>
      <TopBar />
      {show3d && (
        <Suspense fallback={null}>
          <Experience />
        </Suspense>
      )}
      {!simple && <Trace />}
      {!simple && <Cursor />}
      <div className="grain" aria-hidden="true" />
      {/* Filters used by CSS for hand-drawn edges
(see notebook.css). */}
      <svg className="sr-only" aria-hidden="true"
focusable="false">
        <filter id="rough">
          <feTurbulence type="fractalNoise"
baseFrequency="0.035" numOctaves="2" seed="4" />
          <feDisplacementMap in="SourceGraphic"
scale="3.2" />
        </filter>
        <filter id="rough-strong">
          <feTurbulence type="fractalNoise"
baseFrequency="0.06" numOctaves="3" seed="9" />
          <feDisplacementMap in="SourceGraphic"
scale="5" />
        </filter>
      </svg>

      <main id="main">
        {simple && quality.reducedMotion && (
          <p className="notice">
            Your system asks for reduced motion, so
you're seeing the simple view.{' '}
            <button type="button" onClick={() =>
setMode('experience')}>Play the 3D version
anyway</button>
          </p>
        )}
        <Hero />
        <Certs />
        <About />
        {simple && <h2 className="simple-
h">Work</h2>}
        {profile.projects.map((p, i) => (
          <Project key={p.slug} p={p} no={i} />
        ))}
        <Skills />
        <Contact />
      </main>
      <Terminal />
    </>
  )
}
```

src/canvas/CameraRig.jsx — 69 lines

```
/*
 * CameraRig — moves the camera along the spline
every frame.
 *
 * Scroll gives us a target parameter (scroll.camT).
We don't jump to it: we
 * damp toward it, which adds a second layer of
inertia on top of Lenis. The
 * pointer adds a few centimetres of parallax so the
world feels handheld.
 *
 * setViewOffset shifts the *projection*, not the
camera: on "side" stops the
 * scene renders right of centre to leave room for
the HTML panel on the left
 * (on phones, it shifts up to leave room for the
panel at the bottom).
 */
import { useEffect, useRef } from 'react'
import { useFrame, useThree } from '@react-
three/fiber'
import * as THREE from 'three'
import { camCurve, lookCurve, STOPS, STOP_INDEX }
from '../core/route'
import { scroll, useStore } from '../core/store'
import { damp } from './kit'
import { quality } from '../core/quality'

const pos = new THREE.Vector3()
const look = new THREE.Vector3()

export default function CameraRig() {
  const camera = useThree((s) => s.camera)
  const size = useThree((s) => s.size)
  const t = useRef(0)
  const offset = useRef(0)
  const pointer = useRef({ x: 0, y: 0, sx: 0, sy: 0
})

  useEffect(() => {
    if (!quality.finePointer) return
    const onMove = (e) => {
      pointer.current.x = (e.clientX /
window.innerWidth) * 2 - 1
      pointer.current.y = (e.clientY /
window.innerHeight) * 2 - 1
    }
    window.addEventListener('pointermove', onMove, {
passive: true })
    return () =>
window.removeEventListener('pointermove', onMove)
  }, [])

  useFrame((_, dt) => {
    dt = Math.min(dt, 0.05) // a tab coming back
from background shouldn't teleport
    t.current = damp(t.current, scroll.camT, 4.5,
dt)

    camCurve.getPoint(t.current, pos)
    lookCurve.getPoint(t.current, look)

    const p = pointer.current
    p.sx = damp(p.sx, p.x, 2.5, dt)
    p.sy = damp(p.sy, p.y, 2.5, dt)
    pos.x += p.sx * 0.35
    pos.y -= p.sy * 0.2

    camera.position.copy(pos)
    camera.lookAt(look)

    // Frame offset: blend toward the active stop's
framing.
    const stop =
```

```
STOPS[STOP_INDEX[useStore.getState().activeStop]] ||
STOPS[0]
    const target = stop.frame === 'side' ? 1 : 0
    offset.current = damp(offset.current, target, 3,
dt)
    const w = size.width
    const h = size.height
    const portrait = h > w
    const ox = portrait ? 0 : -w * 0.19 *
offset.current
    const oy = portrait ? h * 0.2 * offset.current :
0
    camera.setViewOffset(w, h, ox, oy, w, h)
  })

  return null
}
```

src/canvas/Experience.jsx 96 lines

```
/*
 * Experience — the whole 3D world in one <Canvas>.
 *
 * This file is itself lazy-loaded by App.jsx, so
the HTML content paints
 * before three.js has even downloaded.
 *
 * Scenes mount only when they're near: the active
stop, one behind and two
 * ahead. Each scene is its own code-split chunk;
the one after next is
 * prefetched so it's ready before the camera gets
there.
 */
import { Suspense, lazy, useEffect, useState } from
'react'
import { Canvas, useFrame } from '@react-
three/fiber'
import { PerformanceMonitor } from '@react-
three/drei'
import * as THREE from 'three'
import CameraRig from './CameraRig'
import Ground from './Ground'
import Packet from './Packet'
import Edge from './scenes/Edge'
import { STOPS, STOP_INDEX } from '../core/route'
import { useStore } from '../core/store'
import { quality } from '../core/quality'
import { PALETTE, boil, boilAmp } from './kit'

const loaders = {
  gate: () => import('./scenes/Gate'),
  lb: () => import('./scenes/LoadBalancer'),
  carepilot: () => import('./scenes/CarePilot'),
  rentflow: () => import('./scenes/RentFlow'),
  sensorium: () => import('./scenes/Sensorium'),
  bazaarlens: () => import('./scenes/BazaarLens'),
  'phd-erp': () => import('./scenes/PhdErp'),
  data: () => import('./scenes/DataLayer'),
  response: () => import('./scenes/Response'),
}
const Lazy =
Object.fromEntries(Object.entries(loaders).map(([k,
f]) => [k, lazy(f)]))

/* Steps the line-boil clock at 8 drawings per
second (see kit.js). */
function Boil() {
  useFrame((state) => {
    boil.value = Math.floor(state.clock.elapsedTime
* 8) % 64
  })
  return null
}
if (quality.reducedMotion) boilAmp.value = 0

function World() {
  const active = STOP_INDEX[useStore((s) =>
s.activeStop)] ?? 0

  useEffect(() => {
    STOPS.slice(active + 1, active + 4).forEach((s)
=> loaders[s.id]?.())
  }, [active])

  return (
    <>
      <color attach="background" args=
{[PALETTE.paper]} />
      <fog attach="fog" args={[PALETTE.paper, 14,
46]} />
      <CameraRig />
      <Boil />
      <Ground />
      {active <= 1 && <Edge position={STOPS[0].look}
/>}
      {STOPS.map((s, i) => {
        const C = Lazy[s.id]
        if (!C || i < active - 1 || i > active + 2)
return null
        return (
          <Suspense key={s.id} fallback={null}>
            <C position={s.look} />
          </Suspense>
        )
      })}
      <Packet />
    </>
  )
}

export default function Experience() {
  const [dpr, setDpr] = useState(quality.dpr)
  return (
    <div className="canvas-wrap" aria-hidden="true">
      <Canvas
        dpr={dpr}
        camera={{ fov: 40, near: 0.1, far: 80,
position: STOPS[0].cam }}
        gl={{ antialias: quality.tier !== 'low',
powerPreference: 'high-performance', alpha: false }}
        onCreated={({ gl }) => {
          gl.outputColorSpace = THREE.SRGBColorSpace
        }}
      >
        {/* If fps drops, step resolution down; if
it recovers, step back up. */}
        <PerformanceMonitor
          onDecline={() => setDpr((d) =>
Math.max(0.75, d - 0.25))}
          onIncline={() => setDpr((d) =>
Math.min(quality.dpr, d + 0.25))}
        />
        <World />
      </Canvas>
    </div>
  )
}
```

src/canvas/Ground.jsx 27 lines

```
/* The blueprint grid floor under the whole route.
One plane, one shader. */
import { useMemo } from 'react'
import * as THREE from 'three'
import vert from './shaders/ground.vert'
import frag from './shaders/ground.frag'
import { C } from './kit'

export default function Ground() {
  const mat = useMemo(
    () =>
      new THREE.ShaderMaterial({
        vertexShader: vert,
        fragmentShader: frag,
        transparent: true,
        depthWrite: false,
        side: THREE.DoubleSide,
```

```
        extensions: { derivatives: true },
        uniforms: { uInk: { value: C.ink },
uFadeNear: { value: 8 }, uFadeFar: { value: 38 } },
      }),
    [],
  )
  return (
    <mesh rotation-x={-Math.PI / 2} position={[0,
-1.6, -100]} material={mat} renderOrder={-1}>
      <planeGeometry args={[90, 300, 1, 1]} />
    </mesh>
  )
}
```

src/canvas/Packet.jsx 126 lines

```
/*
 * Packet — the request. It's the only thing that
exists in every scene.
 *
 * It rides a fixed spot in front of the camera
(like a cursor for the
 * journey), but in *world* space, so it physically
passes through gates and
 * leaves a trail behind it. It publishes its world
position to `packetState`
 * so the particle systems can collapse into it /
burst out of it.
 */
import { useMemo, useRef } from 'react'
import { useFrame, useThree } from '@react-
three/fiber'
import * as THREE from 'three'
import vert from './shaders/packet.vert'
import frag from './shaders/packet.frag'
import { C, mats, ringGeo } from './kit'
import { packetState, scroll } from '../core/store'
import { smooth } from '../core/route'

const OFFSET = new THREE.Vector3(0, -0.85, -3.4)
const TRAIL = 48
const RADIUS = 0.1

export default function Packet() {
  const camera = useThree((s) => s.camera)
  const group = useRef()
  const spin = useRef()
  const shell = useRef()
  const ring = useRef()
  const world = useMemo(() => new THREE.Vector3(),
[])

  const mat = useMemo(
    () =>
      new THREE.ShaderMaterial({
        vertexShader: vert,
        fragmentShader: frag,
        transparent: true,
        uniforms: {
          uTime: { value: 0 },
          uScan: { value: 3 },
          uVerified: { value: 0 },
          uOpacity: { value: 0 },
          uAccent: { value: C.accent },
          uInk: { value: C.ink },
          uPaper: { value: C.paper },
        },
      }),
    [],
  )

  // Trail: a ring buffer of past positions drawn as
a fading line.
  const trail = useMemo(() => {
    const g = new THREE.BufferGeometry()
    g.setAttribute('position', new
THREE.BufferAttribute(new Float32Array(TRAIL * 3),
3))
    const m = new THREE.LineBasicMaterial({ color:
C.accent, transparent: true, opacity: 0.6 })
    const line = new THREE.Line(g, m)
    line.frustumCulled = false
    return line
  }, [])
  const filled = useRef(false)
  const ringGeom = useMemo(() => ringGeo(RADIUS *
2.1, 40), [])

  useFrame((state, dt) => {
    const t = state.clock.elapsedTime
    const L = scroll.local

    // World position = a point in front of the
camera.

world.copy(OFFSET).applyMatrix4(camera.matrixWorld)
    group.current.position.copy(world)
    spin.current.rotation.y += dt * 0.6
    spin.current.rotation.x = Math.sin(t * 0.7) *
0.3
    packetState.position[0] = world.x
    packetState.position[1] = world.y
    packetState.position[2] = world.z

    // Visible after the hero name has collapsed,
until the response bursts it.
    const born = smooth(0.4, 0.5, L.edge ?? 0)
    const gone = smooth(0.04, 0.12, L.response ?? 0)
    const opacity = born * (1 - gone)
    packetState.opacity = opacity
    mat.uniforms.uOpacity.value = opacity
    shell.current.material.opacity = opacity * 0.9
    group.current.visible = opacity > 0.01

    // Security scan: two sweeps, one per gate,
timed to the gate section.
    const g = L.gate ?? 0
    const sweep = (a, b) => (g > a && g < b ? 1.2 -
2.4 * ((g - a) / (b - a)) : null)
    const s = sweep(0.02, 0.15) ?? sweep(0.3, 0.43)
    mat.uniforms.uScan.value = s ?? 3
    ring.current.visible = s !== null
    if (s !== null) ring.current.position.y = s *
RADIUS
    const flash = Math.max(smooth(0.15, 0.17, g) -
smooth(0.2, 0.26, g), smooth(0.45, 0.47, g) -
smooth(0.5, 0.56, g))
    mat.uniforms.uVerified.value = flash
    mat.uniforms.uTime.value = t

    // Shift the trail: oldest point drops off,
newest is the packet.
    const arr =
trail.geometry.attributes.position.array
    if (!filled.current) {
      for (let i = 0; i < TRAIL; i++)
arr.set([world.x, world.y, world.z], i * 3)
      filled.current = true
    }
    arr.copyWithin(3, 0, (TRAIL - 1) * 3)
    arr[0] = world.x
    arr[1] = world.y
    arr[2] = world.z
    trail.geometry.attributes.position.needsUpdate =
true
    trail.material.opacity = 0.55 * opacity
  })

  return (
    <>
      <group ref={group}>
        <group ref={spin}>
          <mesh material={mat}>
            <icosahedronGeometry args={[RADIUS, 2]}
/>
          </mesh>
          <lineSegments ref={shell}>
```

```
            <edgesGeometry args={[new
THREE.IcosahedronGeometry(RADIUS * 1.35, 0)]} />
            <lineBasicMaterial color={C.accent}
transparent />
          </lineSegments>
        </group>
        {/* Outside the spinning group so the scan
ring stays level */}
        <lineLoop ref={ring} geometry={ringGeom}
material={mats.ink} visible={false} />
      </group>
      <primitive object={trail} />
    </>
  )
}
```

src/canvas/ParticleText.jsx 170 lines

```
/*
 * ParticleText — text made of particles that can
collapse into the packet.
 *
 * How the text positions are made:
 *   1. Draw the words with the real web font onto
an offscreen 2D canvas.
 *   2. Read the pixels back and keep every pixel
that is "ink".
 *   3. Randomly pick N of those pixels and map them
into world units.
 * No font files or SDF atlases are needed in the 3D
scene at all.
 *
 * Used twice: the hero (name → packet) and the
response (packet → "200 OK").
 */
import { useEffect, useMemo, useRef, useState } from
'react'
import { useFrame, useThree } from '@react-
three/fiber'
import * as THREE from 'three'
import vert from './shaders/particles.vert'
import frag from './shaders/particles.frag'
import { C, rng } from './kit'
import { packetState } from '../core/store'
import { quality } from '../core/quality'

const FONT = '"Instrument Serif", Georgia, serif'

function sampleText(lines, count, seed) {
  const W = 1200
  const H = 300 * lines.length
  const cv = document.createElement('canvas')
  cv.width = W
  cv.height = H
  const ctx = cv.getContext('2d', {
willReadFrequently: true })
  ctx.fillStyle = '#fff'
  ctx.textAlign = 'center'
  ctx.textBaseline = 'middle'
  lines.forEach((line, i) => {
    let size = 290
    ctx.font = `400 ${size}px ${FONT}`
    // Shrink long lines so every line fits the
canvas width.
    const w = ctx.measureText(line).width
    if (w > W * 0.94) size *= (W * 0.94) / w
    ctx.font = `400 ${size}px ${FONT}`
    ctx.fillText(line, W / 2, 150 + i * 300)
  })
  const data = ctx.getImageData(0, 0, W, H).data
  const ink = []
  for (let y = 0; y < H; y += 2) for (let x = 0; x <
W; x += 2) if (data[(y * W + x) * 4] > 140)
ink.push(x, y)

  const r = rng(seed)
  const out = new Float32Array(count * 2)
  const n = ink.length / 2
  for (let i = 0; i < count; i++) {
    const k = Math.floor(r() * n) * 2
    out[i * 2] = (ink[k] / W - 0.5) // -0.5..0.5
across
    out[i * 2 + 1] = -(ink[k + 1] - H / 2) / W //
same scale vertically
  }
  return out
}

export default function ParticleText({
  lines,
  center = [0, 0, 0],
  getIntro, // () => 0..1
  getMorph, // () => 0..1  (0 = text, 1 = packet)
  getOpacity = () => 1,
  seed = 7,
  onReady,
}) {
  const viewport = useThree((s) => s.viewport)
  const camera = useThree((s) => s.camera)
  const [geo, setGeo] = useState(null)
  const count = quality.particles

  // Text needs the web font loaded first, or the
canvas silently uses a fallback.
  useEffect(() => {
    let alive = true
    document.fonts.load(`400 100px
${FONT}`).finally(() => {
      if (!alive) return
      const pts = sampleText(lines, count, seed)
      const r = rng(seed + 1)
      const text = new Float32Array(count * 3)
      const scatter = new Float32Array(count * 3)
      const sphere = new Float32Array(count * 3)
      const rand = new Float32Array(count)
      for (let i = 0; i < count; i++) {
        text[i * 3] = pts[i * 2]
        text[i * 3 + 1] = pts[i * 2 + 1]
        text[i * 3 + 2] = (r() - 0.5) * 0.02
        // Start scattered in a wide, shallow slab
around the text.
        scatter[i * 3] = (r() - 0.5) * 3
        scatter[i * 3 + 1] = (r() - 0.5) * 1.6
        scatter[i * 3 + 2] = (r() - 0.5) * 1.5
        // Uniform point on a unit sphere (for the
packet shape).
        const u = r() * 2 - 1
        const a = r() * Math.PI * 2
        const s = Math.sqrt(1 - u * u)
        sphere.set([s * Math.cos(a), u, s *
Math.sin(a)], i * 3)
        rand[i] = r()
      }
      const g = new THREE.BufferGeometry()
      g.setAttribute('position', new
THREE.BufferAttribute(text, 3)) // needed for
bounds; shader uses aText
      g.setAttribute('aText', new
THREE.BufferAttribute(text, 3))
      g.setAttribute('aScatter', new
THREE.BufferAttribute(scatter, 3))
      g.setAttribute('aSphere', new
THREE.BufferAttribute(sphere, 3))
      g.setAttribute('aRand', new
THREE.BufferAttribute(rand, 1))
      setGeo(g)
      onReady?.()
    })
    return () => {
      alive = false
    }
    // eslint-disable-next-line react-
hooks/exhaustive-deps
  }, [])

  const mat = useMemo(
    () =>
```

```
      new THREE.ShaderMaterial({
        vertexShader: vert,
        fragmentShader: frag,
        transparent: true,
        depthWrite: false,
        uniforms: {
          uTime: { value: 0 },
          uIntro: { value: 0 },
          uMorph: { value: 0 },
          uSize: { value: 30 },
          uPixelRatio: { value:
Math.min(window.devicePixelRatio, quality.dpr || 1)
},
          uPacket: { value: new THREE.Vector3() },
          uPacketRadius: { value: 0.17 },
          uInvScale: { value: 1 },
          uInk: { value: C.ink },
          uAccent: { value: C.accent },
          uOpacity: { value: 1 },
        },
      }),
    [],
  )

  /*
   * The particle positions are normalised (text ~1
unit wide) and the group
   * is scaled up to the real width. So the packet's
world position has to be
   * converted into the group's local space every
frame, and anything sized
   * in world units (packet radius, noise) gets
divided by the scale.
   */
  const group = useRef()
  const local = useMemo(() => new THREE.Vector3(),
[])

  // Text width: 11 units on desktop, narrower on a
portrait phone.
  const dist = new
THREE.Vector3(...center).distanceTo(camera.position)
|| 12
  const visibleW = 2 *
Math.tan(THREE.MathUtils.degToRad(camera.fov / 2)) *
dist * (viewport.aspect || 1)
  const width = Math.min(8.6, visibleW * 0.86)

  useFrame((state) => {
    if (!geo) return
    const u = mat.uniforms
    u.uTime.value = state.clock.elapsedTime
    u.uIntro.value = getIntro()
    u.uMorph.value = getMorph()
    u.uOpacity.value = getOpacity()
    group.current.visible = u.uOpacity.value > 0.01
&& u.uMorph.value < 0.999
    local.fromArray(packetState.position)
    group.current.worldToLocal(local)
    u.uPacket.value.copy(local)
    u.uPacketRadius.value = 0.1 / width
    u.uInvScale.value = 1 / width
  })

  return (
    <group ref={group} position={center} scale=
{width}>
      {geo && <points geometry={geo} material={mat}
frustumCulled={false} />}
    </group>
  )
}
```

src/canvas/kit.js 127 lines

```
/*
 * kit.js — the drawing set for the whole world.
 *
 * The art direction is a field notebook: every
object is drawn in blue
 * ballpoint and pencil on manila paper, so almost
everything is LineSegments with one of three
 * shared materials. Sharing materials means
three.js compiles 3 programs,
 * not 300, and the look stays consistent by
construction.
 */
import * as THREE from 'three'

export const PALETTE = {
  paper: '#ece3d0',  // manila
  ink: '#2a44a8',    // blue ballpoint (in 3D the
"ink" is the pen)
  faded: '#8f8676',  // pencil
  accent: '#c8372d', // red pen
}

export const C = {
  paper: new THREE.Color(PALETTE.paper),
  ink: new THREE.Color(PALETTE.ink),
  faded: new THREE.Color(PALETTE.faded),
  accent: new THREE.Color(PALETTE.accent),
}

/*
 * Line boil. Hand-drawn animation never holds a
line perfectly still: each
 * drawing is re-traced, so it shimmers slightly. We
fake that in the vertex
 * shader: every vertex gets a tiny random offset
that changes 8 times a
 * second (not every frame; the stepping is what
makes it read as "drawn").
 * The offset is hashed from the vertex position, so
two segments that share
 * a corner jitter together and never come apart.
 */
export const boil = { value: 0 }       // time,
stepped; written by <Boil/> in Experience
export const boilAmp = { value: 0.016 } // world
units; 0 for reduced motion

function boilify(mat) {
  mat.onBeforeCompile = (shader) => {
    shader.uniforms.uBoil = boil
    shader.uniforms.uBoilAmp = boilAmp
    shader.vertexShader = shader.vertexShader
      .replace(
        '#include <common>',
        `#include <common>
        uniform float uBoil;
        uniform float uBoilAmp;
        vec3 boilHash(vec3 p) {
          p = fract(p * vec3(0.1031, 0.1030,
0.0973));
          p += dot(p, p.yxz + 33.33);
          return fract((p.xxy + p.yxx) * p.zyx) -
0.5;
        }`,
      )
      .replace(
        '#include <begin_vertex>',
        `#include <begin_vertex>
        transformed += boilHash(position * 7.13 +
uBoil * 1.7) * uBoilAmp;`,
      )
  }
  mat.customProgramCacheKey = () => 'boil'
  return mat
}

/** A boiling line material. Use this instead of
`new LineBasicMaterial`. */
export const lineMat = (color, opacity = 1) =>
  boilify(new THREE.LineBasicMaterial({ color,
transparent: opacity < 1, opacity }))
```

```
/** Copy of a shared line material (when one object
needs its own colour). */
export const cloneLine = (m) =>
lineMat(m.color.clone(), m.opacity)

export const mats = {
  ink: lineMat(C.ink, 0.9),
  faded: lineMat(C.faded, 0.75),
  accent: lineMat(C.accent),
  dotAccent: new THREE.MeshBasicMaterial({ color:
C.accent }),
  dotInk: new THREE.MeshBasicMaterial({ color: C.ink
}),
}

/** Wireframe edges of a box, as reusable geometry.
*/
export const boxEdges = (w, h, d) => new
THREE.EdgesGeometry(new THREE.BoxGeometry(w, h, d))

/** A circle outline in the XZ plane (a "plan view"
ring). */
export function ringGeo(r, seg = 48) {
  const pts = []
  for (let i = 0; i <= seg; i++) {
    const a = (i / seg) * Math.PI * 2
    pts.push(new THREE.Vector3(Math.cos(a) * r, 0,
Math.sin(a) * r))
  }
  return new
THREE.BufferGeometry().setFromPoints(pts)
}

/** Geometry for LineSegments from pairs of points
[[a,b],[a,b]...]. */
export function segmentsGeo(pairs) {
  const arr = new Float32Array(pairs.length * 6)
  pairs.forEach(([a, b], i) => arr.set([...a, ...b],
i * 6))
  const g = new THREE.BufferGeometry()
  g.setAttribute('position', new
THREE.BufferAttribute(arr, 3))
  return g
}

/** Merge many EdgesGeometry boxes into one
LineSegments geometry (one draw call). */
export function mergeBoxes(boxes) {
  const parts = boxes.map(({ size, pos }) => {
    const g = boxEdges(...size)
    g.translate(...pos)
    return g.attributes.position.array
  })
  const total = parts.reduce((n, a) => n + a.length,
0)
  const arr = new Float32Array(total)
  let o = 0
  parts.forEach((a) => {
    arr.set(a, o)
    o += a.length
  })
  const g = new THREE.BufferGeometry()
  g.setAttribute('position', new
THREE.BufferAttribute(arr, 3))
  return g
}

/** Deterministic random, so the world is the same
on every visit. */
export function rng(seed = 1) {
  let s = seed >>> 0
  return () => {
    s = (s * 1664525 + 1013904223) >>> 0
    return s / 4294967296
  }
}
export const damp = (a, b, lambda, dt) => a + (b -
a) * (1 - Math.exp(-lambda * dt))
```

src/canvas/scenes/BazaarLens.jsx 70 lines

```
/*
 * BazaarLens. Four store lanes. A product box races
down each lane; its
 * arrival time is how fast that store answered.
Above each box a bar shows
 * its price. Once every lane has answered, the
CHEAPEST lane lights up,
 * not the fastest one. Then a new race starts. All
values are random.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { C, boxEdges, cloneLine, mats, rng,
segmentsGeo } from '../kit'

const LANES = 4
const LEN = 7.5

export default function BazaarLens({ position }) {
  const r = useMemo(() => rng(5), [])
  const track = useMemo(() => {
    const pairs = []
    for (let i = 0; i <= LANES; i++) pairs.push([[-
LEN / 2, -1.2, i * 1.1 - 2.2], [LEN / 2, -1.2, i *
1.1 - 2.2]])
    pairs.push([[LEN / 2, -1.2, -2.4], [LEN / 2,
-1.2, 2.4]]) // finish line
    pairs.push([[-LEN / 2, -1.2, -2.4], [-LEN / 2,
-1.2, 2.4]]) // start line
    return segmentsGeo(pairs)
  }, [])
  const box = useMemo(() => boxEdges(0.45, 0.45,
0.45), [])
  const bar = useMemo(() => boxEdges(0.12, 1, 0.12),
[])
  const race = useRef(null)
  const boxes = useRef([])
  const bars = useRef([])
  const laneMats = useMemo(() => Array.from({
length: LANES }, () => cloneLine(mats.ink)), [])

  const newRace = (t) => ({
    start: t,
    dur: Array.from({ length: LANES }, () => 1.2 +
r() * 2.2),
    price: Array.from({ length: LANES }, () => 0.3 +
r() * 1.2),
  })

  useFrame((s) => {
    const t = s.clock.elapsedTime
    if (!race.current) race.current = newRace(t)
    const R = race.current
    const e = t - R.start
    const allIn = Math.max(...R.dur)
    const cheapest =
R.price.indexOf(Math.min(...R.price))
    for (let i = 0; i < LANES; i++) {
      const k = Math.min(1, e / R.dur[i])
      const eased = 1 - Math.pow(1 - k, 2)
      const x = -LEN / 2 + 0.3 + eased * (LEN - 0.6)
      const z = i * 1.1 - 1.65
      boxes.current[i].position.set(x, -0.95, z)
      boxes.current[i].rotation.x = k < 1 ? e * 6 :
0
      bars.current[i].position.set(x, -0.6 +
R.price[i] / 2, z)
      bars.current[i].scale.y = R.price[i]
      const won = e > allIn + 0.2 && i === cheapest
      laneMats[i].color.copy(won ? C.accent : k < 1
? C.faded : C.ink)
    }
    if (e > allIn + 2.6) race.current = newRace(t)
```

```
  })

  return (
    <group position={[position[0] - 1.4,
position[1], position[2] + 1]} rotation-y={0.22}>
      <lineSegments geometry={track} material=
{mats.faded} />
      {Array.from({ length: LANES }, (_, i) => (
        <group key={i}>
          <lineSegments ref={(el) =>
(boxes.current[i] = el)} geometry={box} material=
{laneMats[i]} />
          <lineSegments ref={(el) =>
(bars.current[i] = el)} geometry={bar} material=
{laneMats[i]} />
        </group>
      ))}
    </group>
  )
}
```

src/canvas/scenes/CarePilot.jsx 102 lines

```
/*
 * CarePilot. Hospitals as a node graph; records hop
along the edges.
 * Each node is linked to its 2 nearest neighbours.
A record picks a random
 * neighbour when it arrives somewhere, and the node
it lands on pulses.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { mats, ringGeo, rng, segmentsGeo } from
'../kit'
import { quality } from '../../core/quality'

const NODES = 14
const RECORDS = quality.tier === 'low' ? 6 : 12

function buildGraph() {
  const r = rng(42)
  const nodes = []
  for (let i = 0; i < NODES; i++) {
    const a = r() * Math.PI * 2
    const d = 1.2 + r() * 4.2
    nodes.push(new THREE.Vector3(Math.cos(a) * d,
(r() - 0.5) * 1.2, Math.sin(a) * d))
  }
  const adj = nodes.map(() => new Set())
  nodes.forEach((n, i) => {
    nodes
      .map((m, j) => [j, n.distanceTo(m)])
      .filter(([j]) => j !== i)
      .sort((a, b) => a[1] - b[1])
      .slice(0, 2)
      .forEach(([j]) => {
        adj[i].add(j)
        adj[j].add(i)
      })
  })
  const pairs = []
  adj.forEach((s, i) => s.forEach((j) => j > i &&
pairs.push([nodes[i].toArray(),
nodes[j].toArray()])))
  return { nodes, adj: adj.map((s) => [...s]),
edges: segmentsGeo(pairs) }
}

export default function CarePilot({ position }) {
  const graph = useMemo(buildGraph, [])
  const ring = useMemo(() => ringGeo(0.32, 32), [])
  const oct = useMemo(() => new
THREE.EdgesGeometry(new
THREE.OctahedronGeometry(0.16, 0)), [])
  const ringRefs = useRef([])
  const hits = useRef(new Array(NODES).fill(-10))
  const dots = useRef()
  const dummy = useMemo(() => new THREE.Object3D(),
[])
  const r = useMemo(() => rng(7), [])
  // Each record: from node, to node, progress,
speed
  const recs = useMemo(
    () =>
      Array.from({ length: RECORDS }, (_, i) => {
        const from = i % NODES
        return { from, to: graph.adj[from][0], k:
r(), speed: 0.5 + r() * 0.6 }
      }),
    [graph, r],
  )

  useFrame((state, dt) => {
    const t = state.clock.elapsedTime
    recs.forEach((rec, i) => {
      rec.k += dt * rec.speed
      if (rec.k >= 1) {
        hits.current[rec.to] = t
        rec.from = rec.to
        const opts = graph.adj[rec.from]
        rec.to = opts[Math.floor(r() * opts.length)]
        rec.k = 0
      }
      const a = graph.nodes[rec.from]
      const b = graph.nodes[rec.to]
      // A small arc: records "hop" rather than
slide.
      dummy.position.lerpVectors(a, b, rec.k)
      dummy.position.y += Math.sin(rec.k * Math.PI)
* 0.6
      dummy.scale.setScalar(0.07)
      dummy.updateMatrix()
      dots.current.setMatrixAt(i, dummy.matrix)
    })
    dots.current.instanceMatrix.needsUpdate = true
    ringRefs.current.forEach((el, i) => {
      if (!el) return
      const p = Math.exp(-(t - hits.current[i]) * 3)
      el.scale.setScalar(1 + p * 0.9)
      el.material = p > 0.2 ? mats.accent :
mats.faded
    })
  })

  return (
    <group position={position}>
      <lineSegments geometry={graph.edges} material=
{mats.faded} />
      {graph.nodes.map((n, i) => (
        <group key={i} position={n}>
          <lineSegments geometry={oct} material=
{mats.ink} />
          <lineLoop ref={(el) =>
(ringRefs.current[i] = el)} geometry={ring}
material={mats.faded} />
        </group>
      ))}
      <instancedMesh ref={dots} args={[undefined,
undefined, RECORDS]} material={mats.dotAccent}>
        <octahedronGeometry args={[1, 0]} />
      </instancedMesh>
    </group>
  )
}
```

src/canvas/scenes/DataLayer.jsx 48 lines

```
/*
 * Data layer (skills). Below the drafting-table
floor: a column of storage
 * shards (rings with spokes) turning at different
speeds, threaded by index
 * lines. The packet dives through the floor to get
here.
 */
import { useMemo, useRef } from 'react'
```

```
import { useFrame } from '@react-three/fiber'
import { mats, ringGeo, segmentsGeo } from '../kit'

const LAYERS = 5

export default function DataLayer({ position }) {
  const ring = useMemo(() => ringGeo(2.4, 72), [])
  const inner = useMemo(() => ringGeo(1.3, 48), [])
  const spokes = useMemo(() => {
    const pairs = []
    for (let i = 0; i < 12; i++) {
      const a = (i / 12) * Math.PI * 2
      pairs.push([[Math.cos(a) * 1.3, 0, Math.sin(a)
* 1.3], [Math.cos(a) * 2.4, 0, Math.sin(a) * 2.4]])
    }
    return segmentsGeo(pairs)
  }, [])
  const index = useMemo(
    () => segmentsGeo([0, 1, 2, 3].map((i) => {
      const a = (i / 4) * Math.PI * 2 + 0.4
      return [[Math.cos(a) * 1.85, 3.2, Math.sin(a)
* 1.85], [Math.cos(a) * 1.85, -3.2, Math.sin(a) *
1.85]]
    })),
    [],
  )
  const layers = useRef([])

  useFrame((_, dt) => {
    layers.current.forEach((l, i) => l &&
(l.rotation.y += dt * (i % 2 ? -0.12 : 0.18) * (1 +
i * 0.15)))
  })

  return (
    <group position={position}>
      <lineSegments geometry={index} material=
{mats.faded} />
      {Array.from({ length: LAYERS }, (_, i) => (
        <group key={i} ref={(el) =>
(layers.current[i] = el)} position-y={(i - (LAYERS -
1) / 2) * 1.4}>
          <lineLoop geometry={ring} material={i ===
2 ? mats.accent : mats.ink} />
          <lineLoop geometry={inner} material=
{mats.faded} />
          <lineSegments geometry={spokes} material=
{mats.faded} />
        </group>
      ))}
    </group>
  )
}
```

src/canvas/scenes/Edge.jsx 38 lines

```
/*
 * Edge (hero). The name assembles from scattered
ink over ~2.4s once the
 * particles are ready, then scrolling collapses it
into the packet that
 * starts the journey.
 */
import { useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import ParticleText from '../ParticleText'
import { profile } from '../../content/profile'
import { scroll, useStore } from '../../core/store'
import { smooth } from '../../core/route'
import { quality } from '../../core/quality'

export default function Edge({ position }) {
  const ready = useRef(false)
  const start = useRef(null)
  const intro = useRef(quality.reducedMotion ? 1 :
0)

  useFrame((state) => {
    if (!ready.current || quality.reducedMotion)
return
    if (start.current === null) start.current =
state.clock.elapsedTime
    intro.current = Math.min(1,
(state.clock.elapsedTime - start.current) / 2.4)
  })

  return (
    <ParticleText
      lines={profile.particleName}
      center={[position[0], position[1] + 1.15,
position[2]]}
      seed={11}
      getIntro={() => intro.current}
      getMorph={() => smooth(0.02, 0.46,
scroll.local.edge ?? 0)}
      onReady={() => {
        ready.current = true
        useStore.getState().setCanvasReady(true)
      }}
    />
  )
}
```

src/canvas/scenes/Gate.jsx 99 lines

```
/*
 * Security Gate. Two physical gates sit on the
camera path, one per
 * certification. Each holds a hatched "membrane"
(dissolve shader) that burns
 * away once the packet has been scanned and that
cert is checked.
 * The scan itself is drawn by the packet's own
shader (see Packet.jsx).
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import vert from '../shaders/dissolve.vert'
import frag from '../shaders/dissolve.frag'
import { C, cloneLine, mats, segmentsGeo } from
'../kit'
import { camCurve, smooth, tOf, STOP_INDEX } from
'../../core/route'
import { scroll } from '../../core/store'

const W = 3.4
const H = 2.6

// Where along the gate→lb stretch each gate stands,
and when it opens.
const GATES = [
  { along: 0.3, open: [0.15, 0.22] },
  { along: 0.62, open: [0.45, 0.52] },
]

function frameGeo() {
  const w = W / 2
  const h = H / 2
  const t = 0.25 // corner tick length
  return segmentsGeo([
    [[-w, -h, 0], [w, -h, 0]], [[w, -h, 0], [w, h,
0]], [[w, h, 0], [-w, h, 0]], [[-w, h, 0], [-w, -h,
0]],
    // outer registration ticks, like crop marks on
a drawing
    [[-w - t, h + 0.12, 0], [-w - 0.05, h + 0.12,
0]], [[w + 0.05, h + 0.12, 0], [w + t, h + 0.12,
0]],
    [[-w - t, -h - 0.12, 0], [-w - 0.05, -h - 0.12,
0]], [[w + 0.05, -h - 0.12, 0], [w + t, -h - 0.12,
0]],
    // pylons down to the floor
    [[-w, -h, 0], [-w, -h - 0.4, 0]], [[w, -h, 0],
[w, -h - 0.4, 0]],
  ])
}
```

```
function OneGate({ along, open }) {
  const i = STOP_INDEX.gate
  const t = tOf(i) + (tOf(i + 1) - tOf(i)) * along
  const pos = camCurve.getPoint(t)
  const tan = camCurve.getTangent(t)
  const yaw = Math.atan2(tan.x, tan.z)
  pos.y -= 0.75

  const frame = useMemo(frameGeo, [])
  const frameMat = useMemo(() =>
cloneLine(mats.faded), [])
  const mat = useMemo(
    () =>
      new THREE.ShaderMaterial({
        vertexShader: vert,
        fragmentShader: frag,
        transparent: true,
        depthWrite: false,
        side: THREE.DoubleSide,
        uniforms: {
          uTime: { value: 0 },
          uDissolve: { value: 0 },
          uAccent: { value: C.accent },
          uInk: { value: C.ink },
          uAspect: { value: new THREE.Vector2(W, H)
},
        },
      }),
    [],
  )
  const membrane = useRef()

  useFrame((state) => {
    const d = smooth(open[0], open[1],
scroll.local.gate ?? 0)
    mat.uniforms.uDissolve.value = d
    mat.uniforms.uTime.value =
state.clock.elapsedTime
    membrane.current.visible = d < 0.999
    frameMat.color.copy(C.faded).lerp(C.ink, d)
    frameMat.opacity = 0.75 + 0.25 * d
  })

  return (
    <group position={pos} rotation-y={yaw}>
      <lineSegments geometry={frame} material=
{frameMat} />
      <mesh ref={membrane} material={mat}>
        <planeGeometry args={[W, H]} />
      </mesh>
    </group>
  )
}

export default function Gate() {
  const ref = useRef()
  // Hidden while the hero name is on screen, so the
gates don't show through the letters.
  useFrame(() => (ref.current.visible =
(scroll.local.edge ?? 0) > 0.45 ||
(scroll.local.gate ?? 0) > 0))
  return (
    <group ref={ref}>
      {GATES.map((g, k) => (
        <OneGate key={k} {...g} />
      ))}
    </group>
  )
}
```

src/canvas/scenes/LoadBalancer.jsx 66 lines

```
/*
 * Load Balancer (About). A hub routes incoming dots
round-robin to three
 * servers. The three servers are the three things
my time is split between
 * right now (study, teaching, building), which is
what the About panel says.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { boxEdges, cloneLine, mats, ringGeo,
segmentsGeo } from '../kit'

const SERVERS = [
  [-3, 0.4, -3],
  [0, 0.4, -4],
  [3, 0.4, -3],
]
const DOTS = 9

export default function LoadBalancer({ position }) {
  const hubRing = useRef()
  const dots = useRef()
  const boxes = useRef([])
  const dummy = useMemo(() => new THREE.Object3D(),
[])
  const ring1 = useMemo(() => ringGeo(1.1, 64), [])
  const ring2 = useMemo(() => ringGeo(0.7, 6), [])
  const box = useMemo(() => boxEdges(1.2, 1.6, 1.2),
[])
  const links = useMemo(() => segmentsGeo([[[0, 0,
6], [0, 0, 1.1]], ...SERVERS.map((s) => [[0, 0, 0],
s])]), [])
  const lastHit = useRef([0, 0, 0])
  const boxMats = useMemo(() => SERVERS.map(() =>
cloneLine(mats.faded)), [])

  useFrame((state, dt) => {
    const t = state.clock.elapsedTime
    hubRing.current.rotation.y += dt * 0.4
    for (let i = 0; i < DOTS; i++) {
      // Each dot: 0..0.5 travels in to the hub,
0.5..1 travels out to its server.
      const k = (t * 0.35 + i / DOTS) % 1
      const server = SERVERS[i % 3]
      if (k < 0.5) dummy.position.set(0, 0, 6 - 12 *
k)
      else dummy.position.set(server[0] * (k - 0.5)
* 2, server[1] * (k - 0.5) * 2, server[2] * (k -
0.5) * 2)
      if (k > 0.97) lastHit.current[i % 3] = t
      dummy.scale.setScalar(0.07)
      dummy.updateMatrix()
      dots.current.setMatrixAt(i, dummy.matrix)
    }
    dots.current.instanceMatrix.needsUpdate = true
    boxMats.forEach((m, i) => {
      const glow = Math.exp(-(t -
lastHit.current[i]) * 5)
      m.opacity = 0.5 + glow * 0.5
    })
  })

  return (
    <group position={position}>
      <group ref={hubRing}>
        <lineLoop geometry={ring1} material=
{mats.ink} />
        <lineLoop geometry={ring2} material=
{mats.accent} />
      </group>
      <lineSegments geometry={links} material=
{mats.faded} />
      {SERVERS.map((s, i) => (
        <lineSegments key={i} ref={(el) =>
(boxes.current[i] = el)} geometry={box} material=
{boxMats[i]} position={[s[0], s[1] + 0.4, s[2]]} />
      ))}
      <instancedMesh ref={dots} args={[undefined,
undefined, DOTS]} material={mats.dotAccent}>
        <octahedronGeometry args={[1, 0]} />
      </instancedMesh>
    </group>
```

```
  )
}
```

src/canvas/scenes/PhdErp.jsx 70 lines

```
/*
 * DYPIU PhD ERP. A row of stations along a gentle arc. A student token moves
 * through them as you scroll; each station it passes gets "stamped" (a tick
 * appears and its frame turns from faded to ink). Stations are unnamed on
 * purpose: the real workflow stages belong in the case study, not in a guess.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { C, cloneLine, mats, segmentsGeo } from '../kit'
import { scroll } from '../../core/store'
import { smooth } from '../../core/route'

const N = 6

export default function PhdErp({ position }) {
  const stations = useMemo(
    () => Array.from({ length: N }, (_, i) => {
      const a = (i / (N - 1) - 0.5) * 1.6
      return new THREE.Vector3(Math.sin(a) * 6, 0, -Math.cos(a) * 6 + 5)
    }),
    [],
  )
  const frame = useMemo(
    () => segmentsGeo([
      [[-0.55, -1.5, 0], [-0.55, 0.5, 0]], [[0.55, -1.5, 0], [0.55, 0.5, 0]], [[-0.55, 0.5, 0], [0.55, 0.5, 0]],
      [[-0.75, -1.5, 0], [0.75, -1.5, 0]],
    ]),
    [],
  )
  const tick = useMemo(() => segmentsGeo([[[-0.2, 0.85, 0], [-0.05, 0.7, 0]], [[-0.05, 0.7, 0], [0.25, 1.05, 0]]]), [])
  const path = useMemo(() => new THREE.CatmullRomCurve3(stations), [stations])
  const pathGeo = useMemo(() => new THREE.BufferGeometry().setFromPoints(path.getPoints(80).map((p) => p.setY(-1.5))), [path])
  const frameMats = useMemo(() => stations.map(() => cloneLine(mats.faded)), [stations])
  const ticks = useRef([])
  const token = useRef()
  const shown = useRef(0)

  useFrame((s, dt) => {
    const target = smooth(0.05, 0.62, scroll.local['phd-erp'] ?? 0)
    shown.current += (target - shown.current) * (1 - Math.exp(-dt * 5))
    const k = shown.current
    path.getPoint(k, token.current.position)
    token.current.position.y = -1.2 + Math.abs(Math.sin(k * Math.PI * (N - 1))) * 0.25
    token.current.rotation.y += dt
    stations.forEach((_, i) => {
      const done = k >= i / (N - 1) - 0.001
      frameMats[i].color.copy(done ? C.ink : C.faded)
      ticks.current[i].visible = done && k > 0.001
    })
  })

  return (
    <group position={position} rotation-y={0.3}>
      <line geometry={pathGeo} material={mats.faded} />
      {stations.map((p, i) => {
        const yaw = Math.atan2(-p.x, 5 - p.z)
        return (
          <group key={i} position={p} rotation-y={yaw}>
            <lineSegments geometry={frame} material={frameMats[i]} />
            <lineSegments ref={(el) => (ticks.current[i] = el)} geometry={tick} material={mats.accent} visible={false} />
          </group>
        )
      })}
      <mesh ref={token} material={mats.dotAccent}>
        <boxGeometry args={[0.28, 0.28, 0.28]} />
      </mesh>
    </group>
  )
}
```

src/canvas/scenes/RentFlow.jsx 91 lines

```
/*
 * RentFlow. A five-storey building, three units per floor. Each unit's
 * front face is an instance of one plane whose colour shows its rent state.
 * States re-roll as you scroll (one "month" per slice of the section)
 * plus the occasional change on their own, so the building feels lived in.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { C, mats, mergeBoxes, rng, segmentsGeo } from '../kit'
import { scroll } from '../../core/store'

const COLS = 3
const FLOORS = 5
const U = { w: 1.1, h: 0.8, d: 1.1 }
const STATES = [
  { color: C.ink, opacity: 0.32 },     // settled
  { color: C.faded, opacity: 0.3 },    // pending
  { color: C.accent, opacity: 0.75 },  // overdue
  { color: C.paper, opacity: 0.0 },    // vacant
]

export default function RentFlow({ position }) {
  const units = useMemo(() => {
    const list = []
    for (let f = 0; f < FLOORS; f++)
      for (let c = 0; c < COLS; c++) list.push([(c - 1) * U.w, f * U.h + U.h / 2 - 1.6, 0])
    return list
  }, [])
  const frame = useMemo(() => mergeBoxes(units.map((p) => ({ size: [U.w, U.h, U.d], pos: p }))), [units])
  const roof = useMemo(
    () => segmentsGeo([
      [[-1.9, FLOORS * U.h - 1.6, 0.55], [0, FLOORS * U.h - 0.9, 0.55]],
      [[0, FLOORS * U.h - 0.9, 0.55], [1.9, FLOORS * U.h - 1.6, 0.55]],
      [[-3, -1.6, 1.4], [3, -1.6, 1.4]],
    ]),
    [],
  )
  const fills = useRef()
  const r = useMemo(() => rng(3), [])
  const state = useRef(units.map(() => Math.floor(r() * 3)))
  const month = useRef(-1)
  const dummy = useMemo(() => new THREE.Object3D(), [])
  const mat = useMemo(() => new THREE.MeshBasicMaterial({ transparent: true,
```

```
depthWrite: false, side: THREE.DoubleSide }), [])
  const col = useMemo(() => new THREE.Color(), [])
  const nextFlip = useRef(0)

  useFrame((s) => {
    const t = s.clock.elapsedTime
    const m = Math.floor((scroll.local.rentflow ??
0) * 8)
    let dirty = false
    if (m !== month.current) {
      month.current = m
      state.current = state.current.map(() => {
        const x = r()
        return x < 0.55 ? 0 : x < 0.8 ? 1 : x < 0.92
? 2 : 3
      })
      dirty = true
    }
    if (t > nextFlip.current) {
      const i = Math.floor(r() * units.length)
      state.current[i] = (state.current[i] + 1) % 3
      nextFlip.current = t + 0.6 + r() * 1.2
      dirty = true
    }
    if (!dirty && fills.current.userData.init)
return
    fills.current.userData.init = true
    units.forEach((p, i) => {
      const st = STATES[state.current[i]]
      dummy.position.set(p[0], p[1], U.d / 2 +
0.001)
      dummy.scale.set(U.w * 0.86, U.h * 0.78, 1)
      dummy.updateMatrix()
      fills.current.setMatrixAt(i, dummy.matrix)
      // Opacity is faked by mixing toward the paper
colour.
      col.copy(C.paper).lerp(st.color, st.opacity)
      fills.current.setColorAt(i, col)
    })
    fills.current.instanceMatrix.needsUpdate = true
    if (fills.current.instanceColor)
fills.current.instanceColor.needsUpdate = true
  })

  return (
    <group position={position} rotation-y={-0.5}>
      <lineSegments geometry={frame} material=
{mats.ink} />
      <lineSegments geometry={roof} material=
{mats.faded} />
      <instancedMesh ref={fills} args={[undefined,
undefined, units.length]} material={mat}>
        <planeGeometry args={[1, 1]} />
      </instancedMesh>
    </group>
  )
}
```

src/canvas/scenes/Response.jsx 44 lines

```
/*
 * Response. The packet arrives back and bursts into
"200 OK", the same
 * particle system as the hero running backwards.
Two expanding rings mark
 * the return trip.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import ParticleText from '../ParticleText'
import { scroll } from '../../core/store'
import { smooth } from '../../core/route'
import { cloneLine, mats, ringGeo } from '../kit'

export default function Response({ position }) {
  const rings = useRef([])
  const geo = useMemo(() => ringGeo(1, 64), [])
  const ringMats = useMemo(() =>
[cloneLine(mats.faded), cloneLine(mats.faded)], [])

  useFrame((state) => {
    const t = state.clock.elapsedTime
    rings.current.forEach((r, i) => {
      if (!r) return
      const k = ((t * 0.25 + i * 0.5) % 1)
      r.scale.setScalar(1 + k * 7)
      r.material.opacity = (1 - k) * 0.5
    })
  })

  return (
    <group>
      <ParticleText
        lines={['200 OK']}
        center={[position[0], position[1] + 0.6,
position[2]]}
        seed={23}
        getIntro={() => 1}
        getMorph={() => 1 - smooth(0.02, 0.26,
scroll.local.response ?? 0)}
      />
      <group position={position} rotation-x={Math.PI
/ 2}>
        {[0, 1].map((i) => (
          <lineLoop key={i} ref={(el) =>
(rings.current[i] = el)} geometry={geo} material=
{ringMats[i]} />
        ))}
      </group>
    </group>
  )
}
```

src/canvas/scenes/Sensorium.jsx 109 lines

```
/*
 * Sensorium Nexus. Devices on the floor send
readings up into a wall of
 * panels. Each panel holds a live line chart: its
vertex buffer is shifted
 * left by one sample and a new reading is pushed
every few frames.
 * Readings are a mix of slow sine + noise, i.e.
obviously simulated.
 */
import { useMemo, useRef } from 'react'
import { useFrame } from '@react-three/fiber'
import * as THREE from 'three'
import { mats, rng, segmentsGeo } from '../kit'

const PANELS = 6
const SAMPLES = 64
const PW = 2.2
const PH = 1.0

export default function Sensorium({ position }) {
  const r = useMemo(() => rng(9), [])
  const layout = useMemo(
    () =>
      Array.from({ length: PANELS }, (_, i) => {
        const col = i % 3
        const row = Math.floor(i / 3)
        const a = (col - 1) * 0.45
        return {
          pos: [Math.sin(a) * 5.2, 0.9 + row * 1.4,
-Math.cos(a) * 5.2 + 4],
          rotY: -a,
          freq: 0.4 + r() * 1.4,
          phase: r() * 10,
          device: [(col - 1) * 1.6 + (row - 0.5) *
0.7, -1.5, 1.8 + row * 0.6],
        }
      }),
    [r],
  )
  const frame = useMemo(() => {
    const w = PW / 2
    const h = PH / 2
```

```
    return segmentsGeo([
      [[-w, -h, 0], [w, -h, 0]], [[w, -h, 0], [w, h,
0]], [[w, h, 0], [-w, h, 0]], [[-w, h, 0], [-w, -h,
0]],
      [[-w, 0, 0], [-w + 0.08, 0, 0]], [[-w, h *
0.5, 0], [-w + 0.05, h * 0.5, 0]], [[-w, -h * 0.5,
0], [-w + 0.05, -h * 0.5, 0]],
    ])
  }, [])
  const charts = useMemo(
    () =>
      layout.map(() => {
        const g = new THREE.BufferGeometry()
        const arr = new Float32Array(SAMPLES * 3)
        for (let i = 0; i < SAMPLES; i++) arr[i * 3]
= -PW / 2 + 0.06 + (i / (SAMPLES - 1)) * (PW - 0.12)
        g.setAttribute('position', new
THREE.BufferAttribute(arr, 3))
        return g
      }),
    [layout],
  )
  const wires = useMemo(() =>
segmentsGeo(layout.map((l) => [l.device, [l.pos[0],
l.pos[1] - PH / 2, l.pos[2]]])), [layout])
  const dots = useRef()
  const dummy = useMemo(() => new THREE.Object3D(),
[])
  const tick = useRef(0)

  useFrame((s, dt) => {
    const t = s.clock.elapsedTime
    tick.current += dt
    if (tick.current > 1 / 30) {
      tick.current = 0
      charts.forEach((g, p) => {
        const a = g.attributes.position.array
        for (let i = 0; i < SAMPLES - 1; i++) a[i *
3 + 1] = a[(i + 1) * 3 + 1]
        const L = layout[p]
        const v = Math.sin(t * L.freq + L.phase) *
0.22 + Math.sin(t * L.freq * 3.7) * 0.08 + (r() -
0.5) * 0.12
        a[(SAMPLES - 1) * 3 + 1] = Math.max(-PH / 2
+ 0.05, Math.min(PH / 2 - 0.05, v))
        g.attributes.position.needsUpdate = true
      })
    }
    // Readings travel up the wires.
    layout.forEach((l, i) => {
      for (let k = 0; k < 2; k++) {
        const q = (t * 0.6 + i * 0.17 + k * 0.5) % 1
        dummy.position.set(
          l.device[0] + (l.pos[0] - l.device[0]) *
q,
          l.device[1] + (l.pos[1] - PH / 2 -
l.device[1]) * q,
          l.device[2] + (l.pos[2] - l.device[2]) *
q,
        )
        dummy.scale.setScalar(0.05)
        dummy.updateMatrix()
        dots.current.setMatrixAt(i * 2 + k,
dummy.matrix)
      }
    })
    dots.current.instanceMatrix.needsUpdate = true
  })

  return (
    <group position={position} rotation-y={0.35}>
      {layout.map((l, i) => (
        <group key={i} position={l.pos} rotation-y=
{l.rotY}>
          <lineSegments geometry={frame} material=
{mats.faded} />
          <line geometry={charts[i]} material={i ===
1 ? mats.accent : mats.ink} frustumCulled={false} />
        </group>
      ))}
      <lineSegments geometry={wires} material=
{mats.faded} />
      {layout.map((l, i) => (
        <mesh key={i} position={l.device} material=
{mats.dotInk}>
          <boxGeometry args={[0.18, 0.18, 0.18]} />
        </mesh>
      ))}
      <instancedMesh ref={dots} args={[undefined,
undefined, PANELS * 2]} material={mats.dotAccent}>
        <octahedronGeometry args={[1, 0]} />
      </instancedMesh>
    </group>
  )
}
```

src/canvas/shaders/dissolve.frag 32 lines

```
// The locked membrane inside each security gate.
// Drawn as blueprint hatching (diagonal lines).
When a certification checks
// out, uDissolve goes 0→1 and noise decides which
pixels burn away first;
// a thin ink-white rim glows along the burning
edge.
#include "./noise.glsl"

uniform float uTime;
uniform float uDissolve;
uniform vec3 uAccent;
uniform vec3 uInk;
uniform vec2 uAspect;

varying vec2 vUv;

void main() {
  vec2 p = vUv * uAspect;
  float n = snoise(vec3(p * 1.6, 0.0)) * 0.5 + 0.5;
  n = n * 0.75 + 0.25 * (snoise(vec3(p * 6.0, 3.0))
* 0.5 + 0.5);

  float cut = uDissolve * 1.15 - 0.05;
  if (n < cut) discard;

  // 45° hatching, the way a section cut is drawn on
a blueprint.
  float hatch = smoothstep(0.42, 0.5, abs(fract((p.x
+ p.y) * 9.0) - 0.5));
  // A slow scanline sweeping while locked.
  float sweep = smoothstep(0.02, 0.0,
abs(fract(vUv.y - uTime * 0.25) - 0.5)) * (1.0 -
uDissolve);

  float edge = smoothstep(cut + 0.06, cut, n) *
step(0.001, uDissolve);
  vec3 col = mix(uAccent, uInk, max(edge, sweep));
  float a = hatch * 0.55 + sweep * 0.5 + edge;
  gl_FragColor = vec4(col, a * (1.0 - uDissolve *
0.3));
}
```

src/canvas/shaders/dissolve.vert 5 lines

```
varying vec2 vUv;
void main() {
  vUv = uv;
  gl_Position = projectionMatrix * modelViewMatrix *
vec4(position, 1.0);
}
```

src/canvas/shaders/ground.frag 23 lines

```
// The drafting-table floor: a minor 1-unit grid and
a major 5-unit grid,
// anti-aliased with fwidth() so lines stay one
pixel wide at any distance,
// then faded out around the camera so it never
reads as a hard horizon.
```

```
uniform vec3 uInk;
uniform float uFadeNear;
uniform float uFadeFar;

varying vec3 vWorld;

float gridLine(vec2 p, float size) {
  vec2 g = abs(fract(p / size - 0.5) - 0.5) /
fwidth(p / size);
  return 1.0 - min(min(g.x, g.y), 1.0);
}

void main() {
  float minor = gridLine(vWorld.xz, 1.0);
  float major = gridLine(vWorld.xz, 5.0);
  float d = distance(vWorld.xz, cameraPosition.xz);
  float fade = 1.0 - smoothstep(uFadeNear, uFadeFar,
d);
  float a = (minor * 0.06 + major * 0.14) * fade;
  if (a < 0.003) discard;
  gl_FragColor = vec4(uInk, a);
}
```

src/canvas/shaders/ground.vert 6 lines

```
varying vec3 vWorld;
void main() {
  vec4 w = modelMatrix * vec4(position, 1.0);
  vWorld = w.xyz;
  gl_Position = projectionMatrix * viewMatrix * w;
}
```

src/canvas/shaders/noise.glsl 56 lines

```
// Classic 3D simplex noise (Ashima Arts / Stefan
Gustavson, MIT).
// Shared by the particle and dissolve shaders via
#include.
vec3 mod289(vec3 x) { return x - floor(x * (1.0 /
289.0)) * 289.0; }
vec4 mod289(vec4 x) { return x - floor(x * (1.0 /
289.0)) * 289.0; }
vec4 permute(vec4 x) { return mod289(((x * 34.0) +
1.0) * x); }
vec4 taylorInvSqrt(vec4 r) { return 1.79284291400159
- 0.85373472095314 * r; }

float snoise(vec3 v) {
  const vec2 C = vec2(1.0 / 6.0, 1.0 / 3.0);
  const vec4 D = vec4(0.0, 0.5, 1.0, 2.0);
  vec3 i = floor(v + dot(v, C.yyy));
  vec3 x0 = v - i + dot(i, C.xxx);
  vec3 g = step(x0.yzx, x0.xyz);
  vec3 l = 1.0 - g;
  vec3 i1 = min(g.xyz, l.zxy);
  vec3 i2 = max(g.xyz, l.zxy);
  vec3 x1 = x0 - i1 + C.xxx;
  vec3 x2 = x0 - i2 + C.yyy;
  vec3 x3 = x0 - D.yyy;
  i = mod289(i);
  vec4 p = permute(permute(permute(i.z + vec4(0.0,
i1.z, i2.z, 1.0)) + i.y + vec4(0.0, i1.y, i2.y,
1.0)) + i.x + vec4(0.0, i1.x, i2.x, 1.0));
  float n_ = 0.142857142857;
  vec3 ns = n_ * D.wyz - D.xzx;
  vec4 j = p - 49.0 * floor(p * ns.z * ns.z);
  vec4 x_ = floor(j * ns.z);
  vec4 y_ = floor(j - 7.0 * x_);
  vec4 x = x_ * ns.x + ns.yyyy;
  vec4 y = y_ * ns.x + ns.yyyy;
  vec4 h = 1.0 - abs(x) - abs(y);
  vec4 b0 = vec4(x.xy, y.xy);
  vec4 b1 = vec4(x.zw, y.zw);
  vec4 s0 = floor(b0) * 2.0 + 1.0;
  vec4 s1 = floor(b1) * 2.0 + 1.0;
  vec4 sh = -step(h, vec4(0.0));
  vec4 a0 = b0.xzyw + s0.xzyw * sh.xxyy;
  vec4 a1 = b1.xzyw + s1.xzyw * sh.zzww;
  vec3 p0 = vec3(a0.xy, h.x);
  vec3 p1 = vec3(a0.zw, h.y);
  vec3 p2 = vec3(a1.xy, h.z);
  vec3 p3 = vec3(a1.zw, h.w);
  vec4 norm = taylorInvSqrt(vec4(dot(p0, p0),
dot(p1, p1), dot(p2, p2), dot(p3, p3)));
  p0 *= norm.x; p1 *= norm.y; p2 *= norm.z; p3 *=
norm.w;
  vec4 m = max(0.6 - vec4(dot(x0, x0), dot(x1, x1),
dot(x2, x2), dot(x3, x3)), 0.0);
  m = m * m;
  return 42.0 * dot(m * m, vec4(dot(p0, x0), dot(p1,
x1), dot(p2, x2), dot(p3, x3)));
}

// Cheap "curl-like" flow: three offset noise
samples. Not a true curl
// (that needs derivatives), but it swirls
convincingly and costs 3 lookups.
vec3 flow(vec3 p) {
  return vec3(
    snoise(p + vec3(0.0, 13.1, 7.7)),
    snoise(p + vec3(31.4, 0.0, 2.9)),
    snoise(p + vec3(5.3, 21.7, 0.0))
  );
}
```

src/canvas/shaders/packet.frag 30 lines

```
// The request packet + the security scan.
// uScan is the height (-1..1, in packet radii) of
the scanning band. Outside
// the gate it is parked at 3.0, i.e. off the
object.
uniform float uTime;
uniform float uScan;
uniform float uVerified;   // 0..1, flashes ink-
white when a gate accepts it
uniform float uOpacity;
uniform vec3 uAccent;
uniform vec3 uInk;
uniform vec3 uPaper;

varying vec3 vObj;
varying vec3 vNormal;
varying vec3 vView;

void main() {
  float fres = pow(1.0 - max(dot(vNormal, vView),
0.0), 2.2);
  vec3 col = mix(uAccent, uInk, fres * 0.85);

  // Scan band: a bright line plus "readout" stripes
just behind it.
  float y = vObj.y;
  float band = smoothstep(0.12, 0.0, abs(y -
uScan));
  float scanning = step(abs(uScan), 1.2);
  float trail = step(y, uScan) * smoothstep(0.6,
0.0, uScan - y) * scanning;
  float stripes = step(0.5, fract(y * 18.0 - uTime *
2.0)) * trail * 0.35;
  col = mix(col, uInk, clamp(band + stripes, 0.0,
1.0));

  col = mix(col, uInk, uVerified * 0.8);
  gl_FragColor = vec4(col, uOpacity * (0.75 + 0.25 *
fres));
}
```

src/canvas/shaders/packet.vert 11 lines

```
varying vec3 vObj;
varying vec3 vNormal;
varying vec3 vView;

void main() {
  vObj = position;
  vec4 mv = modelViewMatrix * vec4(position, 1.0);
  vNormal = normalize(normalMatrix * normal);
```

```
  vView = normalize(-mv.xyz);
  gl_Position = projectionMatrix * mv;
}
```

src/canvas/shaders/particles.frag 15 lines

```
uniform vec3 uInk;
uniform vec3 uAccent;
uniform float uOpacity;

varying float vAccent;
varying float vAlpha;

void main() {
  // Round, slightly soft dot (ink on paper, so
plain alpha blending). Discarding the corners is
cheaper than blending them.
  vec2 c = gl_PointCoord - 0.5;
  float d = length(c);
  if (d > 0.5) discard;
  float a = smoothstep(0.5, 0.2, d);
  gl_FragColor = vec4(mix(uInk, uAccent, vAccent), a
* vAlpha * uOpacity);
}
```

src/canvas/shaders/particles.vert 50 lines

```
// Hero / response particles.
// Every particle carries three positions and the
shader blends between them:
//   aScatter → aText   (uIntro 0→1, time-based, the
name assembling)
//   aText    → packet  (uMorph 0→1, scroll-based,
the name becoming the packet)
// All of this runs on the GPU, so 11k particles
cost one draw call.
#include "./noise.glsl"

uniform float uTime;
uniform float uIntro;
uniform float uMorph;
uniform float uSize;
uniform float uPixelRatio;
uniform vec3 uPacket;      // live world position of
the packet
uniform float uPacketRadius;
uniform float uInvScale;   // 1 / group scale: keeps
noise sizes in world units

attribute vec3 aText;
attribute vec3 aScatter;
attribute vec3 aSphere;    // unit-sphere point:
where on the packet this one lands
attribute float aRand;

varying float vAccent;
varying float vAlpha;

void main() {
  // Each particle starts at a slightly different
moment → the name "writes" itself.
  float intro = smoothstep(aRand * 0.45, aRand *
0.45 + 0.55, uIntro);
  intro = 1.0 - pow(1.0 - intro, 3.0);
  vec3 p = mix(aScatter, aText, intro);

  // Breathing while the name holds still, so it
never looks like a static image.
  p += 0.025 * uInvScale * flow(p / uInvScale * 0.6
+ uTime * 0.15) * intro;

  // Collapse into the packet. Stagger by aRand
again, and push particles
  // through a noise field in the middle of the trip
(peaks at m = 0.5).
  float m = smoothstep(aRand * 0.35, aRand * 0.35 +
0.65, uMorph);
  float swirl = sin(m * 3.14159);
  vec3 target = uPacket + aSphere * uPacketRadius;
  p = mix(p, target, m * m * (3.0 - 2.0 * m));
  p += flow(p / uInvScale * 0.35 + uTime * 0.2) *
swirl * 1.6 * uInvScale;

  vec4 mv = modelViewMatrix * vec4(p, 1.0);
  gl_Position = projectionMatrix * mv;

  // Size attenuation by distance; particles shrink
as they join the packet.
  gl_PointSize = uSize * uPixelRatio * (0.6 + aRand
* 0.8) * (1.0 - 0.55 * m) / -mv.z;

  vAccent = step(0.94, aRand);      // ~6% are
vermilion, like stray ink marks
  vAlpha = mix(0.25, 1.0, intro) * (1.0 -
smoothstep(0.92, 1.0, uMorph));
}
```

src/content/hero.js 18 lines

```
/*
 * The hero's words, derived from profile.js. Shared
by the React Hero and by
 * the build step in vite.config.js that writes the
same hero straight into
 * index.html, so text shows up before any
JavaScript has loaded.
 */
import { profile } from './profile.js'

const e = profile.education
export const hero = {
  kicker: 'GET /jaykumar-shevate HTTP/1.1 · host:
pune.in',
  title: profile.displayName,
  line: profile.tagline,
  meta1: `${e.degree.replace('Computer Science &
Engineering', 'CSE')} · ${e.school} · ${e.year}`,
  meta2: `Teaching Associate there since Aug 2025 ·
${profile.location.split(',')[0]}`,
  hint: 'scroll. you are the request.',
}

export const ARROW_PATHS = ['M3 6 C 14 2, 30 4, 34
16 S 30 33, 22 34', 'M28 28 L 21 34.5 L 29.5 37']
```

src/content/profile.js 210 lines

```
/*
 * profile.js — the ONE place where facts about me
live.
 *
 * Every section, the terminal and the simple view
read from here, so a fact
 * is never typed twice. Rule for this file: only
what's on my resume.
 *
 *   fill('what')    → shows a hatched [FILL: what]
marker on the page.
 *                     Replace it with a real string
when I have the info.
 *   verify('text')  → shows the text with a
[VERIFY] tag. Use for anything
 *                     (a number, a claim) I still
need to double-check.
 *
 * Searching the codebase for "fill(" lists
everything still missing.
 */

export const fill = (what) => ({ fill: what })
export const verify = (text) => ({ verify: text })

export const profile = {
  name: 'Jaykumar Chandrakant Shevate',
  displayName: 'Jaykumar Shevate',
  // Lines drawn by the hero particles. Kept short
so it fits on a phone.
  particleName: ['JAYKUMAR', 'SHEVATE'],
```

```
  location: 'Pune, Maharashtra, India',
  // Optional. Put the image in /public (e.g.
/me.jpg). null = no photo shown.
  photo: null,
  role: 'Teaching Associate, D. Y. Patil
International University',
  targeting: ['cloud', 'backend', 'junior DevOps',
'security'],
  freelance: true,
  spoken: ['English', 'Hindi', 'Marathi'],
  learning: ['German', 'Japanese', 'Russian'],

  // DRAFT tagline written from resume facts only.
Rewrite in my own words.
  tagline:
    'I work on the parts of a system you only notice
when they fail: the cloud underneath, the backend in
the middle, and the security around both.',

  links: {
    github: fill('GitHub profile URL'),
    linkedin: fill('LinkedIn URL'),
    email: fill('email address'),
    // Put the PDF in /public and write e.g.
'/Jaykumar-Shevate-Resume.pdf'
    resume: fill('resume PDF path'),
  },

  education: {
    degree: 'B.Tech, Computer Science & Engineering
(Cyber Security)',
    school: 'D. Y. Patil International University',
    year: '2026',
    exams: [{ name: 'GATE 2026 (Data Science & AI)',
result: 'Qualified' }],
  },

  certifications: [
    {
      id: 'aws-ccp',
      name: 'AWS Certified Cloud Practitioner',
      short: 'AWS CCP',
      score: '1000/1000',
      date: 'Mar 2026',
      valid: 'Valid through Mar 2029',
      verifyUrl: fill('Credly link'),
      gate: true, // gets a physical gate in the 3D
Security Gate scene
    },
    {
      id: 'ai-900',
      name: 'Microsoft Azure AI Fundamentals (AI-
900)',
      short: 'AI-900',
      score: '929/1000',
      date: 'Mar 2026',
      verifyUrl: fill('verification link'),
      gate: true,
    },
    {
      id: 'restart',
      name: 'AWS re/Start Graduate Program',
      short: 're/Start',
      score: 'Completed',
      date: 'Sep 2025',
      gate: false,
    },
  ],

  experience: [
    {
      title: 'Teaching Associate',
      org: 'D. Y. Patil International University',
      when: 'Aug 2025 – present',
      detail: fill('what I teach / what I
achieved'),
    },
    {
      title: 'Software Developer Intern',
      org: 'Technocia Infotech',
      when: 'May – Aug 2025',
      detail: fill('what I built, tech used,
outcome'),
    },
    {
      title: 'Cybersecurity Research Intern',
      org: 'FirstVidya Pvt. Ltd. (remote)',
      when: 'Feb – Apr 2025',
      detail: fill('what I researched, tools,
outcome'),
    },
  ],

  /*
   * Projects, in the order the camera visits them.
   * `scene` is a plain description of the 3D
metaphor (not a feature claim).
   */
  projects: [
    {
      slug: 'carepilot',
      name: 'CarePilot',
      kind: 'Healthcare SaaS / national health
network',
      scene: 'Hospitals as a node graph. Records hop
between them.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      // Optional: [{ src: '/shots/name-1.webp',
alt: 'what the screenshot shows' }]
      screenshots: [],
    },
    {
      slug: 'rentflow',
      name: 'RentFlow',
      kind: 'Rental property management platform',
      scene: 'One building. Units change state as
rent status changes.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      // Optional: [{ src: '/shots/name-1.webp',
alt: 'what the screenshot shows' }]
      screenshots: [],
    },
    {
      slug: 'sensorium',
      name: 'Sensorium Nexus',
      kind: 'IoT monitoring platform',
      scene: 'Devices stream readings up into panels
that redraw every frame.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      // Optional: [{ src: '/shots/name-1.webp',
alt: 'what the screenshot shows' }]
      screenshots: [],
    },
    {
      slug: 'bazaarlens',
      name: 'BazaarLens',
      kind: 'AI shopping price comparison for
India',
```

```
      scene: 'Stores race in lanes. The fastest
answer is not the winner; the cheapest is.',
      status: fill('status'),
      role: fill('my role'),
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link'),
      demo: fill('demo link'),
      // Optional: [{ src: '/shots/name-1.webp',
alt: 'what the screenshot shows' }]
      screenshots: [],
    },
    {
      slug: 'phd-erp',
      name: 'DYPIU PhD ERP',
      kind: 'Doctoral programme management system
for D Y Patil International University',
      scene: 'A student token moves station to
station, stamped at each approval.',
      status: 'UAT / pilot stage',
      role: 'Sole developer',
      problem: fill('problem it solves'),
      architecture: fill('architecture notes'),
      stack: fill('stack'),
      repo: fill('repo link, or "private"'),
      demo: fill('demo link'),
      // Optional: [{ src: '/shots/name-1.webp',
alt: 'what the screenshot shows' }]
      screenshots: [],
    },
  ],

  // Optional: other repos to feature. [{ name, url,
line }] — hidden while empty.
  otherRepos: [],

  skills: [
    { group: 'Cloud', items: ['AWS', 'Azure AI',
fill('exact services used')] },
    { group: 'Backend · languages · frameworks',
items: [fill('languages and frameworks')] },
    { group: 'Security', items: [fill('tools and
areas')] },
    { group: 'Databases · tools · DevOps', items:
[fill('databases, tools, CI/CD')] },
    { group: 'Learning now', items: ['Python',
'German', 'Japanese', 'Russian'] },
  ],

  contact: {
    // DRAFT call to action. Replace with my own
line.
    cta: "Hiring for cloud, backend, junior DevOps
or security? Or need something built freelance? Send
the request back.",
  },
}

/* Small helpers used across the UI. */
// (Arrays have a .fill() method, so check for a
plain object first.)
const isObj = (v) => v && typeof v === 'object' &&
!Array.isArray(v)
export const isFill = (v) => isObj(v) && 'fill' in v
export const isVerify = (v) => isObj(v) && 'verify'
in v
/** Returns a real URL string, or null if the value
is still a placeholder. */
export const realUrl = (v) => (typeof v === 'string'
&& v.length ? v : null)
/** Plain-text version of a value, for the terminal.
*/
export const plain = (v) =>
  isFill(v) ? `[FILL: ${v.fill}]` : isVerify(v) ?
`${v.verify} [VERIFY]` : Array.isArray(v) ?
v.map(plain).join(', ') : String(v)
```

src/core/ease.js — 9 lines

```
/*
 * The house easing. One curve used everywhere
(Lenis, GSAP, CSS) so the whole
 * site moves with the same "hand": fast out of the
blocks, long soft landing.
 * It's an exponential out with a slight lift, tuned
by eye.
 */
export const requestEase = (t) => (t === 1 ? 1 : 1 -
Math.pow(2, -10.5 * t) * (1 - 0.04 * Math.sin(t *
Math.PI)))

// Same feel as a CSS cubic-bezier, for transitions
in the stylesheet.
export const REQUEST_BEZIER = 'cubic-bezier(0.16, 1,
0.3, 1)'
```

src/core/nav.js — 10 lines

```
/*
 * Navigation helpers that don't force GSAP + Lenis
into the first download.
 * The scroll module is imported on demand: in the
3D view it's already
 * loaded by App, in the simple view it only loads
if someone clicks a link.
 */
export const loadScroll = () => import('./scroll')

export function scrollToStop(id, at) {
  loadScroll().then((m) => m.scrollToStop(id, at))
}
```

src/core/quality.js — 38 lines

```
/*
 * Decide once, at startup, how much this device can
handle.
 * The 3D scenes read `quality.tier` to drop detail;
nothing here is fancy,
 * it just avoids melting a phone.
 */
function hasWebGL() {
  try {
    const c = document.createElement('canvas')
    return !!(c.getContext('webgl2') ||
c.getContext('webgl'))
  } catch {
    return false
  }
}

function detect() {
  const mq = (q) => window.matchMedia(q).matches
  const coarse = mq('(pointer: coarse)')
  const small = Math.min(window.innerWidth,
window.innerHeight) < 700
  const cores = navigator.hardwareConcurrency || 4
  const mem = navigator.deviceMemory || 8

  let tier = 'high'
  if (cores <= 4 || mem <= 4) tier = 'mid'
  if ((coarse && small) || cores <= 2 || mem <= 2)
tier = 'low'

  return {
    tier,
    mobile: coarse && small,
    // DPR cap: retina at 2x would quadruple the
fragment work for little gain.
    dpr: tier === 'low' ? 1 :
Math.min(window.devicePixelRatio || 1, 1.5),
    particles: tier === 'low' ? 3500 : tier ===
'mid' ? 7000 : 11000,
    finePointer: mq('(pointer: fine)'),
    reducedMotion: mq('(prefers-reduced-motion:
reduce)'),
```

```
    webgl: hasWebGL(),
  }
}

export const quality = typeof window !== 'undefined'
? detect() : { tier: 'low', webgl: false }
```

src/core/route.js 74 lines

```
/*
 * route.js — the journey, as data.
 *
 * Each stop has:
 *   cam   — where the camera sits when it arrives
at this stop
 *   look  — what it looks at (also where that
stop's 3D scene is placed)
 *   frame — 'center' puts the scene in the middle
of the screen,
 *           'side' shifts it right so the HTML
panel can sit on the left
 *   move  — how the camera travels during this
section (see easeFor)
 *   vh    — section height in viewport heights (=
how long you scroll here)
 *
 * Two Catmull-Rom splines are built through all the
`cam` and `look` points.
 * Stop k sits exactly at spline parameter k / (N -
1), because
 * CatmullRomCurve3.getPoint(t) maps t linearly onto
its control points.
 */
import { CatmullRomCurve3, Vector3 } from 'three'

export const STOPS = [
  { id: 'edge',       label: 'edge',          cam:
[0, 0.3, 12],    look: [0, 0.2, 0],     frame:
'center', move: 'hero',   vh: 230 },
  { id: 'gate',       label: 'security gate', cam:
[0, 0.2, -4],    look: [0, -0.2, -30],  frame:
'side',   move: 'linear', vh: 320 },
  { id: 'lb',         label: 'load balancer', cam:
[5, 2.6, -38],   look: [10, 0, -48],    frame:
'side',   move: 'dwell',  vh: 210 },
  { id: 'carepilot',  label: 'carepilot',     cam:
[-2, 3.4, -60],  look: [-9, 0, -70],    frame:
'side',   move: 'dwell',  vh: 260 },
  { id: 'rentflow',   label: 'rentflow',      cam:
[3, 2.2, -84],   look: [10, 1.6, -93],  frame:
'side',   move: 'dwell',  vh: 260 },
  { id: 'sensorium',  label: 'sensorium',     cam:
[-2, 2.6, -106], look: [-9, 1.2, -116], frame:
'side',   move: 'dwell',  vh: 260 },
  { id: 'bazaarlens', label: 'bazaarlens',    cam:
[3, 3.6, -128],  look: [10, 0, -138],   frame:
'side',   move: 'dwell',  vh: 260 },
  { id: 'phd-erp',    label: 'phd erp',       cam:
[-2, 2.8, -150], look: [-9, 0, -160],   frame:
'side',   move: 'dwell',  vh: 260 },
  { id: 'data',       label: 'data layer',    cam:
[0, -3, -174],   look: [0, -9, -184],   frame:
'side',   move: 'dwell',  vh: 230 },
  { id: 'response',   label: 'response',      cam:
[0, -7.5, -196], look: [0, -7.5, -210], frame:
'center', move: 'hold',   vh: 0 },
]

export const STOP_INDEX =
Object.fromEntries(STOPS.map((s, i) => [s.id, i]))

const v = (a) => new Vector3(...a)
export const camCurve = new
CatmullRomCurve3(STOPS.map((s) => v(s.cam)), false,
'centripetal')
export const lookCurve = new
CatmullRomCurve3(STOPS.map((s) => v(s.look)), false,
'centripetal')

/** Spline parameter at which stop i sits. */
export const tOf = (i) => i / (STOPS.length - 1)

const clamp01 = (x) => Math.min(1, Math.max(0, x))
const smooth = (a, b, x) => {
  const t = clamp01((x - a) / (b - a))
  return t * t * (3 - 2 * t)
}

/*
 * How far toward the NEXT stop the camera has gone,
given progress in this
 * section. This is where the pacing lives:
 *   hero   — stay put while the name dissolves,
then leave
 *   linear — move the whole time (the gate scene is
a fly-through)
 *   dwell  — drift a little while you read, then
travel in the last third
 */
export function easeFor(move, x) {
  switch (move) {
    case 'hero':   return smooth(0.55, 1, x)
    case 'linear': return x
    case 'dwell':  return 0.12 * x + 0.88 *
smooth(0.62, 1, x)
    default:       return 0
  }
}

/** Turn per-section progress into one parameter
along the spline. */
export function cameraT(local) {
  for (let i = STOPS.length - 1; i >= 0; i--) {
    const p = local[STOPS[i].id] ?? 0
    if (p > 0 || i === 0) {
      const next = Math.min(i + 1, STOPS.length - 1)
      return tOf(i) + (tOf(next) - tOf(i)) *
easeFor(STOPS[i].move, p)
    }
  }
  return 0
}

export { clamp01, smooth }
```

src/core/scroll.js 85 lines

```
/*
 * scroll.js — Lenis + GSAP ScrollTrigger glue.
 *
 * Lenis replaces the browser's wheel scrolling with
an eased version (inertia),
 * but the page still scrolls natively, so
scrollbars, keyboard, find-in-page and
 * screen readers all keep working. ScrollTrigger
watches each
 * <section data-stop="..."> and reports its
progress 0..1, which we write
 * into the non-reactive `scroll` object for the 3D
world to read.
 */
import Lenis from 'lenis'
import gsap from 'gsap'
import { ScrollTrigger } from 'gsap/ScrollTrigger'
import { scroll, useStore } from './store'
import { cameraT } from './route'
import { requestEase } from './ease'

gsap.registerPlugin(ScrollTrigger)

let lenis = null
let triggers = []
const raf = (time) => lenis?.raf(time * 1000)

export function initScroll({ smooth = true } = {}) {
  destroyScroll()

  if (smooth) {
```

```
    lenis = new Lenis({ duration: 1.3, easing:
requestEase, wheelMultiplier: 0.85, touchMultiplier:
1.1 })
    lenis.on('scroll', (e) => {
      scroll.velocity = e.velocity
      ScrollTrigger.update()
    })
    // One clock for everything: GSAP's ticker
drives Lenis.
    gsap.ticker.add(raf)
    gsap.ticker.lagSmoothing(0)
  }

  const { setActiveStop, setGateStage } =
useStore.getState()

  document.querySelectorAll('[data-
stop]').forEach((el) => {
    const id = el.dataset.stop
    scroll.local[id] = 0
    triggers.push(
      ScrollTrigger.create({
        trigger: el,
        start: 'top top',
        end: 'bottom top', // sections are back to
back: no gaps, no overlaps
        onUpdate: (self) => {
          const p = self.progress
          scroll.local[id] = p
          scroll.camT = cameraT(scroll.local)
          // Panels fade in/out with a class instead
of per-frame styles.
          el.classList.toggle('is-on', id === 'edge'
? p < 0.42 : p > 0.03 && p < 0.7)
          if (id === 'gate') {
            const stage = p > 0.46 ? 2 : p > 0.16 ?
1 : 0
            if (stage !==
useStore.getState().gateStage) setGateStage(stage)
          }
        },
        onToggle: (self) => self.isActive &&
setActiveStop(id),
      }),
    )
  })
  ScrollTrigger.refresh()
}

export function destroyScroll() {
  triggers.forEach((t) => t.kill())
  triggers = []
  if (lenis) {
    gsap.ticker.remove(raf)
    lenis.destroy()
    lenis = null
  }
}

/**
 * Jump to a stop. `at` is where inside the section
to land (0..1);
 * 0.3 is past the arrival and into the readable
part.
 */
export function scrollToStop(id, at = 0.3) {
  const el = document.querySelector(`[data-
stop="${id}"]`) || document.getElementById(id)
  if (!el) return
  const y = el.offsetTop + (el.dataset.stop ?
el.offsetHeight * at : 0)
  if (lenis) lenis.scrollTo(y, { duration: 2.2 })
  else window.scrollTo({ top: y, behavior:
useStore.getState().mode === 'simple' ? 'auto' :
'smooth' })
}
```

src/core/sound.js 68 lines

```
/*
 * Ambient sound, generated live with the Web Audio
API, so there is no audio
 * file to download. Two detuned triangle
oscillators and a little filtered noise
 * go through a lowpass filter that breathes slowly.
The filter opens a little
 * at each stop, so the hum gets "brighter" as the
request goes deeper.
 * Off by default; browsers only allow audio after a
click anyway.
 */
let ctx, master, filter

function build() {
  ctx = new (window.AudioContext ||
window.webkitAudioContext)()
  master = ctx.createGain()
  master.gain.value = 0
  filter = ctx.createBiquadFilter()
  filter.type = 'lowpass'
  filter.frequency.value = 420
  filter.Q.value = 6
  filter.connect(master).connect(ctx.destination)

  ;[55, 55.4, 82.4].forEach((f, i) => {
    const o = ctx.createOscillator()
    o.type = i === 2 ? 'sine' : 'triangle'
    o.frequency.value = f
    const g = ctx.createGain()
    g.gain.value = i === 2 ? 0.18 : 0.32
    o.connect(g).connect(filter)
    o.start()
  })

  // Pink-ish noise for "air in the server room".
  const len = ctx.sampleRate * 2
  const buf = ctx.createBuffer(1, len,
ctx.sampleRate)
  const d = buf.getChannelData(0)
  let last = 0
  for (let i = 0; i < len; i++) {
    last = (last + 0.02 * (Math.random() * 2 - 1)) /
1.02
    d[i] = last * 3.5
  }
  const noise = ctx.createBufferSource()
  noise.buffer = buf
  noise.loop = true
  const ng = ctx.createGain()
  ng.gain.value = 0.05
  noise.connect(ng).connect(filter)
  noise.start()

  // Slow LFO on the filter so it never sits still.
  const lfo = ctx.createOscillator()
  lfo.frequency.value = 0.07
  const lg = ctx.createGain()
  lg.gain.value = 120
  lfo.connect(lg).connect(filter.frequency)
  lfo.start()
}

export function setSound(on) {
  if (!ctx) build()
  if (on) ctx.resume()
  const now = ctx.currentTime
  master.gain.cancelScheduledValues(now)
  master.gain.setTargetAtTime(on ? 0.055 : 0, now,
on ? 1.2 : 0.3)
}

/** Called when the active stop changes; index 0..N.
*/
export function setSoundStop(i) {
  if (!ctx) return
  filter.frequency.setTargetAtTime(380 + i * 70,
```

```
ctx.currentTime, 1.5)
}
```

src/core/store.js 49 lines

```
/*
 * Two kinds of state live here, on purpose:
 *
 * 1. `useStore` (zustand) — things React should re-
render for: which stop is
 *    active, is the terminal open, is sound on.
These change rarely.
 *
 * 2. `scroll` — a plain mutable object written on
every scroll event and read
 *    inside useFrame() 60 times a second. Putting
that in React state would
 *    re-render the whole tree every frame, so it
deliberately isn't reactive.
 */
import { create } from 'zustand'

export const useStore = create((set) => ({
  // 'experience' | 'simple'. The inline script in
index.html decides first.
  mode: typeof document !== 'undefined' &&
document.documentElement.classList.contains('is-
simple') ? 'simple' : 'experience',
  setMode: (mode) => set({ mode }),

  canvasReady: false,
  setCanvasReady: (canvasReady) => set({ canvasReady
}),

  activeStop: 'edge',
  setActiveStop: (activeStop) => set({ activeStop
}),

  // How many certification gates the packet has
cleared (0, 1, 2).
  gateStage: 0,
  setGateStage: (gateStage) => set({ gateStage }),

  sound: false,
  setSound: (sound) => set({ sound }),

  terminal: false,
  setTerminal: (terminal) => set({ terminal }),
}))

/** Hot path state. Written by core/scroll.js, read
by the 3D scenes. */
export const scroll = {
  /** 0..1 progress inside each stop's <section>,
keyed by stop id. */
  local: {},
  /** Parameter along the camera spline, 0..1
(before smoothing). */
  camT: 0,
  /** Lenis velocity, used for subtle motion blur-
ish stretching. */
  velocity: 0,
}

/** Shared between Packet and particle scenes: where
the packet is in world space. */
export const packetState = {
  position: [0, 0, 0],
  opacity: 0,
}
```

src/dom/Cursor.jsx 49 lines

```
/*
 * A pen-tip cursor: a red dot that follows
instantly and a pencil ring that
 * follows with inertia (lerp each frame). Over
anything clickable the ring
 * opens up. Only on fine pointers; text fields keep
the native caret.
 */
import { useEffect, useRef } from 'react'
import { quality } from '../core/quality'

export default function Cursor() {
  const ring = useRef(null)
  const cross = useRef(null)

  useEffect(() => {
    if (!quality.finePointer) return
    document.documentElement.classList.add('has-
cursor')
    let mx = -100, my = -100, rx = -100, ry = -100,
raf
    const move = (e) => {
      mx = e.clientX
      my = e.clientY
      const hot = e.target.closest?.('a, button,
[role="button"], input, textarea, summary')
      ring.current.classList.toggle('is-hot', !!hot)
    }
    const loop = () => {
      if (!ring.current) return
      const k = quality.reducedMotion ? 1 : 0.16
      rx += (mx - rx) * k
      ry += (my - ry) * k
      cross.current.style.transform =
`translate3d(${mx}px, ${my}px, 0)`
      ring.current.style.transform =
`translate3d(${rx}px, ${ry}px, 0)`
      raf = requestAnimationFrame(loop)
    }
    window.addEventListener('pointermove', move, {
passive: true })
    loop()
    return () => {
      cancelAnimationFrame(raf)
      window.removeEventListener('pointermove',
move)

document.documentElement.classList.remove('has-
cursor')
    }
  }, [])

  if (!quality.finePointer) return null
  return (
    <div className="cursor" aria-hidden="true">
      <div ref={ring} className="cursor__ring" />
      <div ref={cross} className="cursor__cross">
      </div>
    </div>
  )
}
```

src/dom/Magnetic.jsx 62 lines

```
/*
 * Magnetic — wraps a link/button so it leans toward
the cursor when the
 * cursor is over it, and springs back when it
leaves.
 *
 * A tiny damped spring on requestAnimationFrame,
instead of GSAP, so this
 * file (which every section uses) doesn't pull GSAP
into the first-paint
 * bundle. Capped at `strength` px so it reads as a
nudge. Off for touch and
 * for reduced motion.
 */
import { useEffect, useRef } from 'react'
import { quality } from '../core/quality'

const STIFF = 0.14 // pull toward the target each
frame
const DAMP = 0.72 // how much velocity survives each
frame (springy, not bouncy)
```

```
export default function Magnetic({ children,
strength = 8, as: Tag = 'span', className = '' }) {
  const ref = useRef(null)

  useEffect(() => {
    const el = ref.current
    if (!el || !quality.finePointer ||
quality.reducedMotion) return
    let tx = 0, ty = 0, x = 0, y = 0, vx = 0, vy =
0, raf = 0

    const tick = () => {
      vx = (vx + (tx - x) * STIFF) * DAMP
      vy = (vy + (ty - y) * STIFF) * DAMP
      x += vx
      y += vy
      el.style.transform =
`translate3d(${x.toFixed(2)}px, ${y.toFixed(2)}px,
0)`
      // Stop the loop once it has settled, so idle
links cost nothing.
      if (Math.abs(tx - x) + Math.abs(ty - y) +
Math.abs(vx) + Math.abs(vy) > 0.05) raf =
requestAnimationFrame(tick)
      else raf = 0
    }
    const kick = () => {
      if (!raf) raf = requestAnimationFrame(tick)
    }
    const move = (e) => {
      const r = el.getBoundingClientRect()
      tx = ((e.clientX - (r.left + r.width / 2)) /
r.width) * strength * 2
      ty = ((e.clientY - (r.top + r.height / 2)) /
r.height) * strength * 2
      kick()
    }
    const leave = () => {
      tx = 0
      ty = 0
      kick()
    }
    el.addEventListener('pointermove', move)
    el.addEventListener('pointerleave', leave)
    return () => {
      cancelAnimationFrame(raf)
      el.removeEventListener('pointermove', move)
      el.removeEventListener('pointerleave', leave)
    }
  }, [strength])

  return (
    <Tag ref={ref} className={`magnetic
${className}`}>
      {children}
    </Tag>
  )
}
```

src/dom/Sheet.jsx — 26 lines

```
/*
 * Sheet — every content panel is a page taped into
the notebook.
 *
 *  - The outline is drawn on its own layer and
pushed through an SVG
 *    turbulence filter (#rough, defined in
App.jsx), so the border wobbles
 *    like a pen line but the text inside stays
perfectly crisp.
 *  - Each page sits at a slightly different angle.
The angle comes from the
 *    page number, not Math.random(), so it's the
same on every visit.
 *  - A strip of tape holds it down; the footer is
the page's label.
 */
export default function Sheet({ no, title, children,
className = '' }) {
  const tilt = (((no * 37) % 9) - 4) * 0.18 //
-0.72deg .. +0.72deg
  const tapeX = 30 + ((no * 53) % 40) // tape lands
somewhere along the top
  return (
    <div className={`sheet ${className}`} style={{
'--tilt': `${tilt}deg`, '--tape-x': `${tapeX}%` }}>
      <span className="sheet__edge" aria-
hidden="true" />
      <span className="sheet__tape" aria-
hidden="true" />
      <div className="sheet__body">{children}</div>
      <div className="sheet__block" aria-
hidden="true">
        <span>p. {String(no).padStart(2, '0')}
</span>
        <span>{title.toLowerCase()}</span>
        <span>jcs</span>
      </div>
    </div>
  )
}
```

src/dom/SoundToggle.jsx — 31 lines

```
import { useEffect } from 'react'
import { useStore } from '../core/store'
import { setSound, setSoundStop } from
'../core/sound'
import { STOP_INDEX } from '../core/route'

export default function SoundToggle() {
  const on = useStore((s) => s.sound)
  const set = useStore((s) => s.setSound)
  const stop = useStore((s) => s.activeStop)

  useEffect(() => {
    if (on) setSoundStop(STOP_INDEX[stop] ?? 0)
  }, [on, stop])

  return (
    <button
      type="button"
      className="ctl"
      aria-pressed={on}
      onClick={() => {
        setSound(!on)
        set(!on)
      }}
    >
      <span className="ctl__wave" data-on={on} aria-
hidden="true">
        <i /><i /><i /><i />
      </span>
      sound {on ? 'on' : 'off'}
    </button>
  )
}
```

src/dom/Stop.jsx — 34 lines

```
/*
 * Stop — one stop of the journey as a <section>.
 *
 * Experience mode: the section is tall (vh from
route.js) and its panel is
 * position: sticky, so the text stays put while the
3D world moves behind it.
 * ScrollTrigger reads the section's data-stop
attribute.
 *
 * Simple mode: same content, normal height, no
data-stop, no stickiness.
 */
import { STOPS, STOP_INDEX } from '../core/route'
import { useStore } from '../core/store'
```

```
export default function Stop({ id, children,
className = '', labelledBy }) {
  const simple = useStore((s) => s.mode ===
'simple')
  const stop = STOPS[STOP_INDEX[id]]
  if (simple) {
    return (
      <section id={id} className={`stop stop--simple
${className}`} aria-labelledby={labelledBy}>
        {children}
      </section>
    )
  }
  return (
    <section
      id={id}
      data-stop={id}
      className={`stop ${className} ${id === 'edge'
? 'is-on' : ''}`}
      style={stop.vh ? { height: `${stop.vh}vh` } :
undefined}
      aria-labelledby={labelledBy}
    >
      <div className="stop__sticky">{children}</div>
    </section>
  )
}
```

src/dom/Terminal.jsx 169 lines

```
/*
 * The ~ terminal. A tiny command parser over
profile.js:
 * each command is a function returning lines of
output. `open` and
 * `contact` also move the camera. Up/Down walks
history, Tab completes
 * project names, Esc closes. Output is an aria-live
log for screen readers.
 */
import { useEffect, useRef, useState } from 'react'
import { profile, plain } from '../content/profile'
import { useStore } from '../core/store'
import { scrollToStop } from '../core/nav'

const slugs = profile.projects.map((p) => p.slug)

const COMMANDS = {
  help: () => [
    'whoami            who is on the other end',
    'ls projects       list services',
    'ls repos          other repos worth a look',
    'open <project>    fly to a project (tab
completes)',
    'certs             credentials',
    'contact           how to reach me',
    'resume            where the PDF is',
    'simple            switch to the simple view',
    'sound on|off      ambient hum',
    'clear · exit',
  ],
  whoami: () => [
    profile.name,
    `${profile.role} · ${profile.location}`,
    `${profile.education.degree},
${profile.education.year}`,
    `looking for: ${profile.targeting.join(' / ')}
roles${profile.freelance ? ' · open to freelance' :
''}`,
  ],
  ls: (arg) => {
    if (arg === 'projects' || !arg) return
profile.projects.map((p) => `${p.slug.padEnd(12)}
${p.kind}`)
    if (arg === 'repos')
      return profile.otherRepos.length ?
profile.otherRepos.map((r) => `${r.name.padEnd(18)}
${r.url}`) : ['(nothing here yet)']
    return [`ls: ${arg}: no such directory. try: ls
projects, ls repos`]
  },
  open: (arg) => {
    const p = profile.projects.find((x) => x.slug
=== arg || x.name.toLowerCase() === arg)
    if (!p) return [`open: unknown project "${arg ??
''}". try: ${slugs.join(', ')}`]
    scrollToStop(p.slug, 0.32)
    return [`routing to ${p.name} …`, `stack:
${plain(p.stack)}`, `role: ${plain(p.role)}`]
  },
  certs: () => profile.certifications.map((c) =>
`${c.short.padEnd(9)} ${c.score.padEnd(10)}
${c.date}`),
  contact: () => {
    scrollToStop('response', 0)
    return [`mail
${plain(profile.links.email)}`, `github
${plain(profile.links.github)}`, `linkedin
${plain(profile.links.linkedin)}`]
  },
  resume: () => [`resume:
${plain(profile.links.resume)}`],
  simple: () => {
    setTimeout(() =>
useStore.getState().setMode('simple'), 300)
    return ['switching to simple view …']
  },
  sound: (arg) => {
    const on = arg !== 'off'
    import('../core/sound').then((m) =>
m.setSound(on))
    useStore.getState().setSound(on)
    return [`sound ${on ? 'on' : 'off'}`]
  },
  sudo: () => ['nice try. this incident will be
reported (to nobody).'],
}

export default function Terminal() {
  const open = useStore((s) => s.terminal)
  const setOpen = useStore((s) => s.setTerminal)
  const [log, setLog] = useState([{ k: 'out', t:
'type "help". esc to close.' }])
  const [val, setVal] = useState('')
  const hist = useRef([])
  const hi = useRef(0)
  const input = useRef(null)
  const back = useRef(null)
  const end = useRef(null)

  // ~ (or `) toggles from anywhere, except while
typing in a form field.
  useEffect(() => {
    const onKey = (e) => {
      const typing =
/INPUT|TEXTAREA/.test(document.activeElement?.tagNam
e) && document.activeElement !== input.current
      if ((e.key === '`' || e.key === '~') &&
!typing) {
        e.preventDefault()
        setOpen(!useStore.getState().terminal)
      }
      if (e.key === 'Escape' &&
useStore.getState().terminal) setOpen(false)
    }
    window.addEventListener('keydown', onKey)
    return () =>
window.removeEventListener('keydown', onKey)
  }, [setOpen])

  useEffect(() => {
    if (open) {
      back.current = document.activeElement
      setTimeout(() => input.current?.focus(), 30)
    } else back.current?.focus?.()
  }, [open])
```

```
  // Braces matter: newer Chrome returns a Promise
from scrollIntoView, and an
  // effect must return a cleanup function or
nothing.
  useEffect(() => {
    end.current?.scrollIntoView({ block: 'end' })
  }, [log])

  const run = (line) => {
    const [cmd, ...rest] = line.trim().split(/\s+/)
    const arg = rest.join(' ').toLowerCase() ||
undefined
    if (!cmd) return
    hist.current.push(line)
    hi.current = hist.current.length
    if (cmd === 'clear') return setLog([])
    if (cmd === 'exit') return setOpen(false)
    const f = COMMANDS[cmd.toLowerCase()]
    const out = f ? f(arg) : [`${cmd}: command not
found. type "help".`]
    setLog((l) => [...l, { k: 'in', t: line },
...out.map((t) => ({ k: 'out', t }))])
    if (cmd === 'open' && f &&
!out[0].startsWith('open:')) setTimeout(() =>
setOpen(false), 900)
  }

  const onKeyDown = (e) => {
    if (e.key === 'Enter') {
      run(val)
      setVal('')
    } else if (e.key === 'ArrowUp') {
      e.preventDefault()
      hi.current = Math.max(0, hi.current - 1)
      setVal(hist.current[hi.current] ?? '')
    } else if (e.key === 'ArrowDown') {
      e.preventDefault()
      hi.current = Math.min(hist.current.length,
hi.current + 1)
      setVal(hist.current[hi.current] ?? '')
    } else if (e.key === 'Tab') {
      e.preventDefault()
      const m = val.match(/^open\s+(\S*)$/)
      if (m) {
        const hit = slugs.find((s) =>
s.startsWith(m[1]))
        if (hit) setVal(`open ${hit}`)
      } else {
        const hit = Object.keys(COMMANDS).find((c)
=> c.startsWith(val))
        if (hit) setVal(hit + ' ')
      }
    }
  }

  if (!open) return null
  return (
    <div className="term" role="dialog" aria-
modal="true" aria-label="Terminal" data-lenis-
prevent onClick={() => input.current?.focus()}>
      <div className="term__bar">
        <span>jcs@edge:~</span>
        <button type="button" onClick={() =>
setOpen(false)} aria-label="Close terminal">esc
x</button>
      </div>
      <div className="term__log" aria-live="polite">
        {log.map((l, i) => (
          <p key={i} className={`term__${l.k}`}>{l.k
=== 'in' ? `› ${l.t}` : l.t}</p>
        ))}
        <div ref={end} />
      </div>
      <label className="term__line">
        <span aria-hidden="true">›</span>
        <span className="sr-only">Command</span>
        <input
          ref={input}
          value={val}
          onChange={(e) => setVal(e.target.value)}
          onKeyDown={onKeyDown}
          spellCheck={false}
          autoComplete="off"
          autoCapitalize="off"
        />
      </label>
    </div>
  )
}
```

src/dom/TopBar.jsx 43 lines

```
import { useStore } from '../core/store'
import { scrollToStop } from '../core/nav'
import Magnetic from './Magnetic'
import SoundToggle from './SoundToggle'

const NAV = [
  ['gate', 'certs'],
  ['lb', 'about'],
  ['carepilot', 'work'],
  ['data', 'stack'],
  ['response', 'contact'],
]

export default function TopBar() {
  const mode = useStore((s) => s.mode)
  const setMode = useStore((s) => s.setMode)
  const setTerminal = useStore((s) => s.setTerminal)
  const simple = mode === 'simple'

  return (
    <header className="bar">
      <a className="bar__mark" href="#edge" onClick=
{(e) => { e.preventDefault(); scrollToStop('edge',
0) }}>
        <span className="bar__dot" aria-
hidden="true" /> jcs<span
className="bar__dim">/req</span>
      </a>
      <nav aria-label="Sections"
className="bar__nav">
        {NAV.map(([id, label]) => (
          <Magnetic key={id}>
            <a href={`#${id}`} onClick={(e) => {
e.preventDefault(); scrollToStop(id) }}>{label}</a>
          </Magnetic>
        ))}
      </nav>
      <div className="bar__ctl">
        {!simple && <SoundToggle />}
        <button type="button" className="ctl"
onClick={() => setTerminal(true)} aria-
keyshortcuts="~">
          <kbd>~</kbd> terminal
        </button>
        <button type="button" className="ctl ctl--
em" onClick={() => setMode(simple ? 'experience' :
'simple')}>
          {simple ? '3d view' : 'simple view'}
        </button>
      </div>
    </header>
  )
}
```

src/dom/Trace.jsx 28 lines

```
/*
 * Trace — a request-trace HUD down the right edge:
every hop of the
 * journey, with the current one marked. It doubles
as keyboard navigation
 * (each hop is a button). Desktop only; on phones
the top bar covers it.
 */
import { STOPS } from '../core/route'
```

```
import { useStore } from '../core/store'
import { scrollToStop } from '../core/nav'

export default function Trace() {
  const active = useStore((s) => s.activeStop)
  const idx = STOPS.findIndex((s) => s.id ===
active)
  return (
    <nav className="trace" aria-label="Journey">
      <p className="trace__head" aria-
hidden="true">trace 7f3a·c1</p>
      <ol>
        {STOPS.map((s, i) => (
          <li key={s.id} className={i === idx ? 'is-
now' : i < idx ? 'is-done' : ''}>
            <button type="button" onClick={() =>
scrollToStop(s.id)} aria-current={i === idx ? 'step'
: undefined}>
              <span className="trace__dot" aria-
hidden="true" />
              <span className="trace__label">
{s.label}</span>
            </button>
          </li>
        ))}
      </ol>
    </nav>
  )
}
```

src/dom/Txt.jsx 13 lines

```
/*
 * Renders a profile value. Strings render as-is;
fill() placeholders render
 * as a hatched marker so missing info is impossible
to miss (and impossible
 * to mistake for a real claim).
 */
import { isFill, isVerify } from
'../content/profile'

export default function Txt({ v }) {
  if (isFill(v)) return <mark className="fill">
[FILL: {v.fill}]</mark>
  if (isVerify(v)) return (<>{v.verify} <mark
className="fill fill--verify">[VERIFY]</mark></>)
  if (Array.isArray(v)) return v.map((x, i) =>
(<span key={i}>{i > 0 && ', '}<Txt v={x} /></span>))
  return v ?? null
}
```

src/dom/sections/About.jsx 38 lines

```
import { profile } from '../../content/profile'
import Sheet from '../Sheet'
import Stop from '../Stop'
import Txt from '../Txt'

export default function About() {
  return (
    <Stop id="lb" className="panel-left"
labelledBy="about-title">
      <Sheet no={2} title="LOAD BALANCER">
        <p className="kicker">step 02 · routing</p>
        {profile.photo && (
          <figure className="polaroid">
            <img src={profile.photo} alt={`Photo of
${profile.displayName}`} loading="lazy"
decoding="async" />
          </figure>
        )}
        <h2 id="about-title">Where my time gets
routed right now.</h2>
        <p className="lede">
          Three servers, one hub. I'm finishing a
B.Tech in CSE with a Cyber Security specialisation
at
          D. Y. Patil International University,
teaching there as a Teaching Associate since August
2025, and
          building the university's PhD ERP as its
only developer. I qualified GATE 2026 in Data
Science & AI.
        </p>
        <p>
          I want to work on
{profile.targeting.slice(0, -1).join(', ')} or
{profile.targeting.at(-1)} problems, and I take
          freelance work. I speak
{profile.spoken.join(', ')}, and I'm learning
{profile.learning.join(', ')}.
        </p>
        <ol className="timeline">
          {profile.experience.map((x) => (
            <li key={x.org}>
              <p className="timeline__when">{x.when}
</p>
              <h3>{x.title} <span>· {x.org}</span>
</h3>
              <p><Txt v={x.detail} /></p>
            </li>
          ))}
        </ol>
      </Sheet>
    </Stop>
  )
}
```

src/dom/sections/Certs.jsx 53 lines

```
import { profile, realUrl } from
'../../content/profile'
import { useStore } from '../../core/store'
import Sheet from '../Sheet'
import Stop from '../Stop'
import Txt from '../Txt'

/*
 * Each gated cert shows a state that follows the 3D
scan:
 * pending → verified as the packet clears each
gate. In simple view (and
 * for screen readers) they're simply listed; the
"scan" is decoration.
 */
export default function Certs() {
  const stage = useStore((s) => s.gateStage)
  const simple = useStore((s) => s.mode ===
'simple')
  let gateNo = 0
  return (
    <Stop id="gate" className="panel-left"
labelledBy="certs-title">
      <Sheet no={1} title="SECURITY GATE">
        <p className="kicker">step 01 · auth</p>
        <h2 id="certs-title">Nothing gets in without
credentials.</h2>
        <p className="lede">These are mine. The
first two are the gates you just went through.</p>
        <ul className="certs">
          {profile.certifications.map((c) => {
            const n = c.gate ? ++gateNo : 0
            const ok = simple || !c.gate || stage >=
n
            const url = realUrl(c.verifyUrl)
            return (
              <li key={c.id} className={`cert ${ok ?
'is-ok' : ''}`}>
                <span className={`cert__state ${ok
&& c.gate ? 'stamp' : ''}`} aria-hidden="true">
{c.gate ? (ok ? 'verified' : 'scanning…') : 'on
file'}</span>
                <h3 className="cert__name">{c.name}
</h3>
                <p className="cert__score"><mark
className="hl">{c.score}</mark></p>
                <p className="cert__meta">
```

```
                  {c.date}
                  {c.valid && ` · ${c.valid}`}
                  {c.verifyUrl !== undefined && (
                    <> · {url ? <a href={url}
target="_blank" rel="noreferrer">verify ↗</a> : <Txt
v={c.verifyUrl} />}</>
                  )}
                </p>
              </li>
            )
          })}
          {profile.education.exams.map((x) => (
            <li key={x.name} className="cert is-ok">
              <span className="cert__state" aria-
hidden="true">on file</span>
              <h3 className="cert__name">{x.name}
</h3>
              <p className="cert__score">{x.result}
</p>
            </li>
          ))}
        </ul>
      </Sheet>
    </Stop>
  )
}
```

src/dom/sections/Contact.jsx 105 lines

```
/*
 * The response. CTA, links, resume and a contact
form.
 * The form posts straight to Web3Forms (free, no
backend of mine). The key
 * comes from .env.local (VITE_WEB3FORMS_KEY);
without one the form says so
 * instead of pretending to send.
 */
import { useState } from 'react'
import { profile, realUrl } from
'../../content/profile'
import Magnetic from '../Magnetic'
import Sheet from '../Sheet'
import Stop from '../Stop'
import Txt from '../Txt'

const KEY = import.meta.env.VITE_WEB3FORMS_KEY

function Out({ v, label, href }) {
  const url = realUrl(v)
  if (!url) return <li><span className="out__k">
{label}</span> <Txt v={v} /></li>
  return (
    <li>
      <span className="out__k">{label}</span>{' '}
      <Magnetic><a href={href ? href(url) : url}
target={href ? undefined : '_blank'}
rel="noreferrer">{url.replace(/^https?:\/\//, '')}
</a></Magnetic>
    </li>
  )
}

export default function Contact() {
  const [status, setStatus] = useState('idle') //
idle | sending | sent | error
  const resume = realUrl(profile.links.resume)

  async function submit(e) {
    e.preventDefault()
    if (!KEY) return setStatus('nokey')
    setStatus('sending')
    const data = Object.fromEntries(new
FormData(e.currentTarget))
    try {
      const res = await
fetch('https://api.web3forms.com/submit', {
        method: 'POST',
        headers: { 'Content-Type':
'application/json', Accept: 'application/json' },
        body: JSON.stringify({ ...data, access_key:
KEY, subject: 'Portfolio: new message' }),
      })
      const json = await res.json()
      setStatus(json.success ? 'sent' : 'error')
      if (json.success) e.target.reset()
    } catch {
      setStatus('error')
    }
  }

  return (
    <Stop id="response" className="contact"
labelledBy="contact-title">
      <Sheet no={10} title="RESPONSE 200">
        <p className="kicker">HTTP/1.1 200 OK ·
content-type: human</p>
        <h2 id="contact-title">The request made it
back. Your turn.</h2>
        <p className="lede">{profile.contact.cta}
</p>
        <div className="contact__grid">
          <div>
            <ul className="out">
              <Out v={profile.links.email}
label="mail" href={(u) => `mailto:${u}`} />
              <Out v={profile.links.github}
label="github" />
              <Out v={profile.links.linkedin}
label="linkedin" />
            </ul>
            {resume ? (
              <Magnetic>
                <a className="btn btn--solid" href=
{resume} download>download resume (pdf)</a>
              </Magnetic>
            ) : (
              <p className="btn btn--off">resume:
<Txt v={profile.links.resume} /></p>
            )}
          </div>
          <form className="form" onSubmit={submit}>
            <label>
              <span>name</span>
              <input name="name" required
autoComplete="name" />
            </label>
            <label>
              <span>email</span>
              <input name="email" type="email"
required autoComplete="email" />
            </label>
            <label>
              <span>message</span>
              <textarea name="message" rows={4}
required />
            </label>
            {/* Honeypot for bots; humans never see
it. */}
            <input type="checkbox" name="botcheck"
className="sr-only" tabIndex={-1} aria-hidden="true"
/>
            <Magnetic>
              <button className="btn btn--solid"
type="submit" disabled={status === 'sending'}>
                {status === 'sending' ? 'sending…' :
'send request'}
              </button>
            </Magnetic>
            <p className="form__status"
role="status">
              {status === 'sent' && '202 Accepted. I
will get back to you.'}
              {status === 'error' && '500. It did
not go through. Email works too.'}
              {status === 'nokey' && 'Form not wired
yet: add VITE_WEB3FORMS_KEY to .env.local (see
```

```
README).'}
            </p>
          </form>
        </div>
      </Sheet>
      <footer className="foot">
        <p>Drawn by hand with three.js, React Three
Fiber, GSAP and Lenis. No trackers, no cookies.</p>
        <p>Press <kbd>~</kbd> for a terminal.</p>
      </footer>
    </Stop>
  )
}
```

src/dom/sections/Hero.jsx 27 lines

```
import { hero, ARROW_PATHS } from
'../../content/hero'
import Stop from '../Stop'

/* Keep this markup in step with heroShell() in
vite.config.js. */
export default function Hero() {
  return (
    <Stop id="edge" className="hero"
labelledBy="hero-title">
      <div className="hero__inner">
        <p className="kicker">{hero.kicker}</p>
        {/* Real heading. In the 3D view the
particles draw it; see sections.css. */}
        <h1 id="hero-title" className="hero__title">
{hero.title}</h1>
        <p className="hero__line">{hero.line}</p>
        <p className="hero__meta">
          {hero.meta1}
          <br />
          {hero.meta2}
        </p>
        <p className="hero__hint" aria-
hidden="true">
          <svg width="54" height="40" viewBox="0 0
54 40" className="hero__arrow">
            {ARROW_PATHS.map((d) => <path key={d} d=
{d} />)}
          </svg>
          {hero.hint}
        </p>
      </div>
    </Stop>
  )
}
```

src/dom/sections/Project.jsx 57 lines

```
/*
 * One project district's case study. Fields follow
the same order every
 * time (problem → architecture → stack → role →
links) so a recruiter can
 * compare projects at a glance.
 */
import { isFill, realUrl } from
'../../content/profile'
import Magnetic from '../Magnetic'
import Sheet from '../Sheet'
import Stop from '../Stop'
import Txt from '../Txt'

function LinkOrFill({ v, label }) {
  const url = realUrl(v)
  if (url && /^https?:/.test(url))
    return (
      <Magnetic>
        <a className="btn" href={url}
target="_blank" rel="noreferrer">{label} ↗</a>
      </Magnetic>
    )
  return <span className="btn btn--off">{label}:
{isFill(v) ? <Txt v={v} /> : v}</span>
}

export default function Project({ p, no }) {
  const id = `${p.slug}-title`
  return (
    <Stop id={p.slug} className="panel-left"
labelledBy={id}>
      <Sheet no={no + 3} title=
{p.name.toUpperCase()}>
        <p className="kicker">service {String(no +
1).padStart(2, '0')} / 05 · <Txt v={p.status} /></p>
        <h2 id={id}>{p.name}</h2>
        <p className="lede">{p.kind}.</p>
        <p className="scene-note" aria-
hidden="true">↳ in the drawing: {p.scene}</p>
        <dl className="case" data-lenis-prevent>
          <dt>Problem</dt><dd><Txt v={p.problem} />
</dd>
          <dt>Architecture</dt><dd><Txt v=
{p.architecture} /></dd>
          <dt>Stack</dt><dd><Txt v={p.stack} /></dd>
          <dt>My role</dt><dd><Txt v={p.role} />
</dd>
        </dl>
        <div className="links">
          <LinkOrFill v={p.repo} label="repo" />
          <LinkOrFill v={p.demo} label="demo" />
        </div>
        {/* Screenshots are optional; nothing
renders until profile.js lists some. */}
        {p.screenshots?.length > 0 && (
          <ul className="shots">
            {p.screenshots.map((s) => (
              <li key={s.src}>
                <a href={s.src} target="_blank"
rel="noreferrer">
                  <img src={s.src} alt={s.alt}
loading="lazy" decoding="async" />
                </a>
              </li>
            ))}
          </ul>
        )}
      </Sheet>
    </Stop>
  )
}
```

src/dom/sections/Skills.jsx 38 lines

```
import { profile } from '../../content/profile'
import Sheet from '../Sheet'
import Stop from '../Stop'
import Txt from '../Txt'

export default function Skills() {
  return (
    <Stop id="data" className="panel-left"
labelledBy="skills-title">
      <Sheet no={9} title="DATA LAYER">
        <p className="kicker">step 04 · storage</p>
        <h2 id="skills-title">What's stored down
here.</h2>
        <p className="lede">The stack, in shards.
Only what I've actually used goes in.</p>
        <table className="shards">
          <tbody>
            {profile.skills.map((s, i) => (
              <tr key={s.group}>
                <th scope="row"><span
className="shards__no">{String(i).padStart(2, '0')}
</span>{s.group}</th>
                <td><Txt v={s.items} /></td>
              </tr>
            ))}
          </tbody>
        </table>
        {profile.otherRepos.length > 0 && (
          <>
```

```
            <h3 className="repos__h">Also on
GitHub</h3>
            <ul className="repos">
              {profile.otherRepos.map((r) => (
                <li key={r.url}>
                  <a href={r.url} target="_blank"
rel="noreferrer">{r.name}</a> · {r.line}
                </li>
              ))}
            </ul>
          </>
        )}
      </Sheet>
    </Stop>
  )
}
```

src/main.jsx 12 lines

```
import { createRoot } from 'react-dom/client'
import '@fontsource/young-serif/latin-400.css'
import '@fontsource/courier-prime/latin-400.css'
import '@fontsource/courier-prime/latin-400-
italic.css'
import '@fontsource/courier-prime/latin-700.css'
import './styles/tokens.css'
import './styles/base.css'
import './styles/sections.css'
import './styles/notebook.css'
import App from './App'

createRoot(document.getElementById('root')).render(<
App />)
```

src/styles/base.css 167 lines

```
*, *::before, *::after { box-sizing: border-box; }
html { -webkit-text-size-adjust: 100%; }
html.lenis, html.lenis body { height: auto; }
.lenis.lenis-smooth { scroll-behavior: auto
!important; }
.lenis.lenis-stopped { overflow: hidden; }

body {
  margin: 0;
  background: var(--paper);
  color: var(--ink);
  font: 400 14px/1.7 var(--mono);
  -webkit-font-smoothing: antialiased;
  overflow-x: hidden;
}
h1, h2, h3 { font-family: var(--display); font-
weight: 400; margin: 0; letter-spacing: -0.005em; }
p { margin: 0 0 0.9em; }
a { color: var(--ink); text-decoration-color: var(--
accent); text-underline-offset: 3px; text-
decoration-thickness: 1px; }
a:hover { color: var(--accent); text-decoration-
thickness: 2px; }
button { font: inherit; color: inherit; }
kbd { font: 700 0.85em var(--mono); border: var(--
line); padding: 0 0.35em; border-radius: 2px; }
::selection { background: var(--accent); color:
var(--paper-2); }

:focus-visible { outline: 2px solid var(--accent);
outline-offset: 3px; }

.sr-only {
  position: absolute !important; width: 1px; height:
1px; padding: 0; margin: -1px;
  overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap; border: 0;
}

/* Placeholders for missing facts: hatched so they
can't be mistaken for content. */
.fill {
  color: var(--ink);
  background: repeating-linear-gradient(135deg,
rgb(255 74 28 / 0.18) 0 4px, transparent 4px 8px);
  border: 1px dashed var(--accent);
  padding: 0 0.3em;
  font-size: 0.86em;
}
.fill--verify { border-style: dotted; }

/* Skip link: hidden until focused, first stop for
keyboard users. */
.skip {
  position: fixed; left: var(--gutter); top: 12px;
z-index: 100;
  background: var(--ink); color: var(--paper);
border: 0; padding: 10px 14px;
  transform: translateY(-160%); transition:
transform 0.3s var(--ease);
  cursor: pointer;
}
.skip:focus-visible { transform: none; }

/* The 3D canvas sits behind everything. */
.canvas-wrap { position: fixed; inset: 0; z-index:
0; }
.canvas-wrap canvas { display: block; }
main { position: relative; z-index: 2; }

/* Paper grain, generated by an SVG filter once, not
a downloaded texture. */
.grain {
  position: fixed; inset: -50%; z-index: 3; pointer-
events: none; opacity: 0.13; mix-blend-mode:
multiply;
  background-image: url("data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' width='220'
height='220'%3E%3Cfilter id='n'%3E%3CfeTurbulence
type='fractalNoise' baseFrequency='0.85'
numOctaves='2'
stitchTiles='stitch'/%3E%3C/filter%3E%3Crect
width='100%25' height='100%25'
filter='url(%23n)'/%3E%3C/svg%3E");
}
.grain::after {
  /* vignette: paper darkens toward the edges like a
print */
  content: ''; position: fixed; inset: 0;
  background: radial-gradient(ellipse at 55% 50%,
transparent 60%, rgb(120 100 70 / 0.18));
}

/* ---------- top bar ---------- */
.bar {
  position: fixed; inset: 0 0 auto; z-index: 20;
  display: flex; align-items: center; gap: 24px;
  padding: 14px var(--gutter);
  font-size: 12.5px; letter-spacing: 0.04em;
  /* paper fades in under the bar so content never
collides with it */
  background: linear-gradient(var(--paper) 55%,
color-mix(in srgb, var(--paper) 0%, transparent));
}
.bar a { text-decoration: none; }
.bar__mark { display: inline-flex; align-items:
center; gap: 8px; font-weight: 700; }
.bar__dot { width: 9px; height: 9px; border-radius:
50%; background: var(--accent); box-shadow: 0 0 0
3px rgb(255 74 28 / 0.2); }
.bar__dim { color: var(--faded); }
.bar__nav { display: flex; gap: 20px; margin-left:
auto; }
.bar__nav a { color: var(--ink-2); }
.bar__nav a:hover { color: var(--ink); }
.bar__ctl { display: flex; gap: 8px; }
.ctl {
  display: inline-flex; align-items: center; gap:
7px;
  background: transparent; border: var(--line);
padding: 5px 10px; cursor: pointer;
```

```
  font-size: 12px; letter-spacing: 0.04em; color:
var(--ink-2);
}
.ctl:hover { color: var(--ink); border-color: var(--
ink); }
.ctl--em { background: var(--ink); border-color:
var(--ink); color: var(--paper); }
.ctl--em:hover { background: var(--accent); border-
color: var(--accent); color: var(--paper-2); }
.ctl kbd { border: 0; padding: 0; color: var(--
accent); }
.ctl__wave { display: inline-flex; align-items:
center; gap: 2px; height: 10px; }
.ctl__wave i { width: 2px; height: 3px; background:
currentColor; }
.ctl__wave[data-on='true'] i { animation: wave 0.9s
var(--ease) infinite alternate; }
.ctl__wave i:nth-child(2) { animation-delay: 0.15s;
}
.ctl__wave i:nth-child(3) { animation-delay: 0.3s; }
.ctl__wave i:nth-child(4) { animation-delay: 0.45s;
}
@keyframes wave { to { height: 10px; } }

@media (max-width: 860px) {
  .bar { gap: 10px; flex-wrap: wrap; padding: 10px
16px; background: var(--paper); border-bottom: var(-
-line); }
  .bar__nav { display: none; }
  .bar__ctl { margin-left: auto; }
  .ctl { padding: 4px 7px; font-size: 11px; }
}

/* ---------- trace HUD ---------- */
.trace {
  position: fixed; right: var(--gutter); top: 50%;
transform: translateY(-50%); z-index: 10;
  font-size: 11px; letter-spacing: 0.06em; text-
transform: lowercase;
}
.trace__head { color: var(--faded); margin-bottom:
10px; }
.trace ol { list-style: none; margin: 0; padding: 0;
border-left: var(--line); }
.trace button {
  display: flex; align-items: center; gap: 10px;
background: none; border: 0; cursor: pointer;
  padding: 4px 0 4px 0; color: var(--faded);
transition: color 0.3s;
}
.trace__dot { width: 7px; height: 7px; margin-left:
-4px; background: var(--paper); border: 1px solid
var(--faded); transition: all 0.4s var(--ease); }
.trace .is-done button { color: var(--ink-2); }
.trace .is-done .trace__dot { background: var(--
faded); }
.trace .is-now button { color: var(--ink); }
.trace .is-now .trace__dot { background: var(--
accent); border-color: var(--accent); transform:
scale(1.4); }
.trace__label { opacity: 0; transform:
translateX(-4px); transition: all 0.3s var(--ease);
}
.trace:hover .trace__label, .trace:focus-within
.trace__label, .trace .is-now .trace__label {
opacity: 1; transform: none; }
@media (max-width: 860px), (max-height: 520px) {
.trace { display: none; } }

/* ---------- cursor ---------- */
html.has-cursor:not(.is-simple), html.has-
cursor:not(.is-simple) a, html.has-cursor:not(.is-
simple) button { cursor: none; }
html.has-cursor input, html.has-cursor textarea {
cursor: text; }
.cursor { position: fixed; inset: 0; pointer-events:
none; z-index: 200; }
.cursor__ring, .cursor__cross { position: fixed;
left: 0; top: 0; will-change: transform; }
.cursor__ring {
  width: 34px; height: 34px; margin: -17px 0 0
-17px; border: 1px solid var(--faded); border-
radius: 50%;
  transition: width 0.35s var(--ease), height 0.35s
var(--ease), margin 0.35s var(--ease), border-color
0.3s;
}
.cursor__ring.is-hot { width: 56px; height: 56px;
margin: -28px 0 0 -28px; border-color: var(--
accent); background: rgb(255 74 28 / 0.06); }
.cursor__cross::before, .cursor__cross::after {
content: ''; position: absolute; background: var(--
ink); }
.cursor__cross::before { width: 13px; height: 1px;
left: -6px; top: 0; }
.cursor__cross::after { width: 1px; height: 13px;
left: 0; top: -6px; }
.cursor__label { position: absolute; left: 14px;
top: 10px; font-size: 9.5px; color: var(--faded);
white-space: nowrap; }
html.is-simple .cursor { display: none; }

/* ---------- terminal ---------- */
.term {
  position: fixed; z-index: 150; left: 50%; bottom:
6vh; transform: translateX(-50%);
  width: min(720px, calc(100vw - 32px)); height:
min(420px, 60vh);
  display: flex; flex-direction: column;
  background: var(--ink); color: var(--paper);
border: 1px solid var(--ink); box-shadow: 10px 10px
0 var(--accent);
  font-size: 13px;
  animation: term-in 0.4s var(--ease);
}
@keyframes term-in { from { opacity: 0; transform:
translate(-50%, 16px); } }
.term__bar { display: flex; justify-content: space-
between; padding: 6px 12px; border-bottom: 1px solid
#333; color: #8a847a; font-size: 11.5px; }
.term__bar button { background: none; border: 0;
color: var(--faded); cursor: pointer; }
.term__log { flex: 1; overflow: auto; padding: 12px;
}
.term__log p { margin: 0; white-space: pre-wrap; }
.term__in { color: var(--accent); margin-top: 8px
!important; }
.term__out { color: #d8d2c6; }
.term__line { display: flex; gap: 8px; padding: 10px
12px; border-top: 1px solid #333; color: var(--
accent); }
.term__line input { flex: 1; background: none;
border: 0; color: var(--paper); font: inherit;
outline: none; caret-color: var(--accent); }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration:
0.01ms !important; transition-duration: 0.01ms
!important; }
}

/* The world fades in once it has loaded, instead of
popping in. */
.canvas-wrap { animation: world-in 1.4s var(--ease)
both; }
@keyframes world-in { from { opacity: 0; } }
```

src/styles/notebook.css 196 lines

```
/*
 * notebook.css — the "made by a person" layer, on
top of base + sections.
 *
 * Everything here is about small imperfections:
 *   - page outlines drawn through an SVG wobble
filter
 *   - pages taped down at slightly different angles
```

```
 *   - highlighter that doesn't quite stay inside
the lines
 *   - a rubber stamp instead of a status badge
 * Kept in one file so it's easy to see (and to
delete if it's too much).
 */

/* ---------- pages ---------- */
.sheet {
  border: 0;
  box-shadow: var(--shadow);
  background: var(--paper-2);
  rotate: var(--tilt, 0deg);
  /* faint ruled lines, like the page came out of a
notebook */
  background-image: repeating-linear-gradient(
    to bottom,
    transparent 0 27px,
    rgb(42 68 168 / 0.07) 27px 28px
  );
  background-position: 0 14px;
}
/* The wobbly outline. Separate layer so the filter
never blurs the text. */
.sheet__edge {
  position: absolute; inset: 0; pointer-events:
none;
  border: 1.5px solid var(--ink);
  filter: url(#rough);
}
/* Red margin line on the left, the classic
exercise-book rule. */
.sheet__body { border-left: 1px solid rgb(200 55 45
/ 0.35); margin-left: 18px; padding-left: 22px; }

.sheet__tape {
  position: absolute; top: -11px; left: var(--tape-
x, 40%);
  width: 92px; height: 24px;
  background: rgb(243 223 87 / 0.42);
  border-left: 1px dashed rgb(43 42 40 / 0.12);
  border-right: 1px dashed rgb(43 42 40 / 0.12);
  rotate: calc(var(--tilt, 0deg) * -4 - 2deg);
  mix-blend-mode: multiply;
}
.sheet__mark { display: none; }

.sheet__block {
  grid-template-columns: auto 1fr auto;
  border-top: 1px dashed color-mix(in srgb, var(--
faded) 70%, transparent);
  font-size: 11px; letter-spacing: 0.04em; text-
transform: none;
}
.sheet__block span { border-left: 0; }
.sheet__block span:first-child { color: var(--
accent); font-style: italic; }

.sheet h2 { font-size: clamp(28px, 2.7vw, 40px);
line-height: 1.08; letter-spacing: -0.01em; }
.kicker { font-size: 12px; letter-spacing: 0.06em;
text-transform: none; font-style: italic; color:
var(--faded); }

/* ---------- writing ---------- */
body { font-size: 15px; line-height: 1.6; }
a { color: var(--pen); text-decoration-color: rgb(42
68 168 / 0.45); }
a:hover { color: var(--accent); }

/* Highlighter: slightly crooked, slightly too
short, slightly see-through. */
.hl {
  color: inherit;
  background: linear-gradient(100deg, transparent 0
1%, rgb(243 223 87 / 0.85) 2% 96%, transparent 98%);
  background-size: 100% 62%;
  background-position: 0 78%;
  background-repeat: no-repeat;
  padding: 0 0.2em;
  border-radius: 2px 6px 3px 5px;
}

/* Missing info: written in red pen, circled. The
circle is a pseudo-element
   so only the circle wobbles, not the letters. */
.fill {
  position: relative;
  background: none; border: 0;
  color: var(--accent);
  font-style: italic;
  padding: 0 0.5em;
}
.fill::before {
  content: ''; position: absolute; inset: -2px -1px;
  border: 1.4px solid var(--accent);
  border-radius: 40% 55% 45% 60% / 60% 45% 55% 40%;
  filter: url(#rough);
  pointer-events: none;
}

/* ---------- certs ---------- */
.cert { border: 0; border-bottom: 1px dashed color-
mix(in srgb, var(--faded) 70%, transparent);
padding: 12px 2px 12px; }
.cert.is-ok { border-color: color-mix(in srgb, var(-
-faded) 70%, transparent); }
.cert__state { font-size: 12px; letter-spacing:
0.04em; color: var(--faded); font-style: italic;
top: 14px; }
.cert:not(.is-ok) .cert__state { color: var(--
accent); }
.cert__score { font-size: 21px; }

/* Rubber stamp. Rotated, uneven ink, a bit faded.
*/
.stamp {
  color: var(--accent) !important;
  font: 700 12px/1 var(--mono) !important;
  font-style: normal !important;
  text-transform: uppercase; letter-spacing: 0.18em
!important;
  border: 2px solid var(--accent);
  padding: 5px 7px 4px;
  rotate: -7deg;
  opacity: 0.82;
  filter: url(#rough-strong);
  animation: stamp 0.35s cubic-bezier(0.3, 1.6, 0.5,
1);
}
@keyframes stamp { from { scale: 1.8; opacity: 0; }
}

/* ---------- timeline / case / shards ---------- */
.timeline { border-left: 1.5px solid var(--faded); }
.timeline li::before { width: 9px; height: 9px;
left: -5.5px; border-radius: 50%; background: var(--
paper-2); border: 1.5px solid var(--accent); }
.case { border-top: 0; }
.case dt, .case dd { border-bottom: 1px dashed
color-mix(in srgb, var(--faded) 60%, transparent); }
.case dt { text-transform: none; letter-spacing:
0.02em; font-style: italic; font-size: 13px; }
.scene-note { color: var(--pen) !important; }
.shards th, .shards td { border-bottom: 1px dashed
color-mix(in srgb, var(--faded) 60%, transparent); }
.shards__no { color: var(--accent); font-style:
italic; }

/* ---------- buttons: drawn boxes ---------- */
.btn { position: relative; border: 0; background:
none; color: var(--ink); }
.btn::before { content: ''; position: absolute;
inset: 0; border: 1.5px solid var(--ink); filter:
url(#rough); transition: background 0.2s; }
.btn:hover { background: none; color: var(--accent);
```

```
}
.btn:hover::before { border-color: var(--accent); }
.btn--solid { color: var(--paper-2); }
.btn--solid::before { background: var(--accent);
border-color: var(--accent); z-index: -1; }
.btn--solid { isolation: isolate; }
.btn--solid:hover { color: var(--paper-2); }
.btn--solid:hover::before { background: var(--ink);
border-color: var(--ink); }
.btn--off { color: var(--faded); }
.btn--off::before { border-style: dashed; border-
color: var(--faded); }
.btn--off:hover { color: var(--faded); }
.btn--off:hover::before { border-color: var(--
faded); }

.form input, .form textarea {
  background: transparent; border: 0; border-bottom:
1.5px solid var(--ink-2);
  padding: 6px 2px; font-size: 15px;
}
.form textarea { border: 1.5px solid var(--ink-2);
padding: 8px; }
.form label span, .out__k { text-transform: none;
letter-spacing: 0.02em; font-style: italic; font-
size: 13px; }

/* ---------- top bar & controls ---------- */
.bar__dot { box-shadow: none; border-radius: 48% 52%
45% 55%; }
.ctl { border: 0; position: relative; color: var(--
ink-2); font-size: 13px; }
.ctl::before { content: ''; position: absolute;
inset: 0; border: 1.2px solid var(--faded); filter:
url(#rough); }
.ctl:hover { color: var(--accent); }
.ctl--em { background: none; color: var(--paper-2);
isolation: isolate; }
.ctl--em::before { background: var(--ink); border-
color: var(--ink); z-index: -1; }
.ctl--em:hover { background: none; color: var(--
paper-2); }
.ctl--em:hover::before { background: var(--accent);
border-color: var(--accent); }
.bar__nav a { color: var(--ink-2); font-size:
13.5px; }
.bar__nav a:hover { color: var(--accent); text-
decoration: underline wavy var(--accent) 1px; }

/* ---------- hero ---------- */
.hero__title { color: var(--pen); }
.hero__line { color: var(--ink); }
.hero__meta { color: var(--ink-2); font-size:
13.5px; }
.hero__hint { color: var(--accent); font-size: 14px;
letter-spacing: 0.02em; text-transform: none; font-
style: italic; }
.hero__arrow { fill: none; stroke: var(--accent);
stroke-width: 1.6; stroke-linecap: round; stroke-
linejoin: round; filter: url(#rough); }
.hero__arrow path:first-child { stroke-dasharray:
60; animation: draw 2.4s var(--ease) infinite; }
@keyframes draw { 0% { stroke-dashoffset: 60; } 50%,
100% { stroke-dashoffset: 0; } }

/* ---------- trace & cursor ---------- */
.trace { font-size: 12.5px; text-transform: none;
letter-spacing: 0.02em; }
.trace__head { font-style: italic; }
.trace ol { border-left: 1.5px dashed var(--faded);
}
.trace__dot { border-radius: 50%; background: var(--
paper); }
.cursor__ring { border: 1.5px solid var(--faded);
border-radius: 46% 54% 50% 50%; }
.cursor__cross::before, .cursor__cross::after {
display: none; }
.cursor__cross { width: 6px; height: 6px; margin:
-3px 0 0 -3px; border-radius: 50%; background: var(-
-accent); }

/* ---------- terminal: a graphite slab ----------
*/
.term { background: #262522; box-shadow: 5px 6px 0
rgb(43 42 40 / 0.25); border: 0; rotate: -0.4deg; }
.term__out { color: #ddd5c4; }

/* ---------- simple view: one long notebook page --
-------- */
html.is-simple .sheet { rotate: 0deg; }
html.is-simple .sheet__tape { display: none; }

@media (max-width: 860px) {
  .sheet__body { margin-left: 10px; padding-left:
14px; }
  .sheet { rotate: calc(var(--tilt, 0deg) * 0.5); }
}

/* ---------- optional media (only shows once
profile.js has it) ---------- */
.shots { list-style: none; padding: 0; margin: 16px
0 4px; display: grid; grid-template-columns:
repeat(auto-fill, minmax(120px, 1fr)); gap: 10px; }
.shots img { display: block; width: 100%; aspect-
ratio: 16 / 10; object-fit: cover; border: 1.5px
solid var(--ink); background: var(--paper-3); }
.shots li:nth-child(odd) img { rotate: -1deg; }
.shots li:nth-child(even) img { rotate: 0.8deg; }
.polaroid { float: right; width: 112px; margin: 0 0
10px 14px; padding: 6px 6px 18px; background:
#fbf8f1; box-shadow: var(--shadow); rotate: 3deg; }
.polaroid img { display: block; width: 100%; aspect-
ratio: 1; object-fit: cover; filter: grayscale(0.2)
sepia(0.15); }
.repos__h { font-size: 20px; margin: 20px 0 6px; }
.repos { padding-left: 18px; margin: 0; }
.repos li { padding: 3px 0; color: var(--ink-2); }
```

src/styles/sections.css 163 lines

```
/* ---------- the journey layout ---------- */
.stop { position: relative; }
.stop__sticky {
  position: sticky; top: 0; height: 100vh; height:
100svh;
  display: flex; align-items: center;
  padding: 72px var(--gutter) 40px;
  pointer-events: none;             /* let the
canvas get the cursor... */
}
.stop__sticky > * { pointer-events: auto; }  /*
...except on the panel itself */

/* Panels fade in when their stop is active (class
set by core/scroll.js). */
.stop:not(.stop--simple) .sheet,
.stop:not(.stop--simple) .hero__inner {
  opacity: 0; transform: translateY(26px);
  transition: opacity 0.7s var(--ease), transform
0.9s var(--ease);
}
.stop.is-on .sheet, .stop.is-on .hero__inner,
.stop:focus-within .sheet, .stop:focus-within
.hero__inner { opacity: 1; transform: none; }

/* ---------- the sheet (drawing panel) ----------
*/
.sheet {
  position: relative;
  width: min(540px, 44vw);
  max-height: calc(100svh - 120px);
  display: flex; flex-direction: column;
  background: var(--paper-2);
  border: 1px solid var(--ink);
  box-shadow: var(--shadow);
}
.sheet__body { padding: 26px 28px 18px; overflow:
```

```
auto; overscroll-behavior: contain; scrollbar-width:
thin; }
.sheet__mark { position: absolute; width: 14px;
height: 14px; border-color: var(--accent); border-
style: solid; border-width: 0; }
.sheet__mark--tl { left: -8px; top: -8px; border-
left-width: 1px; border-top-width: 1px; }
.sheet__mark--tr { right: -8px; top: -8px; border-
right-width: 1px; border-top-width: 1px; }
.sheet__mark--bl { left: -8px; bottom: -8px; border-
left-width: 1px; border-bottom-width: 1px; }
.sheet__mark--br { right: -8px; bottom: -8px;
border-right-width: 1px; border-bottom-width: 1px; }
.sheet__block {
  display: grid; grid-template-columns: auto 1fr
auto auto;
  border-top: var(--line); font-size: 10px; letter-
spacing: 0.12em; color: var(--faded);
}
.sheet__block span { padding: 6px 10px; border-left:
var(--line); white-space: nowrap; overflow: hidden;
text-overflow: ellipsis; }
.sheet__block span:first-child { border-left: 0;
color: var(--accent); }

.sheet h2 { font-size: clamp(32px, 3.2vw, 48px);
line-height: 1; margin-bottom: 14px; }
.sheet h2 em, .hero__line em { color: var(--accent);
}
.kicker { font-size: 11px; letter-spacing: 0.14em;
text-transform: uppercase; color: var(--faded);
margin-bottom: 14px; }
.lede { color: var(--ink); }
.sheet p { color: var(--ink-2); }
.sheet .lede { color: var(--ink); }

/* ---------- hero ---------- */
.hero .stop__sticky { align-items: flex-end;
padding-bottom: 6vh; }
.hero__inner { max-width: 640px; }
.hero__title {
  font-size: clamp(60px, 12vw, 180px); line-height:
0.88; margin: 6px 0 22px;
  letter-spacing: -0.02em;
}
/* In the 3D view the particles draw the name, so
the HTML heading is visually
   hidden from the start (hiding it later would make
the page jump). It stays in
   the DOM for search engines and screen readers.
The simple view shows it. */
html:not(.is-simple) .hero__title {
  position: absolute !important; width: 1px; height:
1px; overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap;
}
.hero__line { font-family: var(--display); font-
size: clamp(22px, 1.9vw, 28px); line-height: 1.15;
max-width: 30ch; color: var(--ink); }
.hero__meta { color: var(--faded); font-size:
12.5px; }
.hero__hint { display: flex; align-items: center;
gap: 12px; font-size: 11.5px; letter-spacing: 0.1em;
color: var(--accent); margin-top: 26px; text-
transform: uppercase; }
.hero__hint-line { width: 48px; height: 1px;
background: var(--accent); transform-origin: left;
animation: hint 2.2s var(--ease) infinite; }
@keyframes hint { 0% { transform: scaleX(0); } 60%,
100% { transform: scaleX(1); opacity: 0; } }

/* ---------- certs ---------- */
.certs { list-style: none; padding: 0; margin: 18px
0 0; display: grid; gap: 10px; }
.cert { position: relative; border: var(--line);
padding: 12px 14px 10px; transition: border-color
0.5s, background 0.5s; }
.cert__state { position: absolute; right: 12px; top:
12px; font-size: 10px; letter-spacing: 0.12em;
color: var(--accent); }
.cert.is-ok { border-color: var(--ink); }
.cert.is-ok .cert__state { color: var(--ink); }
.cert:not(.is-ok) .cert__state { animation: blink
0.9s steps(2) infinite; }
@keyframes blink { 50% { opacity: 0.3; } }
.cert__name { font-size: 21px; line-height: 1.15;
padding-right: 110px; }
.cert__score { font: 700 22px/1.2 var(--mono);
color: var(--ink) !important; margin: 4px 0 2px; }
.cert__meta { font-size: 12px; color: var(--faded)
!important; margin: 0; }

/* ---------- about timeline ---------- */
.timeline { list-style: none; padding: 0; margin:
18px 0 0; border-left: var(--line); }
.timeline li { padding: 0 0 12px 18px; position:
relative; }
.timeline li::before { content: ''; position:
absolute; left: -4px; top: 8px; width: 7px; height:
7px; background: var(--accent); }
.timeline h3 { font-size: 21px; }
.timeline h3 span { font-family: var(--mono); font-
size: 12.5px; font-weight: 400; color: var(--faded);
}
.timeline__when { font-size: 11px; letter-spacing:
0.1em; color: var(--faded) !important; margin: 0; }

/* ---------- case study ---------- */
.scene-note { font-size: 12px; color: var(--faded)
!important; font-style: italic; }
.case { display: grid; grid-template-columns: 112px
1fr; margin: 16px 0; border-top: var(--line); }
.case dt, .case dd { margin: 0; padding: 9px 0;
border-bottom: var(--line); }
.case dt { font-size: 11px; letter-spacing: 0.12em;
text-transform: uppercase; color: var(--faded);
padding-top: 11px; }
.links { display: flex; flex-wrap: wrap; gap: 10px;
}
.btn {
  display: inline-block; padding: 8px 14px; border:
1px solid var(--ink); text-decoration: none;
  font-size: 13px; background: transparent; cursor:
pointer;
}
.btn:hover { background: var(--ink); color: var(--
paper); text-decoration: none; }
.btn--solid { background: var(--accent); border-
color: var(--accent); color: var(--paper-2); }
.btn--solid:hover { background: var(--ink); border-
color: var(--ink); color: var(--paper); }
.btn--off { border-style: dashed; border-color:
var(--faded); color: var(--faded); cursor: default;
}
.btn--off:hover { background: none; color: var(--
faded); }
.magnetic { display: inline-block; will-change:
transform; }

/* ---------- skills ---------- */
.shards { width: 100%; border-collapse: collapse;
margin-top: 12px; }
.shards th, .shards td { text-align: left; vertical-
align: top; padding: 10px 0; border-bottom: var(--
line); }
.shards th { font: 400 20px/1.2 var(--display);
width: 46%; padding-right: 14px; }
.shards__no { display: block; font: 400 10px var(--
mono); color: var(--accent); letter-spacing: 0.1em;
}
.shards td { color: var(--ink-2); }

/* ---------- contact ---------- */
.contact { padding: 85vh var(--gutter) 0; }
.contact .stop__sticky { position: static; height:
auto; padding: 0; display: block; pointer-events:
```

```
auto; }
.contact .sheet { width: min(980px, 100%); margin: 0
auto; max-height: none; }
.contact__grid { display: grid; grid-template-
columns: 1fr 1.2fr; gap: 32px; margin-top: 20px; }
.out { list-style: none; padding: 0; margin: 0 0
18px; }
.out li { padding: 8px 0; border-bottom: var(--
line); }
.out__k { display: inline-block; width: 74px; font-
size: 11px; letter-spacing: 0.12em; text-transform:
uppercase; color: var(--faded); }
.form { display: grid; gap: 12px; }
.form label { display: grid; gap: 4px; }
.form label span { font-size: 11px; letter-spacing:
0.12em; text-transform: uppercase; color: var(--
faded); }
.form input, .form textarea {
  background: var(--paper); border: 1px solid var(--
ink); color: var(--ink); font: inherit; padding: 9px
11px; resize: vertical;
}
.form input:focus, .form textarea:focus { outline:
none; border-color: var(--accent); }
.form__status { min-height: 1.4em; font-size:
12.5px; }
.foot { display: flex; justify-content: space-
between; flex-wrap: wrap; gap: 8px; padding: 40px 0
28px; font-size: 11.5px; color: var(--faded); max-
width: 980px; margin: 0 auto; }
.foot p { margin: 0; }

/* ---------- simple view ---------- */
html.is-simple main { max-width: 860px; margin: 0
auto; padding: 84px var(--gutter) 0; }
html.is-simple .grain { opacity: 0.05; }
.stop--simple { margin-bottom: 34px; }
.stop--simple .sheet { width: 100%; max-height:
none; }
.stop--simple.hero { padding: 6vh 0 2vh; }
.stop--simple.contact { padding: 0; }
.simple-h { font-size: 13px; font-family: var(--
mono); font-weight: 500; letter-spacing: 0.14em;
text-transform: uppercase; color: var(--faded);
margin: 44px 0 14px; }
.notice { border: 1px dashed var(--accent); padding:
10px 14px; font-size: 13px; }
.notice button { background: none; border: 0; color:
var(--ink); text-decoration: underline; text-
decoration-color: var(--accent); cursor: pointer;
padding: 0; }

/* ---------- phones: panel becomes a bottom sheet,
world sits above it ---------- */
@media (max-width: 860px) {
  .stop__sticky { align-items: flex-end; padding:
60px 16px 16px; }
  .sheet { width: 100%; max-height: 54svh; }
  .sheet__body { padding: 18px 18px 12px; }
  .sheet h2 { font-size: 24px; }
  .hero .stop__sticky { padding-bottom: 7vh; }
  .cert__name { padding-right: 0; padding-top: 14px;
}
  .case { grid-template-columns: 1fr; }
  .case dt { border-bottom: 0; padding-bottom: 0; }
  .contact__grid { grid-template-columns: 1fr; }
  .contact { padding: 70vh 16px 0; }
  .contact .sheet { max-height: none; }
  .stop--simple .sheet { max-height: none; }
}

/* ---------- pre-rendered hero (index.html, before
React mounts) ---------- */
.shell { height: var(--shell-h, 230vh); }
html.is-simple .shell-main { max-width: 860px;
margin: 0 auto; padding: 84px var(--gutter) 0; }
html.is-simple .shell { height: auto; padding: 6vh 0
2vh; }
html.is-simple .shell .stop__sticky { position:
static; height: auto; padding: 0; display: block; }
```

src/styles/tokens.css 29 lines

```
/*
 * Design tokens.
 *
 * The idea: this portfolio is my field notebook.
Manila paper, graphite for
 * writing, a blue ballpoint for the drawings, a red
pen for the one thing
 * that matters (the request), and a yellow
highlighter I'm only allowed to
 * use on facts I can prove (scores, dates).
 */
:root {
  --paper: #ece3d0;        /* manila */
  --paper-2: #f4eddd;      /* a fresh page taped on
top */
  --paper-3: #e2d7c0;      /* pencil-smudged well */
  --ink: #2b2a28;          /* graphite, for text */
  --ink-2: #4a4740;
  --faded: #676052;        /* soft pencil (dark
enough for 4.5:1 on paper) */
  --pen: #2a44a8;          /* blue ballpoint:
drawings and links */
  --accent: #b3312a;       /* red pen: the request,
live state (4.5:1 for small text) */
  --hl: #f3df57;           /* highlighter */

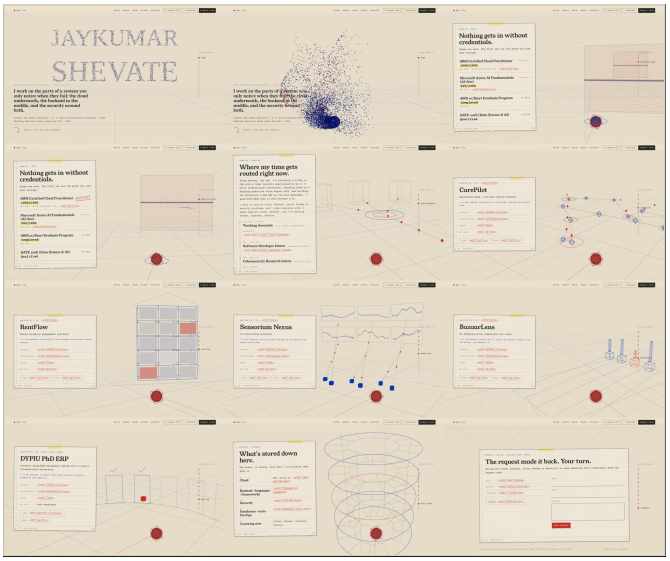
  --display: 'Young Serif', Georgia, serif;
  --mono: 'Courier Prime', 'Courier New', ui-
monospace, monospace;

  --ease: cubic-bezier(0.16, 1, 0.3, 1);
  --gutter: clamp(16px, 4vw, 64px);
  --line: 1px solid color-mix(in srgb, var(--faded)
55%, transparent);
  --shadow: 3px 4px 0 rgb(43 42 40 / 0.14);

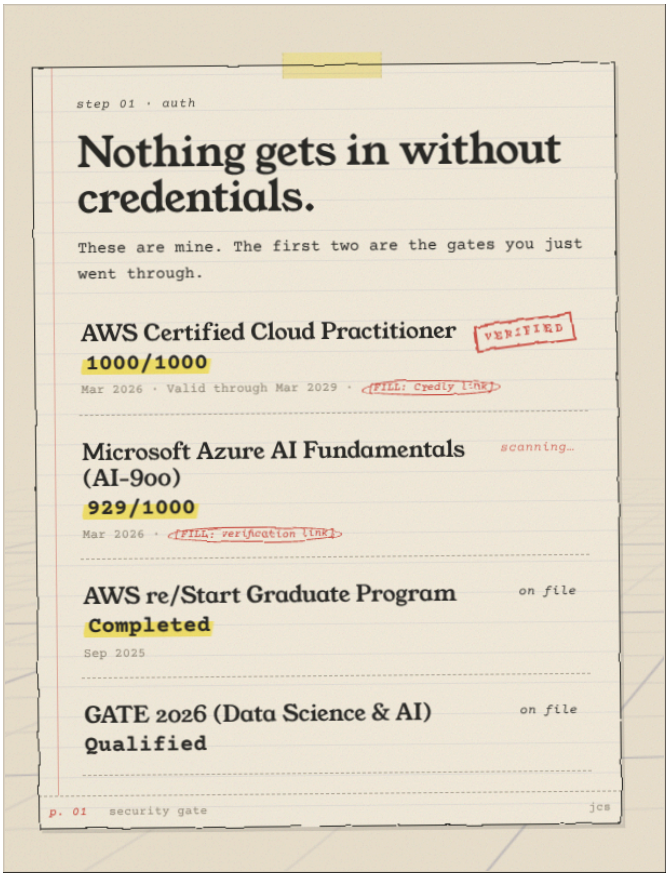
  color-scheme: light;
}
```

APPENDIX D. SCREENSHOT GALLERY

Screenshots captured from the production builds during the project with headless Chromium at 1440 by 900 pixels unless stated. As noted in Section 9, captures of the Cyanotype and Paper and Ink looks were overwritten during the session and could not be included.



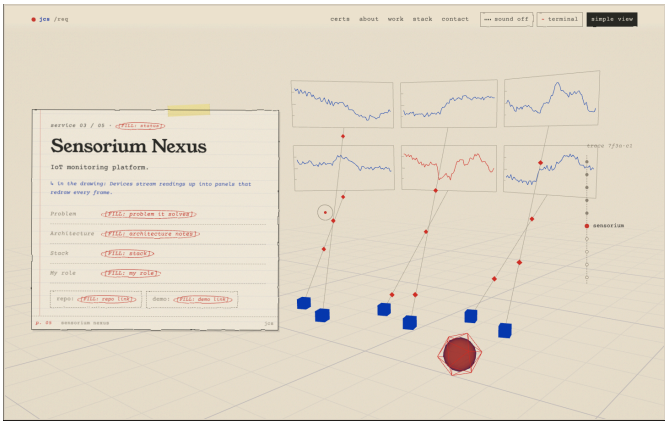
Version 1, Field Notebook look: twelve stops of the 3D journey (hero, name dissolving into the packet, security gate scanning and opening, load balancer, the five project districts, data layer, response), captured from the production build on 4 November.



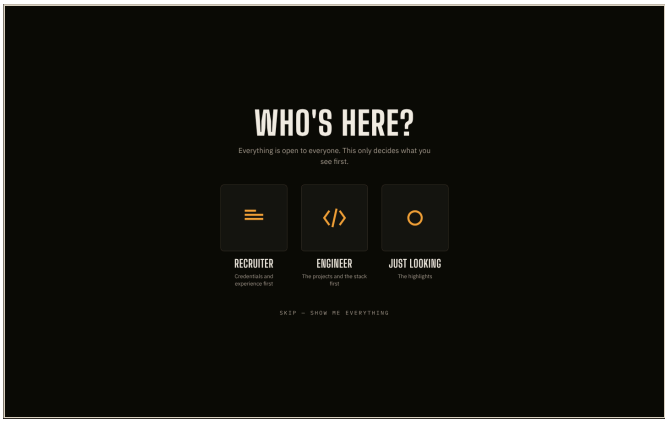
Version 1, close-up of the certifications page: ruled paper, taped page, highlighted scores, rubber "verified" stamp, and circled red placeholders.



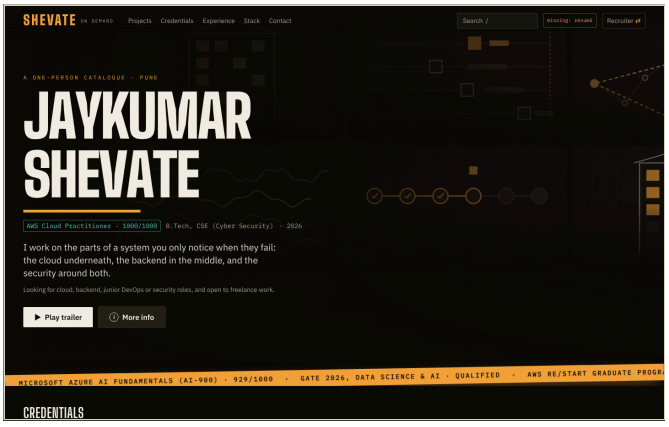
Version 1: the terminal step that exposed defect D9 (blank frame mid-scroll), the simple view for recruiters, and four phone screens.



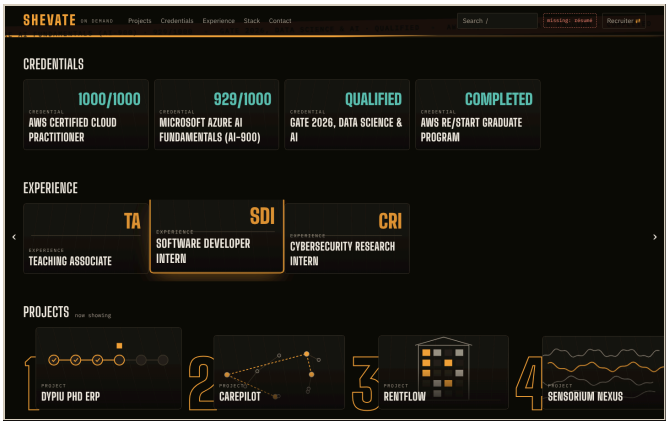
Version 1: the terminal command "open sensorium" flew the camera to the Sensorium Nexus district.



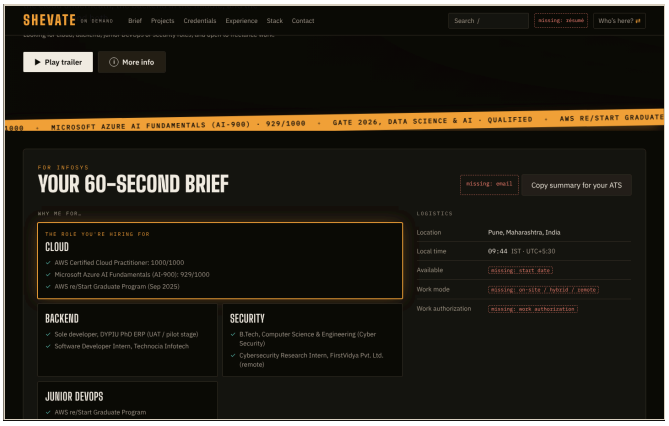
Version 2: "Who's here?", the viewer picker.



Version 2: the billboard with kinetic name, key credential, tagline, buttons and highlights ticker.



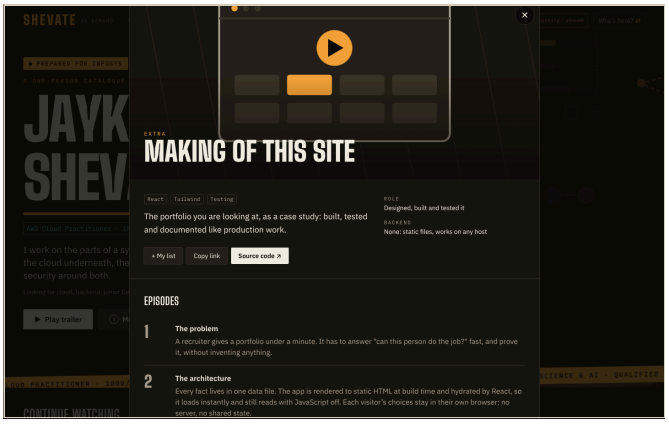
Version 2: rows of titles with SVG posters.



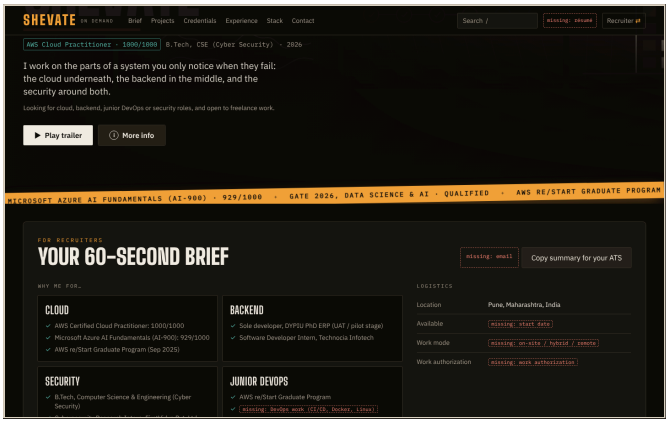
Version 2: the tailored brief with Cloud highlighted and the live Pune clock.



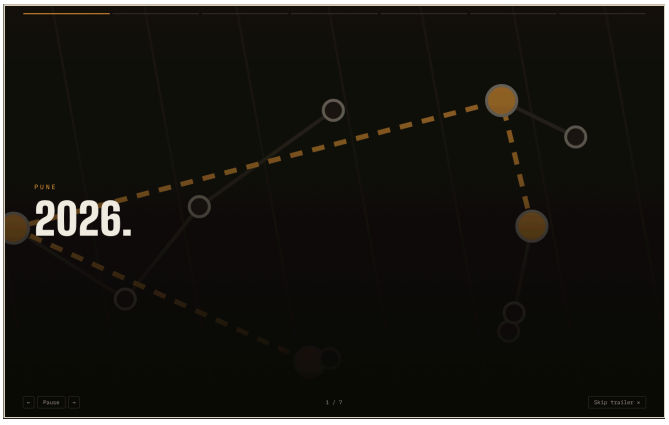
Version 2: a title page with facts, links and episodes; placeholders shown as "missing".



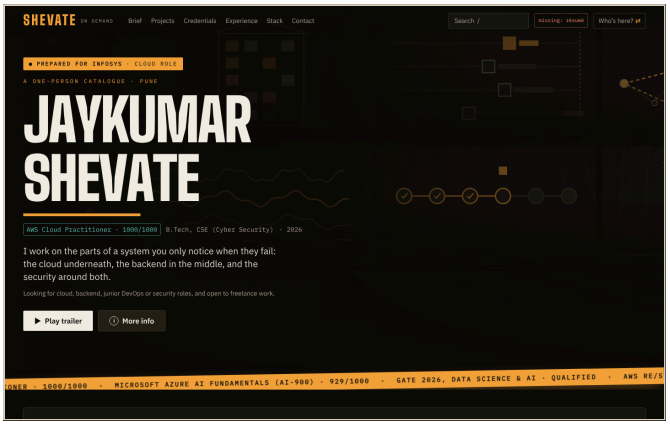
Version 2: the "Making of this site" extra.



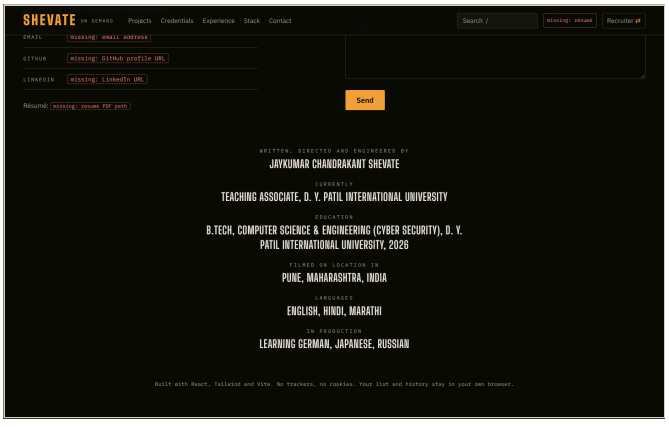
Version 2: the 60-second recruiter brief with role evidence and logistics.



Version 2: the trailer, first shot.



Version 2: a tailored link for Infosys, cloud role.



Version 2: "Roll credits", the contact section and credits roll.

APPENDIX E. HOW THIS DOCUMENT WAS VERIFIED

The owner asked for this document to be created, checked, and checked again before delivery. The checks below were run on the final rendered PDF, and the figures in this appendix are measured from that file, not estimated.

E.1 Facts

Every number in the narrative was traced to a source before inclusion. Commit hashes, dates, file counts and line counts were read from `git log`. Test counts, scenario names, durations and load figures were copied from the logs of the test runs; the final run is reproduced in Appendix A, and its totals (25 unit tests, 18 browser scenarios, two load targets with 0 failed requests, exit code 0) match Sections 3 and 18. Lighthouse figures were read from the twelve saved JSON reports. The missing-facts list was generated by `npm run todo`. Bundle sizes were measured from the build. The remote branch was checked against the local one. During checking, four statements in the draft were found to be wrong and were corrected before rendering: the number of owner messages (20, not 24), the number of clarifying questions (3, asked in 2 prompts), the number of defects (32 plus 4 test corrections, not 27), and the load-test history (7 runs, not 2). One stale test title found while checking ("same four rows", when there are now five) was renamed in the repository, and the unit tests were re-run: 25 passed.

E.2 Layout

The document was rendered by headless Chromium from HTML to A4 PDF. It has exactly 100 pages. The contents page numbers were produced by rendering once, locating every section heading in the extracted text of the PDF, writing the page numbers into the contents table and rendering again; a third extraction confirmed that every section starts on the page the contents table states. To check that no page is left blank, every page was rasterised and the lowest printed line inside the area between the running header and footer was measured. The mean fill across all pages is 98.4 percent of the printable height; 97 of 100 pages are filled to at least 90 percent; the least-filled page is page 100 at 89 percent (the closing page of the document). No page contains an empty band taller than 4 percent of the printable height. No page breaks were forced anywhere except after the cover, so sections run on continuously.

E.3 Completeness

Every message from the owner appears in the chronology (Section 5), every commit in the commit log (Section 24), every push in Section 20.3, every saved Lighthouse report in Section 19, every defect in Sections 10 and 18.7, and every test in Sections 18.2 and 18.3. Every tracked text file of the current version is printed in full in Appendix B and every text file of version 1 in Appendix C, with the lockfile and binary media as the stated exceptions.

E.4 Re-running the checks yourself

Everything this document reports about the product can be re-checked from a fresh clone of the repository with Node.js 20 or newer. Run `git clone https://github.com/Jayshevate/Portfoilo`, then `npm install`, then `npx playwright install chromium` (once, for the browser tests), then `npm run check`. That single command runs the 25 unit tests, builds and pre-renders the site, runs the 18 browser scenarios against the build, and runs the load test; it exits with code 0 only if every one passes. `npm run todo` prints the list in Section 23.1, `git log --stat` prints the commit log in Section 24, and running Lighthouse against `npm run preview` reproduces the measurements in Section 19, allowing for differences between machines.

End of document. Prepared 9 November 2026 from the repository at `https://github.com/Jayshevate/Portfoilo`, the saved test and audit reports, and the working session. Every figure in this record can be re-checked by running `npm run check` in that repository.